Deep Learning: From Foundations to Frontiers

A Comprehensive Tutorial Organized by Model Families

Nurdun Mijit

May 26, 2026

Contents

Preface **ii**

I Foundational Networks 1

1 Feedforward and Deep Neural Networks 2
1.1 Motivation . . . 2
1.2 Mathematical Formulation . . . 2
1.2.1 The Perceptron . . . 2
1.2.2 The XOR Problem . . . 2
1.2.3 Multilayer Perceptron (MLP) . . . 3
1.2.4 Activation Functions . . . 3
1.2.5 Universal Approximation Theorem . . . 4
1.2.6 Depth vs. Width . . . 4
1.2.7 Vanishing and Exploding Gradients . . . 5
1.3 Architecture . . . 5
1.4 Pros and Cons . . . 5
1.4.1 Strengths . . . 5
1.4.2 Limitations . . . 6
1.5 Evolution: What Comes Next . . . 6

2 Training Deep Networks 7
2.1 Motivation . . . 7
2.2 Loss Functions . . . 7
2.2.1 Mean Squared Error . . . 7
2.2.2 Cross-Entropy for Multi-Class Classification . . . 7
2.2.3 Binary Cross-Entropy . . . 8
2.3 Backpropagation . . . 9
2.3.1 Notation and Forward Pass . . . 9
2.3.2 Chain Rule for a Single Layer . . . 9
2.3.3 The Backward Recursion . . . 9
2.3.4 Weight and Bias Gradients . . . 10
2.3.5 Full Backpropagation Algorithm for a 3-Layer Sigmoid Network . . . 10
2.3.6 Vanishing Gradient Problem . . . 11
2.4 Gradient Descent Variants . . . 11
2.4.1 Batch Gradient Descent . . . 11
2.4.2 Stochastic Gradient Descent . . . 11
2.4.3 Mini-Batch SGD . . . 12
2.5 Momentum . . . 12
2.5.1 Classical Momentum . . . 12
2.5.2 Nesterov Accelerated Gradient . . . 13
2.6 Adaptive Learning Rate Methods . . . 13
2.6.1 AdaGrad . . . 13
2.6.2 RMSProp . . . 14
2.6.3 Adam: Adaptive Moment Estimation . . . 14

2.7 Weight Initialization . . . 15
2.7.1 Xavier/Glorot Initialization . . . 15
2.7.2 He/Kaiming Initialization . . . 16
2.8 Batch Normalization . . . 16
2.8.1 Forward Pass . . . 16
2.8.2 Backward Pass Through BatchNorm . . . 17
2.8.3 Inference . . . 18
2.9 Layer Normalization . . . 18
2.10 Dropout . . . 19
2.10.1 Bernoulli Mask Formulation . . . 19
2.10.2 Ensemble Interpretation . . . 19
2.11 Regularization via Parameter Norm Penalties . . . 19
2.11.1 L2 Regularization and Gaussian Prior . . . 19
2.11.2 L1 Regularization and Laplace Prior . . . 20
2.12 Learning Rate Scheduling and Gradient Clipping . . . 20
2.12.1 Learning Rate Schedules . . . 20
2.12.2 Gradient Clipping . . . 20
2.13 Algorithm: Training Loop with Adam and BatchNorm . . . 21
2.14 Pros and Cons . . . 21
2.15 Evolution . . . 22

II Convolutional Architectures 23

3 Convolutional Neural Networks 24
3.1 Motivation . . . 24
3.2 Mathematical Formulation . . . 24
3.2.1 One-Dimensional Convolution . . . 24
3.2.2 Two-Dimensional Convolution . . . 24
3.2.3 Three-Dimensional Convolution . . . 25
3.2.4 Output Size Formula . . . 25
3.2.5 Receptive Field . . . 25
3.2.6 Convolution as Matrix Multiplication: im2col and GEMM . . . 26
3.2.7 FLOPs Calculation . . . 26
3.3 Backpropagation Through Convolution . . . 26
3.3.1 Gradient with Respect to Weights . . . 26
3.3.2 Gradient with Respect to Input . . . 27
3.3.3 Gradient with Respect to Bias . . . 27
3.4 Backpropagation Through Pooling . . . 27
3.4.1 Max Pooling . . . 27
3.4.2 Average Pooling . . . 28
3.4.3 Global Average Pooling . . . 28
3.4.4 Why Pooling? . . . 28
3.5 Special Convolution Variants . . . 28
3.5.1 1×1 Convolutions . . . 28
3.5.2 Depthwise Separable Convolution . . . 29
3.5.3 Dilated (Atrous) Convolution . . . 29
3.6 Feature Hierarchy . . . 29

3.7 Algorithm: Convolutional Layer Forward-Backward Pass 30
3.8 Pros and Cons 31
3.9 Evolution 31

4 AlexNet **33**
4.1 Motivation 33
4.2 Mathematical Formulation 33
4.2.1 Overall Architecture 33
4.2.2 Convolutional Layer Dimensions 34
4.2.3 GPU Split and Cross-GPU Connections 34
4.3 Activation Function: ReLU 35
4.3.1 Definition and Derivative 35
4.3.2 Comparison to Sigmoid and tanh 35
4.3.3 Training Speed 35
4.3.4 Dying ReLU Problem 35
4.4 Local Response Normalization 36
4.4.1 Mathematical Intuition 36
4.5 Overlapping Pooling 36
4.6 Data Augmentation 36
4.6.1 Image Translations and Horizontal Reflections 37
4.6.2 PCA Color Augmentation 37
4.7 Dropout in AlexNet 37
4.8 Training Procedure 38
4.8.1 Optimizer Configuration 38
4.8.2 Weight Initialization 38
4.8.3 Learning Rate Schedule 38
4.8.4 Training Duration and Hardware 39
4.9 Parameter Count and FLOPs Analysis 39
4.9.1 Learnable Parameters 39
4.9.2 FLOPs for Convolutional Layers 39
4.10 Why AlexNet Worked 40
4.10.1 Depth 40
4.10.2 ReLU Activation 40
4.10.3 GPU Implementation 40
4.10.4 Data Augmentation 40
4.10.5 Dropout 40
4.11 Algorithm: AlexNet Forward Pass 42
4.12 Pros and Cons 43
4.13 Evolution 43

5 ResNet — Deep Residual Networks **45**
5.1 Motivation 45
5.2 Mathematical Formulation 45
5.2.1 Residual Learning Hypothesis 45
5.2.2 Basic Residual Block 45
5.2.3 Bottleneck Block 46
5.2.4 Backpropagation Through Residual Connections 46
5.2.5 Full Gradient Flow Analysis 46

5.2.6 Pre-activation vs. Post-activation . . . 47
5.3 Algorithm . . . 47
5.4 Architecture . . . 48
5.5 Pros and Cons . . . 48
5.5.1 Advantages . . . 48
5.5.2 Disadvantages . . . 48
5.6 Evolution . . . 49

6 FCN — Fully Convolutional Networks 50
6.1 Motivation . . . 50
6.2 Mathematical Formulation . . . 50
6.2.1 Convolutionalization of Fully Connected Layers . . . 50
6.2.2 Transposed Convolution for Upsampling . . . 51
6.2.3 Skip Connections for Multi-scale Fusion . . . 51
6.2.4 Loss Function . . . 52
6.2.5 Evaluation Metrics . . . 52
6.3 Algorithm . . . 53
6.4 Architecture . . . 53
6.5 Pros and Cons . . . 54
6.5.1 Advantages . . . 54
6.5.2 Disadvantages . . . 54
6.6 Evolution . . . 54

7 UNet 56
7.1 Motivation . . . 56
7.2 Mathematical Formulation . . . 56
7.2.1 U-shaped Architecture Principle . . . 56
7.2.2 Concatenation vs. Addition . . . 57
7.2.3 Copy and Crop Mechanism . . . 57
7.2.4 Weighted Loss for Touching Objects . . . 57
7.2.5 Data Augmentation with Elastic Deformations . . . 58
7.3 Algorithm . . . 58
7.4 Architecture . . . 59
7.5 Pros and Cons . . . 59
7.5.1 Advantages . . . 59
7.5.2 Disadvantages . . . 60
7.6 Evolution . . . 60

III Sequence Models 62

8 RNN — Recurrent Neural Networks 63
8.1 Motivation . . . 63
8.2 Mathematical Formulation . . . 63
8.2.1 Basic RNN Equations . . . 63
8.2.2 Unrolling Through Time . . . 63
8.2.3 Loss Function for Sequences . . . 64
8.2.4 Backpropagation Through Time (BPTT) . . . 64
8.2.5 Gradient Flow and the Vanishing/Exploding Gradient Problem . . . 65

8.2.6 Bidirectional RNN . . . 65
8.2.7 Deep (Stacked) RNN . . . 66
8.3 Algorithm . . . 66
8.4 Architecture . . . 66
8.5 Pros and Cons . . . 67
8.5.1 Advantages . . . 67
8.5.2 Disadvantages . . . 67
8.6 Evolution . . . 67

9 LSTM — Long Short-Term Memory 69
9.1 Motivation . . . 69
9.2 Mathematical Formulation . . . 69
9.2.1 Full LSTM Equations . . . 69
9.2.2 Gate Semantics . . . 69
9.2.3 Gradient Flow Through the Cell State . . . 70
9.2.4 Backpropagation Through LSTM Gates . . . 71
9.2.5 Gated Recurrent Unit (GRU) . . . 72
9.2.6 Bidirectional and Stacked LSTM . . . 72
9.3 Algorithm . . . 73
9.4 Architecture . . . 73
9.5 Pros and Cons . . . 74
9.5.1 Advantages . . . 74
9.5.2 Disadvantages . . . 74
9.6 Evolution . . . 75

10 Seq2Seq with Attention 77
10.1 Motivation . . . 77
10.2 Mathematical Formulation . . . 77
10.2.1 Encoder . . . 77
10.2.2 Decoder Without Attention . . . 77
10.2.3 Bahdanau Attention (Additive Attention) . . . 78
10.2.4 Luong Attention (Multiplicative Attention) . . . 79
10.2.5 Global vs. Local Attention . . . 79
10.2.6 Attention as Differentiable Memory . . . 80
10.2.7 Beam Search Decoding . . . 80
10.2.8 Teacher Forcing and Scheduled Sampling . . . 80
10.2.9 BLEU Score . . . 81
10.3 Algorithm . . . 82
10.4 Architecture . . . 83
10.5 Pros and Cons . . . 84
10.6 Evolution . . . 84

IV Generative Models 86

11 Autoencoders 87
11.1 Motivation . . . 87
11.2 Mathematical Formulation . . . 87
11.2.1 Architecture . . . 87

11.2.2 Reconstruction Loss . . . 87
11.2.3 Linear Autoencoder and PCA . . . 88
11.2.4 Undercomplete Autoencoders . . . 88
11.2.5 Overcomplete Autoencoders and Regularization . . . 88
11.2.6 Denoising Autoencoder (DAE) . . . 89
11.2.7 Sparse Autoencoder . . . 89
11.2.8 Contractive Autoencoder (CAE) . . . 90
11.2.9 Learning the Data Manifold . . . 90
11.3 Algorithm . . . 91
11.4 Architecture . . . 92
11.5 Pros and Cons . . . 92
11.6 Evolution . . . 93

12 Variational Autoencoders 94
12.1 Motivation . . . 94
12.2 Mathematical Formulation . . . 94
12.2.1 Generative Model . . . 94
12.2.2 Variational Inference . . . 95
12.2.3 ELBO Derivation . . . 95
12.2.4 Analytic KL Divergence . . . 96
12.2.5 Reparameterization Trick . . . 96
12.2.6 Full Loss Function . . . 97
12.2.7 Beta-VAE . . . 97
12.2.8 VAE as Regularized Autoencoder . . . 97
12.3 Algorithm . . . 98
12.4 Architecture . . . 99
12.5 Pros and Cons . . . 99
12.6 Evolution . . . 100

13 Generative Adversarial Networks 102
13.1 Motivation . . . 102
13.2 Mathematical Formulation . . . 102
13.2.1 Two-Player Minimax Game . . . 102
13.2.2 Optimal Discriminator . . . 102
13.2.3 Generator Objective at Optimal D . . . 103
13.2.4 Training Algorithm . . . 103
13.2.5 Non-Saturating Loss . . . 104
13.2.6 Wasserstein GAN . . . 104
13.2.7 Conditional GAN . . . 105
13.2.8 Mode Collapse . . . 105
13.3 Algorithm . . . 106
13.4 Architecture . . . 107
13.4.1 DCGAN Architecture . . . 107
13.4.2 Progressive Growing . . . 108
13.4.3 StyleGAN Architecture . . . 108
13.5 Pros and Cons . . . 108
13.6 Evolution . . . 109

14 Diffusion Models **111**
14.1 Motivation . . . 111
14.2 Mathematical Formulation . . . 111
14.2.1 Forward Diffusion Process . . . 111
14.2.2 Direct Sampling Property . . . 112
14.2.3 Reverse Process . . . 112
14.2.4 Variational Bound Training Objective . . . 113
14.2.5 Simplified Training Objective (DDPM) . . . 113
14.2.6 Connection to Score Matching . . . 114
14.2.7 DDIM: Deterministic Sampling . . . 114
14.2.8 Classifier Guidance . . . 115
14.2.9 Classifier-Free Guidance . . . 115
14.2.10 Latent Diffusion Models . . . 115
14.3 Algorithm . . . 116
14.4 Architecture . . . 117
14.4.1 UNet for Diffusion . . . 117
14.4.2 Noise Schedules . . . 117
14.5 Pros and Cons . . . 118
14.6 Evolution . . . 118

15 Text-to-Image Generation **120**
15.1 Motivation . . . 120
15.2 Mathematical Formulation . . . 120
15.2.1 DALL-E: Autoregressive over Discrete Tokens . . . 120
15.2.2 DALL-E 2 (unCLIP) . . . 121
15.2.3 Stable Diffusion (Latent Diffusion Models) . . . 122
15.2.4 Imagen: Large Frozen Text Encoders . . . 123
15.2.5 DALL-E 3: Caption Improvement . . . 124
15.3 Evaluation Metrics . . . 125
15.3.1 Fréchet Inception Distance (FID) . . . 125
15.3.2 CLIP Score . . . 125
15.4 Algorithm . . . 126
15.5 Pros and Cons . . . 127
15.5.1 Per-Model Comparison . . . 127
15.6 Evolution . . . 128

V Object Detection **130**

16 The RCNN Family: From Region Proposals to End-to-End Detection **131**
16.1 Motivation . . . 131
16.2 RCNN . . . 131
16.2.1 Mathematical Formulation . . . 131
16.2.2 Intersection over Union and Non-Maximum Suppression . . . 133
16.2.3 Algorithm . . . 133
16.2.4 Architecture . . . 134
16.2.5 Pros and Cons . . . 134
16.3 Fast RCNN . . . 134
16.3.1 Mathematical Formulation . . . 134

16.3.2 Algorithm . . . 136
16.3.3 Architecture . . . 136
16.3.4 Pros and Cons . . . 136
16.4 Faster RCNN . . . 137
16.4.1 Mathematical Formulation . . . 137
16.4.2 Algorithm . . . 139
16.4.3 Architecture . . . 139
16.4.4 Pros and Cons . . . 139
16.4.5 Evolution of the RCNN Family . . . 140

17 YOLO: You Only Look Once — Unified, Real-Time Object Detection 141
17.1 Motivation . . . 141
17.2 YOLOv1 . . . 141
17.2.1 Mathematical Formulation . . . 141
17.2.2 Loss Function . . . 142
17.2.3 Architecture . . . 143
17.2.4 Algorithm . . . 143
17.2.5 Pros and Cons . . . 144
17.3 YOLOv2 / YOLO9000 . . . 144
17.3.1 Mathematical Formulation . . . 144
17.3.2 Architecture . . . 145
17.4 YOLOv3 . . . 145
17.4.1 Mathematical Formulation . . . 145
17.4.2 Architecture . . . 146
17.5 YOLOv4 . . . 146
17.5.1 Mathematical Formulation . . . 146
17.5.2 Architecture . . . 147
17.6 YOLOv5 Through YOLOv11: An Evolutionary Table . . . 148
17.7 YOLOv8: Anchor-Free Detection . . . 149
17.7.1 Mathematical Formulation . . . 149
17.7.2 Algorithm . . . 149
17.8 Pros and Cons Across Versions . . . 150
17.9 Evolution of the YOLO Family . . . 150

18 SSD: Single Shot MultiBox Detector — Multi-Scale Default Boxes 151
18.1 Motivation . . . 151
18.2 Mathematical Formulation . . . 151
18.2.1 Default Box Generation . . . 151
18.2.2 Prediction and Encoding . . . 152
18.2.3 Matching Strategy . . . 152
18.2.4 Training Objective . . . 153
18.2.5 Hard Negative Mining . . . 153
18.2.6 Atrous Convolution in the Base Network . . . 154
18.3 Architecture . . . 154
18.3.1 Base Network . . . 154
18.3.2 Auxiliary Convolutional Layers . . . 154
18.3.3 Detection Heads . . . 154
18.3.4 Speed-Accuracy Configurations . . . 155

18.4 Algorithm . . . 156
18.4.1 Training Algorithm with Data Augmentation . . . 157
18.5 Data Augmentation . . . 157
18.6 Pros and Cons . . . 158
18.7 Evolution of Single-Shot Detection . . . 158

VI Vision Transformers & Multimodal Models 160

19 ViT: Vision Transformer 161
19.1 Motivation . . . 161
19.2 Mathematical Formulation . . . 161
19.2.1 Image to Patch Sequence . . . 161
19.2.2 Multi-Head Self-Attention (MSA) . . . 162
19.2.3 Transformer Encoder Block . . . 162
19.2.4 Position Embeddings . . . 162
19.2.5 Self-Attention as Generalized Convolution . . . 163
19.2.6 Hybrid Architecture . . . 163
19.3 Algorithm . . . 163
19.4 Architecture . . . 164
19.4.1 ViT Variants . . . 164
19.4.2 Self-Attention Complexity . . . 164
19.4.3 Pre-training and Fine-tuning . . . 164
19.5 Pros and Cons . . . 164
19.6 Evolution . . . 165

20 CLIP: Contrastive Language-Image Pre-training 167
20.1 Motivation . . . 167
20.2 Mathematical Formulation . . . 167
20.2.1 Joint Embedding Space . . . 167
20.2.2 InfoNCE Loss . . . 167
20.2.3 Information-Theoretic Interpretation . . . 168
20.2.4 Zero-Shot Classification . . . 168
20.2.5 Gradient Analysis . . . 169
20.3 Algorithm . . . 169
20.4 Architecture . . . 169
20.4.1 Image Encoder . . . 169
20.4.2 Text Encoder . . . 170
20.4.3 Training Configuration . . . 170
20.4.4 Prompt Engineering and Ensembling . . . 170
20.5 Pros and Cons . . . 171
20.6 Evolution . . . 172

21 BLIP: Bootstrapping Language-Image Pre-training 174
21.1 Motivation . . . 174
21.2 Mathematical Formulation . . . 174
21.2.1 Multimodal Mixture of Encoder-Decoder (MED) . . . 174
21.2.2 Pre-training Objectives . . . 175
21.2.3 CapFilt: Captioning and Filtering . . . 176

21.3 Algorithm . . . 177
21.4 Architecture . . . 177
21.4.1 BLIP-2: Q-Former . . . 177
21.4.2 Training Details . . . 178
21.4.3 Instruction Tuning (InstructBLIP) . . . 178
21.5 Pros and Cons . . . 179
21.6 Evolution . . . 180

VII Specialized Vision Tasks 181

22 Optical Character Recognition 182
22.1 Motivation . . . 182
22.2 Mathematical Formulation . . . 182
22.2.1 Text Detection: Differential Binarization (DBNet) . . . 182
22.2.2 Text Recognition with CRNN+CTC . . . 183
22.2.3 Attention-Based Sequence Recognition . . . 184
22.2.4 TrOCR: Pure Transformer OCR . . . 184
22.2.5 Thin-Plate Spline (TPS) Rectification . . . 184
22.3 Algorithm . . . 185
22.4 Architecture . . . 185
22.4.1 Detection Architectures . . . 185
22.4.2 Recognition Architectures . . . 185
22.4.3 Metrics . . . 186
22.5 Pros and Cons . . . 186
22.6 Evolution . . . 187

23 Pose Estimation 189
23.1 Motivation . . . 189
23.2 Mathematical Formulation . . . 189
23.2.1 Problem Definition . . . 189
23.2.2 Heatmap Regression . . . 189
23.2.3 Why Heatmaps Outperform Direct Coordinate Regression . . . 190
23.2.4 Stacked Hourglass Networks . . . 190
23.2.5 HRNet: High-Resolution Representation Learning . . . 191
23.2.6 OpenPose: Part Affinity Fields . . . 191
23.2.7 ViTPose: Vision Transformer for Pose Estimation . . . 192
23.2.8 Object Keypoint Similarity (OKS) . . . 193
23.2.9 Associative Embedding . . . 193
23.3 Algorithm . . . 194
23.4 Architecture . . . 194
23.4.1 Top-Down Pipeline . . . 194
23.4.2 Bottom-Up Pipeline . . . 195
23.4.3 Key Model Architectures . . . 195
23.5 Pros and Cons . . . 195
23.6 Evolution . . . 196

24 Object Tracking **198**
24.1 Motivation . . . 198
24.2 Mathematical Formulation . . . 198
24.2.1 Problem Definition . . . 198
24.2.2 Single Object Tracking: Siamese Formulation . . . 199
24.2.3 Multiple Object Tracking: Filtering and Association . . . 200
24.2.4 Tracking Metrics . . . 202
24.3 Algorithms . . . 202
24.4 Architecture . . . 202
24.5 Pros and Cons . . . 204
24.6 Evolution . . . 205

25 Open-Vocabulary Detection and Segmentation **206**
25.1 Motivation . . . 206
25.2 Mathematical Formulation . . . 206
25.2.1 From Closed-Set to Open-Vocabulary Classification . . . 206
25.2.2 CLIP: Vision-Language Pre-training Foundation . . . 207
25.2.3 Open-Vocabulary Object Detection . . . 207
25.2.4 Open-Vocabulary Semantic Segmentation . . . 209
25.3 Algorithms . . . 211
25.4 Architecture . . . 211
25.5 Pros and Cons . . . 212
25.6 Evolution . . . 213

VIII Graph Neural Networks **214**

26 Graph Neural Networks **215**
26.1 Motivation . . . 215
26.2 Mathematical Formulation . . . 215
26.2.1 Graphs: Notation and Fundamentals . . . 215
26.2.2 Message Passing Neural Networks: A Unified Framework . . . 216
26.2.3 Graph Convolutional Network (GCN) . . . 216
26.2.4 GraphSAGE: Inductive Representation Learning . . . 218
26.2.5 Graph Attention Network (GAT) . . . 218
26.2.6 Graph Isomorphism Network (GIN) . . . 219
26.2.7 Training Considerations . . . 219
26.2.8 The Over-Smoothing Problem . . . 220
26.3 Algorithms . . . 220
26.4 Architecture . . . 220
26.5 Pros and Cons . . . 222
26.6 Evolution . . . 223

IX Reinforcement Learning **224**

27 Reinforcement Learning Fundamentals **225**
27.1 Motivation . . . 225
27.2 Mathematical Formulation . . . 225

27.2.1 Markov Decision Processes 225
27.2.2 Value Functions and Bellman Equations 226
27.2.3 Value-Based RL: Q-Learning and DQN 227
27.2.4 Policy Gradient Methods 228
27.2.5 Actor-Critic Methods 229
27.2.6 Proximal Policy Optimization (PPO) 230
27.2.7 Maximum Entropy RL: Soft Actor-Critic (SAC) 231
27.2.8 Reinforcement Learning from Human Feedback (RLHF) 231
27.3 Algorithms 232
27.4 Architecture 234
27.5 Pros and Cons 234
27.6 Evolution 235

Preface

This book is a comprehensive, model-family–organized tutorial on deep learning. It covers the foundational architectures through the state-of-the-art models, organized by intellectual lineage: each family traces a line of ideas—what problem each model solved, what it could not do, and how the next model in the family fixed it.

How to read this book. The chapters are largely self-contained, but within each *Part*, they build on one another. If you are new to deep learning, read Parts I and II first, then follow your interests. Every chapter contains:

- **Motivation** — what came before, and why this model was needed.
- **Mathematical formulation** — equations derived step by step.
- **Algorithm** — pseudocode when a training loop or forward pass is nontrivial.
- **Architecture** — layer-by-layer breakdown.
- **Pros & Cons** — what it can do and what it still cannot.
- **Evolution** — what fixed its shortcomings, and how.

The equations are the primary language; words supplement them. Nothing is omitted for brevity.

Nurdun Mijit

Part I

Foundational Networks

Chapter 1

Feedforward and Deep Neural Networks

1.1 Motivation

The perceptron, introduced by Rosenblatt in 1958, was the first mathematical model of a biological neuron. Its limitation was sobering: a single perceptron can only separate classes that are *linearly separable*. The XOR function—true when exactly one input is 1—cannot be represented by a single perceptron. This was proved by Minsky and Papert in 1969 and froze neural network research for over a decade.

The multilayer perceptron (MLP) solved this by stacking perceptrons. A hidden layer transforms the input into a new representation where linear separability holds. The resulting function class—feedforward neural networks (FNNs)—is a *universal approximator*: with enough hidden units, it can approximate any Borel-measurable function to arbitrary accuracy.

Deep neural networks (DNNs) push this further: instead of one wide hidden layer, they use many narrower layers. This *compositional* structure exploits the hierarchical nature of data and achieves exponentially more representational capacity per parameter.

1.2 Mathematical Formulation

1.2.1 The Perceptron

Given input $\mathbf{x} \in \mathbb{R}^d$, weights $\mathbf{w} \in \mathbb{R}^d$, bias $b \in \mathbb{R}$, the perceptron computes:

$$f(\mathbf{x}) = \mathbb{1}\{\mathbf{w}^\top \mathbf{x} + b > 0\} \tag{1.1}$$

where $\mathbb{1}\{\cdot\}$ is the indicator function. Geometrically, this is a hyperplane with normal vector $\mathbf{w}$ at distance $b/\left\|\mathbf{w}\right\|$ from the origin. Points on one side get label 1; on the other, 0.

The perceptron learning rule updates weights only on misclassified examples:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y_i \mathbf{x}_i, \quad b^{(t+1)} = b^{(t)} + y_i \tag{1.2}$$

where $y_i \in \{-1, +1\}$ is the true label. This rule converges in finite steps *iff* the data is linearly separable (Perceptron Convergence Theorem).

1.2.2 The XOR Problem

XOR maps $(0,0) \mapsto 0$, $(0,1) \mapsto 1$, $(1,0) \mapsto 1$, $(1,1) \mapsto 0$. No single hyperplane separates $\{(0,0),(1,1)\}$ from $\{(0,1),(1,0)\}$. An MLP solves this with one hidden layer of two neurons:

$$h_1 = \mathrm{ReLU}(x_1 + x_2 - 0.5) \tag{1.3}$$

$$h_2 = \mathrm{ReLU}(x_1 + x_2 - 1.5) \tag{1.4}$$

$$y = h_1 - 2h_2 \tag{1.5}$$

The hidden layer maps the four points to a space where they are linearly separable.

1.2.3 Multilayer Perceptron (MLP)

An L-layer MLP computes:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{a}^{(1)} = g_1(\mathbf{z}^{(1)}) \tag{1.6}$$

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{a}^{(\ell)} = g_\ell(\mathbf{z}^{(\ell)}), \quad \ell = 2, \ldots, L-1 \tag{1.7}$$

$$\hat{\mathbf{y}} = \mathbf{W}^{(L)}\mathbf{a}^{(L-1)} + \mathbf{b}^{(L)} \tag{1.8}$$

where $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$, $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$, and g_ℓ is an activation function. The output layer typically has no activation for regression or softmax for classification.

For a mini-batch of m samples $\mathbf{X} \in \mathbb{R}^{m \times d}$, the forward propagation in matrix form is:

$$\mathbf{Z}^{(1)} = \mathbf{X}\mathbf{W}^{(1)\top} + \mathbf{1}_m\mathbf{b}^{(1)\top}, \quad \mathbf{A}^{(1)} = g_1(\mathbf{Z}^{(1)}) \tag{1.9}$$

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)}\mathbf{W}^{(\ell)\top} + \mathbf{1}_m\mathbf{b}^{(\ell)\top}, \quad \mathbf{A}^{(\ell)} = g_\ell(\mathbf{Z}^{(\ell)}) \tag{1.10}$$

$$\hat{\mathbf{Y}} = \mathbf{A}^{(L-1)}\mathbf{W}^{(L)\top} + \mathbf{1}_m\mathbf{b}^{(L)\top} \tag{1.11}$$

1.2.4 Activation Functions

Activation functions introduce nonlinearity. Without them, the composition of linear functions is linear—an L-layer network collapses to a single linear transformation.

Sigmoid

$$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \sigma'(z) = \sigma(z)(1-\sigma(z)) \tag{1.12}$$

Squashes to $(0, 1)$. Saturation at extremes ($|z| \gg 0$) makes gradients near zero—the vanishing gradient problem. Derivative maximum is 0.25 at $z = 0$, halving the gradient at each backward step through sigmoid layers.

Hyperbolic Tangent

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1, \quad \tanh'(z) = 1 - \tanh^2(z) \tag{1.13}$$

Zero-centered (range $(-1, 1)$), which helps optimization. Still saturates; gradient maximum is 1 at $z = 0$.

ReLU (Rectified Linear Unit)

$$\mathrm{ReLU}(z) = \max(0, z), \quad \mathrm{ReLU}'(z) = \begin{cases} 0 & z < 0 \\ 1 & z > 0 \end{cases} \tag{1.14}$$

Non-saturating for $z > 0$—gradient is 1 regardless of input magnitude. This eliminates the vanishing gradient problem in the positive regime. The derivative at $z = 0$ is technically undefined; in practice it is set to 0, 0.5, or 1. The "dying ReLU" problem: neurons with consistently negative pre-activations never activate, and their weights never update. The gradient is zero for all inputs, so they irreversibly die.

Leaky ReLU

$$\text{LeakyReLU}(z) = \max(\alpha z, z), \quad \text{LeakyReLU}'(z) = \begin{cases} \alpha & z < 0 \\ 1 & z > 0 \end{cases} \tag{1.15}$$

with small α (typically 0.01). The small negative slope keeps dead neurons from going completely silent.

GELU (Gaussian Error Linear Unit)

$$\text{GELU}(z) = z \cdot \Phi(z) = z \cdot \frac{1}{2}\left[1 + \text{erf}\left(\frac{z}{\sqrt{2}}\right)\right] \tag{1.16}$$

where $\Phi(z)$ is the standard Gaussian CDF. The derivative:

$$\text{GELU}'(z) = \Phi(z) + z \cdot \phi(z) \tag{1.17}$$

where $\phi(z)$ is the Gaussian PDF. GELU weights inputs by their probability under a Gaussian—a smooth, probabilistic gating. Used in Transformers (BERT, GPT). Approximated efficiently as $\text{GELU}(z) \approx 0.5z\left(1 + \tanh\left[\sqrt{2/\pi}(z + 0.044715z^3)\right]\right)$.

Swish / SiLU

$$\text{Swish}(z) = z \cdot \sigma(\beta z), \quad \text{Swish}'(z) = \sigma(\beta z) + \beta z \cdot \sigma(\beta z)(1 - \sigma(\beta z)) \tag{1.18}$$

When $\beta = 1$, this is SiLU. It is a smooth, non-monotonic function that empirically outperforms ReLU on deep networks.

1.2.5 Universal Approximation Theorem

Theorem 1.1 (Cybenko, 1989; Hornik et al., 1989) *Let σ be a continuous, discriminatory function. For any compact set $K \subset \mathbb{R}^d$, any continuous function $f : K \to \mathbb{R}$, and any $\epsilon > 0$, there exist $N \in \mathbb{N}$, weights $\mathbf{w}_i \in \mathbb{R}^d$, biases $b_i \in \mathbb{R}$, and output weights $c_i \in \mathbb{R}$ such that:*

$$\sup_{\mathbf{x} \in K} \left| f(\mathbf{x}) - \sum_{i=1}^{N} c_i \, \sigma(\mathbf{w}_i^\top \mathbf{x} + b_i) \right| < \epsilon \tag{1.19}$$

Intuition. A single hidden layer with enough sigmoidal neurons can construct an arbitrarily fine "step function" approximation, building a Riemann-sum–style covering of the target function. The layer constructs N half-planes in input space; weighted sums of sigmoids at these half-planes approximate the target.

Caveat. The theorem says nothing about *learnability*. The required N may be exponential in d, and gradient descent may never find those weights. This is where depth helps.

1.2.6 Depth vs. Width

A deep network composes functions:

$$f(\mathbf{x}) = f_L \circ f_{L-1} \circ \cdots \circ f_1(\mathbf{x}) \tag{1.20}$$

Each $f_\ell(\mathbf{z}) = g(\mathbf{W}^{(\ell)}\mathbf{z} + \mathbf{b}^{(\ell)})$ is a simple transformation. Composition creates exponentially more expressiveness than depth-2 networks:

Telgarsky's construction (2016): A ReLU network with $\mathcal{O}(L)$ layers and $\mathcal{O}(1)$ width can represent a function with $\Omega(2^L)$ oscillations. To represent this with a 2-layer network requires $\Omega(2^L)$ hidden units. Depth provides *exponential* efficiency gains in representational capacity.

Linear region counting: A ReLU network with n neurons per layer and L layers partitions input space into at most $\mathcal{O}(n^L)$ linear regions. More layers $\Rightarrow$ exponentially more regions.

Hierarchical composition: Functions arising from physics, images, and language are *hierarchically composed*. An image is edges $\rightarrow$ textures $\rightarrow$ parts $\rightarrow$ objects. Deep networks match this compositional structure.

1.2.7 Vanishing and Exploding Gradients

Consider the gradient of the loss $\mathcal{L}$ with respect to weights at layer ℓ. By the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \cdot \frac{\partial \mathbf{a}^{(L)}}{\partial \mathbf{z}^{(L)}} \cdot \mathbf{W}^{(L)} \cdot \frac{\partial \mathbf{a}^{(L-1)}}{\partial \mathbf{z}^{(L-1)}} \cdot \mathbf{W}^{(L-1)} \cdots \frac{\partial \mathbf{a}^{(\ell)}}{\partial \mathbf{z}^{(\ell)}} \cdot \frac{\partial \mathbf{z}^{(\ell)}}{\partial \mathbf{W}^{(\ell)}} \tag{1.21}$$

Let $\mathbf{D}^{(\ell)} = \operatorname{diag}(g'(\mathbf{z}^{(\ell)}))$ be the Jacobian of the activation. The gradient propagated from layer L back to layer ℓ is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}} = \mathbf{D}^{(\ell)}(\mathbf{W}^{(\ell+1)})^\top \mathbf{D}^{(\ell+1)}(\mathbf{W}^{(\ell+2)})^\top \cdots \mathbf{D}^{(L)}(\mathbf{W}^{(L)})^\top \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \tag{1.22}$$

If g is sigmoid, $|g'(z)| \leq 0.25$, so $\left\|\mathbf{D}^{(\ell)}\right\| \leq 0.25$ for all ℓ. With $L - \ell$ such terms and weight matrices of scale w, the gradient norm scales as:

$$\left\| \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}} \right\| \propto (0.25\, w)^{L-\ell} \tag{1.23}$$

If $0.25w < 1$, gradients *vanish* exponentially in depth. If $0.25w > 1$, they *explode*. ReLU mitigates this by having derivative 1 for active units, removing the 0.25 factor—but weights can still cause explosion.

1.3 Architecture

An L-layer FNN:

- **Input layer:** d neurons (one per feature). No computation; just holds $\mathbf{x}$.
- **Hidden layers:** $L - 1$ layers with arbitrary widths $n_1, n_2, \ldots, n_{L-1}$. Each applies $\mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ followed by g.
- **Output layer:** n_L neurons. For regression: linear. For binary classification: 1 neuron + sigmoid. For multi-class: K neurons + softmax.

Parameters: $\sum_{\ell=1}^{L}(n_\ell n_{\ell-1} + n_\ell)$.

1.4 Pros and Cons

1.4.1 Strengths

- **Universal approximator.** Can represent any Borel-measurable function given enough capacity.

- **End-to-end differentiable.** Trained via backpropagation; no handcrafted features.
- **Compositional.** Deep architectures exploit hierarchical structure in data.
- **Flexible.** Works for regression, classification, density estimation with appropriate output layers.

1.4.2 Limitations

- **No spatial structure.** Fully connected layers treat input as a flat vector; pixel neighborhood information is lost. This is why CNNs were invented.
- **No temporal recurrence.** Each forward pass is feedforward; there's no memory of previous inputs. This is why RNNs were invented.
- **Parameter explosion.** For a $224 \times 224 \times 3$ image, the first hidden layer with 1000 neurons has $224^2 \cdot 3 \cdot 1000 \approx 150$M parameters.
- **Vanishing/exploding gradients.** Deep FNNs are hard to train without careful initialization and normalization (solved by BatchNorm, ResNet).
- **Overfitting.** High capacity requires regularization.

1.5 Evolution: What Comes Next

Problem	Solution	Model
Parameter explosion on images	Weight sharing, local connectivity	CNN (Ch. 3)
Temporal/sequential data	Recurrent connections, hidden state	RNN (Ch. 8), LSTM (Ch. 9)
Vanishing gradients in deep nets	Residual/skip connections	ResNet (Ch. 5)
No output structure (images)	Convolutional upsampling	FCN (Ch. 6), UNet (Ch. 7)

Chapter 2

Training Deep Networks

2.1 Motivation

A neural network with weight matrices $\{\mathbf{W}^{(l)}\}_{l=1}^{L}$ and biases $\{\mathbf{b}^{(l)}\}_{l=1}^{L}$ defines a parameterized function $\mathbf{f}(\mathbf{x};\boldsymbol{\theta})$ where $\boldsymbol{\theta} = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}$. The central optimization problem is

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{n=1}^{N} \ell(\mathbf{f}(\mathbf{x}_n;\boldsymbol{\theta}), \mathbf{y}_n) + \lambda R(\boldsymbol{\theta}) \tag{2.1}$$

where ℓ is a loss function measuring prediction error, R is a regularizer, and λ controls the tradeoff. Solving this requires: (1) choosing an appropriate loss, (2) computing the gradient $\nabla_{\boldsymbol{\theta}}\ell$ efficiently via backpropagation, and (3) descending the loss surface with optimizers that handle non-convexity, ill-conditioning, and stochasticity. This chapter derives each component in full detail.

2.2 Loss Functions

2.2.1 Mean Squared Error

For regression with targets $\mathbf{y} \in \mathbb{R}^d$, the squared error of a prediction $\hat{\mathbf{y}} = \mathbf{f}(\mathbf{x};\boldsymbol{\theta})$ is

$$\ell_{\text{MSE}}(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{2} \|\hat{\mathbf{y}} - \mathbf{y}\|^2 = \frac{1}{2} \sum_{i=1}^{d} (\hat{y}_i - y_i)^2 \tag{2.2}$$

The factor $\frac{1}{2}$ simplifies the gradient to $\nabla_{\hat{\mathbf{y}}}\ell = \hat{\mathbf{y}} - \mathbf{y}$. Under a Gaussian noise model $p(\mathbf{y} \mid \mathbf{x}) = \mathcal{N}(\mathbf{y}; \mathbf{f}(\mathbf{x}), \sigma^2\mathbf{I})$, the negative log-likelihood is

$$-\log p(\mathbf{y}_1, \ldots, \mathbf{y}_N \mid \mathbf{X}) = \sum_{n=1}^{N} \left[\frac{d}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \|\mathbf{f}(\mathbf{x}_n) - \mathbf{y}_n\|^2 \right] \tag{2.3}$$

Maximizing likelihood is equivalent to minimizing $\frac{1}{N}\sum_n \|\mathbf{f}(\mathbf{x}_n) - \mathbf{y}_n\|^2$, establishing MSE as MLE under homoscedastic Gaussian noise.

2.2.2 Cross-Entropy for Multi-Class Classification

For K classes, let $\mathbf{y} \in \{0,1\}^K$ be a one-hot vector with $\sum_k y_k = 1$. The network outputs logits $\mathbf{z} = \mathbf{f}(\mathbf{x}) \in \mathbb{R}^K$. The softmax function converts logits to a valid probability vector:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \triangleq p_k \tag{2.4}$$

Interpreting $\mathbf{p}$ as parameters of a categorical distribution $p(y = k \mid \mathbf{x}) = p_k$, the likelihood of a single observation is

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_{k=1}^{K} p_k^{y_k} \tag{2.5}$$

The negative log-likelihood (cross-entropy loss) for one sample is

$$\ell_{\text{CE}}(\mathbf{p}, \mathbf{y}) = -\sum_{k=1}^{K} y_k \log p_k = -\log p_{k^*} \tag{2.6}$$

where k^* is the index where $y_{k^*} = 1$. For a batch of N i.i.d. samples, the full MLE objective is

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{n=1}^{N} \sum_{k=1}^{K} y_{nk} \log \left(\frac{e^{z_{nk}}}{\sum_j e^{z_{nj}}} \right) \tag{2.7}$$

The gradient with respect to logits admits an elegant form. Let $p_{nk} = \text{softmax}(\mathbf{z}_n)_k$:

$$\begin{aligned} \frac{\partial \ell_n}{\partial z_{nk}} &= -\frac{\partial}{\partial z_{nk}} \sum_{j=1}^{K} y_{nj} \log p_{nj} \\ &= -\sum_{j=1}^{K} y_{nj} \frac{1}{p_{nj}} \frac{\partial p_{nj}}{\partial z_{nk}} \end{aligned} \tag{2.8}$$

The derivative of softmax is

$$\begin{aligned} \frac{\partial p_{nj}}{\partial z_{nk}} &= \frac{\partial}{\partial z_{nk}} \frac{e^{z_{nj}}}{\sum_l e^{z_{nl}}} \\ &= \frac{\delta_{jk} e^{z_{nj}} \sum_l e^{z_{nl}} - e^{z_{nj}} e^{z_{nk}}}{(\sum_l e^{z_{nl}})^2} \\ &= p_{nj}\delta_{jk} - p_{nj}p_{nk} \end{aligned} \tag{2.9}$$

Substituting:

$$\begin{aligned} \frac{\partial \ell_n}{\partial z_{nk}} &= -\sum_j y_{nj} \frac{1}{p_{nj}} (p_{nj}\delta_{jk} - p_{nj}p_{nk}) \\ &= -\sum_j y_{nj} (\delta_{jk} - p_{nk}) \\ &= -\left(y_{nk} - p_{nk} \sum_j y_{nj} \right) = p_{nk} - y_{nk} \end{aligned} \tag{2.10}$$

since $\sum_j y_{nj} = 1$. Thus $\nabla_{\mathbf{z}} \ell_{\text{CE}} = \mathbf{p} - \mathbf{y}$, a remarkably simple gradient.

2.2.3 Binary Cross-Entropy

For binary classification ($K = 2$), let $y \in \{0, 1\}$. The network outputs a single logit z, and the predicted probability is $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$. Under the Bernoulli model $p(y \mid z) = \hat{y}^y (1 - \hat{y})^{1-y}$, the negative log-likelihood is

$$\ell_{\text{BCE}}(\hat{y}, y) = -\left[y \log \hat{y} + (1 - y) \log(1 - \hat{y}) \right] \tag{2.11}$$

$$= -\log \left(\hat{y}^y (1 - \hat{y})^{1-y} \right) \tag{2.12}$$

The derivative with respect to the logit exploits the sigmoid property $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y})$:

$$\begin{aligned}\frac{\partial \ell}{\partial z} &= \frac{\partial \ell}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right) \hat{y}(1-\hat{y}) \\ &= -y(1-\hat{y}) + (1-y)\hat{y} = \hat{y} - y \end{aligned} \tag{2.13}$$

The gradient again takes the simple form $\hat{y} - y$, sharing the same elegant structure as the multi-class case.

2.3 Backpropagation

2.3.1 Notation and Forward Pass

Consider an L-layer feedforward network. For layer $l = 1, \ldots, L$:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad \text{(pre-activation)} \tag{2.14}$$

$$\mathbf{a}^{(l)} = g^{(l)}(\mathbf{z}^{(l)}) \quad \text{(activation)} \tag{2.15}$$

with $\mathbf{a}^{(0)} = \mathbf{x}$ (input) and $\mathbf{a}^{(L)} = \hat{\mathbf{y}}$ (output). The total loss over N samples is $\mathcal{L} = \frac{1}{N}\sum_{n=1}^{N} \ell(\hat{\mathbf{y}}_n, \mathbf{y}_n)$. For clarity, we derive for a single sample and drop the subscript n.

2.3.2 Chain Rule for a Single Layer

Consider layer l with pre-activation $\mathbf{z}^{(l)} \in \mathbb{R}^{d_l}$ and activation $\mathbf{a}^{(l)} = g^{(l)}(\mathbf{z}^{(l)})$. The Jacobian of the activation is

$$\frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} = \operatorname{diag}\left(g^{(l)\prime}(z_1^{(l)}), \ldots, g^{(l)\prime}(z_{d_l}^{(l)})\right) = \mathbf{D}^{(l)} \tag{2.16}$$

where diag constructs a diagonal matrix. The Jacobian of pre-activation with respect to the weight matrix $\mathbf{W}^{(l)}$ requires careful tensor indexing. For a single element:

$$\begin{aligned}\frac{\partial z_i^{(l)}}{\partial W_{jk}^{(l)}} &= \frac{\partial}{\partial W_{jk}^{(l)}}\left(\sum_m W_{im}^{(l)} a_m^{(l-1)} + b_i^{(l)}\right) \\ &= \delta_{ij} a_k^{(l-1)} \end{aligned} \tag{2.17}$$

Thus $\frac{\partial z^{(l)}}{\partial W^{(l)}}$ is a rank-3 tensor. In practice we avoid constructing it explicitly.

2.3.3 The Backward Recursion

Define the error signal at layer l as the gradient of the loss with respect to the pre-activation:

$$\boldsymbol{\delta}^{(l)} \triangleq \frac{\partial \ell}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{d_l} \tag{2.18}$$

At the output layer, $\boldsymbol{\delta}^{(L)}$ depends on the loss function:

$$\boldsymbol{\delta}^{(L)} = \frac{\partial \ell}{\partial \mathbf{a}^{(L)}} \odot g^{(L)\prime}(\mathbf{z}^{(L)}) \quad \text{(element-wise multiplication)} \tag{2.19}$$

For MSE with linear output ($g^{(L)}(z) = z$, so $\mathbf{D}^{(L)} = \mathbf{I}$): $\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$. For cross-entropy with softmax: $\boldsymbol{\delta}^{(L)} = \mathbf{p} - \mathbf{y}$ (derived in §2).

For hidden layers $l = L-1, L-2, \ldots, 1$, we propagate the error backward:

$$\begin{aligned}\boldsymbol{\delta}^{(l)} &= \frac{\partial \ell}{\partial \mathbf{z}^{(l)}} = \frac{\partial \ell}{\partial \mathbf{z}^{(l+1)}} \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \\ &= \left(\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \right)^T \boldsymbol{\delta}^{(l+1)} \odot g^{(l)\prime}(\mathbf{z}^{(l)}) \end{aligned} \tag{2.20}$$

Since $\mathbf{z}^{(l+1)} = \mathbf{W}^{(l+1)}\mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$, the Jacobian is $\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} = \mathbf{W}^{(l+1)}$. Therefore:

$$\boxed{\boldsymbol{\delta}^{(l)} = (\mathbf{W}^{(l+1)})^T \boldsymbol{\delta}^{(l+1)} \odot g^{(l)\prime}(\mathbf{z}^{(l)})} \tag{2.21}$$

This is the **backpropagation recursion**. The gradient signal flows backward through the network: the transposed weight matrix projects the error from layer $l+1$ back to layer l, and the activation derivative gates the flow element-wise.

2.3.4 Weight and Bias Gradients

With $\boldsymbol{\delta}^{(l)}$ computed for all layers, the parameter gradients are:

$$\frac{\partial \ell}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^T \in \mathbb{R}^{d_l \times d_{l-1}} \tag{2.22}$$

$$\frac{\partial \ell}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \in \mathbb{R}^{d_l} \tag{2.23}$$

This follows from $\frac{\partial z_i^{(l)}}{\partial W_{ij}^{(l)}} = a_j^{(l-1)}$, so by the chain rule:

$$\frac{\partial \ell}{\partial W_{ij}^{(l)}} = \frac{\partial \ell}{\partial z_i^{(l)}} \frac{\partial z_i^{(l)}}{\partial W_{ij}^{(l)}} = \delta_i^{(l)} a_j^{(l-1)} \tag{2.24}$$

2.3.5 Full Backpropagation Algorithm for a 3-Layer Sigmoid Network

Consider $L = 3$ with sigmoid activation $g(z) = \sigma(z)$ at hidden layers and linear output. Denote $\mathbf{x} = \mathbf{a}^{(0)}$.

Forward pass:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \tag{2.25}$$

$$\mathbf{a}^{(1)} = \sigma(\mathbf{z}^{(1)}) = \frac{1}{1 + e^{-\mathbf{z}^{(1)}}} \quad \text{(element-wise)} \tag{2.26}$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)} \tag{2.27}$$

$$\mathbf{a}^{(2)} = \sigma(\mathbf{z}^{(2)}) \tag{2.28}$$

$$\mathbf{z}^{(3)} = \mathbf{W}^{(3)}\mathbf{a}^{(2)} + \mathbf{b}^{(3)} \tag{2.29}$$

$$\hat{\mathbf{y}} = \mathbf{a}^{(3)} = \mathbf{z}^{(3)} \quad \text{(linear output)} \tag{2.30}$$

Backward pass:

$$\boldsymbol{\delta}^{(3)} = \frac{\partial \ell}{\partial \hat{\mathbf{y}}} = \hat{\mathbf{y}} - \mathbf{y} \tag{2.31}$$

$$\boldsymbol{\delta}^{(2)} = (\mathbf{W}^{(3)})^T \boldsymbol{\delta}^{(3)} \odot \sigma'(\mathbf{z}^{(2)}) \tag{2.32}$$

$$\boldsymbol{\delta}^{(1)} = (\mathbf{W}^{(2)})^T \boldsymbol{\delta}^{(2)} \odot \sigma'(\mathbf{z}^{(1)}) \tag{2.33}$$

where $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = a(1 - a)$.

Parameter gradients:

$$\nabla_{\mathbf{W}^{(3)}} \ell = \boldsymbol{\delta}^{(3)} (\mathbf{a}^{(2)})^T, \quad \nabla_{\mathbf{b}^{(3)}} \ell = \boldsymbol{\delta}^{(3)} \tag{2.34}$$

$$\nabla_{\mathbf{W}^{(2)}} \ell = \boldsymbol{\delta}^{(2)} (\mathbf{a}^{(1)})^T, \quad \nabla_{\mathbf{b}^{(2)}} \ell = \boldsymbol{\delta}^{(2)} \tag{2.35}$$

$$\nabla_{\mathbf{W}^{(1)}} \ell = \boldsymbol{\delta}^{(1)} \mathbf{x}^T, \quad \nabla_{\mathbf{b}^{(1)}} \ell = \boldsymbol{\delta}^{(1)} \tag{2.36}$$

Observe the pattern: the gradient of $\mathbf{W}^{(l)}$ is the outer product of the error at layer l and the activation from layer $l - 1$ — an elegant Hebbian-like update rule where each synapse adjusts proportionally to pre-synaptic activity and post-synaptic error.

2.3.6 Vanishing Gradient Problem

For a deep network with sigmoid activation, the backward recursion contains the factor $g'(z)$. Since $\sigma'(z) \in (0, 0.25]$, each backward step multiplies the gradient by at most 0.25. After L layers, the gradient at the input layer scales as

$$\left\|\boldsymbol{\delta}^{(1)}\right\| \propto \left\|\boldsymbol{\delta}^{(L)}\right\| \prod_{l=1}^{L-1} \left\|\mathbf{W}^{(l+1)}\right\| \cdot \sigma'(z_{\max}) \leq \left\|\boldsymbol{\delta}^{(L)}\right\| \prod_{l=1}^{L-1} \left\|\mathbf{W}^{(l+1)}\right\| \cdot 0.25^{L-1} \tag{2.37}$$

The geometric decay 0.25^{L-1} causes gradients to vanish exponentially in depth, motivating ReLU and careful initialization.

2.4 Gradient Descent Variants

2.4.1 Batch Gradient Descent

The full gradient over all N samples:

$$\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t) = \frac{1}{N} \sum_{n=1}^{N} \nabla_{\boldsymbol{\theta}} \ell(\mathbf{x}_n, \mathbf{y}_n; \boldsymbol{\theta}_t) \tag{2.38}$$

Update: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{g}_t$. For convex, L-smooth functions with $\eta \leq 1/L$, gradient descent achieves $\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}^*) \leq \frac{\|\boldsymbol{\theta}_0 - \boldsymbol{\theta}^*\|^2}{2\eta t}$. However, computing $\mathbf{g}_t$ over millions of samples is prohibitively expensive per iteration.

2.4.2 Stochastic Gradient Descent

SGD uses a single random sample per update:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla_{\boldsymbol{\theta}} \ell(\mathbf{x}_{i_t}, \mathbf{y}_{i_t}; \boldsymbol{\theta}_t) \tag{2.39}$$

where $i_t \sim \text{Uniform}\{1, \ldots, N\}$. The stochastic gradient $\tilde{\mathbf{g}}_t = \nabla \ell_{i_t}$ is an **unbiased estimator** of the true gradient:

$$\mathbb{E}_{i_t}[\tilde{\mathbf{g}}_t] = \frac{1}{N} \sum_{n=1}^{N} \nabla \ell_n = \mathbf{g}_t \tag{2.40}$$

The variance of this estimator is

$$\text{Var}[\tilde{\mathbf{g}}_t] = \mathbb{E}[\|\tilde{\mathbf{g}}_t - \mathbf{g}_t\|^2] = \frac{1}{N} \sum_{n=1}^{N} \|\nabla \ell_n - \mathbf{g}_t\|^2 \tag{2.41}$$

which can be substantial when individual gradient norms vary widely. SGD convergence requires a decaying learning rate schedule satisfying the Robbins-Monro conditions:

$$\sum_{t=1}^{\infty} \eta_t = \infty, \quad \sum_{t=1}^{\infty} \eta_t^2 < \infty \tag{2.42}$$

The first ensures the algorithm can reach the optimum from any starting point; the second controls the variance accumulation. Under these conditions, SGD converges almost surely to a stationary point for non-convex objectives.

2.4.3 Mini-Batch SGD

Mini-batch SGD strikes a balance, using B samples per update:

$$\mathbf{g}_t^{(B)} = \frac{1}{B} \sum_{b=1}^{B} \nabla_{\boldsymbol{\theta}} \ell(\mathbf{x}_{i_b}, \mathbf{y}_{i_b}; \boldsymbol{\theta}_t) \tag{2.43}$$

The variance of the mini-batch gradient is

$$\text{Var}[\mathbf{g}_t^{(B)}] = \frac{1}{B} \text{Var}[\tilde{\mathbf{g}}_t] = \frac{1}{B} \cdot \frac{1}{N} \sum_{n=1}^{N} \|\nabla \ell_n - \mathbf{g}_t\|^2 \tag{2.44}$$

The $1/B$ variance reduction (assuming i.i.d. sampling without replacement approximates independence for $B \ll N$) enables larger learning rates and more stable convergence. The computational cost per sample is amortized through vectorized operations on GPUs, making $B \in [32, 512]$ optimal in practice.

2.5 Momentum

2.5.1 Classical Momentum

Momentum accumulates a velocity vector that smooths the gradient trajectory:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t) \tag{2.45}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{v}_t \tag{2.46}$$

with $\beta \in [0, 1)$ (typically 0.9). Unrolling the recurrence reveals an exponentially weighted moving average of past gradients:

$$\mathbf{v}_t = \eta \sum_{\tau=0}^{t} \beta^{t-\tau} \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_\tau) \tag{2.47}$$

with an effective window of $1/(1-\beta)$ iterations. In directions of consistent gradient, momentum accelerates (constructive interference); in directions of oscillating gradients (e.g., narrow valleys), successive gradients cancel, damping oscillations. This yields faster convergence in high-curvature directions orthogonal to the gradient.

For a quadratic objective $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^T \mathbf{H} \boldsymbol{\theta}$ with eigenvalues λ_i, the momentum iteration has the characteristic polynomial $z^2 - (1+\beta-\eta\lambda_i)z + \beta = 0$, converging when $|z| < 1$. The optimal momentum for the largest eigenvalue $\lambda_{\max}$ is $\beta^* = \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^2$ where $\kappa = \lambda_{\max}/\lambda_{\min}$, achieving a convergence rate of $O(\sqrt{\kappa})$ versus $O(\kappa)$ for plain gradient descent.

2.5.2 Nesterov Accelerated Gradient

Nesterov momentum evaluates the gradient at a *lookahead* position:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t - \beta \mathbf{v}_{t-1}) \tag{2.48}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{v}_t \tag{2.49}$$

The intuition: if the momentum term $\beta \mathbf{v}_{t-1}$ will move the parameters anyway, we should compute the gradient at the anticipated new position. This correction provably achieves $O(1/t^2)$ convergence rate for convex smooth functions, versus $O(1/t)$ for standard momentum, and reduces oscillations when the momentum overshoots.

2.6 Adaptive Learning Rate Methods

2.6.1 AdaGrad

AdaGrad adapts the learning rate per-parameter based on the historical sum of squared gradients:

$$\mathbf{G}_t = \mathbf{G}_{t-1} + (\nabla \mathcal{L}(\boldsymbol{\theta}_t))^2 \tag{2.50}$$

where the square and square root are element-wise. The update is

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\mathbf{G}_t + \varepsilon}} \odot \nabla \mathcal{L}(\boldsymbol{\theta}_t) \tag{2.51}$$

The i-th parameter's effective learning rate is $\eta_i = \eta/\sqrt{G_{t,ii} + \varepsilon}$. Since $G_{t,ii} = \sum_{\tau=1}^{t} (\nabla_{\theta_i} \mathcal{L}_\tau)^2$ grows monotonically, the learning rate decays *automatically*:

$$\eta_i(t) \propto \frac{1}{\sqrt{\sum_{\tau=1}^{t} (\partial \mathcal{L}_\tau / \partial \theta_i)^2}} \tag{2.52}$$

This is beneficial for sparse features (infrequent parameters get larger effective updates) but problematic for dense optimization because the denominator grows unboundedly, causing the learning rate to vanish prematurely.

2.6.2 RMSProp

RMSProp replaces the cumulative sum with an exponentially weighted moving average:

$$\mathbf{G}_t = \beta \mathbf{G}_{t-1} + (1-\beta)(\nabla \mathcal{L}(\boldsymbol{\theta}_t))^2 \tag{2.53}$$

with $\beta \in [0, 1)$ (typically 0.9). The EMA preserves a running estimate of the recent gradient magnitude, preventing the denominator from growing monotonically:

$$\mathbf{G}_t = (1-\beta) \sum_{\tau=1}^{t} \beta^{t-\tau} (\nabla \mathcal{L}(\boldsymbol{\theta}_\tau))^2 \tag{2.54}$$

The update becomes

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\eta}{\sqrt{\mathbf{G}_t + \varepsilon}} \odot \nabla \mathcal{L}(\boldsymbol{\theta}_t) \tag{2.55}$$

This solves AdaGrad's vanishing learning rate problem and is effective for non-stationary objectives like RNN training.

2.6.3 Adam: Adaptive Moment Estimation

Adam combines momentum (first moment) with RMSProp-style adaptive scaling (second moment). Let $\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t)$.

First moment (mean) estimate:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \tag{2.56}$$

Second moment (uncentered variance) estimate:

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2 \tag{2.57}$$

with typical values $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\mathbf{m}_0 = \mathbf{0}$, $\mathbf{v}_0 = \mathbf{0}$.

Bias correction derivation: Initializing $\mathbf{m}_0 = \mathbf{0}$ biases the estimates toward zero, especially during early iterations. We compute the expectation of $\mathbf{m}_t$:

$$\mathbf{m}_t = (1-\beta_1) \sum_{\tau=1}^{t} \beta_1^{t-\tau} \mathbf{g}_\tau \tag{2.58}$$

$$\mathbb{E}[\mathbf{m}_t] = (1-\beta_1) \sum_{\tau=1}^{t} \beta_1^{t-\tau} \mathbb{E}[\mathbf{g}_\tau] \tag{2.59}$$

If the gradient distribution is stationary, $\mathbb{E}[\mathbf{g}_\tau] = \mathbb{E}[\mathbf{g}]$, then

$$\mathbb{E}[\mathbf{m}_t] = \mathbb{E}[\mathbf{g}](1-\beta_1) \sum_{\tau=1}^{t} \beta_1^{t-\tau} = \mathbb{E}[\mathbf{g}](1-\beta_1^t) \tag{2.60}$$

Thus $\mathbf{m}_t$ underestimates the true first moment by a factor $(1-\beta_1^t)$. The bias-corrected estimate is

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t} \tag{2.61}$$

Similarly for the second moment, assuming stationarity of $\mathbf{g}_\tau^2$:

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \tag{2.62}$$

The Adam update rule is

$$\boxed{\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon}} \tag{2.63}$$

with $\varepsilon = 10^{-8}$ for numerical stability. The ratio $\hat{\mathbf{m}}_t/\sqrt{\hat{\mathbf{v}}_t}$ approximates the signal-to-noise ratio of the gradient, providing an adaptive, scale-invariant step size. For a given parameter, $\frac{|m_i|}{\sqrt{v_i}}$ is bounded by approximately $1/\sqrt{1-\beta_2}$ in steady state, giving an effective step size of roughly η.

2.7 Weight Initialization

2.7.1 Xavier/Glorot Initialization

Consider a linear layer $\mathbf{z} = \mathbf{W}\mathbf{x}$ where $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ with i.i.d. entries such that $\mathbb{E}[x_i] = 0$ and $\text{Var}[x_i] = \sigma_x^2$. Assume $W_{ij} \sim \mathcal{N}(0, \sigma_w^2)$ i.i.d. The variance of a single output unit is

$$\text{Var}[z_i] = \text{Var}\left[\sum_{j=1}^{d_{\text{in}}} W_{ij} x_j\right] = \sum_{j=1}^{d_{\text{in}}} \text{Var}[W_{ij} x_j] \tag{2.64}$$

$$= \sum_{j=1}^{d_{\text{in}}} \mathbb{E}[W_{ij}^2]\mathbb{E}[x_j^2] = d_{\text{in}} \sigma_w^2 \sigma_x^2 \tag{2.65}$$

To preserve variance through the forward pass ($\text{Var}[z_i] = \text{Var}[x_j] = \sigma_x^2$), we require

$$d_{\text{in}} \sigma_w^2 = 1 \quad \Longrightarrow \quad \sigma_w^2 = \frac{1}{d_{\text{in}}} \tag{2.66}$$

Now consider the backward pass. The gradient with respect to the input is $\frac{\partial \ell}{\partial \mathbf{x}} = \mathbf{W}^T \frac{\partial \ell}{\partial \mathbf{z}}$. Let $\delta_i = \frac{\partial \ell}{\partial z_i}$ with $\text{Var}[\delta_i] = \sigma_\delta^2$. Then

$$\text{Var}\left[\frac{\partial \ell}{\partial x_j}\right] = \sum_{i=1}^{d_{\text{out}}} \text{Var}[W_{ij}\delta_i] = d_{\text{out}} \sigma_w^2 \sigma_\delta^2 \tag{2.67}$$

To preserve gradient variance, we require $d_{\text{out}}\sigma_w^2 = 1$, i.e., $\sigma_w^2 = 1/d_{\text{out}}$. The **Xavier/Glorot initialization** compromises:

$$\boxed{\text{Var}[W_{ij}] = \frac{2}{d_{\text{in}} + d_{\text{out}}}} \tag{2.68}$$

For uniform distribution $\mathcal{U}[-a, a]$, $\text{Var} = a^2/3$, so $a = \sqrt{6/(d_{\text{in}} + d_{\text{out}})}$.

Critical assumption: The derivation assumes linear activation. For sigmoid/tanh, the variance-preserving property holds approximately because $\tanh'(0) = 1$ and the function is approximately linear near zero, where activations tend to concentrate with proper initialization.

2.7.2 He/Kaiming Initialization

For ReLU activation $g(z) = \max(0, z)$, the linearity assumption breaks: half of the units are zeroed. Re-derive variance propagation for ReLU:

$$\mathrm{Var}[z_i] = \mathrm{Var}\left[\sum_j W_{ij} x_j\right] = d_{\text{in}} \sigma_w^2 \mathbb{E}[x_j^2] \tag{2.69}$$

If $x_j = \mathrm{ReLU}(z_j^{(l-1)})$ and $z_j^{(l-1)}$ has a symmetric distribution around zero, then

$$\mathbb{E}[x_j^2] = \mathbb{E}[\mathrm{ReLU}(z)^2] = \int_0^\infty z^2 p(z)\, dz = \frac{1}{2}\mathbb{E}[z^2] = \frac{1}{2}\,\mathrm{Var}[z] \tag{2.70}$$

where the factor $1/2$ comes from zeroing the negative half. Therefore

$$\mathrm{Var}[z_i^{(l)}] = \frac{1}{2} d_{\text{in}} \sigma_w^2\, \mathrm{Var}[z^{(l-1)}] \tag{2.71}$$

For variance preservation through L layers, we need $\frac{1}{2} d_{\text{in}} \sigma_w^2 = 1$, giving

$$\boxed{\mathrm{Var}[W_{ij}] = \frac{2}{d_{\text{in}}}} \tag{2.72}$$

The uniform version is $\mathcal{U}[-\sqrt{6/d_{\text{in}}}, \sqrt{6/d_{\text{in}}}]$. Note the factor of 2 compensates for the $1/2$ attenuation from ReLU. For the backward pass, a similar analysis gives $\mathrm{Var}[W] = 2/d_{\text{out}}$. In practice, $2/d_{\text{in}}$ (fan-in mode) is the standard.

2.8 Batch Normalization

2.8.1 Forward Pass

Given a mini-batch $\mathcal{B} = \{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ of activations (for a single feature dimension), batch normalization computes:

Mini-batch mean:

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i \tag{2.73}$$

Mini-batch variance:

$$\sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \tag{2.74}$$

Normalize:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} \tag{2.75}$$

Scale and shift (learnable parameters γ, β):

$$y_i = \gamma \hat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \tag{2.76}$$

The learnable affine transformation restores the network's representational capacity — without γ, β, the normalized distribution would always have zero mean and unit variance, severely limiting expressiveness.

2.8.2 Backward Pass Through BatchNorm

We need $\frac{\partial \ell}{\partial x_i}$, $\frac{\partial \ell}{\partial \gamma}$, and $\frac{\partial \ell}{\partial \beta}$. The computation graph is:

$$x_i \to \mu_{\mathcal{B}}, \sigma^2_{\mathcal{B}} \to \hat{x}_i \to y_i \to \ell \tag{2.77}$$

with x_i affecting the loss through three paths: directly through $\hat{x}_i$, and indirectly through $\mu_{\mathcal{B}}$ and $\sigma^2_{\mathcal{B}}$ (since they depend on all x_i in the batch).

First, the easy gradients:

$$\frac{\partial \ell}{\partial \gamma} = \sum_{i=1}^{m} \frac{\partial \ell}{\partial y_i} \frac{\partial y_i}{\partial \gamma} = \sum_{i=1}^{m} \frac{\partial \ell}{\partial y_i} \hat{x}_i \tag{2.78}$$

$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^{m} \frac{\partial \ell}{\partial y_i} \tag{2.79}$$

For $\frac{\partial \ell}{\partial \hat{x}_i}$:

$$\frac{\partial \ell}{\partial \hat{x}_i} = \frac{\partial \ell}{\partial y_i} \gamma \tag{2.80}$$

For $\frac{\partial \ell}{\partial \sigma^2_{\mathcal{B}}}$ (via all $\hat{x}_j$):

$$\frac{\partial \ell}{\partial \sigma^2_{\mathcal{B}}} = \sum_{j=1}^{m} \frac{\partial \ell}{\partial \hat{x}_j} \frac{\partial \hat{x}_j}{\partial \sigma^2_{\mathcal{B}}} \tag{2.81}$$

$$\frac{\partial \hat{x}_j}{\partial \sigma^2_{\mathcal{B}}} = \frac{\partial}{\partial \sigma^2_{\mathcal{B}}} \frac{x_j - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \varepsilon}} = -\frac{1}{2}(x_j - \mu_{\mathcal{B}})(\sigma^2_{\mathcal{B}} + \varepsilon)^{-3/2} \tag{2.82}$$

$$\frac{\partial \ell}{\partial \sigma^2_{\mathcal{B}}} = \sum_{j=1}^{m} \frac{\partial \ell}{\partial \hat{x}_j} (x_j - \mu_{\mathcal{B}}) \cdot \left(-\frac{1}{2}\right) (\sigma^2_{\mathcal{B}} + \varepsilon)^{-3/2} \tag{2.83}$$

For $\frac{\partial \ell}{\partial \mu_{\mathcal{B}}}$ (via $\hat{x}_j$ and $\sigma^2_{\mathcal{B}}$):

$$\frac{\partial \ell}{\partial \mu_{\mathcal{B}}} = \sum_{j=1}^{m} \frac{\partial \ell}{\partial \hat{x}_j} \frac{\partial \hat{x}_j}{\partial \mu_{\mathcal{B}}} + \frac{\partial \ell}{\partial \sigma^2_{\mathcal{B}}} \frac{\partial \sigma^2_{\mathcal{B}}}{\partial \mu_{\mathcal{B}}} \tag{2.84}$$

$$\frac{\partial \hat{x}_j}{\partial \mu_{\mathcal{B}}} = -\frac{1}{\sqrt{\sigma^2_{\mathcal{B}} + \varepsilon}} \tag{2.85}$$

$$\frac{\partial \sigma^2_{\mathcal{B}}}{\partial \mu_{\mathcal{B}}} = \frac{1}{m} \sum_{j=1}^{m} 2(x_j - \mu_{\mathcal{B}})(-1) = -\frac{2}{m} \sum_{j=1}^{m} (x_j - \mu_{\mathcal{B}}) = 0 \tag{2.86}$$

The last term vanishes because $\sum_j (x_j - \mu_{\mathcal{B}}) = 0$ by definition of the mean. Thus

$$\frac{\partial \ell}{\partial \mu_{\mathcal{B}}} = \sum_{j=1}^{m} \frac{\partial \ell}{\partial \hat{x}_j} \cdot \frac{-1}{\sqrt{\sigma^2_{\mathcal{B}} + \varepsilon}} \tag{2.87}$$

Finally, combining all paths for $\frac{\partial \ell}{\partial x_i}$:

$$\frac{\partial \ell}{\partial x_i} = \frac{\partial \ell}{\partial \hat{x}_i}\frac{\partial \hat{x}_i}{\partial x_i} + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2}\frac{\partial \sigma_{\mathcal{B}}^2}{\partial x_i} + \frac{\partial \ell}{\partial \mu_{\mathcal{B}}}\frac{\partial \mu_{\mathcal{B}}}{\partial x_i} \tag{2.88}$$

$$\frac{\partial \hat{x}_i}{\partial x_i} = \frac{1}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} \tag{2.89}$$

$$\frac{\partial \sigma_{\mathcal{B}}^2}{\partial x_i} = \frac{2}{m}(x_i - \mu_{\mathcal{B}}) \tag{2.90}$$

$$\frac{\partial \mu_{\mathcal{B}}}{\partial x_i} = \frac{1}{m} \tag{2.91}$$

Assembling:

$$\boxed{\frac{\partial \ell}{\partial x_i} = \frac{\partial \ell}{\partial \hat{x}_i}\frac{1}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2}\frac{2(x_i - \mu_{\mathcal{B}})}{m} + \frac{\partial \ell}{\partial \mu_{\mathcal{B}}}\frac{1}{m}} \tag{2.92}$$

2.8.3 Inference

During inference, we cannot compute batch statistics. Instead, we maintain running averages during training:

$$\mathbb{E}[\mathbf{x}] \leftarrow \alpha \mathbb{E}[\mathbf{x}] + (1-\alpha)\mu_{\mathcal{B}} \tag{2.93}$$

$$\mathrm{Var}[\mathbf{x}] \leftarrow \alpha\, \mathrm{Var}[\mathbf{x}] + (1-\alpha)\frac{m}{m-1}\sigma_{\mathcal{B}}^2 \tag{2.94}$$

with momentum α (e.g., 0.99). The factor $m/(m-1)$ corrects for the biased variance estimator. At test time:

$$y = \gamma \frac{x - \mathbb{E}[\mathbf{x}]}{\sqrt{\mathrm{Var}[\mathbf{x}] + \varepsilon}} + \beta \tag{2.95}$$

2.9 Layer Normalization

LayerNorm normalizes across features instead of batch:

$$\mu^{(l)} = \frac{1}{H}\sum_{i=1}^{H} x_i^{(l)} \quad \text{(mean over hidden units)} \tag{2.96}$$

$$\sigma^{(l)2} = \frac{1}{H}\sum_{i=1}^{H}(x_i^{(l)} - \mu^{(l)})^2 \tag{2.97}$$

$$\hat{x}_i^{(l)} = \frac{x_i^{(l)} - \mu^{(l)}}{\sqrt{\sigma^{(l)2} + \varepsilon}} \tag{2.98}$$

$$y_i^{(l)} = \gamma_i \hat{x}_i^{(l)} + \beta_i \tag{2.99}$$

Key differences from BatchNorm: (1) the statistics are computed *independently for each sample*, making the operation identical at training and test time; (2) it is invariant to per-sample scaling and shifting. For RNNs, where batch statistics are ill-defined due to varying sequence lengths and recurrent dependencies, LayerNorm works naturally. For Transformers, where the batch dimension interacts with the sequence dimension, LayerNorm applies consistently across tokens.

2.10 Dropout

2.10.1 Bernoulli Mask Formulation

During training, each hidden unit is retained with probability p:

$$\mathbf{m} \sim \text{Bernoulli}(p)^d, \quad \tilde{\mathbf{a}} = \mathbf{m} \odot \mathbf{a} \tag{2.100}$$

The output is scaled to maintain the same expected value at test time. In **inverted dropout**, scaling is done at training time:

$$\tilde{\mathbf{a}} = \frac{1}{p}\mathbf{m} \odot \mathbf{a} \tag{2.101}$$

so $\mathbb{E}[\tilde{a}_i] = \frac{1}{p} \cdot p \cdot a_i = a_i$, and at test time the layer is identity. The original dropout performs scaling at test time: train with $\tilde{\mathbf{a}} = \mathbf{m} \odot \mathbf{a}$, test with $\mathbb{E}[\tilde{\mathbf{a}}] = p\mathbf{a}$.

2.10.2 Ensemble Interpretation

Dropout samples a subnetwork for each training step. With d units, there are 2^d possible subnetworks. Training with dropout approximates training an exponential ensemble:

$$\hat{y}_{\text{ensemble}} = \frac{1}{2^d} \sum_{\mathbf{m} \in \{0,1\}^d} f(\mathbf{x}; \mathbf{m}) \tag{2.102}$$

At test time, using the full network with scaled weights approximates the geometric mean of the ensemble predictions. For a single linear layer $\mathbf{y} = \mathbf{W}(\mathbf{m} \odot \mathbf{x})$ with dropout,

$$\mathbb{E}_{\mathbf{m}}[y_k] = \mathbb{E}\left[\sum_j W_{kj} m_j x_j\right] = \sum_j W_{kj} p x_j = \sum_j (pW_{kj}) x_j \tag{2.103}$$

confirming that test-time weight scaling $p\mathbf{W}$ produces the correct expected output.

2.11 Regularization via Parameter Norm Penalties

2.11.1 L2 Regularization and Gaussian Prior

L2 weight decay adds $\frac{\lambda}{2}\|\mathbf{W}\|_F^2$ to the loss. From a Bayesian perspective, we impose a Gaussian prior $p(\mathbf{W}) = \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$ with $\sigma^2 = 1/\lambda$:

$$p(\mathbf{W} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \mathbf{W}) p(\mathbf{W}) \tag{2.104}$$

$$-\log p(\mathbf{W} \mid \mathcal{D}) = -\log p(\mathcal{D} \mid \mathbf{W}) - \log p(\mathbf{W}) + \text{const} \tag{2.105}$$

$$= -\log p(\mathcal{D} \mid \mathbf{W}) + \frac{1}{2\sigma^2}\|\mathbf{W}\|^2 + \text{const} \tag{2.106}$$

Thus MAP estimation with a zero-mean isotropic Gaussian prior yields L2 regularization. The Gaussian prior encodes the belief that weights should be small and concentrated near zero.

2.11.2 L1 Regularization and Laplace Prior

L1 regularization adds $\lambda \|\mathbf{W}\|_1 = \lambda \sum_{ij} |W_{ij}|$. The Laplace prior $p(W_{ij}) = \frac{\lambda}{2} e^{-\lambda |W_{ij}|}$ gives:

$$-\log p(\mathbf{W}) = -\sum_{ij} \log \left(\frac{\lambda}{2} e^{-\lambda |W_{ij}|} \right) = \lambda \sum_{ij} |W_{ij}| + \text{const} \tag{2.107}$$

The Laplace distribution has heavier tails than a Gaussian and a sharp peak at zero, encouraging exact sparsity (many weights become exactly zero) due to the non-differentiability of $|W_{ij}|$ at $W_{ij} = 0$.

2.12 Learning Rate Scheduling and Gradient Clipping

2.12.1 Learning Rate Schedules

Step decay: $\eta_t = \eta_0 \cdot \gamma^{\lfloor t/T \rfloor}$, reducing the learning rate by factor γ every T epochs.

Exponential decay: $\eta_t = \eta_0 \cdot e^{-kt}$.

Cosine annealing:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t}{T_{\max}} \pi \right) \right) \tag{2.108}$$

smoothly decaying from $\eta_{\max}$ to $\eta_{\min}$ over $T_{\max}$ iterations.

Warmup: For the first T_w iterations, linearly increase the learning rate from 0 to η_0:

$$\eta_t = \frac{t}{T_w} \eta_0, \quad t \leq T_w \tag{2.109}$$

Warmup prevents destabilization from large initial gradients when using large batch sizes or adaptive optimizers, giving Adam's moment estimates time to calibrate before large parameter updates.

2.12.2 Gradient Clipping

When gradients explode (common in RNNs), gradient clipping rescales the gradient:

$$\tilde{\mathbf{g}} = \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\| \leq c \\ c \frac{\mathbf{g}}{\|\mathbf{g}\|} & \text{if } \|\mathbf{g}\| > c \end{cases} \tag{2.110}$$

This ensures $\|\tilde{\mathbf{g}}\| \leq c$ while preserving the gradient direction. The clipping threshold c is a hyperparameter, typically in $[1, 10]$.

2.13 Algorithm: Training Loop with Adam and BatchNorm

Algorithm 1: Training a Deep Network with Adam, BatchNorm, and Gradient Clipping

Input : Network f_θ, data $\{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^N$, learning rate η, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, batch size B, epochs E, clip threshold c

Output: Trained parameters θ

Initialize θ with He init;
$\mathbf{m}_0 \leftarrow \mathbf{0}$, $\mathbf{v}_0 \leftarrow \mathbf{0}$, $t \leftarrow 0$;
for *epoch* $= 1$ **to** E **do**
  Shuffle dataset;
  for *each mini-batch* $\mathcal{B}$ *of size* B **do**
    $t \leftarrow t + 1$;
    Forward pass with BatchNorm (training mode): compute $\hat{\mathbf{y}}_n$ for $n \in \mathcal{B}$;
    Compute loss $\mathcal{L}_\mathcal{B} = \frac{1}{B}\sum_{n\in\mathcal{B}} \ell(\hat{\mathbf{y}}_n, \mathbf{y}_n) + \frac{\lambda}{2}\|\theta\|_2^2$;
    Backpropagate: compute $\mathbf{g}_t = \nabla_\theta \mathcal{L}_\mathcal{B}$ (including BatchNorm backward);
    if $\|\mathbf{g}_t\| > c$ **then**
      $\mathbf{g}_t \leftarrow c\frac{\mathbf{g}_t}{\|\mathbf{g}_t\|}$;
    $\mathbf{m}_t \leftarrow \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$;
    $\mathbf{v}_t \leftarrow \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$;
    $\hat{\mathbf{m}}_t \leftarrow \mathbf{m}_t/(1-\beta_1^t)$;
    $\hat{\mathbf{v}}_t \leftarrow \mathbf{v}_t/(1-\beta_2^t)$;
    $\theta \leftarrow \theta - \eta\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t}+\varepsilon}$;
  Update learning rate via cosine annealing;
  Evaluate on validation set (BatchNorm inference mode);
return θ

2.14 Pros and Cons

Pros: The backpropagation + SGD framework scales to networks with hundreds of millions of parameters. Adam provides robust, hyperparameter-insensitive convergence. BatchNorm enables training of very deep networks by controlling internal covariate shift. Dropout and L2 regularization provide effective overfitting prevention without architectural constraints. Momentum and adaptive methods navigate pathological loss landscapes.

Cons: SGD requires careful learning rate tuning; Adam can generalize worse than well-tuned SGD with momentum on some tasks. BatchNorm introduces a dependency between samples in a mini-batch, causing issues for small batches and recurrent networks. The backpropagation chain can still suffer from vanishing/exploding gradients despite careful initialization. All first-order methods struggle with sharp minima and saddle points in high-dimensional non-convex landscapes.

2.15 Evolution

The optimization of neural networks evolved through distinct phases. Early attempts used plain SGD (Robbins-Monro, 1951) with hand-tuned learning rates, which was slow and brittle. Momentum (Polyak, 1964) and Nesterov acceleration (1983) improved convergence rates dramatically, but still required per-layer tuning. The breakthrough came with adaptive methods: AdaGrad (Duchi et al., 2011) automated per-parameter learning rates for sparse problems; RMSProp (Tieleman & Hinton, 2012) fixed AdaGrad's vanishing rate issue; Adam (Kingma & Ba, 2015) unified momentum and adaptive scaling, becoming the default optimizer. Parallel to optimization advances, initialization theory progressed from heuristic random weights through Xavier/Glorot (2010) to He initialization (2015), each enabling deeper architectures. BatchNorm (Ioffe & Szegedy, 2015) was transformative: by reducing internal covariate shift, it allowed 10-100× higher learning rates, reduced initialization sensitivity, and acted as a regularizer. LayerNorm (Ba et al., 2016) adapted normalization to RNNs and Transformers. Dropout (Srivastava et al., 2014) provided an extremely effective, computationally cheap regularizer. What remains unsolved: all current optimizers are first-order; second-order methods (Newton, K-FAC) are theoretically superior but computationally prohibitive. Sharp vs. flat minima, convergence to global optima in overparameterized networks, and understanding the implicit regularization of SGD remain active research areas.

Part II

Convolutional Architectures

Chapter 3

Convolutional Neural Networks

3.1 Motivation

A fully-connected layer mapping an image of size $H \times W \times C$ to D hidden units requires $H \cdot W \cdot C \cdot D$ parameters. For a modest $224 \times 224 \times 3$ input and $D = 4096$, this is $224^2 \cdot 3 \cdot 4096 \approx 6.2 \times 10^8$ parameters in a single layer — computationally prohibitive and statistically doomed to overfitting. Natural images exhibit *spatial locality* (nearby pixels are highly correlated) and *translation equivariance* (a cat shifted by a few pixels is still a cat). A convolutional layer encodes these inductive biases: local connectivity restricts each neuron to a small receptive field, and weight sharing (the same filter slides across the entire spatial extent) enforces translation equivariance. This reduces parameters from $HWC \cdot D$ to $k_h k_w C \cdot C_{\text{out}}$, where the kernel size $k \times k$ is typically 3×3 or 5×5, yielding orders-of-magnitude parameter savings.

3.2 Mathematical Formulation

3.2.1 One-Dimensional Convolution

For a 1D input signal $x : \mathbb{Z} \to \mathbb{R}$ and filter w of size k, the discrete convolution is

$$(x * w)[t] = \sum_{m=-\infty}^{\infty} x[m]\, w[t-m] \tag{3.1}$$

For finite-length signals with 0-indexing and filter of size k:

$$(x * w)[t] = \sum_{m=0}^{k-1} w[m]\, x[t-m] \tag{3.2}$$

In deep learning, we typically implement **cross-correlation**, which is convolution without flipping the kernel:

$$(x \star w)[t] = \sum_{m=0}^{k-1} w[m]\, x[t+m] \tag{3.3}$$

Since the weights are learned, the distinction is irrelevant — the network will learn flipped kernels if needed. This text uses $\star$ to denote the cross-correlation operation, as implemented by all modern frameworks.

3.2.2 Two-Dimensional Convolution

For a 2D input $\mathbf{X} \in \mathbb{R}^{H \times W}$ and kernel $\mathbf{K} \in \mathbb{R}^{k_h \times k_w}$, the single-channel convolution is

$$\mathbf{S}[i,j] = (\mathbf{X} \star \mathbf{K})[i,j] = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \mathbf{K}[u,v] \cdot \mathbf{X}[i \cdot s + u,\ j \cdot s + v] \tag{3.4}$$

where s is the stride. For a multichannel input $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ and C_{out} filters, each filter $\mathbf{W}^{(c)} \in \mathbb{R}^{C_{\text{in}} \times k_h \times k_w}$ produces one output channel:

$$\mathbf{Z}[c_{\text{out}}, i, j] = \sum_{c_{\text{in}}=0}^{C_{\text{in}}-1} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \mathbf{W}[c_{\text{out}}, c_{\text{in}}, u, v] \cdot \mathbf{X}[c_{\text{in}},\ i \cdot s + u,\ j \cdot s + v] + \mathbf{b}[c_{\text{out}}] \quad (3.5)$$

where $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w}$ and $\mathbf{b} \in \mathbb{R}^{C_{\text{out}}}$. The resulting output has shape $C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$.

3.2.3 Three-Dimensional Convolution

For volumetric data (e.g., video or medical CT scans), the input is $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times D \times H \times W}$ and kernels are $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_d \times k_h \times k_w}$:

$$\mathbf{Z}[c_{\text{out}}, i, j, k] = \sum_{c_{\text{in}}, u, v, w} \mathbf{W}[c_{\text{out}}, c_{\text{in}}, u, v, w] \cdot \mathbf{X}[c_{\text{in}},\ is_d + u,\ js_h + v,\ ks_w + w] + \mathbf{b}[c_{\text{out}}] \quad (3.6)$$

3.2.4 Output Size Formula

For input size i, kernel size k, padding p, stride s, and dilation d, the output spatial dimension is

$$o = \left\lfloor \frac{i + 2p - d(k-1) - 1}{s} \right\rfloor + 1 \quad (3.7)$$

For non-dilated convolution ($d = 1$): $o = \lfloor (i + 2p - k)/s \rfloor + 1$. Common settings:

- **Valid** ($p = 0$): $o = \lfloor (i - k)/s \rfloor + 1$, output smaller than input.
- **Same** (p chosen so $o = \lceil i/s \rceil$): $p = \lfloor (s(i-1) - i + k)/2 \rfloor$, with potential asymmetric padding.
- **Full** ($p = k - 1$): output larger than input, including all partial overlaps.

3.2.5 Receptive Field

The receptive field r_l of a unit in layer l is the number of input pixels that affect its value. For a stack of layers with kernel sizes k_i and strides s_i (with j_i as the effective stride):

$$r_0 = 1 \quad (3.8)$$

$$r_l = r_{l-1} + (k_l - 1) \prod_{i=1}^{l-1} s_i \quad (3.9)$$

The effective stride (cumulative subsampling factor) at layer l is

$$j_l = \prod_{i=1}^{l} s_i \quad (3.10)$$

For a deep network like VGG-16 (all 3×3 kernels, stride 1, interleaved max-pooling stride 2), the receptive field grows rapidly:

$$r_0 = 1 \tag{3.11}$$
$$r_1 = 1 + (3-1)\cdot 1 = 3 \tag{3.12}$$
$$r_2 = 3 + (3-1)\cdot 1 = 5 \tag{3.13}$$
$$r_3 = 5 + (3-1)\cdot 1 = 7 \quad \text{(after maxpool stride 2: } r_4 = 7 + (2+1-1)\cdot 2 = ...) \tag{3.14}$$

This arithmetic shows that a stack of small kernels can achieve the same receptive field as a large kernel with fewer parameters and more nonlinearities.

3.2.6 Convolution as Matrix Multiplication: im2col and GEMM

The convolution operation can be recast as a single matrix multiplication for efficient GPU execution. The `im2col` operation rearranges the input into columns, each column containing the receptive field for one output position:

For input $\mathbf{X} \in \mathbb{R}^{C_{\text{in}}\times H\times W}$ and kernel $\mathbf{W} \in \mathbb{R}^{C_{\text{out}}\times C_{\text{in}}\times k_h\times k_w}$:

1. Flatten each $C_{\text{in}} \times k_h \times k_w$ filter into a row vector of length $C_{\text{in}} \cdot k_h \cdot k_w$, forming matrix $\mathbf{W}_{\text{mat}} \in \mathbb{R}^{C_{\text{out}}\times(C_{\text{in}}k_hk_w)}$.
2. For each output position (i, j), extract the $C_{\text{in}} \times k_h \times k_w$ patch from $\mathbf{X}$, flatten it into a column vector. Collect all $H_{\text{out}} \cdot W_{\text{out}}$ columns into $\mathbf{X}_{\text{col}} \in \mathbb{R}^{(C_{\text{in}}k_hk_w)\times(H_{\text{out}}W_{\text{out}})}$.
3. Compute $\mathbf{Z}_{\text{mat}} = \mathbf{W}_{\text{mat}}\mathbf{X}_{\text{col}} + \mathbf{b}$, giving $\mathbf{Z}_{\text{mat}} \in \mathbb{R}^{C_{\text{out}}\times(H_{\text{out}}W_{\text{out}})}$.
4. Reshape to $\mathbf{Z} \in \mathbb{R}^{C_{\text{out}}\times H_{\text{out}}\times W_{\text{out}}}$.

This reduces convolution to a GEMM (General Matrix Multiply) call, for which optimized BLAS libraries (cuBLAS, MKL) achieve near-peak hardware utilization.

3.2.7 FLOPs Calculation

The number of multiply-add operations for a single convolutional layer:

$$\text{FLOPs} = 2 \cdot C_{\text{out}} \cdot C_{\text{in}} \cdot k_h \cdot k_w \cdot H_{\text{out}} \cdot W_{\text{out}} \tag{3.15}$$

The factor of 2 counts one multiply and one add per weight-input pair (assuming FMA — fused multiply-add). For a fully-connected layer mapping $d_{\text{in}} \to d_{\text{out}}$:

$$\text{FLOPs}_{\text{FC}} = 2 \cdot d_{\text{in}} \cdot d_{\text{out}} \tag{3.16}$$

The ratio $\text{FLOPs}_{\text{FC}}/\text{FLOPs}_{\text{conv}} = \frac{H\cdot W\cdot d_{\text{out}}}{C_{\text{out}}\cdot k_h\cdot k_w\cdot H_{\text{out}}\cdot W_{\text{out}}}$ demonstrates the efficiency of convolution when $k_hk_w \ll HW$.

3.3 Backpropagation Through Convolution

3.3.1 Gradient with Respect to Weights

The loss ℓ depends on $\mathbf{W}$ through every output position. The partial derivative is

$$\frac{\partial \ell}{\partial \mathbf{W}[c_{\text{out}}, c_{\text{in}}, u, v]} = \sum_{i,j} \frac{\partial \ell}{\partial \mathbf{Z}[c_{\text{out}}, i, j]} \cdot \frac{\partial \mathbf{Z}[c_{\text{out}}, i, j]}{\partial \mathbf{W}[c_{\text{out}}, c_{\text{in}}, u, v]} \tag{3.17}$$

From the forward equation, $\frac{\partial \mathbf{Z}[c_{\text{out}},i,j]}{\partial \mathbf{W}[c_{\text{out}},c_{\text{in}},u,v]} = \mathbf{X}[c_{\text{in}}, is+u, js+v]$. Thus

$$\boxed{\frac{\partial \ell}{\partial \mathbf{W}[c_{\text{out}}, c_{\text{in}}, u, v]} = \sum_{i,j} \boldsymbol{\delta}_{\mathbf{Z}}[c_{\text{out}}, i, j] \cdot \mathbf{X}[c_{\text{in}}, is+u, js+v]} \tag{3.18}$$

where $\boldsymbol{\delta}_{\mathbf{Z}} \triangleq \frac{\partial \ell}{\partial \mathbf{Z}}$. This is a **convolution** of the output gradient $\boldsymbol{\delta}_{\mathbf{Z}}$ with the (suitably padded) input $\mathbf{X}$. Specifically, $\frac{\partial \ell}{\partial \mathbf{W}}$ equals the cross-correlation of $\mathbf{X}$ with $\boldsymbol{\delta}_{\mathbf{Z}}$, which is the convolution of $\mathbf{X}$ with the flipped $\boldsymbol{\delta}_{\mathbf{Z}}$.

3.3.2 Gradient with Respect to Input

The gradient propagates to the previous layer:

$$\frac{\partial \ell}{\partial \mathbf{X}[c_{\text{in}}, p, q]} = \sum_{c_{\text{out}}} \sum_{u,v} \sum_{i,j} \boldsymbol{\delta}_{\mathbf{Z}}[c_{\text{out}}, i, j] \cdot \mathbf{W}[c_{\text{out}}, c_{\text{in}}, u, v] \cdot \frac{\partial \mathbf{Z}[c_{\text{out}}, i, j]}{\partial \mathbf{X}[c_{\text{in}}, p, q]} \tag{3.19}$$

The indicator $\frac{\partial \mathbf{Z}[c_{\text{out}},i,j]}{\partial \mathbf{X}[c_{\text{in}},p,q]}$ is 1 when $p = is+u$ and $q = js+v$, and 0 otherwise. This gives

$$\boxed{\frac{\partial \ell}{\partial \mathbf{X}} = \boldsymbol{\delta}_{\mathbf{Z}} \star_{\text{trans}} \mathbf{W}} \tag{3.20}$$

where $\star_{\text{trans}}$ denotes the **transposed convolution** (also called deconvolution or fractionally-strided convolution). Operationally, this corresponds to:

1. Rotating each filter by 180° (flip both spatial dimensions).
2. Performing a full convolution of $\boldsymbol{\delta}_{\mathbf{Z}}$ with the rotated filters, with appropriate stride and padding.

The transposed convolution can also be understood through the im2col formulation: $\frac{\partial \ell}{\partial \mathbf{X}_{\text{col}}} = \mathbf{W}_{\text{mat}}^T \frac{\partial \ell}{\partial \mathbf{Z}_{\text{mat}}}$, and the `col2im` operation reverses the columnization.

3.3.3 Gradient with Respect to Bias

Since $\mathbf{b}[c_{\text{out}}]$ is added to every spatial position:

$$\frac{\partial \ell}{\partial \mathbf{b}[c_{\text{out}}]} = \sum_{i,j} \boldsymbol{\delta}_{\mathbf{Z}}[c_{\text{out}}, i, j] \tag{3.21}$$

3.4 Backpropagation Through Pooling

3.4.1 Max Pooling

For max pooling with kernel $k \times k$, the forward pass computes:

$$\mathbf{P}[c, i, j] = \max_{u,v \in \{0,\ldots,k-1\}} \mathbf{X}[c, is+u, js+v] \tag{3.22}$$

During the forward pass, we record the argmax positions (u^*, v^*) (the *switch* variables). The backward pass routes the entire gradient to the winning position:

$$\frac{\partial \ell}{\partial \mathbf{X}[c, is+u, js+v]} = \begin{cases} \frac{\partial \ell}{\partial \mathbf{P}[c,i,j]} & \text{if } (u,v) = (u^*, v^*) \\ 0 & \text{otherwise} \end{cases} \tag{3.23}$$

This sparse gradient routing is a subgradient of the max operation.

3.4.2 Average Pooling

Average pooling computes:

$$\mathbf{P}[c,i,j] = \frac{1}{k_h k_w} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} \mathbf{X}[c, is+u, js+v] \tag{3.24}$$

The gradient is uniformly distributed:

$$\frac{\partial \ell}{\partial \mathbf{X}[c, is+u, js+v]} = \frac{1}{k_h k_w} \frac{\partial \ell}{\partial \mathbf{P}[c,i,j]} \tag{3.25}$$

for all (u, v) in the pooling window.

3.4.3 Global Average Pooling

Global average pooling (GAP) reduces each feature map to a scalar:

$$\mathbf{P}[c] = \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \mathbf{X}[c,i,j] \tag{3.26}$$

The gradient is $\frac{\partial \ell}{\partial \mathbf{X}[c,i,j]} = \frac{1}{HW} \frac{\partial \ell}{\partial \mathbf{P}[c]}$. GAP replaces the fully-connected layers at the end of a CNN, drastically reducing parameters while providing structural regularization (each feature map is forced to be a confidence map for the corresponding class). GAP is essential to architectures like ResNet and is used in place of flatten+FC combinations.

3.4.4 Why Pooling?

Pooling provides approximate translation invariance: shifting the input by a small amount changes the pool output only if the maximum or average within the window changes significantly. By progressively downsampling ($s \geq 2$), pooling reduces spatial dimensions, decreasing computation in deeper layers and expanding the receptive field exponentially. The combination of convolution (which extracts features) and pooling (which summarizes over spatial regions and adds robustness to small translations) forms the backbone of nearly all CNN architectures.

3.5 Special Convolution Variants

3.5.1 1×1 Convolutions

A 1×1 convolution with C_{out} filters applied to $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ is

$$\mathbf{Z}[c_{\text{out}}, i, j] = \sum_{c_{\text{in}}=0}^{C_{\text{in}}-1} \mathbf{W}[c_{\text{out}}, c_{\text{in}}, 0, 0] \cdot \mathbf{X}[c_{\text{in}}, i, j] + \mathbf{b}[c_{\text{out}}] \tag{3.27}$$

This is a per-pixel fully-connected layer applied identically across all spatial positions — it performs **channel mixing** without spatial aggregation. 1×1 convolutions serve two key purposes: (1) **dimensionality reduction** — compressing C_{in} channels to a smaller C_{out} (a bottleneck), reducing the computational cost of subsequent 3×3 or 5×5 convolutions; (2) **adding nonlinearity** — interleaving 1×1 conv + ReLU between larger convolutions increases network depth and representational capacity with minimal parameter overhead. The Inception architecture (GoogLeNet) popularized this technique.

3.5.2 Depthwise Separable Convolution

A standard convolution applies C_{out} filters of shape $C_{\text{in}} \times k \times k$, requiring $C_{\text{out}} \cdot C_{\text{in}} \cdot k^2$ parameters. **Depthwise separable convolution** factorizes this into two steps:

Depthwise convolution: Each input channel is convolved independently with a single $k \times k$ filter (no cross-channel mixing). This produces C_{in} output channels using $C_{\text{in}} \cdot k^2$ parameters:

$$\hat{\mathbf{Z}}[c, i, j] = \sum_{u,v} \mathbf{W}_{\text{depth}}[c, u, v] \cdot \mathbf{X}[c, is + u, js + v] \tag{3.28}$$

Pointwise convolution (1×1): The C_{in} depthwise outputs are linearly combined to produce C_{out} channels:

$$\mathbf{Z}[c_{\text{out}}, i, j] = \sum_{c=0}^{C_{\text{in}}-1} \mathbf{W}_{\text{point}}[c_{\text{out}}, c] \cdot \hat{\mathbf{Z}}[c, i, j] \tag{3.29}$$

This requires $C_{\text{out}} \cdot C_{\text{in}}$ parameters.

Total parameters: $C_{\text{in}}k^2 + C_{\text{out}}C_{\text{in}}$. The parameter ratio compared to standard convolution is

$$\frac{C_{\text{in}}k^2 + C_{\text{out}}C_{\text{in}}}{C_{\text{out}}C_{\text{in}}k^2} = \frac{1}{C_{\text{out}}} + \frac{1}{k^2} \tag{3.30}$$

For typical values $k = 3$, $C_{\text{out}} = 128$, this is approximately $1/128 + 1/9 \approx 0.119$, a roughly $8.4\times$ reduction. MobileNet and Xception architectures are built entirely from depthwise separable convolutions, enabling mobile and embedded deployment.

3.5.3 Dilated (Atrous) Convolution

Dilation inserts gaps between kernel elements, controlled by the dilation rate d:

$$\mathbf{Z}[c, i, j] = \sum_{u,v} \mathbf{W}[c, u, v] \cdot \mathbf{X}[c, i + u \cdot d, j + v \cdot d] \tag{3.31}$$

The **effective kernel size** with dilation d is

$$k_{\text{eff}} = k + (k - 1)(d - 1) \tag{3.32}$$

For $k = 3$ and $d = 2$, $k_{\text{eff}} = 5$, meaning the kernel covers a 5×5 region in the input while using only 9 parameters. Dilated convolutions exponentially expand the receptive field without increasing parameters or losing resolution (unlike pooling/striding). A stack with dilation rates $[1, 2, 4, 8]$ achieves a receptive field of size $1 + (3 - 1)(1 + 2 + 4 + 8) = 31$ in just 4 layers, while maintaining full spatial resolution. This is the foundation of semantic segmentation architectures (DeepLab), where high-resolution outputs are critical.

3.6 Feature Hierarchy

Trained CNNs develop a hierarchical representation that mirrors the visual cortex. Empirical studies (visualizing filters, analyzing activation patterns) reveal a consistent progression:

- **Layer 1:** Gabor-like oriented edge detectors and color blobs. The filters resemble the receptive fields of V1 simple cells, responding to edges at specific orientations and spatial frequencies. Mathematically, these emerge as the network learns to represent the local gradient structure: $\nabla I(x, y) = [\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y}]^T$.

- **Layer 2–3:** Texture detectors combining edges into corners, curves, and simple patterns. Units respond to conjunctions of layer-1 features, forming basis functions for mid-level texture representation.

- **Layer 4–5:** Part detectors — wheels, eyes, handles, beaks. These units achieve some viewpoint and deformation invariance through the composition of lower-level features across spatial positions.

- **Layer 6+:** Object-category detectors — faces, cars, dogs. These neurons are highly selective for specific object classes and invariant to considerable variation in pose, lighting, and background.

This hierarchy is not explicitly programmed — it is an **emergent property** of optimizing a deep convolutional architecture for classification. The inductive biases of local connectivity, weight sharing, and spatial pooling naturally give rise to this compositional structure through stochastic gradient descent.

3.7 Algorithm: Convolutional Layer Forward-Backward Pass

Algorithm 2: Convolutional Layer: Efficient Forward and Backward via im2col

Input : Input $\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$, filters $\mathbf{W} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w}$, bias $\mathbf{b} \in \mathbb{R}^{C_{\text{out}}}$, stride s, padding p, upstream gradient $\boldsymbol{\delta}_{\mathbf{Z}}$

Output: Output $\mathbf{Z}$, $\frac{\partial \ell}{\partial \mathbf{W}}$, $\frac{\partial \ell}{\partial \mathbf{b}}$, $\frac{\partial \ell}{\partial \mathbf{X}}$

`// Forward pass (im2col + GEMM)`

$\mathbf{X}_{\text{pad}} \leftarrow$ pad $\mathbf{X}$ with p zeros on each spatial side;

$\mathbf{X}_{\text{col}} \leftarrow \text{im2col}(\mathbf{X}_{\text{pad}}, k_h, k_w, s)$ `// shape:` $(C_{\texttt{in}} k_h k_w) \times (H_{\texttt{out}} W_{\texttt{out}})$

$\mathbf{W}_{\text{mat}} \leftarrow$ reshape $\mathbf{W}$ to $(C_{\text{out}}) \times (C_{\text{in}} k_h k_w)$;

$\mathbf{Z}_{\text{mat}} \leftarrow \mathbf{W}_{\text{mat}} \mathbf{X}_{\text{col}} + \mathbf{b}[:, \text{None}]$;

$\mathbf{Z} \leftarrow$ reshape $\mathbf{Z}_{\text{mat}}$ to $C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$;

`// Backward pass`

$\boldsymbol{\delta}_{\mathbf{Z}\text{mat}} \leftarrow$ reshape $\boldsymbol{\delta}_{\mathbf{Z}}$ to $C_{\text{out}} \times (H_{\text{out}} W_{\text{out}})$;

$\frac{\partial \ell}{\partial \mathbf{W}_{\text{mat}}} \leftarrow \boldsymbol{\delta}_{\mathbf{Z}\text{mat}} \mathbf{X}_{\text{col}}^T$;

$\frac{\partial \ell}{\partial \mathbf{W}} \leftarrow$ reshape $\frac{\partial \ell}{\partial \mathbf{W}_{\text{mat}}}$ to $C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w$;

$\frac{\partial \ell}{\partial \mathbf{b}} \leftarrow \sum_{ij} \boldsymbol{\delta}_{\mathbf{Z}}[c, i, j]$;

$\frac{\partial \ell}{\partial \mathbf{X}_{\text{col}}} \leftarrow \mathbf{W}_{\text{mat}}^T \boldsymbol{\delta}_{\mathbf{Z}\text{mat}}$;

$\frac{\partial \ell}{\partial \mathbf{X}_{\text{pad}}} \leftarrow \text{col2im}(\frac{\partial \ell}{\partial \mathbf{X}_{\text{col}}}, H, W, k_h, k_w, s, p)$;

$\frac{\partial \ell}{\partial \mathbf{X}} \leftarrow$ remove padding from $\frac{\partial \ell}{\partial \mathbf{X}_{\text{pad}}}$;

return $\mathbf{Z}$, $\frac{\partial \ell}{\partial \mathbf{W}}$, $\frac{\partial \ell}{\partial \mathbf{b}}$, $\frac{\partial \ell}{\partial \mathbf{X}}$

3.8 Pros and Cons

Pros: Convolutional layers are parameter-efficient via weight sharing, achieving $O(C_{\text{in}}C_{\text{out}}k^2)$ parameters independent of spatial dimensions. Translation equivariance is baked into the architecture, eliminating the need to learn shifted copies of features. The hierarchical feature learning mirrors biological vision, leading to highly interpretable intermediate representations. The im2col + GEMM formulation enables highly optimized GPU execution that saturates memory bandwidth. Depthwise separable convolutions and 1×1 bottlenecks enable deployment on resource-constrained devices.

Cons: The fixed receptive field size limits the network's ability to model long-range dependencies (a problem partially addressed by attention mechanisms). Convolutional layers are not equivariant to rotation, scaling, or other non-translational transformations — a rotated cat must be learned as a different pattern, requiring extensive data augmentation. Pooling discards precise spatial information, making CNNs suboptimal for dense prediction tasks requiring pixel-level accuracy (addressed by dilated convolutions, skip connections, and U-Net architectures). The im2col operation duplicates memory for overlapping patches, creating a memory bottleneck for large batches or high-resolution inputs.

3.9 Evolution

The convolutional neural network traces its lineage to Fukushima's Neocognitron (1980) and LeCun's LeNet-5 (1998) for handwritten digit recognition. The modern CNN era began with AlexNet (Krizhevsky et al., 2012), which demonstrated that deep CNNs trained on GPUs with ReLU and dropout could dominate ImageNet. AlexNet's success catalyzed a sequence of architectural innovations, each addressing specific limitations:

VGG (Simonyan & Zisserman, 2014) demonstrated that depth alone, using uniform 3×3 convolutions, could outperform wider, shallower architectures — establishing that a stack of small filters achieves a large receptive field with fewer parameters. However, VGG's 138M parameters were unwieldy.

GoogLeNet/Inception (Szegedy et al., 2014) introduced multi-scale processing (Inception modules with 1×1, 3×3, 5×5 convolutions in parallel) and aggressive 1×1 bottlenecking, achieving 22 layers with only 5M parameters. The auxiliary classifiers addressed vanishing gradients in early layers.

ResNet (He et al., 2015) solved the degradation problem — the counterintuitive observation that deeper networks performed *worse* than shallower ones, even on the training set. Residual connections ($\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$) enabled effective training of 152-layer (and later 1000+ layer) networks by providing gradient highways that circumvent vanishing gradients entirely.

DenseNet (Huang et al., 2016) took skip connections to the extreme: each layer receives the feature maps of all preceding layers, promoting feature reuse and extremely deep supervision. **MobileNet** (Howard et al., 2017) made CNNs practical for mobile devices through depthwise separable convolutions. **EfficientNet** (Tan & Le, 2019) used neural architecture search to discover optimal scaling laws, jointly scaling depth, width, and resolution with a compound coefficient.

The progression reveals a clear pattern: each generation identifies a bottleneck in the previous architecture — parameter count, gradient flow, computational efficiency,

or receptive field coverage — and proposes a structural modification to overcome it. The next frontier merges convolutions with attention (Vision Transformers, ConvNeXt), questioning whether the inductive biases of convolution are still necessary given sufficient data.

Chapter 4

AlexNet

4.1 Motivation

In 2010–2011, the state of the art in image classification on ImageNet (1.2 million training images, 1000 classes) achieved top-5 error rates around 25%–26% using hand-crafted SIFT features, Fisher vectors, and shallow classifiers. The prevailing wisdom held that neural networks, while theoretically appealing, were impractical for large-scale vision: they were too slow to train, prone to overfitting given limited data (by today's standards), and difficult to optimize beyond a few layers. AlexNet (Krizhevsky, Sutskever, and Hinton, 2012) shattered these assumptions, achieving a top-5 error rate of 15.3% — a relative improvement of over 40% — and ushering in the deep learning revolution in computer vision. The paper introduced no single novel algorithm but rather *engineered a system* combining existing components (CNNs, ReLU, dropout, GPU parallelism) at unprecedented scale, demonstrating that depth, data, and compute were sufficient ingredients for breakthrough performance.

4.2 Mathematical Formulation

4.2.1 Overall Architecture

AlexNet consists of 8 parameterized layers: 5 convolutional layers followed by 3 fully-connected layers. The output of the final FC layer feeds a 1000-way softmax. The architecture is summarized as:

Layer	Type	Filters	Kernel	Stride	Pad	Output Size
Input	—	—	—	—	—	$3 \times 227 \times 227$
Conv1	Conv + ReLU	96	11×11	4	0	$96 \times 55 \times 55$
Pool1	MaxPool	—	3×3	2	0	$96 \times 27 \times 27$
Conv2	Conv + ReLU	256	5×5	1	2	$256 \times 27 \times 27$
Pool2	MaxPool	—	3×3	2	0	$256 \times 13 \times 13$
Conv3	Conv + ReLU	384	3×3	1	1	$384 \times 13 \times 13$
Conv4	Conv + ReLU	384	3×3	1	1	$384 \times 13 \times 13$
Conv5	Conv + ReLU	256	3×3	1	1	$256 \times 13 \times 13$
Pool5	MaxPool	—	3×3	2	0	$256 \times 6 \times 6$
FC6	FC + ReLU	—	—	—	—	4096
FC7	FC + ReLU	—	—	—	—	4096
FC8	FC	—	—	—	—	1000
Output	Softmax	—	—	—	—	1000

4.2.2 Convolutional Layer Dimensions

Conv1: Input $3 \times 227 \times 227$. With 96 kernels of size 11×11, stride 4, no padding:

$$H_{\text{out}} = \left\lfloor \frac{227 - 11}{4} \right\rfloor + 1 = \lfloor 54.0 \rfloor + 1 = 55 \tag{4.1}$$

The spatial dimension calculation for each pixel: $(227 - 11)/4 = 54$, floor gives 54, $+1 = 55$. Verify: the kernel covers positions 0–10, 4–14, ..., reaching 216–226, which is 55 positions — correct.

MaxPool1: Kernel 3×3, stride 2. Output: $\lfloor (55 - 3)/2 \rfloor + 1 = \lfloor 52/2 \rfloor + 1 = 27$.

Conv2: Input $96 \times 27 \times 27$ (after pooling, but note AlexNet uses 27×27 due to the original 224×224 input with slightly different arithmetic — the canonical formulation uses 27×27 after Pool1). With 256 kernels of 5×5, stride 1, padding 2:

$$H_{\text{out}} = \left\lfloor \frac{27 + 2 \cdot 2 - 5}{1} \right\rfloor + 1 = \lfloor 26 \rfloor + 1 = 27 \tag{4.2}$$

The "same" padding preserves the spatial size.

Conv3: 384 kernels, 3×3, padding 1, stride 1: output remains 13×13.

Conv4, Conv5: Same as Conv3 (384 and 256 filters respectively), output stays 13×13.

FC6: Input flattened: $256 \times 6 \times 6 = 9216$. Output: 4096. **FC7:** $4096 \to 4096$. **FC8:** $4096 \to 1000$.

4.2.3 GPU Split and Cross-GPU Connections

The entire network is split across two NVIDIA GTX 580 GPUs (each with 3GB memory), with kernels divided evenly between GPUs. Conv1 has 48 filters per GPU; Conv2 has 128 per GPU. Cross-GPU communication occurs at specific layers:

Conv2: Each GPU's Conv2 receives input from the pooled output of *both* GPUs' Conv1 (a cross-GPU connection), connecting all 96 feature maps from Conv1 to each GPU's 128 Conv2 kernels. Mathematically, for GPU $g \in \{0, 1\}$:

$$\mathbf{Z}^{(g)}_{\text{Conv2}}[c_{\text{out}}, i, j] = \sum_{g'=0}^{1} \sum_{c_{\text{in}}=0}^{47} \sum_{u,v} \mathbf{W}^{(g, c_{\text{out}}, c_{\text{in}}^{g'}, u, v)}_{\text{Conv2}} \cdot \mathbf{X}^{(g')}_{\text{Conv1}}[c_{\text{in}}, i + u, j + v] \tag{4.3}$$

where $c_{\text{in}}^{g'}$ indexes the 48 channels produced by GPU g''s Conv1.

Conv4 and Conv5: No cross-GPU connections — each GPU's kernels only see the feature maps on the same GPU from the previous layer.

FC6–8: Fully connected layers connect to *both* GPUs' outputs from the previous layer, meaning all cross-GPU communication is restored at the fully-connected stage:

$$\mathbf{z}^{(g)}_{\text{FC6}} = \mathbf{W}^{(g,:)}_{\text{FC6}} \begin{bmatrix} \mathbf{a}^{(0)}_{\text{Conv5}} \\ \mathbf{a}^{(1)}_{\text{Conv5}} \end{bmatrix} + \mathbf{b}^{(g)}_{\text{FC6}} \tag{4.4}$$

where the concatenated vector has dimension $256 \times 6 \times 6 \times 2 = 18432$ input features going to each GPU's 2048 FC6 neurons (4096 total across 2 GPUs).

This hybrid parallelism scheme (data parallelism for compute-heavy convolutions, model parallelism for memory-heavy FC layers) was a pragmatic solution to GPU memory constraints of the era.

4.3 Activation Function: ReLU

4.3.1 Definition and Derivative

AlexNet popularized the Rectified Linear Unit:

$$\mathrm{ReLU}(x) = \max(0, x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases} \tag{4.5}$$

Its derivative is the step function:

$$\mathrm{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases} \tag{4.6}$$

At $x = 0$, the subgradient is typically set to 0 (though $[0, 1]$ is the full subdifferential; any value works in practice for SGD).

4.3.2 Comparison to Sigmoid and tanh

The sigmoid $\sigma(x) = 1/(1 + e^{-x})$ saturates at both extremes ($\sigma'(x) \to 0$ as $|x| \to \infty$), causing vanishing gradients. Its derivative is bounded by 0.25. The tanh function $\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$ improves matters with a zero-centered output range $(-1, 1)$ and a maximum derivative of 1, but still saturates. ReLU's derivative is 1 for all positive inputs — no saturation in the active region — enabling gradients to flow through many layers without decay. This property is critical: deep sigmoid networks exhibit the vanishing gradient problem, where $\left\|\frac{\partial \ell}{\partial \mathbf{W}^{(1)}}\right\| \propto \prod_{l=2}^{L} \sigma'(z_{\max}) \leq 0.25^{L-1}$, becoming negligible for $L \geq 10$. ReLU eliminates this exponential attenuation for positive pre-activations.

4.3.3 Training Speed

AlexNet with ReLUs reached 25% training error on CIFAR-10 6× faster than an equivalent tanh network. The computational advantage is two-fold: (1) ReLU and its gradient are simple thresholding operations — no exponentials or divisions — making them $O(1)$ with very small constants; (2) ReLU induces sparsity, with approximately 50% of units outputting zero, reducing effective computation and encouraging distributed representations where each example activates a sparse subset of neurons.

4.3.4 Dying ReLU Problem

If a ReLU neuron receives consistently negative inputs, its gradient is always zero and its weights will never update — the neuron "dies." This occurs when: (a) learning rates are too high; (b) a large negative bias pushes the pre-activation permanently negative; or (c) poor initialization causes a large fraction of units to start in the zero-gradient region. AlexNet mitigated this through careful initialization ($\mathcal{N}(0, 0.01)$, small learning rates, and data-driven weight convergence. Later architectures adopt Leaky ReLU ($f(x) = \max(\alpha x, x)$ with $\alpha = 0.01$) or GELU(x) $= x \cdot \Phi(x)$ to eliminate the dying problem entirely.

4.4 Local Response Normalization

After ReLU activation in Conv1 and Conv2, AlexNet applies Local Response Normalization (LRN):

$$b^i_{x,y} = \frac{a^i_{x,y}}{\left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a^j_{x,y})^2\right)^\beta} \tag{4.7}$$

where $a^i_{x,y}$ is the activation of kernel i at spatial position (x, y), N is the total number of kernels in that layer, and the hyperparameters are $k = 2$, $n = 5$, $\alpha = 10^{-4}$, $\beta = 0.75$. The sum runs over n adjacent kernel maps at the same spatial position.

This implements a form of **lateral inhibition** inspired by real neurons: a strongly activated feature map suppresses its neighbors, encouraging competition among features and improving generalization. The effect is modest — LRN reduces top-1 error by about 1.4% — and subsequent architectures (VGG, ResNet) found it unnecessary. Batch Normalization later provided a far more principled channel-wise normalization.

4.4.1 Mathematical Intuition

Consider two feature maps i and j at the same (x, y) with activations a_i, a_j. The LRN output for each is:

$$b_i = \frac{a_i}{(k + \alpha(a^2_{i-2} + a^2_{i-1} + a^2_i + a^2_{i+1} + a^2_{i+2}))^{0.75}} \tag{4.8}$$

If $\|\mathbf{a}\|^2 = \sum a_j^2$ is large in a local neighborhood, all b_i are scaled down, preventing any single filter from dominating. This is a simplified, input-independent approximation of the divisive normalization observed in the visual cortex.

4.5 Overlapping Pooling

Traditional pooling uses non-overlapping windows ($s = k$). AlexNet uses $k = 3$, $s = 2$, creating overlap:

$$\text{Overlap} = k - s = 3 - 2 = 1 \tag{4.9}$$

The overlapping pooling reduces the top-1 error by 0.4% and top-5 error by 0.3% compared to non-overlapping $s = 3, k = 3$ pooling (with the same stride, overlapping means larger effective window). The effect is attributed to the increased difficulty of overfitting: overlapping windows create smoother, more redundant representations where the exact position of a feature matters less. However, overlapping pooling also increases computation with no increase in parameters.

4.6 Data Augmentation

AlexNet employs two forms of data augmentation that are computed on-the-fly on the CPU, so no augmented data is stored on disk.

4.6.1 Image Translations and Horizontal Reflections

During training, 224×224 patches are randomly extracted from 256×256 images, along with their horizontal flips. This generates $(256 - 224 + 1)^2 \times 2 = 1089 \times 2 = 2178$ distinct views per training image by translation and reflection alone. The network sees an exponentially large number of unique training samples over many epochs without increasing dataset size on disk. At test time, the network makes predictions on five 224×224 patches (four corners + center) and their horizontal flips (10 views total), and the softmax probabilities are averaged:

$$p(y = k \mid \mathbf{x}) = \frac{1}{10} \sum_{v=1}^{10} \text{softmax}(\mathbf{f}(\mathbf{x}_v))_k \tag{4.10}$$

This multi-view evaluation reduces top-1 error by about 2%.

4.6.2 PCA Color Augmentation

AlexNet introduces a novel augmentation: altering the intensities of RGB channels using PCA of the ImageNet training pixel values. For each training image, add a multiple of the principal components:

$$[\tilde{R}, \tilde{G}, \tilde{B}]^T = [R, G, B]^T + [\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3][\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^T \tag{4.11}$$

where $\mathbf{p}_i$ are the 3×1 eigenvectors of the 3×3 RGB covariance matrix, λ_i are the corresponding eigenvalues, and $\alpha_i \sim \mathcal{N}(0, 0.1)$ is drawn randomly per image. This captures the fact that object identity is invariant to changes in illumination color and intensity. The PCA-based noise respects the natural color correlations in the dataset, unlike independent per-channel noise. This augmentation reduces top-1 error by over 1%.

4.7 Dropout in AlexNet

Dropout with $p = 0.5$ is applied to the outputs of FC6 and FC7 (the two 4096-dimensional fully-connected layers). For a given training example, each of the 4096 neurons is retained with probability 0.5:

$$\tilde{\mathbf{a}} = \frac{1}{0.5} \cdot \mathbf{m} \odot \mathbf{a} = 2 \cdot \mathbf{m} \odot \mathbf{a}, \quad m_i \sim \text{Bernoulli}(0.5) \tag{4.12}$$

where the factor $1/0.5 = 2$ compensates for the dropped units (inverted dropout). At test time, no dropout is applied — the full network is used.

The FC6 layer has $9216 \times 4096 = 37.7$M parameters and FC7 has $4096 \times 4096 = 16.8$M parameters, together accounting for 54.5M of the network's 62.3M total parameters (87%). Dropout on these layers prevents co-adaptation: no neuron can rely on the presence of specific other neurons, forcing each to learn individually useful features. Without dropout, AlexNet overfits severely on the 1.2M-example ImageNet training set.

4.8 Training Procedure

4.8.1 Optimizer Configuration

AlexNet is trained using **stochastic gradient descent** (mini-batch) with the following hyperparameters:

$$\eta_0 = 0.01 \quad \text{(initial learning rate)} \tag{4.13}$$
$$\beta = 0.9 \quad \text{(momentum coefficient)} \tag{4.14}$$
$$\lambda = 0.0005 \quad \text{(L2 weight decay coefficient)} \tag{4.15}$$
$$B = 128 \quad \text{(batch size)} \tag{4.16}$$

The momentum update at iteration t is:

$$\mathbf{v}_{t+1} = 0.9 \cdot \mathbf{v}_t - 0.0005 \cdot \eta \cdot \boldsymbol{\theta}_t - \eta \cdot \langle \frac{\partial \ell}{\partial \boldsymbol{\theta}} \rangle_{\mathcal{B}_t} \tag{4.17}$$
$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \mathbf{v}_{t+1} \tag{4.18}$$

Note the weight decay term $0.0005 \cdot \eta \cdot \boldsymbol{\theta}_t$ is applied inside the velocity update rather than as a separate additive term to the loss gradient. This distinction matters for adaptive optimizers but is equivalent for plain SGD with momentum:

$$\frac{\partial}{\partial \boldsymbol{\theta}} \left(\ell(\boldsymbol{\theta}) + \frac{0.0005}{2} \left\| \boldsymbol{\theta} \right\|^2 \right) = \nabla_\theta \ell + 0.0005 \cdot \boldsymbol{\theta} \tag{4.19}$$

4.8.2 Weight Initialization

All weights are initialized from a zero-mean Gaussian:

$$W_{ij} \sim \mathcal{N}(0, 0.01) \tag{4.20}$$

Biases in Conv2, Conv4, Conv5, and all FC layers are initialized to 1 (providing positive initial ReLU inputs). All other biases are initialized to 0.

4.8.3 Learning Rate Schedule

The learning rate is manually reduced when validation error plateaus:

$$\eta_t = \begin{cases} 0.01 & t \leq T_1 \\ 0.001 & T_1 < t \leq T_2 \\ 0.00001 & t > T_2 \end{cases} \tag{4.21}$$

where T_1 and T_2 are determined by monitoring validation error. The original training ran for approximately 90 epochs (roughly $90 \times 1.2\text{M}/128 \approx 844,000$ iterations), with the learning rate divided by 10 at validation error plateaus. This heuristic schedule requires human monitoring but works well when training duration is measured in days rather than hours.

4.8.4 Training Duration and Hardware

Training took 5–6 days on two NVIDIA GTX 580 GPUs. Each GPU had 3GB of memory, severely constraining the model: the 1.2M training images required the split-GPU architecture. The forward pass processes 128 images in parallel (batch size 128), with each GPU handling 64. Backpropagation computes gradients, which are then averaged across GPUs.

4.9 Parameter Count and FLOPs Analysis

4.9.1 Learnable Parameters

$$\text{Conv1 params} = (11 \times 11 \times 3) \times 96 + 96 = 34,944 \tag{4.22}$$

$$\text{Conv2 params} = (5 \times 5 \times 48) \times 256 + 256 = 307,456 \quad \text{(per GPU: 128 filters see 48 channels)} \tag{4.23}$$

$$\text{Conv3 params} = (3 \times 3 \times 256) \times 384 + 384 = 885,120 \tag{4.24}$$

$$\text{Conv4 params} = (3 \times 3 \times 192) \times 384 + 384 = 663,936 \quad \text{(per GPU: 192 channels)} \tag{4.25}$$

$$\text{Conv5 params} = (3 \times 3 \times 192) \times 256 + 256 = 442,624 \tag{4.26}$$

$$\text{FC6 params} = 9216 \times 4096 + 4096 = 37,752,832 \tag{4.27}$$

$$\text{FC7 params} = 4096 \times 4096 + 4096 = 16,781,312 \tag{4.28}$$

$$\text{FC8 params} = 4096 \times 1000 + 1000 = 4,097,000 \tag{4.29}$$

Total: $\approx$ 60.96M parameters (the commonly cited 60M; the exact count depends on the cross-GPU connectivity accounting). The FC layers account for 58.6M (96%) of all parameters, demonstrating that the convolutional layers achieve massive spatial processing with relatively few parameters.

4.9.2 FLOPs for Convolutional Layers

Using the formula $\text{FLOPs} = 2 \cdot C_{\text{out}} \cdot C_{\text{in}} \cdot k_h \cdot k_w \cdot H_{\text{out}} \cdot W_{\text{out}}$ (assuming per-GPU values need to be doubled for total network):

$$\text{FLOPs}_{\text{Conv1}} = 2 \times 96 \times 3 \times 11^2 \times 55^2 \approx 2 \times 96 \times 3 \times 121 \times 3025 \approx 2.11 \times 10^8 \tag{4.30}$$

$$\text{FLOPs}_{\text{Conv2}} = 2 \times 256 \times 96 \times 5^2 \times 27^2 \approx 2 \times 256 \times 96 \times 25 \times 729 \approx 8.96 \times 10^8 \tag{4.31}$$

$$\text{FLOPs}_{\text{Conv3}} = 2 \times 384 \times 256 \times 3^2 \times 13^2 \approx 2 \times 384 \times 256 \times 9 \times 169 \approx 2.99 \times 10^8 \tag{4.32}$$

$$\text{FLOPs}_{\text{Conv4}} = 2 \times 384 \times 384 \times 3^2 \times 13^2 \approx 2 \times 384 \times 384 \times 9 \times 169 \approx 4.48 \times 10^8 \tag{4.33}$$

$$\text{FLOPs}_{\text{Conv5}} = 2 \times 256 \times 384 \times 3^2 \times 13^2 \approx 2 \times 256 \times 384 \times 9 \times 169 \approx 2.99 \times 10^8 \tag{4.34}$$

Total convolutional FLOPs $\approx 2.15 \times 10^9$ (2.15 GFLOPs). For FC layers:

$$\text{FLOPs}_{\text{FC6}} = 2 \times 9216 \times 4096 \approx 7.55 \times 10^7 \tag{4.35}$$

$$\text{FLOPs}_{\text{FC7}} = 2 \times 4096 \times 4096 \approx 3.36 \times 10^7 \tag{4.36}$$

$$\text{FLOPs}_{\text{FC8}} = 2 \times 4096 \times 1000 \approx 8.19 \times 10^6 \tag{4.37}$$

Total FC FLOPs $\approx 1.17 \times 10^8$. The convolutional layers dominate compute (95% of FLOPs) despite having only 4% of parameters. This asymmetry — FC layers are parameter-heavy but compute-light, conv layers are compute-heavy but parameter-light — is a defining characteristic of CNN architectures.

4.10 Why AlexNet Worked

4.10.1 Depth

With 8 parameterized layers, AlexNet was significantly deeper than previous CNNs (LeNet-5 had 5 layers; other contemporary models were typically < 5 layers). The depth hierarchy enabled learning of complex, compositional features from raw pixels. The receptive field at the final conv layer covers the majority of the input image, enabling detection of objects at multiple scales and positions.

4.10.2 ReLU Activation

The non-saturating activation accelerated training by 6× compared to tanh, making 90-epoch training feasible. More importantly, it mitigated the vanishing gradient problem that had prevented effective training of deep networks, enabling gradients to propagate from the softmax loss back through all 8 layers.

4.10.3 GPU Implementation

The dual-GPU implementation with careful communication patterns demonstrated that GPU parallelism could train deep networks on datasets previously considered too large. The 3GB GPU memory limit motivated the split architecture, which although pragmatic, inadvertently introduced architectural innovations (cross-GPU connectivity patterns) that presaged later multi-branch architectures.

4.10.4 Data Augmentation

Aggressive on-the-fly data augmentation (translations, flips, PCA color jitter) effectively increased the training set size by orders of magnitude, preventing the heavily-parameterized FC layers from overfitting despite the relatively small (1.2M images) training set. The augmentation encodes invariance to common image transformations directly into the training signal.

4.10.5 Dropout

Applying dropout to the two largest parameter layers (37.7M + 16.8M parameters) provided essential regularization for the FC layers, which otherwise memorized training examples. The 0.5 dropout rate meant that, in expectation, each training example was processed by 2^{4096} different FC subnetworks, approximating an ensemble of exponential size.

4.11 Algorithm: AlexNet Forward Pass

Algorithm 3: AlexNet: Forward Pass

Input : RGB image $\mathbf{X} \in \mathbb{R}^{3\times227\times227}$, trained parameters $\boldsymbol{\theta}$
Output: Class probabilities $\mathbf{p} \in \mathbb{R}^{1000}$

`// Convolutional layers`

$\mathbf{a}^{(0)} \leftarrow \mathbf{X}$ `// Input`
$\mathbf{z}^{(1)} \leftarrow \text{Conv2D}(\mathbf{a}^{(0)}; \mathbf{W}^{(1)}, \mathbf{b}^{(1)}, k = 11, s = 4, p = 0, C_{\text{out}} = 96)$;
$\mathbf{a}^{(1)} \leftarrow \text{ReLU}(\mathbf{z}^{(1)})$;
$\mathbf{a}^{(1)} \leftarrow \text{MaxPool2D}(\mathbf{a}^{(1)}; k = 3, s = 2)$;
$\mathbf{a}^{(1)} \leftarrow \text{LRN}(\mathbf{a}^{(1)}; n = 5, \alpha = 10^{-4}, \beta = 0.75, k = 2)$;
;
$\mathbf{z}^{(2)} \leftarrow \text{Conv2D}(\mathbf{a}^{(1)}; \mathbf{W}^{(2)}, \mathbf{b}^{(2)}, k = 5, s = 1, p = 2, C_{\text{out}} = 256)$;
$\mathbf{a}^{(2)} \leftarrow \text{ReLU}(\mathbf{z}^{(2)})$;
$\mathbf{a}^{(2)} \leftarrow \text{MaxPool2D}(\mathbf{a}^{(2)}; k = 3, s = 2)$;
$\mathbf{a}^{(2)} \leftarrow \text{LRN}(\mathbf{a}^{(2)}; n = 5, \alpha = 10^{-4}, \beta = 0.75, k = 2)$;
;
$\mathbf{z}^{(3)} \leftarrow \text{Conv2D}(\mathbf{a}^{(2)}; \mathbf{W}^{(3)}, \mathbf{b}^{(3)}, k = 3, s = 1, p = 1, C_{\text{out}} = 384)$;
$\mathbf{a}^{(3)} \leftarrow \text{ReLU}(\mathbf{z}^{(3)})$;
;
$\mathbf{z}^{(4)} \leftarrow \text{Conv2D}(\mathbf{a}^{(3)}; \mathbf{W}^{(4)}, \mathbf{b}^{(4)}, k = 3, s = 1, p = 1, C_{\text{out}} = 384)$;
$\mathbf{a}^{(4)} \leftarrow \text{ReLU}(\mathbf{z}^{(4)})$;
;
$\mathbf{z}^{(5)} \leftarrow \text{Conv2D}(\mathbf{a}^{(4)}; \mathbf{W}^{(5)}, \mathbf{b}^{(5)}, k = 3, s = 1, p = 1, C_{\text{out}} = 256)$;
$\mathbf{a}^{(5)} \leftarrow \text{ReLU}(\mathbf{z}^{(5)})$;
$\mathbf{a}^{(5)} \leftarrow \text{MaxPool2D}(\mathbf{a}^{(5)}; k = 3, s = 2)$;
;

`// Fully connected layers`

$\mathbf{a}_{\text{flat}} \leftarrow \text{Flatten}(\mathbf{a}^{(5)})$ `// Size:` $256 \times 6 \times 6 = 9216$
$\mathbf{z}^{(6)} \leftarrow \mathbf{W}^{(6)}\mathbf{a}_{\text{flat}} + \mathbf{b}^{(6)}$ `// 9216 → 4096`
$\mathbf{a}^{(6)} \leftarrow \text{ReLU}(\mathbf{z}^{(6)})$;
if *training* **then**
  $\mathbf{a}^{(6)} \leftarrow \text{Dropout}(\mathbf{a}^{(6)}; p = 0.5)$ `// Scaled by` $1/p = 2$
else
  ;
$\mathbf{z}^{(7)} \leftarrow \mathbf{W}^{(7)}\mathbf{a}^{(6)} + \mathbf{b}^{(7)}$ `// 4096 → 4096`
$\mathbf{a}^{(7)} \leftarrow \text{ReLU}(\mathbf{z}^{(7)})$;
if *training* **then**
  $\mathbf{a}^{(7)} \leftarrow \text{Dropout}(\mathbf{a}^{(7)}; p = 0.5)$
else
  ;
$\mathbf{z}^{(8)} \leftarrow \mathbf{W}^{(8)}\mathbf{a}^{(7)} + \mathbf{b}^{(8)}$ `// 4096 → 1000`
$\mathbf{p} \leftarrow \text{softmax}(\mathbf{z}^{(8)})$;
;
return $\mathbf{p}$

4.12 Pros and Cons

Pros: AlexNet provided the first empirical proof that deep CNNs could achieve state-of-the-art results on a large-scale natural image dataset. The careful system engineering — combining GPU parallelism, ReLU, dropout, overlapping pooling, and aggressive augmentation — established a template for deep learning experimentation that persists today. The architecture is conceptually simple (feedforward, no skip connections, no branching), making it an excellent pedagogical entry point. The 15.3% top-5 error represented a qualitative threshold: deep learning had definitively surpassed shallow computer vision methods.

Cons: The 60M parameter count is dominated by FC layers (94% of parameters), making the network large and slow. The 11×11 Conv1 filters are computationally expensive and capture mostly low-level features that could be learned by stacks of smaller filters. LRN adds complexity with marginal benefit. The dual-GPU split is an artifact of hardware constraints, not a principled design choice, and complicates reproduction. The manual learning rate schedule is ad hoc and requires monitoring. At 5–6 days of training on expensive hardware, the network was inaccessible to most researchers.

4.13 Evolution

AlexNet's 2012 ImageNet victory ignited an explosion of CNN architectures, each addressing specific weaknesses:

ZFNet (Zeiler & Fergus, 2013) refined AlexNet's hyperparameters: reducing Conv1 filters from 11×11 to 7×7 with stride 2 instead of 4, preserving more spatial information. More importantly, ZFNet introduced novel visualization techniques (deconvolutional networks) that made CNN features interpretable, showing that layer 1 learns Gabor filters, layer 2 learns corners and patterns, and deeper layers detect object parts and whole objects. This visualization work was crucial for building intuition and trust in deep networks.

VGG (Simonyan & Zisserman, 2014) addressed the filter size issue systematically: by replacing large filters (11×11, 5×5) with stacks of 3×3 filters, VGG demonstrated that deeper, narrower networks with uniform filter sizes outperform AlexNet. A stack of two 3×3 convolutions has the same 5×5 receptive field as one 5×5 convolution but with $2 \times 9 = 18$ effective parameters vs. 25, plus an extra nonlinearity. VGG-19 achieved 7.3% top-5 error with 144M parameters. However, VGG's massive FC layers (123M parameters) made it the last of the "heavy FC" architectures.

GoogLeNet/Inception (Szegedy et al., 2014) tackled AlexNet's parameter inefficiency. By replacing FC layers with global average pooling and using bottleneck 1×1 convolutions throughout, GoogLeNet achieved only 5M parameters (12× fewer than AlexNet) while being 22 layers deep (nearly 3× deeper). The Inception module processed features at multiple scales simultaneously, addressing AlexNet's fixed receptive field limitation. GoogLeNet won ILSVRC 2014 with 6.7% top-5 error.

ResNet (He et al., 2015) made an even deeper architectural leap: at 152 layers (vs. AlexNet's 8), residual connections enabled training by providing identity shortcuts that alleviated the degradation problem AlexNet's depth could not reach. ResNet achieved 3.57% top-5 error, surpassing human-level performance (5.1%). The key insight — that the optimization difficulty, not the representational capacity, limited depth — fundamen-

tally changed how networks are designed.

Batch Normalization (Ioffe & Szegedy, 2015) replaced AlexNet's LRN and made training of very deep networks robust. By normalizing layer inputs, BatchNorm allowed 10–100× higher learning rates and reduced the sensitivity to initialization, which had been critical pain points in AlexNet training.

Modern descendents: The AlexNet lineage continues through ResNeXt (grouped convolutions), DenseNet (dense skip connections), SENet (channel attention), MobileNet (depthwise separable convs), EfficientNet (compound scaling), and ConvNeXt (modernized ResNet with transformer-inspired design choices). Each step either improves accuracy efficiency or both, but all trace their conceptual roots to AlexNet's demonstration that deep CNNs, properly trained, can learn hierarchical visual representations from pixels to objects.

The most remarkable aspect of AlexNet's legacy is that its *architectural* innovations (deep ReLU CNNs trained with dropout and massive augmentation) proved more enduring than its specific *design choices* (large initial filters, LRN, GPU splitting, heavy FC layers). The paper's true contribution was methodological: it established the experimental template — train on ImageNet, report top-1 and top-5 error, compare against both shallow baselines and human performance — that guided computer vision research for the following decade.

Chapter 5

ResNet — Deep Residual Networks

5.1 Motivation

The VGG and GoogLeNet architectures demonstrated that increasing network depth improves representational capacity and accuracy. However, experiments with plain (non-residual) convolutional networks revealed a counterintuitive phenomenon: a 20-layer network outperformed a 56-layer network on both training and test error. This is the **degradation problem** — not caused by overfitting (training error itself degrades), but by optimization difficulty in very deep networks.

Consider a plain stack of layers. Each layer applies a transformation $\mathcal{F}_l$:

$$x_l = \mathcal{F}_l(x_{l-1}) = \sigma(W_l x_{l-1} + b_l). \tag{5.1}$$

The gradient of the loss $\mathcal{L}$ with respect to the input at layer 1 is:

$$\frac{\partial \mathcal{L}}{\partial x_1} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=2}^{L} \frac{\partial x_l}{\partial x_{l-1}} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=2}^{L} \operatorname{diag}(\sigma'(a_l)) W_l. \tag{5.2}$$

If the spectral norms $\|W_l\| < 1$, the product vanishes exponentially in L. If $\|W_l\| > 1$, it explodes. Even with careful initialization and BatchNorm, very deep plain networks struggle because each layer must learn a useful transformation — there is no "default" path for the gradient.

5.2 Mathematical Formulation

5.2.1 Residual Learning Hypothesis

Instead of learning an underlying mapping $\mathcal{H}(x)$, let each stacked layer learn a **residual** mapping:

$$\mathcal{F}(x) \coloneqq \mathcal{H}(x) - x. \tag{5.3}$$

The forward pass becomes:

$$y = \mathcal{F}(x, \{W_i\}) + x. \tag{5.4}$$

The hypothesis: if identity mappings are optimal, driving weights to zero is easier than learning identity directly. When $W \to 0$, $\mathcal{F}(x) \to 0$ and the block defaults to identity. This makes adding layers never harmful: at worst, the residual learns zero and the layer is transparent.

5.2.2 Basic Residual Block

A two-layer residual block is defined as:

$$\mathcal{F}(x) = W_2\, \sigma(W_1 x + b_1) + b_2, \tag{5.5}$$

$$y = \sigma(\mathcal{F}(x) + x). \tag{5.6}$$

Here σ denotes ReLU. The dimensions of x and $\mathcal{F}(x)$ must match for the element-wise addition.

When dimensions change (e.g., after strided convolutions or channel expansion), a linear projection W_s aligns dimensions:

$$y = \sigma(\mathcal{F}(x, \{W_i\}) + W_s x). \tag{5.7}$$

The projection W_s is typically a 1×1 convolution.

5.2.3 Bottleneck Block

For very deep networks (ResNet-50 and beyond), a bottleneck design reduces parameters:

$$\text{Bottleneck}(x) = W_3\,\sigma(W_2\,\sigma(W_1 x)) + x, \tag{5.8}$$

where $W_1 \in \mathbb{R}^{d\times 256}$ (reduce), $W_2 \in \mathbb{R}^{d\times d}$ (3×3 conv), $W_3 \in \mathbb{R}^{256\times d}$ (expand), with $d = 64$.

Parameter count comparison. Two 3×3 convolutions with 256 input and output channels:

$$\text{Params}_{3\times3} = 2 \times (3 \times 3 \times 256 \times 256) = 1,179,648. \tag{5.9}$$

Bottleneck with $d = 64$:

$$\text{Params}_{\text{BN}} = (1{\times}1{\times}256{\times}64){+}(3{\times}3{\times}64{\times}64){+}(1{\times}1{\times}64{\times}256) = 16,384{+}36,864{+}16,384 = 69,632. \tag{5.10}$$

The bottleneck uses $\approx 17\times$ fewer parameters while maintaining representation capacity.

5.2.4 Backpropagation Through Residual Connections

Let x be the input to a residual block and $y = \mathcal{F}(x) + x$ its output. The gradient is:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \cdot \left(\frac{\partial \mathcal{F}}{\partial x} + I\right) = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial \mathcal{F}}{\partial x} + \frac{\partial \mathcal{L}}{\partial y}. \tag{5.11}$$

The additive identity term $\frac{\partial \mathcal{L}}{\partial y}$ provides a **direct gradient highway** that is never multiplicatively attenuated by weight matrices.

5.2.5 Full Gradient Flow Analysis

For a plain network of L layers:

$$\frac{\partial \mathcal{L}}{\partial x_1} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=2}^{L} \frac{\partial x_l}{\partial x_{l-1}}. \tag{5.12}$$

The product of Jacobians causes exponential vanishing or explosion.

For a residual network defined recursively as $x_{l+1} = x_l + \mathcal{F}_l(x_l, W_l)$:

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_{l+1}} \left(I + \frac{\partial \mathcal{F}_l}{\partial x_l}\right) = \frac{\partial \mathcal{L}}{\partial x_{l+1}} + \frac{\partial \mathcal{L}}{\partial x_{l+1}} \frac{\partial \mathcal{F}_l}{\partial x_l}. \tag{5.13}$$

Unrolling from layer L back to layer 1:

$$\frac{\partial \mathcal{L}}{\partial x_1} = \frac{\partial \mathcal{L}}{\partial x_L} \left(I + \sum_{l=1}^{L-1} \prod_{k=1}^{l} \frac{\partial \mathcal{F}_{L-k}}{\partial x_{L-k-1}}\right). \tag{5.14}$$

The term $\frac{\partial \mathcal{L}}{\partial x_L}$ always appears additively, guaranteeing that the gradient signal from layer L reaches every earlier layer directly. The residual gradient never vanishes completely because there is always at least this direct additive path.

5.2.6 Pre-activation vs. Post-activation

The original ResNet places BatchNorm and ReLU **after** the weight layers (post-activation). The pre-activation variant applies them **before**:

$$\text{Post:} \quad y = \sigma(\text{BN}(W_2\,\sigma(\text{BN}(W_1 x))) + x), \tag{5.15}$$

$$\text{Pre:} \quad y = W_2\,\sigma(\text{BN}(W_1\,\sigma(\text{BN}(x)))) + x. \tag{5.16}$$

Pre-activation makes the entire path from input to output a pure identity (no activation on the skip connection), further improving gradient flow and enabling training of 1001-layer networks.

5.3 Algorithm

Input : Input image $\mathbf{X}$, ground truth y
Output: Trained ResNet parameters $\{W_l\}$
Initialize weights W_l with He initialization;
Normalize input to $[0, 1]$ with mean subtraction;
for *epoch* $= 1$ **to** *max_ epochs* **do**
    for *each mini-batch of size 256* **do**
        Forward pass through residual blocks;
        Compute softmax cross-entropy loss $\mathcal{L}$;
        Backpropagate gradients through residual connections:
        $\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L} + \frac{\partial \mathcal{L}}{\partial x_L} \sum \cdots$;
        Update weights via SGD with momentum $\mu = 0.9$;
        Apply weight decay $\lambda = 10^{-4}$;
    if *iteration* $= 30{,}000$ **then**
        $\eta \leftarrow \eta/10$;
    if *iteration* $= 60{,}000$ **then**
        $\eta \leftarrow \eta/10$;
Output: Return trained model

5.4 Architecture

Layer	Output	ResNet-18	ResNet-34	ResNet-50	ResNet-101
conv1	112^2	$7 \times 7, 64$, stride 2			
conv2_x	56^2	3×3 max pool, stride 2			
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix}$
conv3_x	28^2	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix}$
conv4_x	14^2	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix}$
conv5_x	7^2	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix}$
	1^2	global average pool, 1000-d FC, softmax			

Table 5.1: ResNet architectures. Brackets denote residual block structure; $\times N$ indicates number of stacked blocks. The first bottleneck of each stage uses stride 2 and a 1×1 projection shortcut.

Training hyperparameters: SGD with momentum 0.9, weight decay 10^{-4}, mini-batch 256, initial learning rate 0.1 divided by 10 at iterations 30,000 and 60,000. BatchNorm after each convolution, before ReLU. He (MSRA) initialization for all weights.

5.5 Pros and Cons

5.5.1 Advantages

- Solves degradation: adding layers never hurts; identity is the default.
- Gradient highway eliminates vanishing gradient: $\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L} + \cdots$.
- Enables training of extremely deep networks (152, 1001+ layers).
- Bottleneck design is parameter-efficient.
- Strong generalization; won ILSVRC 2015 (3.57% top-5 error, ensemble).
- Modular: residual blocks are plug-and-play in many architectures.

5.5.2 Disadvantages

- Many layers $\rightarrow$ high memory and FLOPs during training.
- Skip connections require matching dimensions, adding design complexity.

- Post-activation still has some gradient attenuation from ReLUs on the residual path.
- Simple identity skip is suboptimal for very fine-grained feature reuse (cf. DenseNet).
- Does not address cardinality (width of grouped convolutions) — see ResNeXt.

5.6 Evolution

What ResNet solved: The degradation problem in plain networks. By reformulating learning as residual learning, depth up to 152+ layers became trainable with monotonically improving accuracy. The gradient highway eliminated vanishing gradient as a bottleneck.

What ResNet still couldn't solve: The residual block uses only additive skip connections, which implicitly assumes features are combined by summation. This limits feature reuse: later layers cannot easily access earlier features except through the accumulated sum. The constant-width design (each block has fixed channel count internally) limits representational diversity.

What came next:

- **Pre-activation ResNet** (He et al., 2016): BN $\rightarrow$ ReLU $\rightarrow$ Conv order creates a fully identity skip path, enabling 1001-layer networks with improved accuracy. The gradient flows through pure identity from loss to input.
- **ResNeXt** (Xie et al., 2017): Introduces *cardinality* — split the residual path into C parallel branches (grouped convolutions): $\mathcal{F}(x) = \sum_{i=1}^{C} \mathcal{T}_i(x)$. Same FLOPs as ResNet but higher accuracy.
- **DenseNet** (Huang et al., 2017): Replaces additive skip with concatenation: $x_l = [x_0, x_1, \ldots, x_{l-1}]$. Every layer has direct access to all previous features, improving gradient flow and feature reuse at the cost of higher memory.
- **Wide ResNet** (Zagoruyko & Komodakis, 2016): Shows that increasing width (channels) is more effective than increasing depth beyond a certain point, with better GPU utilization.

Chapter 6

FCN — Fully Convolutional Networks

6.1 Motivation

Standard CNN classifiers (AlexNet, VGG, GoogLeNet, ResNet) process an input image through convolutional and pooling layers, then flatten the spatial feature map into a fixed-length vector fed to fully connected layers. The output is a single class label or a distribution over K categories. For semantic segmentation — classifying every pixel into a semantic category — we need a dense spatial output map.

Before FCNs, the dominant approach was **patch-based classification**: extract a patch around each pixel, classify it independently, and stitch results. Given an image with $H \times W$ pixels, this requires $O(HW)$ forward passes. The receptive field is limited to the patch size, discarding global context. The computational cost is prohibitive.

The key insight of FCNs: a fully connected layer on a spatial feature map is mathematically equivalent to a 1×1 convolution.

6.2 Mathematical Formulation

6.2.1 Convolutionalization of Fully Connected Layers

Consider the final feature map of VGG-16 before FC layers: a tensor of shape $7 \times 7 \times 512$ (width $\times$ height $\times$ channels). A fully connected layer with 4096 units computes:

$$y_j = \sum_{i=1}^{7\times7\times512} W_{ji}\, x_i + b_j, \quad j = 1, \ldots, 4096. \tag{6.1}$$

Flattening the $7 \times 7 \times 512$ tensor into a vector of length 25,088, the weight matrix $W \in \mathbb{R}^{4096\times25088}$.

A 1×1 convolution with 4096 filters, each of shape $1 \times 1 \times 512$, applied to the same tensor:

$$y_{j,h,w} = \sum_{c=1}^{512} w_{j,c}\, x_{c,h,w} + b_j. \tag{6.2}$$

This is identical to the FC computation element-wise at each spatial position (h, w). The output is $1 \times 1 \times 4096$ for a 7×7 input, or $H' \times W' \times 4096$ for any spatial input size.

The FC layers are **replaced by**:

- Conv $1 \times 1, 4096$ (replaces FC-4096)
- Conv $1 \times 1, 4096$ (replaces FC-4096)
- Conv $1 \times 1, C$ (replaces FC-1000 / class prediction layer)

where C is the number of semantic classes. The network becomes **fully convolutional**: it can accept arbitrary-sized inputs and produces a spatial output map.

6.2.2 Transposed Convolution for Upsampling

The output of the convolutionalized network is a coarse score map with spatial resolution reduced by the total stride (e.g., 32× smaller for VGG-16). To produce pixel-wise predictions, we must upsample this map. The learned upsampling uses **transposed convolution** (also called deconvolution or fractionally-strided convolution).

Forward pass interpretation. A regular convolution computes $y = x * w$ with stride s. The transposed convolution takes a small input map and produces a larger output. For a $s\times$ upsampling, it uses fractional stride $1/s$, implemented as:

1. Insert $s-1$ zeros between every pair of adjacent input elements (in both height and width).
2. Pad the expanded input appropriately.
3. Apply a standard convolution.

Output size formula. Given input size $H_{\text{in}} \times W_{\text{in}}$, kernel size k, stride s, padding p:

$$H_{\text{out}} = s(H_{\text{in}} - 1) + k - 2p, \qquad W_{\text{out}} = s(W_{\text{in}} - 1) + k - 2p. \tag{6.3}$$

Backward pass. The gradient of a transposed convolution is a regular convolution. Let $\mathcal{L}$ be the loss. The gradient with respect to the input x of a transposed convolution with kernel w and stride s is:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} *_{\text{conv},s} w', \tag{6.4}$$

where w' is the kernel rotated by 180° and $*_{\text{conv},s}$ is a standard convolution with stride s. Conversely, the gradient w.r.t. the kernel weights uses the zero-inserted input convolved with the output gradient.

Matrix representation. Let the convolution operate by a sparse matrix C such that $\vec{y} = C\vec{x}$. The transposed convolution computes $\vec{y}_{\text{up}} = C^T \vec{x}_{\text{coarse}}$, giving the name "transposed."

6.2.3 Skip Connections for Multi-scale Fusion

The coarse score map from the final layer has rich semantics but imprecise spatial localization. Earlier layers have finer spatial resolution but weaker semantics. FCN fuses them via additive skip connections at three granularities:

FCN-32s

Upsample the pool5 score map (stride 32) directly to input resolution with a learned transposed convolution (kernel size 64, stride 32):

$$\hat{y}^{32\text{s}}_{i,j} = \text{softmax}\left(\text{transposed_conv}_{32\times}(S_{\text{pool5}})\right)_{i,j}. \tag{6.5}$$

The output is coarse with blurred boundaries.

FCN-16s

Fuse pool4 with an upsampled pool5 prediction:

$$S_{\text{pool5}}^{(2\times)} = \text{transposed_conv}_{2\times}(S_{\text{pool5}}), \tag{6.6}$$

$$S_{\text{fused}} = S_{\text{pool5}}^{(2\times)} + \text{conv}_{1\times1}(S_{\text{pool4}}), \tag{6.7}$$

$$\hat{y}^{16\text{s}} = \text{softmax}\left(\text{transposed_conv}_{16\times}(S_{\text{fused}})\right). \tag{6.8}$$

The 1×1 conv on pool4 produces class predictions at that resolution. Adding pool5 (upsampled $2\times$) brings semantic guidance to finer spatial features.

FCN-8s

Further fuse pool3:

$$S_{\text{fused16}}^{(2\times)} = \text{transposed_conv}_{2\times}(S_{\text{fused}}), \tag{6.9}$$

$$S_{\text{fused8}} = S_{\text{fused16}}^{(2\times)} + \text{conv}_{1\times1}(S_{\text{pool3}}), \tag{6.10}$$

$$\hat{y}^{8\text{s}} = \text{softmax}\left(\text{transposed_conv}_{8\times}(S_{\text{fused8}})\right). \tag{6.11}$$

Each fusion step refines boundaries with higher-resolution features.

6.2.4 Loss Function

Per-pixel softmax cross-entropy, averaged over all labeled pixels:

$$\mathcal{L} = -\frac{1}{N_{\text{labeled}}} \sum_{i=1}^{N} \sum_{c=1}^{C} \mathbf{1}[y_i = c] \cdot \log \frac{\exp(s_{i,c})}{\sum_{k=1}^{C} \exp(s_{i,k})}, \tag{6.12}$$

where $s_{i,c}$ is the score of pixel i for class c, and N_{labeled} excludes ignore/unlabeled pixels (e.g., void class in PASCAL VOC).

6.2.5 Evaluation Metrics

Pixel accuracy:

$$\text{PA} = \frac{\sum_i \text{TP}_i}{\sum_i (\text{TP}_i + \text{FP}_i)}. \tag{6.13}$$

Intersection over Union (IoU) per class:

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c} = \frac{|P_c \cap G_c|}{|P_c \cup G_c|}, \tag{6.14}$$

where P_c is the set of pixels predicted as class c, G_c is the ground truth set for class c. **Mean IoU (mIoU)** averages over all classes: $\text{mIoU} = \frac{1}{C} \sum_{c=1}^{C} \text{IoU}_c$.

6.3 Algorithm

Input : Image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$, pretrained VGG-16 weights
Output: Pixel-wise class predictions $\hat{\mathbf{Y}} \in \{1, \ldots, C\}^{H \times W}$

Initialize VGG-16 backbone with ImageNet pretrained weights;
Replace FC layers with 1×1 convolutions (conv6: 4096, conv7: 4096, conv8: C);
Attach scoring layers: conv $1 \times 1 \to C$ on pool4 and pool3;
for *epoch* $= 1$ **to** *max_ epochs* **do**
  for *each mini-batch of images with ground truth masks* **do**
    Forward:;
    Extract features through conv1–conv5, pool3, pool4, pool5;
    Compute pool5 score map $S_{\text{pool5}} = \text{conv8}(\text{conv7}(\text{conv6}(\text{pool5})))$;
    Compute FCN-32s: $\hat{Y}_{32s} = \text{transposed_conv}_{32\times}(S_{\text{pool5}})$;
    Compute FCN-16s: $S_{\text{fused}} = \text{transposed_conv}_{2\times}(S_{\text{pool5}}) + \text{conv}_{1\times1}(S_{\text{pool4}})$;
    Compute FCN-8s: $S_{\text{fused8}} = \text{transposed_conv}_{2\times}(S_{\text{fused}}) + \text{conv}_{1\times1}(S_{\text{pool3}})$;
    $\hat{Y}_{\text{final}} = \text{transposed_conv}_{8\times}(S_{\text{fused8}})$;
    Compute per-pixel softmax cross-entropy loss $\mathcal{L}$;
    Backward: Compute $\frac{\partial \mathcal{L}}{\partial \hat{Y}}$ and backprop through transposed convs;
    Update weights via SGD with momentum 0.9;
    if *using FCN-32s only* **then**
      Fine-tune all FC-replacement + upsampling layers;
    else
      Stage-wise: first train FCN-32s, then add pool4 fusion and fine-tune, then add pool3;

Output: Trained fully convolutional network with skip fusion

6.4 Architecture

The FCN architecture consists of three components:

1. **Encoder (convolutionalized backbone):** VGG-16 conv1 through conv5, producing feature maps at strides 2, 4, 8, 16, 32. Key intermediate tensors:
   pool3 : $H/8 \times W/8 \times 256$, pool4 : $H/16 \times W/16 \times 512$, pool5 : $H/32 \times W/32 \times 512$.

2. **Prediction heads (1×1 convs):** Each of pool3, pool4, pool5 passes through a 1×1 convolution producing C-dimensional score maps. The fully connected replacement on pool5 is: Conv 7×7→1×1 (4096 filters), then another 1×1 (4096 filters), then 1×1 (C filters). Because pool5 is 7×7 at training resolution, the 7×7 conv degenerates to 1×1, but at test time with larger inputs it generalizes.

3. **Skip fusion and upsampling:** As described in FCN-32s/16s/8s, progressively fuse and upsample using transposed convolutions with bilinear interpolation initialization for the upsampling kernels.

6.5 Pros and Cons

6.5.1 Advantages

- Dense pixel prediction in a single forward pass; $O(1)$ passes vs. $O(HW)$ for patch-based methods.
- Arbitrary input sizes due to full convolutionality.
- Skip connections (FCN-8s) effectively combine coarse semantics and fine appearance.
- Leverages ImageNet pretraining via convolutionalization of classification networks.
- End-to-end trainable; no separate proposal/refinement stages.
- Transposed convolution is learned, adapting upsampling to the task.

6.5.2 Disadvantages

- Output resolution is limited by the encoder stride (8× downsampled for FCN-8s), resulting in blurred boundaries even after skip fusion.
- Additive skip fusion is a weak mechanism for spatial recovery compared to learned decoder architectures.
- Global context is limited to the receptive field of the encoder (no explicit global pooling or attention).
- The transposed convolution upsampling of 8× or 16× in one step produces checkerboard artifacts.
- No mechanism for instance segmentation (cannot separate instances of the same class).

6.6 Evolution

What FCN solved: End-to-end semantic segmentation without patch-based classification. The convolutionalization trick made it possible to repurpose any classification network for dense prediction. Skip connections improved spatial accuracy by fusing multi-scale features.

What FCN still couldn't solve:

- Coarse output: even FCN-8s outputs are 8× smaller than input; bilinear upsampling to full resolution loses detail. The additive skip fusion provides limited spatial refinement.
- No dedicated decoder: the upsampling path is a single transposed convolution step per fusion, not a learned multi-stage decoder that progressively refines features.
- Boundary precision: object boundaries remain imprecise because the encoder's max pooling discards fine spatial information.

What came next:

- **UNet** (Ronneberger et al., 2015): Introduces a symmetric encoder-decoder with concatenative skip connections (not additive) and progressive upsampling through a multi-stage decoder. Each decoder stage halves the spatial gap via learned upsampling, producing precise boundaries. Designed for biomedical images with limited training data.
- **DeepLab** (Chen et al., 2014–2018): Introduces atrous (dilated) convolutions to control the encoder's receptive field without reducing resolution, combined with Conditional Random Fields (CRFs) for boundary refinement (v1–v2), then Atrous Spatial Pyramid Pooling (ASPP) for multi-scale context and depthwise separable convolutions (v3+).
- **PSPNet** (Zhao et al., 2017): Adds a Pyramid Pooling Module that pools features at multiple grid scales and concatenates them, capturing global and sub-region context before the final prediction.

Chapter 7

UNet

7.1 Motivation

FCN introduced end-to-end dense prediction, but its skip connections are additive and its upsampling path is shallow — a single transposed convolution step. For biomedical image segmentation, two challenges demand a different design:

1. **Limited training data:** Biomedical datasets may contain only dozens of annotated images. The network must learn from very few examples, requiring strong architectural priors for spatial precision.

2. **Precise localization:** Biomedical tasks (cell boundary detection, organ segmentation) require pixel-perfect boundaries. FCN's $8\times$ upsampling cannot recover the fine detail lost by max pooling.

The UNet architecture addresses both with two key innovations: a **symmetric encoder-decoder** that progressively recovers spatial resolution over multiple stages, and **concatenative skip connections** that give the decoder direct access to encoder features at each resolution.

7.2 Mathematical Formulation

7.2.1 U-shaped Architecture Principle

The network consists of a contracting path (encoder) $\mathcal{E}$ and an expanding path (decoder) $\mathcal{D}$. Let $x^{(l)}$ denote the feature maps at resolution level l (where $l = 0$ is input resolution, $l = 4$ is the bottleneck). The encoder applies:

$$x^{(l)} = \mathcal{E}_l(x^{(l-1)}) = \text{MaxPool}_{2\times 2}(\text{ConvBlock}(x^{(l-1)})), \quad l = 1, \ldots, 4, \tag{7.1}$$

where ConvBlock applies two 3×3 convolutions with ReLU.

The decoder recovers resolution stage by stage:

$$x^{(l)}_{\text{up}} = \text{TransposedConv}_{2\times 2}(x^{(l+1)}), \quad l = 3, \ldots, 0. \tag{7.2}$$

Crucially, encoder features from the same resolution are concatenated:

$$x^{(l)} = \text{ConvBlock}\big(\big[x^{(l)}_{\text{up}} \parallel x^{(l)}_{\text{enc}}\big]\big)\,, \tag{7.3}$$

where $[\cdot\|\cdot]$ denotes channel-wise concatenation and $x^{(l)}_{\text{enc}}$ is the encoder output at level l (center-cropped to match the decoder's spatial size).

7.2.2 Concatenation vs. Addition

FCN uses additive fusion $y = \mathcal{D}(x) + \mathcal{E}(x)$, which forces a direct channel-wise merge. UNet concatenates, producing a feature tensor with $C_{\text{dec}} + C_{\text{enc}}$ channels:

$$y = \text{Conv}\left(\left[\frac{\mathcal{D}(x)}{\mathcal{E}(x)}\right]\right). \tag{7.4}$$

The subsequent convolution learns how to combine these features. Concatenation preserves both paths independently; the decoder can selectively use or ignore each encoder channel. Since early encoder features contain fine textural and edge information while decoder features carry coarse semantics, concatenation avoids forced blending and gives the network more representational freedom.

7.2.3 Copy and Crop Mechanism

Due to valid (unpadded) convolutions, each 3×3 conv reduces spatial size by 2 pixels per dimension. The encoder feature map at level l is therefore larger than the decoder input at that level. UNet **center-crops** the encoder features to match the decoder spatial dimensions before concatenation. Let $d_{\text{enc}}^{(l)}$ and $d_{\text{dec}}^{(l)}$ be the spatial sizes. The crop removes:

$$\Delta = \frac{d_{\text{enc}}^{(l)} - d_{\text{dec}}^{(l)}}{2} \tag{7.5}$$

pixels from each border.

7.2.4 Weighted Loss for Touching Objects

Biomedical images often contain touching objects of the same class (e.g., adjacent cells). A simple cross-entropy loss cannot separate them; the network learns to merge touching instances. UNet introduces a **weight map** that penalizes errors on border pixels between touching objects:

$$w(x) = w_c(x) + w_0 \cdot \exp\left(-\frac{(d_1(x) + d_2(x))^2}{2\sigma^2}\right), \tag{7.6}$$

where:

- $w_c(x)$ is the class-balancing weight (inverse frequency of the pixel's class).
- $d_1(x)$: Euclidean distance from pixel x to the border of the nearest cell.
- $d_2(x)$: Euclidean distance from pixel x to the border of the second-nearest cell.
- $w_0 \approx 10$ and $\sigma \approx 5$ pixels are hyperparameters.

The exponential term is largest in the thin gap between two touching cells (where $d_1 + d_2$ is small, d_1 being small near one cell and d_2 near the other). This forces the network to learn separating boundaries.

The per-pixel weighted cross-entropy loss:

$$\mathcal{L} = -\sum_{x \in \Omega} w(x) \sum_{c=1}^{C} \mathbf{1}[y_x = c] \cdot \log\left(\text{softmax}(s_x)\right)_c. \tag{7.7}$$

7.2.5 Data Augmentation with Elastic Deformations

With few training images, augmentation is essential. UNet's primary augmentation is **elastic deformation**: the image is warped by a smooth random displacement field $\Phi(x) = x + \delta(x)$, where $\delta(x)$ is sampled from a Gaussian with standard deviation σ (typically 10 pixels) and smoothed by a Gaussian filter of size σ. This simulates tissue deformations and shape variations without requiring additional annotated data.

Let $I(x)$ be the original image and $M(x)$ the ground truth mask. The deformed pair is:

$$\tilde{I}(x) = I(\Phi^{-1}(x)), \qquad \tilde{M}(x) = M(\Phi^{-1}(x)), \tag{7.8}$$

computed via bicubic interpolation for images and nearest-neighbor for masks.

7.3 Algorithm

Input : Images $\{I_i\}_{i=1}^N$, masks $\{M_i\}_{i=1}^N$, hyperparameters w_0, σ
Output: Trained UNet parameters θ

Pretrain on similar data or initialize with He initialization;
for *epoch* $= 1$ **to** *max_ epochs* **do**
  for *each training image* I *with mask* M **do**
    Apply random elastic deformation to (I, M);
    Apply random rigid transformations (rotation, scaling, flipping);
    Compute weight map $w(x)$ using distance transform d_1, d_2;
    Encoder forward pass:;
    $x^{(0)} \leftarrow I$;
    for $l = 1$ **to** 4 **do**
      $x^{(l)} \leftarrow \text{MaxPool}_{2\times2}(\text{Conv}_{3\times3}(\text{ReLU}(\text{Conv}_{3\times3}(\text{ReLU}(x^{(l-1)})))))$;
    Bottleneck:;
    $x^{(5)} \leftarrow \text{Conv}_{3\times3}(\text{ReLU}(\text{Conv}_{3\times3}(\text{ReLU}(x^{(4)}))))$;
    Decoder forward pass:;
    for $l = 4$ ***downto*** 1 **do**
      $x_{\text{up}} \leftarrow \text{TransposedConv}_{2\times2}(x^{(l)})$;
      Crop encoder features $x_{\text{enc}}^{(l-1)}$ to match x_{up} spatial size;
      $x^{(l-1)} \leftarrow \text{Conv}_{3\times3}(\text{ReLU}(\text{Conv}_{3\times3}(\text{ReLU}([x_{\text{up}} \| x_{\text{enc}}^{(l-1)}]))))$;
    Output: $\hat{Y} \leftarrow \text{softmax}(\text{Conv}_{1\times1}(x^{(0)}))$;
    Compute weighted cross-entropy loss $\mathcal{L}$;
    Backpropagate through all decoder and encoder layers;
    Update θ via SGD with momentum 0.99;

Output: Segmentation network θ

7.4 Architecture

Stage	Operation	Output Size	Channels
Input	—	572×572	1/3
conv1_1 + conv1_2	3×3 conv, ReLU	$570 \to 568$	64
pool1	2×2 max pool	284×284	64
conv2_1 + conv2_2	3×3 conv, ReLU	$282 \to 280$	128
pool2	2×2 max pool	140×140	128
conv3_1 + conv3_2	3×3 conv, ReLU	$138 \to 136$	256
pool3	2×2 max pool	68×68	256
conv4_1 + conv4_2	3×3 conv, ReLU	$66 \to 64$	512
pool4	2×2 max pool	32×32	512
Bottleneck			
conv5_1 + conv5_2	3×3 conv, ReLU	$30 \to 28$	1024
Decoder			
up-conv4	2×2 transposed conv	56×56	512
concat (crop conv4_2)	concatenate	56×56	$512 + 512 = 1024$
conv6_1 + conv6_2	3×3 conv, ReLU	$54 \to 52$	512
up-conv3	2×2 transposed conv	104×104	256
concat (crop conv3_2)	concatenate	104×104	$256 + 256 = 512$
conv7_1 + conv7_2	3×3 conv, ReLU	$102 \to 100$	256
up-conv2	2×2 transposed conv	200×200	128
concat (crop conv2_2)	concatenate	200×200	$128 + 128 = 256$
conv8_1 + conv8_2	3×3 conv, ReLU	$198 \to 196$	128
up-conv1	2×2 transposed conv	392×392	64
concat (crop conv1_2)	concatenate	392×392	$64 + 64 = 128$
conv9_1 + conv9_2	3×3 conv, ReLU	$390 \to 388$	64
Output	1×1 conv, softmax	388×388	C

Table 7.1: UNet architecture (original). Input 572×572, output 388×388 due to valid convolutions. Modern implementations often use padded convolutions to maintain spatial size.

Channel progression: The encoder doubles channels at each downsampling step: $1 \to 64 \to 128 \to 256 \to 512 \to 1024$. The decoder halves channels symmetrically: $1024 \to 512 \to 256 \to 128 \to 64 \to C$.

Spatial reduction: Each 2×2 max pooling halves spatial dimensions. The total spatial reduction from input to bottleneck is $2^4 = 16\times$ (with valid convolutions further reducing by border loss). The decoder recovers resolution in four stages of $2\times$ upsampling.

7.5 Pros and Cons

7.5.1 Advantages

- Concatenative skip connections preserve fine spatial detail that additive fusion loses.

- Symmetric encoder-decoder with multiple upsampling stages recovers resolution gradually.
- Works exceptionally well with very limited training data (original paper: 30 images).
- Weighted loss separates touching objects without instance-level supervision.
- Elastic deformations are a powerful, domain-appropriate augmentation.
- U-shaped design has become the default architecture for medical imaging segmentation.
- Training from scratch is feasible (no ImageNet pretraining needed for medical modalities).

7.5.2 Disadvantages

- Valid convolutions waste border pixels; input size must be carefully chosen for tile-based inference.
- The 1024-channel bottleneck is computationally heavy for 3D extensions.
- Fixed receptive field (no multi-scale context aggregation like ASPP).
- Fixed architecture: the number of downsampling stages is not adaptive to input resolution.
- Simple skip connections directly copy encoder features; no gating or attention on which features to propagate.
- Instance segmentation of touching objects still requires post-processing (watershed, etc.).

7.6 Evolution

What UNet solved: FCN's coarse output and shallow upsampling, especially for small-data regimes. The symmetric encoder-decoder with concatenative skip connections enables precise localization of fine structures. The weighted loss and elastic deformations made it the standard for biomedical segmentation.

What UNet still couldn't solve:

- Fixed receptive field: the network has one scale of analysis; multi-scale features are only captured implicitly through the encoder hierarchy.
- Naive concatenation propagates all encoder features to the decoder, including potentially irrelevant or noisy ones.
- The architecture depth (number of stages) is fixed, which may be suboptimal for some input resolutions or tasks.
- No built-in mechanism for handling class imbalance beyond the simple weight map.

What came next:

- **UNet++** (Zhou et al., 2018): Introduces nested and dense skip connections at multiple semantic levels between encoder and decoder, reducing the semantic gap. Convolution blocks on skip paths allow progressive feature refinement rather than direct copy-paste.

- **Attention UNet** (Oktay et al., 2018): Adds attention gates to skip connections, allowing the decoder to focus on relevant spatial regions of the encoder features and suppress irrelevant activations. The gating signal comes from the coarser decoder level:

$$\alpha_l = \sigma_2(\psi^T(\sigma_1(W_x^T x_{\text{enc}}^{(l)} + W_g^T g + b_g)) + b_\psi). \tag{7.9}$$

- **3D UNet** (Çiçek et al., 2016): Extends the architecture to volumetric data ($3 \times 3 \times 3$ convolutions, $2 \times 2 \times 2$ max pooling, $2 \times 2 \times 2$ transposed convs), enabling segmentation of 3D biomedical volumes (CT, MRI).

- **nnUNet** (Isensee et al., 2021): A self-configuring framework that automatically adapts the UNet architecture, preprocessing, and post-processing to any new dataset, removing the need for manual architecture design.

Part III

Sequence Models

Chapter 8

RNN — Recurrent Neural Networks

8.1 Motivation

Feedforward neural networks process each input independently, assuming no temporal structure. Each input x_t is mapped to an output y_t without reference to past inputs. This fails for sequential data: language, speech, video, time series, and any domain where the order and dependence between elements matters.

Consider a sentence: the meaning of word t depends on preceding words. A feedforward network would treat "The cat sat on the mat" as six independent observations, unable to model subject-verb agreement or long-range context. Recurrent neural networks introduce a **hidden state** h_t that summarizes the past sequence and is updated at each time step. Crucially, the same parameters (W_{xh}, W_{hh}, W_{hy}) are shared across all time steps, giving the model translational invariance in the time dimension and a fixed parameter count regardless of sequence length.

8.2 Mathematical Formulation

8.2.1 Basic RNN Equations

Given a sequence $\{x_1, x_2, \ldots, x_T\}$ with $x_t \in \mathbb{R}^d$, the RNN maintains a hidden state $h_t \in \mathbb{R}^H$:

$$a_t = W_{hh}h_{t-1} + W_{xh}x_t + b_h, \tag{8.1}$$
$$h_t = g(a_t) = g(W_{hh}h_{t-1} + W_{xh}x_t + b_h), \tag{8.2}$$
$$y_t = W_{hy}h_t + b_y, \tag{8.3}$$

where $W_{hh} \in \mathbb{R}^{H\times H}$, $W_{xh} \in \mathbb{R}^{H\times d}$, $W_{hy} \in \mathbb{R}^{K\times H}$, and g is an element-wise nonlinearity (typically tanh). The initial state h_0 is usually initialized to zero. Parameter count: $H(H + d + K) + H + K$, independent of T.

8.2.2 Unrolling Through Time

An RNN can be viewed as a very deep feedforward network with tied weights across layers. Unrolling T steps produces a computation graph of depth T:

$$h_1 = g(W_{hh}h_0 + W_{xh}x_1 + b_h), \tag{8.4}$$
$$h_2 = g(W_{hh}h_1 + W_{xh}x_2 + b_h), \tag{8.5}$$
$$\vdots \tag{8.6}$$
$$h_T = g(W_{hh}h_{T-1} + W_{xh}x_T + b_h). \tag{8.7}$$

Every "layer" (time step) shares the identical parameters (W_{hh}, W_{xh}, b_h). This unrolling perspective is central to understanding training dynamics.

8.2.3 Loss Function for Sequences

For sequence-to-sequence tasks, the loss aggregates over all time steps:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t(y_t, \hat{y}_t), \tag{8.8}$$

where $\mathcal{L}_t$ is typically cross-entropy (classification) or mean squared error (regression). For sequence classification, only the final output y_T may be used: $\mathcal{L} = \mathcal{L}(y_T, \hat{y})$.

8.2.4 Backpropagation Through Time (BPTT)

To compute $\frac{\partial \mathcal{L}}{\partial \theta}$ for parameters $\theta \in \{W_{hh}, W_{xh}, W_{hy}, b_h, b_y\}$, we unroll the network through all time steps and apply backpropagation.

The gradient of the loss with respect to hidden state h_t receives contributions from the immediate output at time t and from the future state at $t+1$:

$$\frac{\partial \mathcal{L}}{\partial h_t} = \frac{\partial \mathcal{L}_t}{\partial y_t} \cdot \frac{\partial y_t}{\partial h_t} + \frac{\partial \mathcal{L}}{\partial h_{t+1}} \cdot \frac{\partial h_{t+1}}{\partial h_t}. \tag{8.9}$$

Substituting the derivatives:

$$\frac{\partial y_t}{\partial h_t} = W_{hy}^T, \tag{8.10}$$

$$\frac{\partial h_{t+1}}{\partial h_t} = \frac{\partial}{\partial h_t} g(W_{hh} h_t + W_{xh} x_{t+1} + b_h) = \operatorname{diag}(g'(a_{t+1})) \cdot W_{hh}^T. \tag{8.11}$$

Thus:

$$\frac{\partial \mathcal{L}}{\partial h_t} = \frac{\partial \mathcal{L}_t}{\partial y_t} W_{hy}^T + \frac{\partial \mathcal{L}}{\partial h_{t+1}} \cdot \operatorname{diag}(g'(a_{t+1})) \cdot W_{hh}^T. \tag{8.12}$$

This recurrence runs backward from $t = T$ to $t = 1$. The computation starts with $\frac{\partial \mathcal{L}}{\partial h_T} = \frac{\partial \mathcal{L}_T}{\partial y_T} W_{hy}^T$ (assuming $\frac{\partial \mathcal{L}}{\partial h_{T+1}} = 0$).

Gradients with respect to parameters: After computing all $\frac{\partial \mathcal{L}}{\partial h_t}$, the parameter gradients are:

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial h_t} \odot g'(a_t) \cdot h_{t-1}^T, \tag{8.13}$$

$$\frac{\partial \mathcal{L}}{\partial W_{xh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial h_t} \odot g'(a_t) \cdot x_t^T, \tag{8.14}$$

$$\frac{\partial \mathcal{L}}{\partial b_h} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial h_t} \odot g'(a_t), \tag{8.15}$$

$$\frac{\partial \mathcal{L}}{\partial W_{hy}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial y_t} \cdot h_t^T. \tag{8.16}$$

Truncated BPTT: For long sequences, BPTT through all T steps is computationally expensive and suffers from vanishing/exploding gradients. Truncated BPTT limits the backpropagation to K steps: at each time step t, states h_{t-K+1} through h_t are treated as a K-step network and gradients flow only within this window.

8.2.5 Gradient Flow and the Vanishing/Exploding Gradient Problem

The core dynamic is the Jacobian of the hidden state transition. Consider how a perturbation in h_t propagates to h_T:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k} = \prod_{k=t}^{T-1} \operatorname{diag}(g'(a_{k+1})) \cdot W_{hh}^T. \tag{8.17}$$

Eigenanalysis. Diagonalize $W_{hh}^T = Q\Lambda Q^{-1}$ where $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_H)$ are eigenvalues sorted by magnitude $|\lambda_1| \geq \cdots \geq |\lambda_H|$. Then:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t}^{T-1} \operatorname{diag}(g'(a_{k+1})) \cdot Q\Lambda Q^{-1}. \tag{8.18}$$

The term $\prod_k \Lambda$ contributes factors $|\lambda_i|^{T-t}$. If $|\lambda_{\max}| < 1$, then $\|\frac{\partial h_T}{\partial h_t}\| \to 0$ exponentially in $T - t$ — the gradient **vanishes**. If $|\lambda_{\max}| > 1$, the gradient **explodes** exponentially.

Bounding with activation derivatives. For tanh activation, $\sup |g'(x)| = 1$ (attained at $x = 0$). The norm of each Jacobian factor is bounded:

$$\left\| \frac{\partial h_{k+1}}{\partial h_k} \right\| \leq \left\| W_{hh}^T \right\| \cdot \|\operatorname{diag}(g'(a_{k+1}))\| \leq \left\| W_{hh}^T \right\| \cdot 1. \tag{8.19}$$

Thus $\left\| \frac{\partial h_T}{\partial h_t} \right\| \lesssim \|W_{hh}\|^{T-t}$. For long-term dependencies ($T-t$ large), this requires $\|W_{hh}\| \approx 1$, which is a difficult optimization constraint.

Consequence for the loss gradient. For the full loss gradient:

$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{k=1}^{T-1} \operatorname{diag}(g'(a_{k+1})) W_{hh}^T + (\text{contributions from intermediate outputs}). \tag{8.20}$$

If any eigenvalue $|\lambda| \neq 1$, the long-range contribution to $\frac{\partial \mathcal{L}}{\partial h_1}$ is either exponentially small (cannot learn dependencies spanning many steps) or exponentially large (training diverges).

Gradient clipping addresses explosion by scaling gradients when their norm exceeds a threshold τ:

$$\tilde{g} \leftarrow g \cdot \min\left(1, \frac{\tau}{\|g\|}\right). \tag{8.21}$$

This does not solve vanishing, only explosion.

8.2.6 Bidirectional RNN

For tasks where both past and future context matter (e.g., sequence labeling, machine translation), a bidirectional RNN processes the sequence in both directions:

$$h_t^{\rightarrow} = g(W_{hh}^{\rightarrow} h_{t-1}^{\rightarrow} + W_{xh}^{\rightarrow} x_t + b_h^{\rightarrow}), \tag{8.22}$$

$$h_t^{\leftarrow} = g(W_{hh}^{\leftarrow} h_{t+1}^{\leftarrow} + W_{xh}^{\leftarrow} x_t + b_h^{\leftarrow}), \tag{8.23}$$

$$y_t = W_{hy}^{\rightarrow} h_t^{\rightarrow} + W_{hy}^{\leftarrow} h_t^{\leftarrow} + b_y. \tag{8.24}$$

The forward RNN runs left-to-right, the backward RNN runs right-to-left. The concatenation $[h_t^{\rightarrow}; h_t^{\leftarrow}]$ captures context from both sides.

8.2.7 Deep (Stacked) RNN

Multiple RNN layers can be stacked for increased capacity. At layer l:

$$h_t^{(l)} = g\Big(W_{hh}^{(l)} h_{t-1}^{(l)} + W_{xh}^{(l)} h_t^{(l-1)} + b_h^{(l)}\Big), \tag{8.25}$$

where $h_t^{(0)} = x_t$. The output layer uses the topmost hidden states: $y_t = W_{hy} h_t^{(L)} + b_y$.

8.3 Algorithm

Input : Sequence $\{x_1, \ldots, x_T\}$, ground truth $\{\hat{y}_1, \ldots, \hat{y}_T\}$
Output: Trained RNN parameters

Initialize W_{hh}, W_{xh}, W_{hy} with Xavier initialization; $b_h, b_y \leftarrow 0$;
Set initial hidden state $h_0 \leftarrow \mathbf{0}$;
for *epoch* $= 1$ **to** *max_ epochs* **do**
  for *each sequence in mini-batch* **do**
    Forward pass (unroll):;
    $h_0 \leftarrow \mathbf{0}$;
    for $t = 1$ **to** T **do**
      $a_t \leftarrow W_{hh} h_{t-1} + W_{xh} x_t + b_h$;
      $h_t \leftarrow g(a_t)$;
      $y_t \leftarrow W_{hy} h_t + b_y$;
      $\mathcal{L}_t \leftarrow \ell(y_t, \hat{y}_t)$;
    $\mathcal{L} \leftarrow \sum_{t=1}^{T} \mathcal{L}_t$;
    BPTT (unroll backward):;
    $\frac{\partial \mathcal{L}}{\partial h_T} \leftarrow \frac{\partial \mathcal{L}_T}{\partial y_T} W_{hy}^T$;
    for $t = T - 1$ **to** 1 **do**
      $\frac{\partial \mathcal{L}}{\partial h_t} \leftarrow \frac{\partial \mathcal{L}_t}{\partial y_t} W_{hy}^T + \frac{\partial \mathcal{L}}{\partial h_{t+1}} \cdot \mathrm{diag}(g'(a_{t+1})) \cdot W_{hh}^T$;
    Accumulate parameter gradients $\frac{\partial \mathcal{L}}{\partial W_{hh}}, \frac{\partial \mathcal{L}}{\partial W_{xh}}, \frac{\partial \mathcal{L}}{\partial W_{hy}}, \ldots$;
    if $\|\nabla_\theta \mathcal{L}\| > \tau$ *(gradient clipping)* **then**
      $\nabla_\theta \mathcal{L} \leftarrow \nabla_\theta \mathcal{L} \cdot \frac{\tau}{\|\nabla_\theta \mathcal{L}\|}$;
    else
      ;
    Update parameters via SGD or Adam;
Output: Return trained RNN

8.4 Architecture

Basic RNN cell. The computational graph at each time step involves three matrix multiplications:

$$W_{xh} \in \mathbb{R}^{H \times d}: \quad \text{input-to-hidden}, \tag{8.26}$$

$$W_{hh} \in \mathbb{R}^{H \times H}: \quad \text{hidden-to-hidden (recurrent)}, \tag{8.27}$$

$$W_{hy} \in \mathbb{R}^{K \times H}: \quad \text{hidden-to-output}. \tag{8.28}$$

Bidirectional RNN. Two independent RNNs with separate parameter sets. Forward RNN: $(W_{hh}^{\rightarrow}, W_{xh}^{\rightarrow}, b_h^{\rightarrow})$. Backward RNN: $(W_{hh}^{\leftarrow}, W_{xh}^{\leftarrow}, b_h^{\leftarrow})$. Output concatenation: $[h_t^{\rightarrow}; h_t^{\leftarrow}] \in \mathbb{R}^{2H}$.

Stacked RNN. L recurrent layers. Hidden dimension typically constant across layers, though some designs taper outward. Each layer receives the hidden states from the layer below as its input at each time step.

Activation function: tanh is standard: $g(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, satisfying $g'(x) = 1 - g(x)^2 \leq 1$, with $\sup |g'| = 1$.

8.5 Pros and Cons

8.5.1 Advantages

- Natural inductive bias for sequential data: temporal order matters.
- Parameter sharing across time: handles variable-length sequences with fixed parameter count.
- Theoretically Turing-complete given sufficient hidden dimension.
- Bidirectional variant captures both past and future context.
- Stacking increases representational capacity hierarchically.
- Simple to implement and understand; foundational to sequence modeling.

8.5.2 Disadvantages

- Vanishing gradient prevents learning dependencies beyond ∼10–20 steps in practice.
- Exploding gradient requires clipping, which is a heuristic fix, not a solution to the underlying optimization problem.
- Sequential computation along time: h_t depends on h_{t-1}, so parallelization across time is impossible. Training on long sequences is slow.
- The hidden state is a fixed-size vector bottleneck; it must compress the entire past.
- No explicit memory mechanism: the hidden state is overwritten at each step, and there is no mechanism to selectively retain or forget information.
- Tanh saturation: when $g'(x) \approx 0$, gradient flow through that step is blocked regardless of W_{hh}.

8.6 Evolution

What RNN solved: Processing of variable-length sequences with parameter sharing across time. Enabled early successes in language modeling, machine translation (seq2seq with RNN encoder-decoder), speech recognition, and time series forecasting. The bidirectional variant was particularly effective for sequence labeling.

What RNN still couldn't solve:

- Long-range dependencies: the vanishing gradient means errors from time T cannot influence learning at time 1 if T is large. Empirically, effective memory is limited to a few dozen steps.
- The hidden state h_t is a monolithic vector with no mechanism to selectively write, read, or erase – every update overwrites the entire state.
- Sequential computation along time prohibits GPU-level parallelism over the time dimension.
- No gating: the network cannot learn *when* to remember and *when* to forget.

What came next:

- **LSTM** (Hochreiter & Schmidhuber, 1997): Introduces a memory cell c_t with gated additive updates. The cell state evolves via $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, where gates f_t, i_t, o_t are learned. The forget gate f_t allows selective memory retention, and the gradient $\frac{\partial c_t}{\partial c_{t-1}} = \text{diag}(f_t)$ can be close to identity, enabling gradient flow over hundreds of steps. LSTM directly addresses the vanishing gradient problem through gating.
- **GRU** (Cho et al., 2014): A simplified gated architecture with two gates (reset and update) and a merged cell/hidden state. Fewer parameters than LSTM, similar empirical performance.
- **Attention mechanisms** (Bahdanau et al., 2015): Instead of compressing the entire input sequence into a fixed-length vector (as in RNN encoder-decoder), attention allows the decoder to attend to all encoder hidden states with learnable weights, creating direct gradient paths to every time step.
- **Transformers** (Vaswani et al., 2017): Eliminate recurrence entirely. Self-attention computes pairwise interactions between all positions in $O(1)$ sequential operations (fully parallelizable across time). Positional encodings inject order information. Transformer blocks with multi-head attention and feedforward layers process the entire sequence simultaneously, achieving superior long-range modeling.

Chapter 9

LSTM — Long Short-Term Memory

9.1 Motivation

The RNN's vanishing gradient prevents learning dependencies spanning more than ∼10–20 time steps. The fundamental reason: the hidden state $h_t = g(W_{hh}h_{t-1} + \cdots)$ is a composite nonlinear mapping. The Jacobian $\frac{\partial h_{t+1}}{\partial h_t} = \text{diag}(g'(a_{t+1}))W_{hh}^T$ contracts each dimension independently through both the weight matrix and the activation derivative. Over many steps, this product either vanishes or explodes.

LSTM re-engineers the state update to be **additive** rather than multiplicative. The key insight is the **Constant Error Carousel** (CEC): a memory cell c_t that evolves as $c_t = c_{t-1} + \Delta c$, where Δc is gated. The gradient $\frac{\partial c_t}{\partial c_{t-1}} = 1$ (for the identity path), providing a perfect gradient highway that never attenuates. Gating mechanisms then regulate what information enters (input gate), what leaves (output gate), and what is discarded (forget gate), while preserving the additive update structure.

9.2 Mathematical Formulation

9.2.1 Full LSTM Equations

At each time step t, given input x_t and previous hidden state h_{t-1}, the LSTM computes:

$$f_t = \sigma\left(W_f \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_f\right), \tag{9.1}$$

$$i_t = \sigma\left(W_i \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_i\right), \tag{9.2}$$

$$o_t = \sigma\left(W_o \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_o\right), \tag{9.3}$$

$$\tilde{c}_t = \tanh\left(W_c \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_c\right), \tag{9.4}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \tag{9.5}$$

$$h_t = o_t \odot \tanh(c_t). \tag{9.6}$$

All gate activations are sigmoid (σ), producing values in $(0, 1)$. The candidate $\tilde{c}_t$ uses tanh, producing values in $(-1, 1)$. All weight matrices $W_f, W_i, W_o, W_c \in \mathbb{R}^{H\times(H+d)}$, where d is the input dimension and H is the hidden (cell) dimension.

Shape summary: $f_t, i_t, o_t, c_t, \tilde{c}_t, h_t \in \mathbb{R}^H$.

9.2.2 Gate Semantics

- **Forget gate f_t:** Controls what proportion of the previous cell state to retain. $f_t \to 1$ means "keep everything"; $f_t \to 0$ means "erase". The forget gate enables

the LSTM to reset memory when the context changes (e.g., end of sentence, scene change). Typical initialization: b_f is initialized to 1 (or a positive constant) to bias the network toward remembering by default before learning to forget.

- **Input gate i_t:** Controls what proportion of the candidate values $\tilde{c}_t$ to write into the cell state. $i_t \to 1$ means "store new information"; $i_t \to 0$ means "ignore this input."

- **Output gate o_t:** Controls what proportion of the (transformed) cell state to expose as the hidden output. $o_t \to 1$ means "reveal the cell state"; $o_t \to 0$ means "hide it" (the cell may store information without affecting the output). This decouples memory from output.

9.2.3 Gradient Flow Through the Cell State

The cell state update in Equation (9.5) is the core innovation. Computing the gradient:

$$\frac{\partial c_t}{\partial c_{t-1}} = \text{diag}(f_t) + \frac{\partial (i_t \odot \tilde{c}_t)}{\partial c_{t-1}}. \tag{9.7}$$

The second term involves gradients of gates and candidate values with respect to c_{t-1} through h_{t-1}. But the dominant term is $\text{diag}(f_t)$, which is a diagonal matrix with entries in $(0, 1)$. When $f_t \approx 1$ (the typical default state), $\frac{\partial c_t}{\partial c_{t-1}} \approx I$, and the gradient flows unchanged.

For the full loss gradient with respect to the cell state, we use the chain rule backward through time:

$$\frac{\partial \mathcal{L}}{\partial c_t} = \underbrace{\frac{\partial \mathcal{L}}{\partial h_t}\frac{\partial h_t}{\partial c_t}}_{\text{from output at } t} + \underbrace{\frac{\partial \mathcal{L}}{\partial c_{t+1}}\frac{\partial c_{t+1}}{\partial c_t}}_{\text{from future}}. \tag{9.8}$$

Substituting the derivatives:

$$\frac{\partial h_t}{\partial c_t} = \text{diag}(o_t \odot \tanh'(c_t)) = \text{diag}(o_t) \cdot \text{diag}(1 - \tanh^2(c_t)), \tag{9.9}$$

$$\frac{\partial c_{t+1}}{\partial c_t} = \text{diag}(f_{t+1}). \tag{9.10}$$

Thus:

$$\frac{\partial \mathcal{L}}{\partial c_t} = \frac{\partial \mathcal{L}}{\partial h_t} \odot o_t \odot (1 - \tanh^2(c_t)) + \frac{\partial \mathcal{L}}{\partial c_{t+1}} \odot f_{t+1}. \tag{9.11}$$

The second term $\frac{\partial \mathcal{L}}{\partial c_{t+1}} \odot f_{t+1}$ is the gradient highway. It propagates error backward through time with multiplicative factor f_{t+1} at each step. Since $f_{t+1} \in (0, 1)^H$, the gradient can still slowly decay if $f_t < 1$ consistently, but:

1. The decay is element-wise and learnable (each dimension can retain its gradient independently).

2. Near 1, the decay rate is much slower than the $\|W_{hh}\|^T$ decay in standard RNNs.

3. The input gate i_t can open to inject new gradient paths forward, partially compensating for forgetting.

Unrolling the highway: Starting from time T:

$$\frac{\partial \mathcal{L}}{\partial c_t} = \frac{\partial \mathcal{L}}{\partial c_T} \odot \prod_{k=t+1}^{T} f_k + \sum_{\tau=t+1}^{T} \frac{\partial \mathcal{L}}{\partial h_\tau} \odot o_\tau \odot (1 - \tanh^2(c_\tau)) \odot \prod_{k=\tau+1}^{T} f_k. \tag{9.12}$$

Each term is modulated by a product of forget gate values. With b_f initialized to ≈ 1, the forget gate learns to be ≈ 1 for relevant information and < 1 only when forgetting is appropriate.

9.2.4 Backpropagation Through LSTM Gates

To train the LSTM, we need gradients through all gates. Define $\delta h_t \coloneqq \frac{\partial \mathcal{L}}{\partial h_t}$ and $\delta c_t \coloneqq \frac{\partial \mathcal{L}}{\partial c_t}$ (with $\delta c_{T+1} = 0$, $\delta h_{T+1} = 0$; note δh_t is also affected by the next step's recurrent connection).

Gate gradients (at time t):

$$\frac{\partial \mathcal{L}}{\partial o_t} = \delta h_t \odot \tanh(c_t) \odot \sigma'(o_t), \tag{9.13}$$

$$\frac{\partial \mathcal{L}}{\partial f_t} = \delta c_t \odot c_{t-1} \odot \sigma'(f_t), \tag{9.14}$$

$$\frac{\partial \mathcal{L}}{\partial i_t} = \delta c_t \odot \tilde{c}_t \odot \sigma'(i_t), \tag{9.15}$$

$$\frac{\partial \mathcal{L}}{\partial \tilde{c}_t} = \delta c_t \odot i_t \odot \tanh'(\tilde{c}_t), \tag{9.16}$$

where $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ and $\tanh'(x) = 1 - \tanh^2(x)$.

Input weight gradients: For each gate type $\{ f, i, o, c \}$:

$$\frac{\partial \mathcal{L}}{\partial W_*} = \sum_{t=1}^{T} \delta *_t \ [h_{t-1}^T \ x_t^T], \tag{9.17}$$

where $\delta f_t, \delta i_t, \delta o_t$ denote the gate-specific errors above and δc_t (for $\tilde{c}_t$) is treated analogously.

Recurrent gradient to h_{t-1}:

$$\delta h_{t-1} = \sum_{*\in\{f,i,o,c\}} W_{*h}^T \, (\delta *_t \odot *_t \odot (1 - *_t)), \tag{9.18}$$

where W_{*h} denotes the submatrix of W_* corresponding to h_{t-1} (the first H columns), and $*_t$ is the respective gate activation. More explicitly:

$$\delta h_{t-1} = W_{fh}^T(\delta c_t \odot c_{t-1} \odot f_t \odot (1-f_t)) + W_{ih}^T(\delta c_t \odot \tilde{c}_t \odot i_t \odot (1-i_t)) + W_{oh}^T(\delta h_t \odot \tanh(c_t) \odot o_t \odot (1-o_t)) + W_{ch}^T(\delta c_t \odot \tag{9.19}$$

9.2.5 Gated Recurrent Unit (GRU)

The GRU simplifies LSTM by merging the cell and hidden states and using two gates instead of three. The GRU equations:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z), \tag{9.20}$$
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r), \tag{9.21}$$
$$\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t] + b), \tag{9.22}$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \tag{9.23}$$

Update gate z_t: Controls the interpolation between old state h_{t-1} and new candidate $\tilde{h}_t$. When $z_t \to 0$, the state is copied ($h_t \approx h_{t-1}$). When $z_t \to 1$, the state is overwritten ($h_t \approx \tilde{h}_t$).

Reset gate r_t: Controls how much of the previous state to use when computing the candidate. When $r_t \to 0$, the candidate $\tilde{h}_t$ ignores the past state and acts like a feedforward transformation of x_t.

Gradient flow in GRU:

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(1 - z_t) + \cdots (\text{terms involving derivatives of gates and candidates}). \tag{9.24}$$

The $\text{diag}(1-z_t)$ term provides the same additive gradient highway as LSTM's forget gate, but the GRU eschews a separate memory cell and uses z_t both for forgetting $(1 - z_t)$ and updating (z_t).

LSTM vs. GRU comparison:

Property	LSTM	GRU
Number of gates	3 (f, i, o)	2 (z, r)
Separate memory cell	Yes (c_t)	No (merged into h_t)
Parameters	$4H(H + d)$	$3H(H + d)$
Gradient highway	$\text{diag}(f_t)$ on c_t	$\text{diag}(1 - z_t)$ on h_t
Explicit output control	Output gate o_t	None (direct exposure)

GRU has $\sim$25% fewer parameters but comparable empirical performance. LSTM's separate cell + output gate provides finer control over memory exposure. In practice, both perform similarly; the choice is often dataset-dependent, with LSTM being more common in early work and GRU gaining adoption for efficiency.

9.2.6 Bidirectional and Stacked LSTM

Bidirectional LSTM: Two independent LSTMs process the sequence forward and backward. At time t, the output is $[h_t^{\rightarrow}; h_t^{\leftarrow}]$, concatenating the forward and backward hidden states. This provides full context for each position.

Stacked LSTM: Multiple LSTM layers are stacked, where layer l receives the hidden states of layer $l - 1$ as input:

$$x_t^{(l)} = h_t^{(l-1)}, \quad h_t^{(l)} = \text{LSTM}^{(l)}(x_t^{(l)}, h_{t-1}^{(l)}, c_{t-1}^{(l)}). \tag{9.25}$$

Each layer has independent parameters. Stacked LSTMs learn temporal abstractions at different levels: lower layers capture local patterns, higher layers capture longer-range dependencies.

9.3 Algorithm

Input : Sequence $\{x_1, \ldots, x_T\}$, ground truth $\{\hat{y}_1, \ldots, \hat{y}_T\}$
Output: Trained LSTM parameters $(W_f, W_i, W_o, W_c, b_f, b_i, b_o, b_c, W_{hy}, b_y)$

Initialize forget gate bias $b_f \leftarrow 1$ (encourage remembering);
Initialize all other weights with Xavier/uniform initialization; other biases $\leftarrow 0$;
Set $c_0 \leftarrow \mathbf{0}$, $h_0 \leftarrow \mathbf{0}$;
for *epoch* $= 1$ **to** *max_ epochs* **do**
  for *each sequence in mini-batch* **do**
    Forward pass:;
    for $t = 1$ **to** T **do**
      $f_t \leftarrow \sigma(W_f[h_{t-1}, x_t] + b_f)$;
      $i_t \leftarrow \sigma(W_i[h_{t-1}, x_t] + b_i)$;
      $o_t \leftarrow \sigma(W_o[h_{t-1}, x_t] + b_o)$;
      $\tilde{c}_t \leftarrow \tanh(W_c[h_{t-1}, x_t] + b_c)$;
      $c_t \leftarrow f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$;
      $h_t \leftarrow o_t \odot \tanh(c_t)$;
      $y_t \leftarrow W_{hy} h_t + b_y$;
      $\mathcal{L}_t \leftarrow \ell(y_t, \hat{y}_t)$;
    $\mathcal{L} \leftarrow \sum_{t=1}^{T} \mathcal{L}_t$;
    Backward pass (BPTT through LSTM):;
    Initialize $\delta c_{T+1} \leftarrow \mathbf{0}$, $\delta h_{T+1} \leftarrow \mathbf{0}$;
    for $t = T$ **to** 1 **do**
      $\delta h_t \leftarrow \frac{\partial \mathcal{L}_t}{\partial h_t} + \delta h_t$;
      $\delta c_t \leftarrow \delta h_t \odot o_t \odot (1 - \tanh^2(c_t)) + \delta c_{t+1} \odot f_{t+1}$;
      Compute gate errors $\delta o_t, \delta f_t, \delta i_t, \delta \tilde{c}_t$;
      $\delta h_{t-1} \leftarrow$ gradient to h_{t-1} through all four weight matrices;
      Accumulate parameter gradients $\frac{\partial \mathcal{L}}{\partial W_*}, \frac{\partial \mathcal{L}}{\partial b_*}, \frac{\partial \mathcal{L}}{\partial W_{hy}}$;
    if $\|\nabla_\theta \mathcal{L}\| > \tau$ **then**
      $\nabla_\theta \mathcal{L} \leftarrow \nabla_\theta \mathcal{L} \cdot \frac{\tau}{\|\nabla_\theta \mathcal{L}\|}$;
    Update parameters via SGD, Adam, or RMSProp;

Output: Return trained LSTM

9.4 Architecture

LSTM cell internals. The cell contains:

- **Cell state c_t:** The long-term memory. Updated via additive gating; gradient flows diagonally through forget gates.
- **Hidden state h_t:** The short-term output. Computed as a gated, squashed version of c_t.
- **Four weight matrices:** $W_f, W_i, W_o, W_c \in \mathbb{R}^{H \times (H+d)}$, each mapping the concatenated $[h_{t-1}; x_t]$ to a gate or candidate vector. Total recurrent parameters: $4H(H + d)$.

- **Activation functions:** σ for gates (output in $(0, 1)$), tanh for candidate and hidden (output in $(-1, 1)$).

Forget gate bias initialization: Standard practice initializes b_f to 1 (or a vector of ones), which biases $\sigma(b_f) \approx 0.73$ initially. This means the default behavior is to remember, and the network learns when to deviate from this. Without this bias, $b_f = 0$ gives $\sigma(0) = 0.5$, causing an initial tendency to forget half the cell state at each step.

9.5 Pros and Cons

9.5.1 Advantages

- Additive cell state update provides a gradient highway that eliminates vanishing gradients for relevant information.
- Gating mechanisms learn to control information flow: what to remember, what to forget, what to output.
- Empirically captures dependencies spanning hundreds of time steps (vs. $\sim$10–20 for vanilla RNN).
- Separate cell state decouples memory from output, enabling the network to store information without exposing it.
- Bidirectional and stacked variants provide rich contextual representations.
- Robust to gradient explosion when combined with clipping.
- GRU variant offers comparable performance with fewer parameters.

9.5.2 Disadvantages

- Sequential computation: h_t still depends on h_{t-1} and c_{t-1}; no parallelization across time steps during training or inference.
- High parameter count: $4H(H + d)$ per layer vs. $H(H + d)$ for vanilla RNN. For large H, this is substantial.
- The forget gate still causes slow multiplicative gradient decay if f_t is consistently < 1.
- Learning can saturate gates (f_t, i_t, o_t stuck at 0 or 1) if gradients through sigmoid saturate.
- Fixed memory size H: the cell state must compress an indefinitely long past into a fixed-size vector.
- Still suffers from limited memory for very long sequences (> 1000 steps).
- No mechanism for direct, content-based retrieval of past information (unlike attention).

9.6 Evolution

What LSTM solved: The vanishing gradient problem in RNNs. By replacing the composite nonlinear recurrence $h_t = g(W_{hh}h_{t-1} + \cdots)$ with an additive cell state update $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, LSTM provided a direct gradient path through time. The forget gate (added later to the original LSTM) enabled selective retention, making the network learn what to remember rather than being forced to remember everything. LSTM became the standard for sequence modeling, powering breakthroughs in machine translation, speech recognition, language modeling, and handwriting recognition.

What LSTM still couldn't solve:

- Sequential computation bottleneck: each time step must wait for the previous one, limiting training speed and inference latency.
- Fixed-size memory: the cell state is a fixed-size vector. For very long sequences, information is inevitably overwritten or diluted.
- No direct access to arbitrary past states: to retrieve information from step t, the LSTM must carry it forward through all intermediate states, with cumulative loss from the forget gate.
- Difficulty modeling hierarchical structure: flat recurrence does not naturally model tree-structured or multi-scale temporal dependencies.
- Despite gating, gradients still decay over very long horizons (f_t product over thousands of steps eventually vanishes).

What came next:

- **GRU** (Cho et al., 2014): Simplified LSTM with two gates, merged cell and hidden state, and ∼25% fewer parameters. Showed that the separate memory cell is not strictly necessary; the additive update structure is the critical component.
- **Attention mechanisms** (Bahdanau et al., 2015; Luong et al., 2015): In sequence-to-sequence models, attention allows the decoder to directly access every encoder hidden state with learnable weights $\alpha_{t,s}$, creating direct gradient paths from the decoder loss to every encoder time step. This bypasses the LSTM's sequential bottleneck for information retrieval across positions.
- **Transformer** (Vaswani et al., 2017): Eliminates recurrence entirely. Self-attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{9.26}$$

  providing direct pairwise connections between all positions. Positional encodings (sin/cos or learned) inject order information. The Transformer processes all positions in parallel ($O(1)$ sequential operations vs. $O(T)$ for LSTM), captures dependencies across arbitrary distances with constant path length, and stacks multiple attention heads for multi-faceted representation. This architecture displaced LSTMs as the dominant sequence model, leading to BERT, GPT, and all modern large language models.

- **State Space Models / Mamba** (Gu & Dao, 2023): A recent line of work revisiting recurrent computation with structured state spaces that are parallelizable during training (via convolution mode) and efficient during inference (via recurrent mode), aiming to combine the best of both worlds.

Chapter 10

Seq2Seq with Attention

10.1 Motivation

Recurrent neural networks, particularly LSTM, can process sequences of arbitrary length, making them suitable for sequence transduction tasks such as machine translation, speech recognition, and text summarization. The encoder-decoder architecture decomposes the task: an encoder reads the input sequence $\mathbf{x} = (x_1, x_2, \ldots, x_T)$ and compresses it into a fixed-dimensional context vector $\mathbf{c}$, from which a decoder generates the output sequence $\mathbf{y} = (y_1, y_2, \ldots, y_{T'})$.

The fundamental limitation of this architecture is the information bottleneck imposed by the fixed-length context vector $\mathbf{c}$. As the source sequence length grows, the encoder must compress increasingly more information into the same fixed-dimensional representation. Empirical evidence shows that BLEU scores degrade substantially when source sentences exceed approximately 30 words. The decoder has no direct access to individual encoder hidden states, relying entirely on the single summary vector $\mathbf{c}$.

Attention mechanisms address this bottleneck by allowing the decoder to access all encoder hidden states at each decoding step, learning a soft, differentiable weighting over the source positions. This creates a dynamic context vector that adapts to the current decoding needs.

10.2 Mathematical Formulation

10.2.1 Encoder

The encoder processes the input sequence through a recurrent neural network, typically a bidirectional LSTM or GRU. For a standard unidirectional LSTM:

$$\mathbf{h}_t = \text{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1}), \quad t = 1, \ldots, T \tag{10.1}$$

where $\mathbf{x}_t \in \mathbb{R}^{d_x}$ is the input embedding at position t, and $\mathbf{h}_t \in \mathbb{R}^{d_h}$ is the hidden state. For bidirectional encoders, we concatenate forward and backward states:

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{2d_h} \tag{10.2}$$

The encoder produces the sequence of annotations $(\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_T)$. In the basic model, the context vector is simply $\mathbf{c} = \mathbf{h}_T$.

10.2.2 Decoder Without Attention

The decoder generates the output sequence one token at a time. At each step t, the decoder hidden state is:

$$\mathbf{s}_t = \text{LSTM}(\mathbf{s}_{t-1}, \mathbf{y}_{t-1}, \mathbf{c}) \tag{10.3}$$

where $\mathbf{y}_{t-1} \in \mathbb{R}^{d_y}$ is the embedding of the previously generated token. The probability distribution over the output vocabulary is:

$$P(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_{\text{out}}\mathbf{s}_t + \mathbf{b}_{\text{out}}) \tag{10.4}$$

The fundamental problem is that $\mathbf{c}$ is identical for all decoding steps t, forcing it to encode all necessary information about the entire source sentence.

10.2.3 Bahdanau Attention (Additive Attention)

Bahdanau et al. (2015) introduced the attention mechanism to create a distinct context vector $\mathbf{c}_t$ for each decoding step. The attention mechanism performs a soft, differentiable lookup over the encoder annotations.

Alignment Score

For decoder state $\mathbf{s}_{t-1}$ and each encoder annotation $\mathbf{h}_i$, compute the alignment score (energy):

$$e_{t,i} = \mathbf{v}^\top \tanh(\mathbf{W}_a\mathbf{s}_{t-1} + \mathbf{U}_a\mathbf{h}_i) \tag{10.5}$$

where $\mathbf{W}_a \in \mathbb{R}^{d_a \times d_s}$, $\mathbf{U}_a \in \mathbb{R}^{d_a \times d_h}$, and $\mathbf{v} \in \mathbb{R}^{d_a}$ are learned parameters. The score $e_{t,i}$ measures the relevance of source position i to generating target position t.

Attention Weights

The alignment scores are normalized via softmax to yield attention weights:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})}, \quad \sum_{i=1}^{T} \alpha_{t,i} = 1 \tag{10.6}$$

The vector $\boldsymbol{\alpha}_t = (\alpha_{t,1}, \ldots, \alpha_{t,T})$ forms a probability distribution over source positions.

Context Vector

The context vector is the weighted sum of encoder annotations:

$$\mathbf{c}_t = \sum_{i=1}^{T} \alpha_{t,i}\mathbf{h}_i \tag{10.7}$$

This can be interpreted as the expected encoder annotation under the attention distribution:

$$\mathbf{c}_t = \mathbb{E}_{i\sim\boldsymbol{\alpha}_t}[\mathbf{h}_i] = \sum_{i=1}^{T} \alpha_{t,i}\mathbf{h}_i \tag{10.8}$$

Decoder with Attention

The decoder now incorporates the dynamic context vector:

$$\mathbf{s}_t = \text{LSTM}(\mathbf{s}_{t-1}, [\mathbf{y}_{t-1}; \mathbf{c}_t]) \tag{10.9}$$

The output probability becomes:

$$P(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_{\text{out}}[\mathbf{s}_t; \mathbf{c}_t] + \mathbf{b}_{\text{out}}) \tag{10.10}$$

Matrix Form

For efficient computation, the alignment scores for all T source positions can be computed simultaneously:

$$\mathbf{e}_t = \mathbf{v}^\top \tanh(\mathbf{W}_a \mathbf{s}_{t-1} \mathbf{1}^\top + \mathbf{U}_a \mathbf{H}) \in \mathbb{R}^{1 \times T} \tag{10.11}$$

where $\mathbf{H} = [\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_T] \in \mathbb{R}^{d_h \times T}$.

10.2.4 Luong Attention (Multiplicative Attention)

Luong et al. (2015) proposed several scoring variants that use multiplicative interactions:

Dot Attention

$$e_{t,i} = \mathbf{s}_t^\top \mathbf{h}_i \tag{10.12}$$

Requires that $\dim(\mathbf{s}_t) = \dim(\mathbf{h}_i)$. Computationally efficient with no additional parameters.

General Attention

$$e_{t,i} = \mathbf{s}_t^\top \mathbf{W}_a \mathbf{h}_i \tag{10.13}$$

where $\mathbf{W}_a \in \mathbb{R}^{d_s \times d_h}$ is a learned compatibility matrix.

Concat Attention

$$e_{t,i} = \mathbf{v}^\top \tanh(\mathbf{W}_a [\mathbf{s}_t; \mathbf{h}_i]) \tag{10.14}$$

Similar to Bahdanau's formulation but uses the current decoder state $\mathbf{s}_t$ rather than $\mathbf{s}_{t-1}$.

10.2.5 Global vs. Local Attention

Global attention: The decoder attends to all source positions, as described above. The context vector uses all encoder states:

$$\mathbf{c}_t = \sum_{i=1}^{T} \alpha_{t,i} \mathbf{h}_i \tag{10.15}$$

Local attention: The decoder attends to a window of source positions centered at a predicted aligned position p_t:

$$\alpha_{t,i} = (\mathbf{s}_t, \mathbf{h}_i) \cdot \exp\left(-\frac{(i - p_t)^2}{2\sigma^2}\right)$$

where $p_t = T \cdot \sigma(\mathbf{v}_p^\top \tanh(\mathbf{W}_p \mathbf{s}_t))$. This reduces computational cost for long sequences.

10.2.6 Attention as Differentiable Memory

The attention mechanism can be understood as a soft, differentiable content-addressable memory retrieval:

- **Query:** The decoder state $\mathbf{s}_t$ serves as a query vector.
- **Keys:** The encoder annotations $\mathbf{h}_i$ serve as key vectors.
- **Values:** The same encoder annotations $\mathbf{h}_i$ serve as value vectors.
- **Retrieval:** The output is $\mathbf{c}_t = \sum_i \text{softmax}(\text{score}(\mathbf{q}_t, \mathbf{k}_i))_i \cdot \mathbf{v}_i$.

The scoring function $\text{score}(\mathbf{q}, \mathbf{k})$ measures compatibility between query and key. This query-key-value abstraction generalizes directly to the Transformer's self-attention mechanism.

10.2.7 Beam Search Decoding

At inference time, greedy decoding (selecting $\arg\max$ at each step) is suboptimal because it cannot recover from early mistakes. Beam search maintains a set of the B most promising partial hypotheses.

Let $\mathcal{H}_t$ be the set of B hypotheses at step t, each with a cumulative score. For each hypothesis $h \in \mathcal{H}_t$, extend by every vocabulary token and select the top B:

$$\mathcal{H}_{t+1} = \arg\max_B \{(h \circ y, \text{score}(h) + \log P(y \mid h, \mathbf{x})) \mid h \in \mathcal{H}_t, y \in \mathcal{V}\} \tag{10.17}$$

To prevent bias toward short sequences, apply length normalization:

$$\text{score}(h) = \frac{1}{|h|^\gamma} \sum_{t=1}^{|h|} \log P(y_t \mid y_{<t}, \mathbf{x}) \tag{10.18}$$

where $\gamma \in [0, 1]$ controls the strength of the penalty. With $\gamma = 0$, no penalty; with $\gamma = 1$, average log-probability.

10.2.8 Teacher Forcing and Scheduled Sampling

Teacher forcing: During training, the ground-truth previous token y_{t-1}^* is fed to the decoder regardless of the model's prediction:

$$\mathbf{s}_t = \text{LSTM}(\mathbf{s}_{t-1}, \mathbf{y}_{t-1}^*, \mathbf{c}_t) \tag{10.19}$$

This stabilizes training but creates a discrepancy between training and inference, where the model must condition on its own predictions.

Scheduled sampling (Bengio et al., 2015): With probability ϵ_i at iteration i, use the model's own prediction $\hat{y}_{t-1}$ instead of the ground-truth y^*_{t-1}:

$$\mathbf{s}_t = \text{LSTM}(\mathbf{s}_{t-1}, \tilde{\mathbf{y}}_{t-1}, \mathbf{c}_t), \quad \tilde{y}_{t-1} = \begin{cases} y^*_{t-1} & \text{with probability } 1 - \epsilon_i \\ \hat{y}_{t-1} & \text{with probability } \epsilon_i \end{cases} \tag{10.20}$$

The schedule typically decays ϵ_i from 0 toward 1 during training, e.g., linear decay or inverse sigmoid decay:

$$\epsilon_i = \frac{k}{k + \exp(i/k)} \tag{10.21}$$

10.2.9 BLEU Score

The Bilingual Evaluation Understudy (BLEU) measures translation quality through modified n-gram precision:

$$p_n = \frac{\sum_{C \in \text{candidates}} \sum_{\text{n-gram} \in C} \min(\text{count}_C(\text{n-gram}), \max_{\text{ref} \in R_C} \text{count}_{\text{ref}}(\text{n-gram}))}{\sum_{C \in \text{candidates}} \sum_{\text{n-gram}' \in C} \text{count}_C(\text{n-gram}')} \tag{10.22}$$

The brevity penalty ensures that translations are not excessively short:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ \exp(1 - r/c) & \text{if } c \leq r \end{cases} \tag{10.23}$$

where c is the candidate length and r is the effective reference length. The final BLEU score is:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n \right) \tag{10.24}$$

With uniform weights, $w_n = 1/N$.

10.3 Algorithm

Algorithm 4: Training Seq2Seq with Bahdanau Attention

Input: Source sequence $\mathbf{x} = (x_1, \ldots, x_T)$, target sequence $\mathbf{y}^*$ (training)
Output: Model parameters $\boldsymbol{\theta}$

`// Encoder forward pass`
$\mathbf{h}_0 \leftarrow \mathbf{0}$;
for $t \leftarrow 1$ **to** T **do**
  $\mathbf{h}_t \leftarrow \mathrm{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1})$;
end

`// Decoder forward pass with attention (training with teacher forcing)`
$\mathbf{s}_0 \leftarrow \mathbf{h}_T$;
$\mathcal{L} \leftarrow 0$;
for $t \leftarrow 1$ **to** T' **do**
  `// Compute attention`
  for $i \leftarrow 1$ **to** T **do**
    $e_{t,i} \leftarrow \mathbf{v}^\top \tanh(\mathbf{W}_a \mathbf{s}_{t-1} + \mathbf{U}_a \mathbf{h}_i)$;
  end
  $\boldsymbol{\alpha}_t \leftarrow \mathrm{softmax}(\mathbf{e}_t)$;
  $\mathbf{c}_t \leftarrow \sum_{i=1}^{T} \alpha_{t,i} \mathbf{h}_i$;
  `// Decoder step`
  $\mathbf{s}_t \leftarrow \mathrm{LSTM}(\mathbf{s}_{t-1}, [\mathbf{y}^*_{t-1}; \mathbf{c}_t])$;
  $\mathbf{p}_t \leftarrow \mathrm{softmax}(\mathbf{W}_{\text{out}}[\mathbf{s}_t; \mathbf{c}_t] + \mathbf{b}_{\text{out}})$;
  $\mathcal{L} \leftarrow \mathcal{L} - \log p_t(y^*_t)$;
end

`// Backpropagation`
Compute $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ and update parameters via SGD/Adam;

Algorithm 5: Beam Search Decoding

Input: Encoder states $(\mathbf{h}_1, \ldots, \mathbf{h}_T)$, beam size B, max length $L_{\max}$
Output: Best output sequence $\hat{\mathbf{y}}$

$\mathcal{B} \leftarrow \{(\text{BOS}, 0)\}$; `// Each hypothesis: (sequence, log-score)`
for $t \leftarrow 1$ **to** $L_{\max}$ **do**
  $\mathcal{C} \leftarrow \emptyset$;
  foreach $(\mathbf{y}_{<t}, s) \in \mathcal{B}$ **do**
    foreach $y \in \mathcal{V}$ **do**
      `// Compute attention context` $\mathbf{c}_t$ `using current decoder state`
      $\mathbf{s}_t \leftarrow \text{LSTM}(\mathbf{s}_{t-1}, [\mathbf{y}_{t-1}; \mathbf{c}_t])$;
      $p \leftarrow \text{softmax}(\mathbf{W}_{\text{out}}[\mathbf{s}_t; \mathbf{c}_t])$;
      $s' \leftarrow s + \log p(y)$;
      $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\mathbf{y}_{<t} \circ y, s')\}$;
    end
  end
  $\mathcal{B} \leftarrow \arg\max_{B} \mathcal{C}$; `// Keep top B by score`
  if *all hypotheses in* $\mathcal{B}$ *end with EOS* **then**
    break;
  end
end
$\hat{\mathbf{y}} \leftarrow \arg\max_{h \in \mathcal{B}} \text{score}(h)$; `// Select best with length penalty`

10.4 Architecture

The typical attention-based Seq2Seq architecture for neural machine translation consists of:

Embedding layers: Both source and target tokens are mapped to dense vectors via learned embedding matrices $\mathbf{E}_{\text{src}} \in \mathbb{R}^{V_{\text{src}} \times d_{\text{emb}}}$ and $\mathbf{E}_{\text{tgt}} \in \mathbb{R}^{V_{\text{tgt}} \times d_{\text{emb}}}$.

Encoder: A multi-layer bidirectional LSTM with residual connections between layers. The bidirectionality ensures that each $\mathbf{h}_t$ captures context from both directions:

$$\mathbf{h}_t^{(l)} = \text{LSTM}^{(l)}\left(\mathbf{h}_t^{(l-1)}, \mathbf{h}_{t-1}^{(l)}\right) \tag{10.25}$$

Attention module: Receives encoder output $\mathbf{H} \in \mathbb{R}^{T \times 2d_h}$ and decoder state $\mathbf{s}_{t-1}$, produces context $\mathbf{c}_t$. Parameters: $\mathbf{W}_a$, $\mathbf{U}_a$, $\mathbf{v}$.

Decoder: A multi-layer unidirectional LSTM. The input to each layer at step t is the concatenation of the embedding of y_{t-1} and the context vector $\mathbf{c}_t$. A projection layer maps the concatenation of the final decoder state and the context vector to vocabulary logits.

Output layer: A linear projection followed by softmax:

$$P(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_{\text{proj}} \tanh(\mathbf{W}_{\text{out}}[\mathbf{s}_t^{(L)}; \mathbf{c}_t] + \mathbf{b}_{\text{out}}) + \mathbf{b}_{\text{proj}}) \tag{10.26}$$

The full model has approximately $4\times$ the parameters of non-attentional models due to the attention parameters and the larger output projection.

10.5 Pros and Cons

Pros:

- *Interpretability:* Attention weights $\alpha_{t,i}$ provide a natural visualization of which source words influence each target word, offering interpretable alignments.
- *Long-range dependencies:* Unlike fixed-context encoders, attention directly connects the decoder to any source position, substantially improving performance on long sentences (no more BLEU degradation beyond 30 words).
- *Soft alignment:* Learns a differentiable, end-to-end alignment without requiring pre-computed word alignments from tools like GIZA++.
- *Efficient training:* The attention mechanism adds only $O(T \cdot T' \cdot d)$ computation, which is quadratic but manageable for typical sentence lengths.

Cons:

- *Sequential decoding:* The decoder generates tokens one at a time with $O(T')$ sequential LSTM steps, preventing parallelization at inference time.
- *Quadratic attention cost:* Computing attention over the full source requires $O(T \cdot T')$ operations, problematic for very long sequences (e.g., document-level translation).
- *Vanishing signal:* Although attention mitigates the long-range dependency problem of RNNs, the recurrent encoder and decoder still suffer from vanishing gradients over long sequences.
- *Coverage issues:* No built-in mechanism to prevent attending to the same source positions repeatedly (over-translation) or missing positions (under-translation), requiring explicit coverage mechanisms.
- *Exposure bias:* Discrepancy between teacher-forced training and autoregressive inference leads to error accumulation at test time.

10.6 Evolution

The attention-based Seq2Seq model catalyzed the deep learning revolution in NLP and led directly to the Transformer architecture, which eliminates recurrence entirely:

- **Coverage models:** Tu et al. (2016) added a coverage vector $\sum_{t'<t} \alpha_{t',i}$ to track which source words have been attended to, penalizing over-translation.
- **Input-feeding:** Luong et al. introduced passing the attention context $\mathbf{c}_t$ as input to the next LSTM step, ensuring the model is aware of past alignment decisions.
- **Transformer (Vaswani et al., 2017):** Replaced recurrence with multi-head self-attention. Encoder: self-attention over $\mathbf{x}$; Decoder: self-attention over $\mathbf{y}_{<t}$ plus cross-attention to encoder output. The decoder's cross-attention is the direct descendant of Seq2Seq attention: queries from decoder, keys/values from encoder. Complete parallelization of training.

- **BERT pretraining:** Transformer encoders pre-trained with masked language modeling, then fine-tuned for diverse tasks, eliminating the need for explicit encoder-decoder architectures for many tasks.
- **GPT autoregressive pretraining:** Transformer decoders pre-trained with causal language modeling, capable of zero-shot and few-shot translation.
- **RNNsearch to GNMT:** Google's Neural Machine Translation system combined attention with deep stacked LSTMs, residual connections, and subword units (WordPiece/BPE), achieving near-human translation quality for several language pairs by 2016.

The fundamental query-key-value abstraction introduced by attention remains central to all modern transformer architectures, demonstrating the enduring value of this architectural innovation.

Part IV

Generative Models

Chapter 11

Autoencoders

11.1 Motivation

Supervised learning requires labeled data, which is expensive and scarce for most real-world problems. Autoencoders provide a framework for unsupervised representation learning: train a neural network to reconstruct its input through an information bottleneck, forcing the model to discover a compressed, meaningful representation.

The core idea is that by constraining the latent representation—through dimensionality reduction, sparsity, or noise injection—the model cannot simply learn the identity function and must instead extract the most salient features of the data distribution. Autoencoders form the foundation of modern generative modeling, from VAEs to latent diffusion models.

11.2 Mathematical Formulation

11.2.1 Architecture

An autoencoder consists of two components:

Encoder $f_{\boldsymbol{\theta}} : \mathcal{X} \to \mathcal{Z}$ maps input $\mathbf{x} \in \mathbb{R}^d$ to latent representation $\mathbf{z} \in \mathbb{R}^k$:

$$\mathbf{z} = f_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{11.1}$$

for a single-layer encoder, or more generally:

$$\mathbf{z} = f_{\boldsymbol{\theta}}^{(L)} \circ f_{\boldsymbol{\theta}}^{(L-1)} \circ \cdots \circ f_{\boldsymbol{\theta}}^{(1)}(\mathbf{x}) \tag{11.2}$$

Decoder $g_{\boldsymbol{\phi}} : \mathcal{Z} \to \mathcal{X}$ reconstructs $\hat{\mathbf{x}}$ from $\mathbf{z}$:

$$\hat{\mathbf{x}} = g_{\boldsymbol{\phi}}(\mathbf{z}) = \sigma'(\mathbf{W}'\mathbf{z} + \mathbf{b}') \tag{11.3}$$

The parameters are $\boldsymbol{\theta}$ (encoder) and $\boldsymbol{\phi}$ (decoder).

11.2.2 Reconstruction Loss

The model is trained to minimize the discrepancy between input and reconstruction:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}) = \frac{1}{m} \sum_{i=1}^{m} \ell(\mathbf{x}^{(i)}, \hat{\mathbf{x}}^{(i)}) \tag{11.4}$$

For real-valued inputs, the mean squared error is standard:

$$\ell_{\text{MSE}}(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 = \sum_{j=1}^{d} (x_j - \hat{x}_j)^2 \tag{11.5}$$

For binary inputs or values in $[0, 1]$, cross-entropy is appropriate:

$$\ell_{\text{BCE}}(\mathbf{x}, \hat{\mathbf{x}}) = -\sum_{j=1}^{d} [x_j \log \hat{x}_j + (1 - x_j) \log(1 - \hat{x}_j)] \tag{11.6}$$

11.2.3 Linear Autoencoder and PCA

Consider a linear autoencoder with no activation: $\mathbf{z} = \mathbf{W}\mathbf{x}$ and $\hat{\mathbf{x}} = \mathbf{V}\mathbf{z} = \mathbf{V}\mathbf{W}\mathbf{x}$, where $\mathbf{W} \in \mathbb{R}^{k \times d}$, $\mathbf{V} \in \mathbb{R}^{d \times k}$.

The objective is:

$$\min_{\mathbf{W},\mathbf{V}} \frac{1}{m} \sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - \mathbf{V}\mathbf{W}\mathbf{x}^{(i)} \right\|_2^2 \tag{11.7}$$

Let $\mathbf{X} \in \mathbb{R}^{d \times m}$ be the data matrix. The reconstruction can be written as:

$$\mathcal{L} = \frac{1}{m} \|\mathbf{X} - \mathbf{V}\mathbf{W}\mathbf{X}\|_F^2 \tag{11.8}$$

The optimal solution is: $\mathbf{W} = \mathbf{U}_k^\top$ and $\mathbf{V} = \mathbf{U}_k$, where $\mathbf{U}_k \in \mathbb{R}^{d \times k}$ contains the k eigenvectors corresponding to the k largest eigenvalues of the covariance matrix $\frac{1}{m}\mathbf{X}\mathbf{X}^\top$.

Derivation: The product $\mathbf{V}\mathbf{W}$ is a rank-k projection matrix. Minimizing the Frobenius norm of the residual is equivalent to finding the best rank-k approximation via the Eckart-Young-Mirsky theorem:

$$\min_{\text{rank}(\mathbf{M}) \leq k} \|\mathbf{X} - \mathbf{M}\mathbf{X}\|_F^2 \tag{11.9}$$

The solution is $\mathbf{M} = \mathbf{U}_k\mathbf{U}_k^\top$, the projection onto the principal subspace. Thus, the linear autoencoder exactly recovers PCA.

With nonlinear activations, the autoencoder learns a nonlinear generalization of PCA, capable of discovering curved manifolds.

11.2.4 Undercomplete Autoencoders

When $k < d$ (bottleneck narrower than input), the autoencoder is *undercomplete*. The model cannot learn the identity mapping and must prioritize which features to preserve.

The loss surface exhibits saddle points but no spurious local minima when the encoder and decoder have sufficient capacity. The learned representation captures the manifold of the data distribution.

For Gaussian data, the optimal undercomplete autoencoder learns exactly the PCA subspace. For data on a nonlinear k-dimensional manifold embedded in $\mathbb{R}^d$, a sufficiently expressive autoencoder can approximately learn the manifold coordinates.

11.2.5 Overcomplete Autoencoders and Regularization

When $k \geq d$, the autoencoder can trivially learn the identity (e.g., $\mathbf{W} = \mathbf{I}$, $\mathbf{V} = \mathbf{I}$ with linear activation, or a passthrough with nonlinear activations). To prevent this, we must add regularization that forces the representation to be meaningful.

11.2.6 Denoising Autoencoder (DAE)

The denoising autoencoder corrupts the input before encoding and is trained to reconstruct the clean original:

$$\tilde{\mathbf{x}} \sim q(\tilde{\mathbf{x}} \mid \mathbf{x}) \tag{11.10}$$

where q is a corruption process: additive Gaussian noise ($\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \sigma^2\mathbf{I})$), masking noise (randomly set components to 0), or salt-and-pepper noise.

The training objective becomes:

$$\mathcal{L}_{\text{DAE}} = \frac{1}{m}\sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - g_{\boldsymbol{\phi}}(f_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}^{(i)})) \right\|_2^2 \tag{11.11}$$

Manifold learning interpretation: For small corruption $\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\delta}$, the DAE approximately learns the score of the data distribution. Consider the Taylor expansion:

$$g(f(\mathbf{x} + \boldsymbol{\delta})) \approx \mathbf{x} + \frac{\partial g(f(\mathbf{x}))}{\partial \mathbf{x}}\boldsymbol{\delta} \tag{11.12}$$

Minimizing $\mathbb{E}[\|\mathbf{x} - g(f(\mathbf{x} + \boldsymbol{\delta}))\|^2]$ encourages:

$$\frac{\partial g(f(\mathbf{x}))}{\partial \mathbf{x}} \approx \mathbf{I} \tag{11.13}$$

in directions along the manifold, and near-zero in orthogonal directions, effectively projecting noisy points back onto the data manifold.

Score matching connection (Vincent, 2011): For Gaussian corruption with variance σ^2, as $\sigma \to 0$, the optimal DAE reconstruction function $r(\mathbf{x}) = g(f(\mathbf{x}))$ satisfies:

$$r(\mathbf{x}) - \mathbf{x} \approx \sigma^2 \nabla_{\mathbf{x}} \log p(\mathbf{x}) \tag{11.14}$$

This means the DAE implicitly learns the score function (gradient of the log-density), connecting it to score-based generative models and diffusion models.

11.2.7 Sparse Autoencoder

Sparse autoencoders impose a sparsity constraint on the latent activations, encouraging each unit to be active for only a small fraction of inputs.

Let $a_j(\mathbf{x})$ be the activation of hidden unit j for input $\mathbf{x}$. The average activation over the training set is:

$$\hat{\rho}_j = \frac{1}{m}\sum_{i=1}^{m} a_j(\mathbf{x}^{(i)}) \tag{11.15}$$

We enforce $\hat{\rho}_j \approx \rho$, where ρ is a small target sparsity parameter (typically $\rho = 0.05$ or 0.1).

The sparsity penalty uses the KL divergence between a Bernoulli random variable with mean ρ and one with mean $\hat{\rho}_j$:

$$\text{KL}(\rho \parallel \hat{\rho}_j) = \rho \log \frac{\rho}{\hat{\rho}_j} + (1-\rho)\log \frac{1-\rho}{1-\hat{\rho}_j} \tag{11.16}$$

The total loss become

s:

$$\mathcal{L}_{\text{sparse}} = \mathcal{L}_{\text{recon}} + \beta \sum_{j=1}^{k} \text{KL}(\rho \parallel \hat{\rho}_j) \tag{11.17}$$

where β controls the sparsity trade-off.

Gradient of the sparsity penalty:

$$\frac{\partial}{\partial \hat{\rho}_j} \text{KL}(\rho \parallel \hat{\rho}_j) = -\frac{\rho}{\hat{\rho}_j} + \frac{1-\rho}{1-\hat{\rho}_j} \tag{11.18}$$

This penalty encourages $\hat{\rho}_j$ toward ρ: if $\hat{\rho}_j$ is too large, the gradient is positive (increase loss, decrease activation); if $\hat{\rho}_j$ is too small, the gradient is negative.

11.2.8 Contractive Autoencoder (CAE)

The contractive autoencoder penalizes the Frobenius norm of the Jacobian of the encoder:

$$\mathcal{L}_{\text{CAE}} = \frac{1}{m} \sum_{i=1}^{m} \left(\left\| \mathbf{x}^{(i)} - g_{\boldsymbol{\phi}}(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) \right\|_2^2 + \lambda \left\| \frac{\partial f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})}{\partial \mathbf{x}} \right\|_F^2 \right) \tag{11.19}$$

The Jacobian penalty is:

$$\left\| \frac{\partial f_{\boldsymbol{\theta}}(\mathbf{x})}{\partial \mathbf{x}} \right\|_F^2 = \sum_{j=1}^{k} \sum_{i=1}^{d} \left(\frac{\partial f_j(\mathbf{x})}{\partial x_i} \right)^2 \tag{11.20}$$

For a single-layer encoder with sigmoid activation $f(\mathbf{x}) = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$:

$$\frac{\partial f_j}{\partial x_i} = f_j(\mathbf{x})(1 - f_j(\mathbf{x}))W_{ji} \tag{11.21}$$

$$\left\| \frac{\partial f}{\partial \mathbf{x}} \right\|_F^2 = \sum_{j=1}^{k} [f_j(1-f_j)]^2 \sum_{i=1}^{d} W_{ji}^2 \tag{11.22}$$

The penalty encourages the encoder to be insensitive to small variations in the input—the representation contracts around the data manifold. Directions orthogonal to the manifold are strongly penalized (derivative must be small), while directions along the manifold can have larger derivatives to preserve reconstruction capability.

11.2.9 Learning the Data Manifold

All three variants—DAE, sparse AE, and CAE—can be understood as different strategies for learning the data manifold:

- **DAE:** Explicitly corrupt input, reconstruct clean. The model learns to "project" points back onto the manifold.
- **Sparse AE:** Enforce that only a small subset of features responds to each input. This leads to distributed, factorized representations where each feature captures a specific pattern.

- **CAE:** Explicitly penalize sensitivity orthogonal to the manifold through the Jacobian norm. This is the most direct geometric regularization.

All three encourage the encoder to discard irrelevant variation (noise) while preserving the information necessary for reconstruction.

11.3 Algorithm

Algorithm 6: Training a Denoising Autoencoder

Input: Training data $\{\mathbf{x}^{(i)}\}_{i=1}^{m}$, corruption process $q(\tilde{\mathbf{x}} \mid \mathbf{x})$
Output: Trained encoder $f_{\boldsymbol{\theta}}$, decoder $g_{\boldsymbol{\phi}}$

Initialize parameters $\boldsymbol{\theta}$, $\boldsymbol{\phi}$;
for *epoch* $\leftarrow 1$ **to** *max_ epochs* **do**
  for *each mini-batch* $\mathcal{B} \subset \{1, \ldots, m\}$ **do**
    foreach $i \in \mathcal{B}$ **do**
      $\tilde{\mathbf{x}}^{(i)} \sim q(\tilde{\mathbf{x}} \mid \mathbf{x}^{(i)})$;
      $\mathbf{z}^{(i)} \leftarrow f_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}^{(i)})$;
      $\hat{\mathbf{x}}^{(i)} \leftarrow g_{\boldsymbol{\phi}}(\mathbf{z}^{(i)})$;
    end
    $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left\| \mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)} \right\|_2^2$;
    `// Add regularization if applicable`
    $\mathcal{L} \leftarrow \mathcal{L} + \lambda \cdot$ regularizer;
    Compute $\nabla_{\boldsymbol{\theta},\boldsymbol{\phi}} \mathcal{L}$ via backpropagation;
    Update $\boldsymbol{\theta}, \boldsymbol{\phi}$ with Adam optimizer;
  end
end

Algorithm 7: Training a Sparse Autoencoder

Input: Training data $\{\mathbf{x}^{(i)}\}_{i=1}^{m}$, target sparsity ρ, weight β
Output: Trained sparse autoencoder

Initialize parameters $\boldsymbol{\theta}$, $\boldsymbol{\phi}$;
for *epoch* $\leftarrow 1$ **to** *max_ epochs* **do**
  Compute forward pass for all data: $\mathbf{a}^{(i)} \leftarrow f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$, $i = 1, \ldots, m$;
  for $j \leftarrow 1$ **to** k **do**
    $\hat{\rho}_j \leftarrow \frac{1}{m} \sum_{i=1}^{m} a_j^{(i)}$;
  end
  for *each mini-batch* $\mathcal{B}$ **do**
    $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left\| \mathbf{x}^{(i)} - g_{\boldsymbol{\phi}}(f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})) \right\|_2^2$;
    $\mathcal{L} \leftarrow \mathcal{L} + \beta \sum_{j=1}^{k} \left(\rho \log \frac{\rho}{\hat{\rho}_j} + (1-\rho) \log \frac{1-\rho}{1-\hat{\rho}_j} \right)$;
    Compute gradients and update;
  end
end

11.4 Architecture

A typical deep autoencoder stacks multiple layers for both encoder and decoder:

Encoder:

$$\mathbf{z}^{(1)} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \tag{11.23}$$

$$\mathbf{z}^{(l)} = \text{ReLU}(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}), \quad l = 2, \ldots, L \tag{11.24}$$

$$\mathbf{z} = \mathbf{W}^{(L+1)}\mathbf{z}^{(L)} + \mathbf{b}^{(L+1)} \tag{11.25}$$

Decoder (mirror of encoder):

$$\hat{\mathbf{z}}^{(L)} = \text{ReLU}(\mathbf{W}'^{(1)}\mathbf{z} + \mathbf{b}'^{(1)}) \tag{11.26}$$

$$\hat{\mathbf{z}}^{(l-1)} = \text{ReLU}(\mathbf{W}'^{(L-l+2)}\hat{\mathbf{z}}^{(l)} + \mathbf{b}'^{(L-l+2)}) \tag{11.27}$$

$$\hat{\mathbf{x}} = \sigma(\mathbf{W}'^{(L+1)}\hat{\mathbf{z}}^{(1)} + \mathbf{b}'^{(L+1)}) \tag{11.28}$$

The latent dimension k at the bottleneck is typically much smaller than the input dimension (e.g., 30–100 for MNIST). Often, tied weights are used: $\mathbf{W}'^{(l)} = (\mathbf{W}^{(L-l+2)})^\top$.

For convolutional autoencoders, the encoder uses strided convolutions or pooling for downsampling, and the decoder uses transposed convolutions or upsampling layers.

11.5 Pros and Cons

Pros:

- *Unsupervised:* Learns representations without labeled data, making it applicable to vast amounts of unlabeled data.
- *Dimensionality reduction:* Nonlinear generalization of PCA, capable of discovering curved manifolds that PCA cannot capture.
- *Pretraining:* Stacked autoencoders can be used for greedy layer-wise pretraining of deep networks, helping with optimization in the pre-BatchNorm era.
- *Feature learning:* Denoising and sparse autoencoders learn robust, interpretable features that generalize well to downstream tasks.
- *Anomaly detection:* The reconstruction error on anomalous inputs is typically higher than on normal inputs, providing an unsupervised anomaly detector.

Cons:

- *No generative capability:* Standard autoencoders learn a deterministic mapping; they cannot sample new data points from the learned distribution.
- *Latent space structure:* The latent space of a standard autoencoder is not regularized to be smooth or continuous; interpolating between encodings may produce nonsensical reconstructions.
- *Reconstruction quality:* MSE-trained autoencoders produce blurry reconstructions, averaging over posterior modes.

- *Identity shortcut:* Overcomplete autoencoders without regularization learn the identity function, providing no useful representation.
- *Curse of dimensionality:* In high dimensions, MSE loss focuses on low-frequency components, ignoring fine details.

11.6 Evolution

The autoencoder has evolved from a simple compression tool into the backbone of generative AI:

- **Stacked Denoising Autoencoders (Vincent et al., 2010):** Layer-wise training of denoising autoencoders for deep unsupervised pretraining, showing competitive results with Deep Belief Networks.
- **Variational Autoencoder (Kingma & Welling, 2014):** Replaced the deterministic bottleneck with a probabilistic latent variable model, enabling sampling and generation. The VAE regularizes the latent space to follow a prior distribution, solved via variational inference. (See Chapter 12.)
- **VQ-VAE (van den Oord et al., 2017):** Replaced the continuous latent space with discrete codebook vectors, combined with an autoregressive prior (PixelCNN) over the discrete latents. This produces much sharper images than VAEs.
- **VQ-VAE-2:** Hierarchical discrete latents at multiple resolutions, capturing both global structure and local details.
- **Latent Diffusion Models (Rombach et al., 2022):** The autoencoder compresses images into a lower-dimensional latent space, where diffusion models operate much more efficiently than in pixel space. The VAE (or VQ-GAN) encoder-decoder is the crucial first stage that makes Stable Diffusion computationally feasible.
- **Masked Autoencoders (He et al., 2022):** The autoencoder idea applied to Vision Transformers: mask a large fraction of image patches, encode the visible ones, and reconstruct the missing patches. This asymmetric design enables efficient self-supervised pretraining of large vision models.

The journey from PCA-like compression to the foundation of billion-parameter generative models illustrates the remarkable depth of the simple idea: learning to reconstruct teaches us what matters.

Chapter 12

Variational Autoencoders

12.1 Motivation

Standard autoencoders learn a deterministic mapping from input $\mathbf{x}$ to latent code $\mathbf{z}$ and back. While useful for dimensionality reduction, this deterministic framework has fundamental limitations: the latent space has no imposed structure, there is no principled way to sample new data, and interpolating between latent points can produce nonsensical outputs.

The Variational Autoencoder (VAE) addresses these limitations by casting the autoencoder as a probabilistic generative model. Instead of learning a deterministic encoding, the VAE learns the parameters of a probability distribution over the latent space. This probabilistic formulation (1) regularizes the latent space to follow a prior distribution, (2) enables sampling of new data by drawing from the prior, and (3) provides a principled variational inference framework grounded in Bayesian statistics.

12.2 Mathematical Formulation

12.2.1 Generative Model

The VAE defines a generative model over observed variables $\mathbf{x}$ and latent variables $\mathbf{z}$:

$$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) = p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z}) \tag{12.1}$$

The prior over latent variables is typically a standard multivariate Gaussian:

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}) \tag{12.2}$$

The likelihood (decoder) is chosen based on the data type. For continuous data:

$$p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{z}), \sigma^2 \mathbf{I}) \tag{12.3}$$

where $\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{z})$ is the output of the decoder network and σ^2 is a fixed or learned variance. For binary data:

$$p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z}) = \text{Bernoulli}(\mathbf{x}; \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{z})) \tag{12.4}$$

The marginal likelihood is the integral over latent variables:

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \int p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z} \tag{12.5}$$

This integral is intractable because the decoder $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$ is a complex neural network, rendering the posterior $p_{\boldsymbol{\theta}}(\mathbf{z} \mid \mathbf{x}) = p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})p(\mathbf{z})/p_{\boldsymbol{\theta}}(\mathbf{x})$ also intractable.

12.2.2 Variational Inference

Since the true posterior is intractable, we approximate it with a variational distribution $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$ (the encoder), parameterized by $\boldsymbol{\phi}$:

$$q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x}), \boldsymbol{\sigma}^2_{\boldsymbol{\phi}}(\mathbf{x}) \odot \mathbf{I}) \tag{12.6}$$

where $\boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x}) \in \mathbb{R}^k$ and $\boldsymbol{\sigma}^2_{\boldsymbol{\phi}}(\mathbf{x}) \in \mathbb{R}^k_+$ are the encoder outputs. For numerical stability, the encoder typically outputs $\log \boldsymbol{\sigma}^2$ rather than $\boldsymbol{\sigma}^2$ directly.

12.2.3 ELBO Derivation

The evidence lower bound (ELBO) provides a tractable lower bound on the log marginal likelihood. Starting from the log-likelihood:

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) = \log \int p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})\, d\mathbf{z} \tag{12.7}$$

Introduce the variational distribution $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$:

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) = \log \int q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) \frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})}{q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})}\, d\mathbf{z} \tag{12.8}$$

$$= \log \mathbb{E}_{\mathbf{z} \sim q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})}{q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})} \right] \tag{12.9}$$

By Jensen's inequality ($\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$ since log is concave):

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) \geq \mathbb{E}_{\mathbf{z} \sim q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})}{q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})} \right] \equiv \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}) \tag{12.10}$$

This lower bound $\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x})$ is the ELBO. We can decompose it:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\log p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z}) + \log p(\mathbf{z}) - \log q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) \right] \tag{12.11}$$

$$= \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\log p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z}) \right] - \mathrm{KL}(q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})) \tag{12.12}$$

The first term is the expected reconstruction log-likelihood (negative reconstruction error). The second term is the KL divergence between the approximate posterior and the prior, regularizing the latent space.

Gap to the true log-likelihood:

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) - \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})} \left[\log \frac{q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})}{p_{\boldsymbol{\theta}}(\mathbf{z} \mid \mathbf{x})} \right] \tag{12.13}$$

$$= \mathrm{KL}(q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) \parallel p_{\boldsymbol{\theta}}(\mathbf{z} \mid \mathbf{x})) \geq 0 \tag{12.14}$$

Thus, maximizing the ELBO simultaneously maximizes the log-likelihood and minimizes the KL gap between the approximate and true posteriors.

12.2.4 Analytic KL Divergence

For Gaussian encoder and standard Gaussian prior, the KL divergence has a closed form. Let $\boldsymbol{\mu} = \boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x})$ and $\boldsymbol{\sigma}^2 = \boldsymbol{\sigma}^2_{\boldsymbol{\phi}}(\mathbf{x})$:

$$\mathrm{KL}(q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})) = \mathrm{KL}(\mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2)) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})) \tag{12.15}$$

$$= \frac{1}{2} \sum_{j=1}^{k} \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right) \tag{12.16}$$

Derivation: The KL divergence between two k-dimensional Gaussians $\mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)$ and $\mathcal{N}(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$ is:

$$\mathrm{KL}(\mathcal{N}_1 \parallel \mathcal{N}_2) = \frac{1}{2} \left[\log \frac{|\boldsymbol{\Sigma}_2|}{|\boldsymbol{\Sigma}_1|} - k + \mathrm{Tr}(\boldsymbol{\Sigma}_2^{-1} \boldsymbol{\Sigma}_1) + (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^\top \boldsymbol{\Sigma}_2^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \right] \tag{12.17}$$

Substituting $\boldsymbol{\mu}_1 = \boldsymbol{\mu}$, $\boldsymbol{\Sigma}_1 = \mathrm{diag}(\boldsymbol{\sigma}^2)$, $\boldsymbol{\mu}_2 = \mathbf{0}$, $\boldsymbol{\Sigma}_2 = \mathbf{I}$:

$$\mathrm{KL} = \frac{1}{2} \left[\log \frac{1}{\prod_j \sigma_j^2} - k + \sum_j \sigma_j^2 + \sum_j \mu_j^2 \right] \tag{12.18}$$

$$= \frac{1}{2} \sum_{j=1}^{k} \left(-\log \sigma_j^2 - 1 + \sigma_j^2 + \mu_j^2 \right) \tag{12.19}$$

$$= \frac{1}{2} \sum_{j=1}^{k} \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right) \tag{12.20}$$

12.2.5 Reparameterization Trick

The expectation $\mathbb{E}_{q_{\boldsymbol{\phi}}}[\log p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})]$ cannot be directly differentiated w.r.t. $\boldsymbol{\phi}$ because $\mathbf{z}$ is sampled from a distribution parameterized by $\boldsymbol{\phi}$. The reparameterization trick resolves this by expressing $\mathbf{z}$ as a deterministic function of $\boldsymbol{\mu}$, $\boldsymbol{\sigma}$, and an independent noise variable:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{12.21}$$

Now $\mathbf{z} = g_{\boldsymbol{\phi}}(\boldsymbol{\epsilon}, \mathbf{x})$ with gradients:

$$\frac{\partial \mathbf{z}}{\partial \boldsymbol{\mu}} = \mathbf{I}, \quad \frac{\partial \mathbf{z}}{\partial \boldsymbol{\sigma}} = \mathrm{diag}(\boldsymbol{\epsilon}) \tag{12.22}$$

The gradient of the ELBO w.r.t. $\boldsymbol{\phi}$ becomes:

$$\nabla_{\boldsymbol{\phi}} \mathcal{L} = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})} \left[\nabla_{\boldsymbol{\phi}} \log p_{\boldsymbol{\theta}}(\mathbf{x} \mid \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}) - \nabla_{\boldsymbol{\phi}} \mathrm{KL} \right] \tag{12.23}$$

The KL gradient is computed analytically. The reconstruction gradient is estimated via Monte Carlo:

$$\nabla_{\boldsymbol{\phi}} \mathbb{E}_{q_{\boldsymbol{\phi}}}[\log p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})] \approx \frac{1}{L} \sum_{l=1}^{L} \nabla_{\boldsymbol{\phi}} \log p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z}^{(l)}) \tag{12.24}$$

where $\mathbf{z}^{(l)} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}^{(l)}$, $\boldsymbol{\epsilon}^{(l)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. In practice, $L = 1$ sample per datapoint during training is sufficient due to the stochastic nature of mini-batch training.

12.2.6 Full Loss Function

The VAE is trained by minimizing the negative ELBO:

$$\mathcal{L}_{\text{VAE}}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}) = \text{KL}(q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})) - \mathbb{E}_{\mathbf{z} \sim q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})] \tag{12.25}$$

With the Gaussian likelihood $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x}; \hat{\mathbf{x}}, \mathbf{I})$:

$$\log p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z}) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \left\| \mathbf{x} - \hat{\mathbf{x}} \right\|_2^2 \tag{12.26}$$

The reconstruction term becomes $\frac{1}{2} \left\| \mathbf{x} - \hat{\mathbf{x}} \right\|_2^2$ (MSE), ignoring constants. The full loss for a mini-batch $\mathcal{B}$:

$$\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left[\frac{1}{2} \left\| \mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)} \right\|_2^2 + \frac{1}{2} \sum_{j=1}^{k} \left(\mu_j^{(i)2} + \sigma_j^{(i)2} - \log \sigma_j^{(i)2} - 1 \right) \right] \tag{12.27}$$

12.2.7 Beta-VAE

The β-VAE introduces a weight on the KL term to control the trade-off between reconstruction fidelity and latent regularization:

$$\mathcal{L}_{\beta\text{-VAE}} = -\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}[\log p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})] + \beta \ \text{KL}(q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})) \tag{12.28}$$

For $\beta = 1$, this is the standard VAE. For $\beta > 1$, stronger pressure toward factorized latent representations, encouraging *disentangled* representations where each latent dimension corresponds to a semantically meaningful and independent factor of variation.

The original β-VAE paper (Higgins et al., 2017) showed that with $\beta > 1$, the latent dimensions become disentangled: each dimension controls a single generative factor (e.g., azimuth, lighting, object identity for faces). The trade-off is reduced reconstruction quality due to the bottlenecked information flow.

12.2.8 VAE as Regularized Autoencoder

The VAE loss can be understood as a regularized autoencoder. The reconstruction term is the standard autoencoder loss. The KL term can be decomposed:

$$\text{KL}(q \parallel p) = \underbrace{-\mathbb{E}_q[\log p(\mathbf{z})]}_{\text{prior matching}} - \underbrace{\mathcal{H}[q]}_{\text{entropy bonus}} \tag{12.29}$$

where $\mathcal{H}[q]$ is the entropy of $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$:

$$\mathcal{H}[q] = \frac{k}{2}(1 + \log 2\pi) + \frac{1}{2} \sum_{j=1}^{k} \log \sigma_j^2 \tag{12.30}$$

The entropy term encourages σ_j^2 to increase (higher variance), preventing posterior collapse where $\sigma_j^2 \to 0$ and the VAE degenerates to a standard autoencoder.

12.3 Algorithm

Algorithm 8: Training a Variational Autoencoder

Input: Training data $\{\mathbf{x}^{(i)}\}_{i=1}^{m}$, latent dimension k, number of MC samples L
Output: Trained encoder $(\boldsymbol{\mu}_{\boldsymbol{\phi}}, \log \boldsymbol{\sigma}_{\boldsymbol{\phi}}^2)$ and decoder $\boldsymbol{\mu}_{\boldsymbol{\theta}}$

Initialize parameters $\boldsymbol{\theta}$, $\boldsymbol{\phi}$;
for *epoch* $\leftarrow 1$ **to** *max_ epochs* **do**
  for *each mini-batch* $\mathcal{B}$ **do**
    `// Encoder forward pass`
    foreach $\mathbf{x}^{(i)} \in \mathcal{B}$ **do**
      $\boldsymbol{\mu}^{(i)} \leftarrow \mathrm{NN}_{\boldsymbol{\phi},\mu}(\mathbf{x}^{(i)})$;
      $\log \boldsymbol{\sigma}^{2(i)} \leftarrow \mathrm{NN}_{\boldsymbol{\phi},\sigma}(\mathbf{x}^{(i)})$;
    end
    `// Reparameterization and Monte Carlo estimation`
    $\mathcal{L}_{\text{recon}} \leftarrow 0$;
    for $l \leftarrow 1$ **to** L **do**
      foreach $i \in \mathcal{B}$ **do**
        $\boldsymbol{\epsilon}^{(i,l)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$;
        $\mathbf{z}^{(i,l)} \leftarrow \boldsymbol{\mu}^{(i)} + \exp(\frac{1}{2} \log \boldsymbol{\sigma}^{2(i)}) \odot \boldsymbol{\epsilon}^{(i,l)}$;
      end
      foreach $i \in \mathcal{B}$ **do**
        $\hat{\mathbf{x}}^{(i,l)} \leftarrow \mathrm{NN}_{\boldsymbol{\theta}}(\mathbf{z}^{(i,l)})$;
        $\mathcal{L}_{\text{recon}} \leftarrow \mathcal{L}_{\text{recon}} + \left\|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i,l)}\right\|_2^2$;
      end
    end
    $\mathcal{L}_{\text{recon}} \leftarrow \mathcal{L}_{\text{recon}} / (L \cdot |\mathcal{B}|)$;
    `// KL divergence`
    $\mathcal{L}_{\text{KL}} \leftarrow \frac{1}{2|\mathcal{B}|} \sum_{i \in \mathcal{B}} \sum_{j=1}^{k} \left(\mu_j^{(i)2} + \exp(\log \sigma_j^{2(i)}) - \log \sigma_j^{2(i)} - 1 \right)$;
    `// Total loss and update`
    $\mathcal{L} \leftarrow \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{KL}}$;
    Compute $\nabla_{\boldsymbol{\theta},\boldsymbol{\phi}} \mathcal{L}$;
    Update $\boldsymbol{\theta}, \boldsymbol{\phi}$ with Adam;
  end
end

Algorithm 9: Generation from VAE

Input: Trained decoder $\boldsymbol{\mu}_{\boldsymbol{\theta}}$
Output: Generated sample $\mathbf{x}$

$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$;
$\mathbf{x} \leftarrow \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{z})$;
if *binary data* **then**
  $\mathbf{x} \sim \text{Bernoulli}(\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{z}))$;
end
return $\mathbf{x}$

12.4 Architecture

The VAE architecture is symmetric, with the encoder and decoder typically mirroring each other:

Encoder $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$: A neural network that outputs the parameters of the variational posterior. Given input $\mathbf{x}$, the encoder produces two vectors of dimension k:

$$\boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x}) = \mathbf{W}_{\mu}\mathbf{h}^{(L)} + \mathbf{b}_{\mu}, \quad \log \boldsymbol{\sigma}^2_{\boldsymbol{\phi}}(\mathbf{x}) = \mathbf{W}_{\sigma}\mathbf{h}^{(L)} + \mathbf{b}_{\sigma} \tag{12.31}$$

where $\mathbf{h}^{(L)}$ is the final hidden representation from stacked layers. The encoder outputs $\log \boldsymbol{\sigma}^2$ (not $\boldsymbol{\sigma}^2$) for numerical stability.

Reparameterization layer: A deterministic non-trainable layer:

$$\mathbf{z} = \boldsymbol{\mu} + \exp(\log \boldsymbol{\sigma}^2/2) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{12.32}$$

Decoder $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$: Mirrors the encoder. Takes $\mathbf{z} \in \mathbb{R}^k$ and produces reconstruction parameters. For Gaussian likelihood, outputs $\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{z})$. For Bernoulli likelihood, outputs probabilities $\in [0,1]^d$ via sigmoid activation.

Convolutional VAE: For image data, the architecture uses:

- Encoder: Convolutional layers with stride-2 downsampling, followed by flatten and dense layers producing $\boldsymbol{\mu}$ and $\log \boldsymbol{\sigma}^2$.
- Decoder: Dense layer projecting $\mathbf{z}$ to a feature map, followed by transposed convolutions for upsampling to original resolution.

Hierarchical VAE (NVAE): Stacks multiple stochastic layers: $\mathbf{z} = (\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_L)$, with $\mathbf{z}_l$ conditioned on $\mathbf{z}_{<l}$. Each layer contributes to progressively finer details. The prior becomes $p(\mathbf{z}) = p(\mathbf{z}_1) \prod_{l=2}^{L} p(\mathbf{z}_l \mid \mathbf{z}_{<l})$.

12.5 Pros and Cons

Pros:

- *Principled generative model:* Built on solid variational inference foundations; maximizes a well-defined lower bound on data likelihood.
- *Structured latent space:* The KL regularization ensures the latent space approximately follows a standard Gaussian, making sampling and interpolation straightforward and meaningful.
- *Stable training:* Unlike GANs, VAE training is stable and converges reliably with standard optimizers.
- *Encoder included:* Provides both inference ($\mathbf{x} \to \mathbf{z}$) and generation ($\mathbf{z} \to \mathbf{x}$) in one model, unlike GANs which lack an encoder.
- *Disentanglement:* With $\beta > 1$, the VAE can learn disentangled representations where latent dimensions correspond to interpretable factors.

Cons:

- *Blurry samples:* The combination of Gaussian likelihood and mode-covering behavior leads to averaging over plausible outputs, producing blurry images that lack sharp details.
- *Posterior collapse:* The decoder may learn to ignore $\mathbf{z}$ entirely, causing $\mathrm{KL} \to 0$ and the model to degenerate to an unconditional generative model. This is especially common with powerful autoregressive decoders.
- *Hole problem:* The VAE assigns probability mass to regions far from training data (e.g., between modes in a multimodal distribution), potentially generating unrealistic samples.
- *Limited expressivity:* The Gaussian prior and posterior may be insufficient to model complex latent structures. Normalizing flows and hierarchical VAEs address this but add complexity.
- *ELBO is a bound, not the log-likelihood:* There is no guarantee on the tightness of the bound; a high ELBO does not guarantee a good model.

12.6 Evolution

The VAE has evolved from a proof-of-concept on MNIST to a critical component of state-of-the-art generative systems:

- **Conditional VAE (Sohn et al., 2015):** Conditions both encoder and decoder on auxiliary information $\mathbf{y}$: $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}, \mathbf{y})$ and $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z}, \mathbf{y})$. Enables controlled generation.
- **Importance Weighted Autoencoder (IWAE, Burda et al., 2016):** Uses multiple importance-weighted samples to tighten the bound: $\mathcal{L}_K = \mathbb{E}_{\mathbf{z}_1,\ldots,\mathbf{z}_K}[\log \frac{1}{K} \sum_k p(\mathbf{x}, \mathbf{z}_k)/q(\mathbf{z}_k|\mathbf{x})]$ As $K \to \infty$, the bound approaches the true log-likelihood.
- **VQ-VAE (van den Oord et al., 2017):** Replaces continuous latent with discrete codebook: $\mathbf{z}_q = \arg\min_{\mathbf{e} \in \mathcal{E}} \|\mathbf{z}_e - \mathbf{e}\|$. Trained with straight-through gradient estimation and codebook/commitment losses. Combined with PixelCNN prior for generation.
- **VQ-VAE-2 (Razavi et al., 2019):** Hierarchical VQ-VAE with top-level global latent and bottom-level local latents. Generates high-fidelity 1024×1024 images.
- **NVAE (Vahdat & Kautz, 2020):** Deep hierarchical VAE with carefully designed residual cells, achieving SOTA likelihood on images while producing high-quality samples.
- **Stable Diffusion (Rombach et al., 2022):** Uses a VAE to compress images 8× into latent space, where a diffusion model operates. The VAE encoder/decoder is the crucial first stage enabling efficient diffusion in a compact latent space. This marriage of VAE compression and diffusion generation powers modern text-to-image models.
- **DALL-E (Ramesh et al., 2021):** Uses dVAE (discrete VAE) to tokenize images into a grid of discrete codes, then models them autoregressively with a Transformer.

The VAE's fundamental insight—that we can learn both an inference network and a generative model by maximizing a variational lower bound—remains central to unsupervised learning. Its role in latent diffusion models has made it an indispensable component of the most capable generative AI systems today.

Chapter 13

Generative Adversarial Networks

13.1 Motivation

Variational autoencoders maximize a lower bound on the data likelihood, which encourages mode-covering behavior: the model spreads probability mass over all modes of the data distribution, producing samples that average over multiple plausible outputs. This results in characteristically blurry generations.

Generative Adversarial Networks (GANs) take a fundamentally different approach. Instead of maximizing likelihood, GANs frame generation as a two-player minimax game between a generator and a discriminator. The discriminator learns to distinguish real from generated samples, while the generator learns to produce samples that fool the discriminator. This adversarial objective naturally encourages mode-seeking behavior and produces sharp, realistic samples. The game-theoretic formulation eliminates the need for an explicit likelihood function and enables implicit generative modeling of arbitrarily complex distributions.

13.2 Mathematical Formulation

13.2.1 Two-Player Minimax Game

The GAN framework consists of:

- **Generator** $G_{\boldsymbol{\theta}_g} : \mathcal{Z} \to \mathcal{X}$ maps a latent code $\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})$ to a generated sample $G(\mathbf{z})$. The induced distribution is denoted p_g.
- **Discriminator** $D_{\boldsymbol{\theta}_d} : \mathcal{X} \to [0, 1]$ outputs the probability that a given sample came from the real data distribution p_{data} rather than from p_g.

The value function of the minimax game is:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))] \tag{13.1}$$

The discriminator aims to maximize $V(D, G)$: high $D(\mathbf{x})$ for real $\mathbf{x}$ and low $D(G(\mathbf{z}))$ for generated samples. The generator aims to minimize V, i.e., maximize $D(G(\mathbf{z}))$ to fool the discriminator.

13.2.2 Optimal Discriminator

For a fixed generator G, what is the optimal discriminator D^*? The objective for D is:

$$V(D, G) = \int_{\mathbf{x}} [p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x}))] \, d\mathbf{x} \tag{13.2}$$

This is a functional optimization problem. For each $\mathbf{x}$, the integrand is maximized when:

$$\frac{\partial}{\partial D(\mathbf{x})} \left[p_{\text{data}}(\mathbf{x}) \log D(\mathbf{x}) + p_g(\mathbf{x}) \log(1 - D(\mathbf{x})) \right] = 0 \tag{13.3}$$

$$\frac{p_{\text{data}}(\mathbf{x})}{D(\mathbf{x})} - \frac{p_g(\mathbf{x})}{1 - D(\mathbf{x})} = 0 \tag{13.4}$$

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})} \tag{13.5}$$

The optimal discriminator is the ratio of the real density to the sum of real and generated densities. At D^*, the discriminator outputs $1/2$ when $p_{\text{data}} = p_g$.

13.2.3 Generator Objective at Optimal D

Plugging D^* into the value function:

$$C(G) = \max_D V(D, G) = V(D^*, G) \tag{13.6}$$

$$= \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \left[\log \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})} \right] + \mathbb{E}_{\mathbf{x} \sim p_g} \left[\log \frac{p_g(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_g(\mathbf{x})} \right] \tag{13.7}$$

Manipulating this expression:

$$C(G) = \int p_{\text{data}} \log \frac{p_{\text{data}}}{p_{\text{data}} + p_g} \, d\mathbf{x} + \int p_g \log \frac{p_g}{p_{\text{data}} + p_g} \, d\mathbf{x} \tag{13.8}$$

$$= \int p_{\text{data}} \log \frac{p_{\text{data}}}{(p_{\text{data}} + p_g)/2} \cdot \frac{1}{2} \, d\mathbf{x} + \int p_g \log \frac{p_g}{(p_{\text{data}} + p_g)/2} \cdot \frac{1}{2} \, d\mathbf{x} \tag{13.9}$$

$$= -\log 4 + \text{KL}\left(p_{\text{data}} \,\middle\|\, \frac{p_{\text{data}} + p_g}{2} \right) + \text{KL}\left(p_g \,\middle\|\, \frac{p_{\text{data}} + p_g}{2} \right) \tag{13.10}$$

$$= -\log 4 + 2 \cdot \text{JSD}(p_{\text{data}} \parallel p_g) \tag{13.11}$$

where $\text{JSD}(P \parallel Q) = \frac{1}{2}\text{KL}(P \parallel \frac{P+Q}{2}) + \frac{1}{2}\text{KL}(Q \parallel \frac{P+Q}{2})$ is the Jensen-Shannon divergence.

The global minimum $C(G) = -\log 4$ is achieved if and only if $p_g = p_{\text{data}}$, i.e., the generator perfectly captures the data distribution. Thus, minimizing the GAN objective is equivalent to minimizing the Jensen-Shannon divergence between the real and generated distributions.

13.2.4 Training Algorithm

In practice, rather than optimizing D to completion (which would be computationally prohibitive and lead to vanishing gradients for G), we alternate between k steps of optimizing D and one step of optimizing G:

1. Sample minibatch of m noise samples $\{\mathbf{z}^{(i)}\}_{i=1}^m \sim p_{\mathbf{z}}$ and m real samples $\{\mathbf{x}^{(i)}\}_{i=1}^m \sim p_{\text{data}}$.

2. Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\boldsymbol{\theta}_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D(\mathbf{x}^{(i)}) + \log(1 - D(G(\mathbf{z}^{(i)}))) \right] \tag{13.12}$$

3. Sample minibatch of m noise samples $\{\mathbf{z}^{(i)}\}_{i=1}^{m} \sim p_{\mathbf{z}}$.

4. Update the generator by descending its stochastic gradient:

$$\nabla_{\boldsymbol{\theta}_g} \frac{1}{m} \sum_{i=1}^{m} \log(1 - D(G(\mathbf{z}^{(i)}))) \tag{13.13}$$

13.2.5 Non-Saturating Loss

The original generator loss $\min_G \mathbb{E}[\log(1 - D(G(\mathbf{z})))]$ suffers from vanishing gradients early in training. When the discriminator is confident, $D(G(\mathbf{z})) \approx 0$, and:

$$\frac{\partial}{\partial \boldsymbol{\theta}_g} \log(1 - D(G(\mathbf{z}))) = -\frac{1}{1 - D(G(\mathbf{z}))} \approx -1 \tag{13.14}$$

is small, providing weak learning signal.

The **non-saturating loss** replaces minimization of $\log(1 - D(G(\mathbf{z})))$ with maximization of $\log D(G(\mathbf{z}))$:

$$\max_G \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}}[\log D(G(\mathbf{z}))] \tag{13.15}$$

or equivalently in the minimization form:

$$\min_G -\mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}}[\log D(G(\mathbf{z}))] \tag{13.16}$$

When $D(G(\mathbf{z})) \approx 0$, the gradient becomes:

$$\frac{\partial}{\partial \boldsymbol{\theta}_g} \log D(G(\mathbf{z})) = \frac{1}{D(G(\mathbf{z}))} \frac{\partial D}{\partial \boldsymbol{\theta}_g} \to \infty \tag{13.17}$$

providing a much stronger gradient signal. This is the loss used in all practical GAN implementations.

13.2.6 Wasserstein GAN

The Jensen-Shannon divergence can be problematic: when the supports of p_{data} and p_g are disjoint or have low-dimensional overlap (common in high dimensions), JSD is constant ($\log 2$), providing no useful gradient.

The Wasserstein GAN (WGAN) replaces JSD with the Earth Mover's (Wasserstein-1) distance:

$$W(p_r, p_g) = \inf_{\gamma \in \Pi(p_r, p_g)} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \gamma} \left[\|\mathbf{x} - \mathbf{y}\| \right] \tag{13.18}$$

where $\Pi(p_r, p_g)$ is the set of all joint distributions $\gamma(\mathbf{x}, \mathbf{y})$ whose marginals are p_r and p_g. Intuitively, $W(p_r, p_g)$ is the minimum cost of transporting probability mass from p_g to p_r.

Kantorovich-Rubinstein duality: The Wasserstein-1 distance has a dual representation:

$$W(p_r, p_g) = \sup_{\|f\|_L \leq 1} \left(\mathbb{E}_{\mathbf{x} \sim p_r}[f(\mathbf{x})] - \mathbb{E}_{\mathbf{x} \sim p_g}[f(\mathbf{x})] \right) \tag{13.19}$$

where the supremum is over all 1-Lipschitz functions $f : \mathcal{X} \to \mathbb{R}$. We approximate f with a neural network (called the *critic*), and enforce the 1-Lipschitz constraint.

Weight clipping (original WGAN): Clip critic weights to $[-c, c]$ after each update. This is a crude but simple way to enforce Lipschitz continuity. Disadvantage: limits model capacity, can cause vanishing/exploding gradients.

Gradient penalty (WGAN-GP): A much more effective approach penalizes the gradient norm directly:

$$\mathcal{L}_{\text{GP}} = \lambda \, \mathbb{E}_{\hat{\mathbf{x}} \sim p_{\hat{\mathbf{x}}}} \left[\left(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1 \right)^2 \right] \tag{13.20}$$

where $\hat{\mathbf{x}} = \epsilon \mathbf{x} + (1 - \epsilon) G(\mathbf{z})$ is a random interpolation between real and generated samples, with $\epsilon \sim \mathcal{U}[0, 1]$. The penalty encourages the gradient norm to be 1 everywhere on the interpolation line between p_r and p_g. The full critic loss is:

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}}[D(G(\mathbf{z}))] - \mathbb{E}_{\mathbf{x} \sim p_r}[D(\mathbf{x})] + \lambda \, \mathbb{E}_{\hat{\mathbf{x}} \sim p_{\hat{\mathbf{x}}}} \left[\left(\|\nabla_{\hat{\mathbf{x}}} D(\hat{\mathbf{x}})\|_2 - 1 \right)^2 \right] \tag{13.21}$$

Note the sign reversal: the critic *minimizes* this loss, while the generator minimizes $-\mathbb{E}[D(G(\mathbf{z}))]$.

13.2.7 Conditional GAN

In many applications, we want to control the generated output. The conditional GAN provides class label $\mathbf{y}$ to both generator and discriminator:

$$\min_G \max_D \, \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim p_{\text{data}}}[\log D(\mathbf{x}, \mathbf{y})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}, \mathbf{y} \sim p_{\mathbf{y}}}[\log(1 - D(G(\mathbf{z}, \mathbf{y}), \mathbf{y}))] \tag{13.22}$$

The generator now models $p(\mathbf{x} \mid \mathbf{y})$. The discriminator evaluates the joint compatibility of $(\mathbf{x}, \mathbf{y})$.

13.2.8 Mode Collapse

A notorious failure mode of GANs is mode collapse: the generator maps multiple different $\mathbf{z}$ values to the same (or very similar) output, covering only a subset of the data distribution's modes. Mathematically, this occurs when the generator finds a strategy that fools the discriminator while p_g does not match p_{data} in support.

Diagnosis: Mode collapse is detectable by generating many samples and checking diversity (e.g., via Inception Score or by training a classifier on generated images).

Mitigations:

- **Minibatch discrimination:** Allow the discriminator to compare samples within a minibatch, detecting lack of diversity. Compute a minibatch similarity matrix and append features summarizing cross-sample distances.

- **Unrolled GANs:** Unroll the generator optimization for K steps when updating the discriminator, so the discriminator sees where the generator is *going* rather than only where it *is*.
- **WGAN:** The Wasserstein distance is continuous and differentiable almost everywhere, providing meaningful gradients even when p_{data} and p_g have disjoint supports, significantly reducing mode collapse.
- **Packing (PacGAN):** Feed multiple samples jointly to the discriminator as a "pack," which must distinguish real vs. generated packs, naturally encouraging within-pack diversity.

13.3 Algorithm

Algorithm 10: Training a GAN with Non-Saturating Loss

Input: Training data $\{\mathbf{x}^{(i)}\}_{i=1}^{m}$, latent distribution $p_{\mathbf{z}}$, steps k, batch size m
Output: Trained generator $G_{\boldsymbol{\theta}_g}$

Initialize $\boldsymbol{\theta}_d$, $\boldsymbol{\theta}_g$;
for *training iterations* **do**
  `// Update discriminator` k `times`
  for *step* $\leftarrow 1$ **to** k **do**
    Sample $\{\mathbf{z}^{(i)}\}_{i=1}^{m} \sim p_{\mathbf{z}}$;
    Sample $\{\mathbf{x}^{(i)}\}_{i=1}^{m} \sim p_{\text{data}}$;
    $\mathcal{L}_D \leftarrow -\frac{1}{m}\sum_{i=1}^{m}\left[\log D(\mathbf{x}^{(i)}) + \log(1 - D(G(\mathbf{z}^{(i)})))\right]$;
    $\boldsymbol{\theta}_d \leftarrow \boldsymbol{\theta}_d - \eta\nabla_{\boldsymbol{\theta}_d}\mathcal{L}_D$;
  end
  `// Update generator once`
  Sample $\{\mathbf{z}^{(i)}\}_{i=1}^{m} \sim p_{\mathbf{z}}$;
  $\mathcal{L}_G \leftarrow -\frac{1}{m}\sum_{i=1}^{m}\log D(G(\mathbf{z}^{(i)}))$; `// Non-saturating loss`
  $\boldsymbol{\theta}_g \leftarrow \boldsymbol{\theta}_g - \eta\nabla_{\boldsymbol{\theta}_g}\mathcal{L}_G$;
end

Algorithm 11: Training a WGAN with Gradient Penalty

Input: Training data, latent distribution, gradient penalty λ, critic iterations n_{critic}

Output: Trained WGAN-GP generator

Initialize $\boldsymbol{\theta}$, $\boldsymbol{\phi}$ (critic and generator);
for *training iterations* **do**
  for $t \leftarrow 1$ **to** n_{critic} **do**
    Sample $\{\mathbf{x}^{(i)}\}_{i=1}^{m} \sim p_{\text{data}}$, $\{\mathbf{z}^{(i)}\}_{i=1}^{m} \sim p_{\mathbf{z}}$;
    $\tilde{\mathbf{x}}^{(i)} \leftarrow G(\mathbf{z}^{(i)})$;
    foreach i **do**
      $\epsilon_i \sim \mathcal{U}[0, 1]$;
      $\hat{\mathbf{x}}^{(i)} \leftarrow \epsilon_i \mathbf{x}^{(i)} + (1 - \epsilon_i)\tilde{\mathbf{x}}^{(i)}$;
    end
    $\mathcal{L}_C \leftarrow \frac{1}{m} \sum_{i=1}^{m} \left[D(\tilde{\mathbf{x}}^{(i)}) - D(\mathbf{x}^{(i)}) + \lambda(\left\| \nabla_{\hat{\mathbf{x}}^{(i)}} D(\hat{\mathbf{x}}^{(i)}) \right\|_2 - 1)^2 \right]$;
    Update $\boldsymbol{\theta}$ to minimize $\mathcal{L}_C$ (Adam);
  end
  Sample $\{\mathbf{z}^{(i)}\}_{i=1}^{m} \sim p_{\mathbf{z}}$;
  $\mathcal{L}_G \leftarrow -\frac{1}{m} \sum_{i=1}^{m} D(G(\mathbf{z}^{(i)}))$;
  Update $\boldsymbol{\phi}$ to minimize $\mathcal{L}_G$ (Adam);
end

13.4 Architecture

13.4.1 DCGAN Architecture

Radford et al. (2016) introduced DCGAN, establishing architectural guidelines that remain standard:

Generator:

- Input: $\mathbf{z} \in \mathbb{R}^{100} \sim \mathcal{N}(0, \mathbf{I})$ or $\mathcal{U}[-1, 1]$
- Project and reshape to $4 \times 4 \times 1024$ feature maps
- Fractionally-strided (transposed) convolutions: $4 \to 8 \to 16 \to 32 \to 64$
- BatchNorm after each transposed convolution (except output)
- ReLU activation in all layers except output
- Output: tanh activation (images in $[-1, 1]$)
- No fully connected layers (fully convolutional)

Discriminator:

- Strided convolutions (mirror of generator): $64 \to 32 \to 16 \to 8 \to 4$
- BatchNorm after each convolution (except first layer)
- LeakyReLU ($\alpha = 0.2$) throughout
- Output: single sigmoid unit, or no activation (for WGAN)

13.4.2 Progressive Growing

Karras et al. (2018) proposed Progressive GAN (ProGAN), which grows both generator and discriminator gradually:

- Start at low resolution (4×4)
- Train until stable, then fade in new layers doubling resolution
- Continue until target resolution (e.g., 1024×1024)
- This incremental training dramatically stabilizes high-resolution generation

13.4.3 StyleGAN Architecture

StyleGAN (Karras et al., 2019) introduced the style-based generator:

- **Mapping network:** 8-layer MLP maps $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ to intermediate latent $\mathbf{w} \in \mathcal{W}$
- **Synthesis network:** Starting from a learned constant input $4 \times 4 \times 512$, $\mathbf{w}$ is injected via Adaptive Instance Normalization (AdaIN) at each resolution:

$$\text{AdaIN}(\mathbf{x}, \mathbf{w}) = \mathbf{y}_s(\mathbf{w}) \cdot \frac{\mathbf{x} - \mu(\mathbf{x})}{\sigma(\mathbf{x})} + \mathbf{y}_b(\mathbf{w}) \tag{13.23}$$

- **Noise injection:** Per-pixel Gaussian noise added before each AdaIN, controlling stochastic variation
- **Style mixing:** During training, randomly switch $\mathbf{w}$ at a layer, encouraging scale-specific style disentanglement

StyleGAN produces a disentangled latent space where $\mathcal{W}$ enables semantically meaningful interpolation.

13.5 Pros and Cons

Pros:

- *Sharp, realistic samples:* GANs produce the sharpest images among generative models, without the blurring characteristic of VAEs and early diffusion models.
- *No explicit likelihood:* The implicit generative model can represent arbitrarily complex distributions without specifying a parametric likelihood function.
- *Fast sampling:* A single forward pass through the generator produces a sample, compared to iterative denoising in diffusion models.
- *Latent space manipulation:* StyleGAN's $\mathcal{W}$ space enables smooth, semantically meaningful interpolation and vector arithmetic (e.g., "smile vector").
- *Unsupervised representation learning:* The discriminator learns rich feature representations that transfer well to downstream tasks.

Cons:

- *Training instability:* The minimax game does not converge to a stable equilibrium; training oscillates and requires careful balancing of G and D.
- *Mode collapse:* The generator may produce limited diversity, ignoring entire regions of the data distribution.
- *No encoder:* Standard GANs lack an inference network ($\mathbf{x} \to \mathbf{z}$), requiring additional techniques (e.g., BiGAN, ALI) to obtain latent representations.
- *Evaluation difficulty:* No single metric reliably captures both fidelity and diversity. Inception Score and FID are common but imperfect proxies.
- *Vanishing gradients:* When D becomes too good, the generator receives negligible gradient, stalling training.
- *Hyperparameter sensitivity:* Architecture choices, learning rates, and optimizer settings critically affect training outcomes.

13.6 Evolution

GANs have undergone rapid evolution since their introduction in 2014:

- **GAN (Goodfellow et al., 2014):** Original formulation with minimax game and JS divergence. Demonstrated on MNIST and CIFAR-10.
- **DCGAN (Radford et al., 2016):** Introduced the convolutional architecture template that enabled stable training on natural images. Demonstrated vector arithmetic in latent space.
- **WGAN (Arjovsky et al., 2017):** Replaced JS divergence with Wasserstein distance, providing meaningful gradients everywhere and dramatically improving training stability.
- **WGAN-GP (Gulrajani et al., 2017):** Gradient penalty as a superior alternative to weight clipping. Became the de facto standard for GAN training.
- **ProGAN (Karras et al., 2018):** Progressive growing enabled generation at unprecedented resolutions (1024×1024).
- **StyleGAN (Karras et al., 2019):** Style-based generator with AdaIN, mapping network, and noise injection. The $\mathcal{W}$ space demonstrated remarkable disentanglement.
- **StyleGAN2 (Karras et al., 2020):** Redesigned normalization (weight demodulation), eliminated droplet artifacts, and improved path length regularization. Achieved near-photorealistic face generation.
- **StyleGAN3 (Karras et al., 2021):** Alias-free generator ensuring translation and rotation equivariance, enabling seamless video generation and texture synthesis.

- **BigGAN (Brock et al., 2019):** Scaled GANs to enormous batch sizes (2048) on TPU pods, achieving conditional ImageNet generation with record Inception Scores using class-conditional BatchNorm and orthogonal regularization.
- **GigaGAN (Kang et al., 2023):** Billion-parameter GAN demonstrating that GANs remain competitive with diffusion models for text-to-image generation at scale.

The GAN framework demonstrated that adversarial training can produce extraordinarily realistic samples, fundamentally changing the landscape of generative modeling. While diffusion models have recently surpassed GANs in sample diversity and training stability, the adversarial paradigm remains influential, and GANs retain advantages in sampling speed for real-time applications.

Chapter 14

Diffusion Models

14.1 Motivation

Generative models face a fundamental trade-off. GANs produce sharp, high-fidelity samples but suffer from training instability, mode collapse, and non-convergence due to the adversarial minimax game. VAEs are stable to train and provide a principled likelihood-based objective, but their samples are characteristically blurry because the Gaussian likelihood leads to mode-covering averaging.

Diffusion models offer a resolution to this dilemma. They define a generative process as the reverse of a fixed Markov chain that gradually adds Gaussian noise to data. By learning to reverse this diffusion process, the model denoises pure noise into realistic samples through a sequence of simple steps. This formulation:

- Eliminates adversarial training entirely—the objective is a simple regression loss.
- Provides a well-defined likelihood bound with stable optimization.
- Decomposes the hard one-shot generation into many easy denoising subtasks.
- Achieves state-of-the-art sample quality across image, audio, and video domains.

14.2 Mathematical Formulation

14.2.1 Forward Diffusion Process

The forward (diffusion) process is a fixed Markov chain that incrementally destroys the data structure by adding Gaussian noise:

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \tag{14.1}$$

where each transition is:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\left(\mathbf{x}_t; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t \mathbf{I}\right) \tag{14.2}$$

The $\beta_t \in (0, 1)$ constitute a *variance schedule*. Common choices include:

- **Linear schedule:** β_t increases linearly from β_1 to β_T (e.g., 10^{-4} to 0.02).
- **Cosine schedule:** $\beta_t = 1 - \cos^2\left(\frac{t/T+s}{1+s} \cdot \frac{\pi}{2}\right) / \cos^2\left(\frac{s}{1+s} \cdot \frac{\pi}{2}\right)$, which preserves more information in early steps.
- **Sigmoid schedule:** β_t follows a sigmoid curve for smooth transition.

14.2.2 Direct Sampling Property

A crucial property of the Gaussian diffusion process is that we can sample $\mathbf{x}_t$ directly from $\mathbf{x}_0$ without iterating through all intermediate steps. Define:

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s \tag{14.3}$$

Using the reparameterization trick $\mathbf{x}_t = \sqrt{\alpha_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\boldsymbol{\epsilon}_{t-1}$ where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, we expand recursively:

$$\begin{aligned}
\mathbf{x}_t &= \sqrt{\alpha_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\boldsymbol{\epsilon}_{t-1} && (14.4)\\
&= \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}}\,\mathbf{x}_{t-2} + \sqrt{\beta_{t-1}}\,\boldsymbol{\epsilon}_{t-2}\right) + \sqrt{\beta_t}\,\boldsymbol{\epsilon}_{t-1} && (14.5)\\
&= \sqrt{\alpha_t\alpha_{t-1}}\,\mathbf{x}_{t-2} + \sqrt{\alpha_t\beta_{t-1}}\,\boldsymbol{\epsilon}_{t-2} + \sqrt{\beta_t}\,\boldsymbol{\epsilon}_{t-1} && (14.6)
\end{aligned}$$

The sum of independent Gaussians is Gaussian. The variance of the combined noise term is:

$$\alpha_t\beta_{t-1} + \beta_t = \alpha_t(1 - \alpha_{t-1}) + (1 - \alpha_t) = 1 - \alpha_t\alpha_{t-1} \tag{14.7}$$

Continuing this recursion:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}\right) \tag{14.8}$$

Equivalently, we can write:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\,\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{14.9}$$

As $t \to T$, $\bar{\alpha}_T \to 0$, so $q(\mathbf{x}_T \mid \mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$, i.e., the data is completely corrupted to pure noise.

14.2.3 Reverse Process

The reverse process is a learned Markov chain that inverts the forward diffusion:

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) \tag{14.10}$$

where $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$ and each transition is:

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}\left(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t, t), \boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\right) \tag{14.11}$$

In the DDPM formulation, the variance is fixed to a time-dependent constant:

$$\boldsymbol{\Sigma}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) = \sigma_t^2 \mathbf{I} \tag{14.12}$$

where $\sigma_t^2 = \beta_t$ or $\sigma_t^2 = \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$.

14.2.4 Variational Bound Training Objective

We train the model by maximizing the variational lower bound on $\log p_{\boldsymbol{\theta}}(\mathbf{x}_0)$:

$$
\begin{aligned}
-\log p_{\boldsymbol{\theta}}(\mathbf{x}_0) &\leq -\log p_{\boldsymbol{\theta}}(\mathbf{x}_0) + \mathrm{KL}(q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) \parallel p_{\boldsymbol{\theta}}(\mathbf{x}_{1:T} \mid \mathbf{x}_0)) && (14.13)\\
&= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)}{p_{\boldsymbol{\theta}}(\mathbf{x}_{0:T})} \right] && (14.14)\\
&= \mathbb{E}_q \left[-\log p(\mathbf{x}_T) - \sum_{t=1}^{T} \log \frac{p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)}{q(\mathbf{x}_t \mid \mathbf{x}_{t-1})} \right] =: L && (14.15)
\end{aligned}
$$

After algebraic manipulation, the loss decomposes into:

$$
\begin{aligned}
L = \mathbb{E}_q \Bigg[& \underbrace{\mathrm{KL}(q(\mathbf{x}_T \mid \mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} && (14.16)\\
& + \sum_{t=2}^{T} \underbrace{\mathrm{KL}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1} \mid \mathbf{x}_t))}_{L_{t-1}} && (14.17)\\
& - \underbrace{\log p_{\boldsymbol{\theta}}(\mathbf{x}_0 \mid \mathbf{x}_1)}_{L_0} \Bigg] && (14.18)
\end{aligned}
$$

The term L_T is constant (independent of $\boldsymbol{\theta}$, since the forward process is fixed and $p(\mathbf{x}_T)$ is the standard Gaussian). The crucial insight is that $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ has a closed-form Gaussian distribution:

$$
q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\left(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}\right) \tag{14.19}
$$

where:

$$
\begin{aligned}
\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) &= \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\mathbf{x}_t && (14.20)\\
\tilde{\beta}_t &= \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t && (14.21)
\end{aligned}
$$

14.2.5 Simplified Training Objective (DDPM)

Ho et al. (2020) proposed a simplified objective. First, reparameterize $\mathbf{x}_0$ in terms of $\mathbf{x}_t$ and noise:

$$
\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}\left(\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}\right) \tag{14.22}
$$

Substituting into $\tilde{\boldsymbol{\mu}}_t$:

$$
\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}\right) \tag{14.23}
$$

Rather than predicting $\boldsymbol{\mu}_{\boldsymbol{\theta}}$, the model predicts the noise $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$:

$$\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \right) \tag{14.24}$$

The L_{t-1} KL terms then reduce to a simple weighted MSE:

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\epsilon}} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1-\bar{\alpha}_t)} \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \right\|_2^2 \right] \tag{14.25}$$

Ho et al. found that simply removing the weighting yields better sample quality:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \left[\left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \right\|_2^2 \right] \tag{14.26}$$

where $t \sim \mathcal{U}\{1, \ldots, T\}$, $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, and $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

14.2.6 Connection to Score Matching

The noise prediction objective has a deep connection to score-based generative modeling. The score function is the gradient of the log-density:

$$\mathbf{s}(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x}) \tag{14.27}$$

For the noised distribution $q(\mathbf{x}_t \mid \mathbf{x}_0)$, the score is:

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t \mid \mathbf{x}_0) = -\frac{\mathbf{x}_t - \sqrt{\bar{\alpha}_t}\mathbf{x}_0}{1-\bar{\alpha}_t} = -\frac{\boldsymbol{\epsilon}}{\sqrt{1-\bar{\alpha}_t}} \tag{14.28}$$

Therefore, the noise predictor is proportional to the negative score:

$$\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \approx -\sqrt{1-\bar{\alpha}_t}\, \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \tag{14.29}$$

The model implicitly learns the score function of the noised data distribution at each noise level. This directly connects DDPM to score-based models (Song & Ermon, 2019) and the unified framework of stochastic differential equations.

14.2.7 DDIM: Deterministic Sampling

Denoising Diffusion Implicit Models (DDIM, Song et al., 2021) generalize the reverse process to be non-Markovian, enabling faster deterministic sampling:

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\, \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)}{\sqrt{\bar{\alpha}_t}} \right)}_{\hat{\mathbf{x}}_0} + \sqrt{1-\bar{\alpha}_{t-1}-\sigma_t^2}\, \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) + \sigma_t \mathbf{z} \tag{14.30}$$

where $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and:

$$\sigma_t = \eta \sqrt{\frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}} \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}} \tag{14.31}$$

When $\eta = 0$ (and hence $\sigma_t = 0$), the process is *deterministic*: given $\mathbf{x}_T$, the generated $\mathbf{x}_0$ is uniquely determined. This property enables:

- **Fast sampling:** With a subsequence of $S \ll T$ timepoints (e.g., 50 instead of 1000), DDIM produces high-quality samples, making generation 20–50 times faster.
- **Inversion:** The deterministic mapping $\mathbf{x}_0 \to \mathbf{x}_T$ is invertible (up to discretization error), enabling image editing by running the reverse of DDIM to obtain the latent, modifying it, and regenerating.

14.2.8 Classifier Guidance

To generate samples from a specific class y, we can guide the diffusion process using a pre-trained classifier $p_{\boldsymbol{\phi}}(y \mid \mathbf{x}_t)$:

$$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t \mid y) = \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log q(y \mid \mathbf{x}_t) \tag{14.32}$$

In terms of the noise predictor:

$$\hat{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) = \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) - w\sqrt{1-\bar{\alpha}_t}\, \nabla_{\mathbf{x}_t} \log p_{\boldsymbol{\phi}}(y \mid \mathbf{x}_t) \tag{14.33}$$

where w is the guidance scale. Higher w trades diversity for fidelity to the class condition.

14.2.9 Classifier-Free Guidance

Classifier-free guidance (Ho & Salimans, 2021) eliminates the need for a separate classifier by jointly training a conditional and unconditional model. During training, the conditioning c is randomly dropped (e.g., 10–20% of samples). At test time, the guided prediction is:

$$\hat{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\mathbf{x}_t, t, c) = \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \varnothing) + w\left(\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t, c) - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \varnothing)\right) \tag{14.34}$$

where $\varnothing$ denotes the null conditioning and $w \geq 1$ is the guidance scale. When $w = 1$, this is standard conditional generation. For $w > 1$, the model is pushed away from the unconditional distribution toward the conditional one, sharpening the output.

This formulation naturally extends to text conditioning (e.g., for Stable Diffusion) and image conditioning (e.g., for image-to-image translation).

14.2.10 Latent Diffusion Models

Running diffusion in pixel space is computationally prohibitive for high-resolution images. Latent diffusion models (Rombach et al., 2022) solve this by first training an autoencoder that compresses images into a lower-dimensional latent space:

$$\mathbf{z} = \mathcal{E}(\mathbf{x}), \quad \hat{\mathbf{x}} = \mathcal{D}(\mathbf{z}) \tag{14.35}$$

The diffusion process operates entirely in the latent space, where the resolution is reduced by a factor of f (typically $f = 4$ or 8). The objective becomes:

$$\mathcal{L}_{\text{LDM}} = \mathbb{E}_{\mathcal{E}(\mathbf{x}),\boldsymbol{\epsilon}\sim\mathcal{N}(0,1),t,y}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{z}_t, t, \boldsymbol{\tau}_{\boldsymbol{\theta}}(y))\|_2^2\right] \tag{14.36}$$

where $\boldsymbol{\tau}_{\boldsymbol{\theta}}(y)$ encodes conditioning text y. Conditioning is implemented via cross-attention layers in the UNet denoiser. This is the core of Stable Diffusion.

14.3 Algorithm

Algorithm 12: Training a DDPM (Simplified Objective)

Input: Training data $\{\mathbf{x}^{(i)}\}$, variance schedule $\beta_1, \dots, \beta_T$, model $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}$
Output: Trained noise prediction model

Initialize $\boldsymbol{\theta}$;
Precompute $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ for $t = 1, \dots, T$;
repeat
  Sample $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ from training data;
  Sample $t \sim \mathcal{U}\{1, \dots, T\}$;
  Sample $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$;
  $\mathbf{x}_t \leftarrow \sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\, \boldsymbol{\epsilon}$;
  $\mathcal{L} \leftarrow \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\|_2^2$;
  Compute $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ and update $\boldsymbol{\theta}$ with Adam;
until *converged*;

Algorithm 13: DDPM Sampling (Ancestral)

Input: Trained model $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}$, total steps T, variance schedule
Output: Generated sample $\mathbf{x}_0$

$\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$;
for $t \leftarrow T$ **to** 1 **do**
  $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} \leftarrow \mathbf{0}$;
  `// DDPM sampling step`
  $\mathbf{x}_{t-1} \leftarrow \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$;
  `// where` $\sigma_t^2 = \beta_t$ `or` $\tilde{\beta}_t$
end
return $\mathbf{x}_0$

Algorithm 14: DDIM Sampling (Deterministic, $\eta = 0$)

Input: Trained model $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}$, DDIM steps $S \ll T$, time indices $t_1 > t_2 > \cdots > t_S$
Output: Generated sample $\mathbf{x}_0$

$\mathbf{x}_{t_1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$;
for $i \leftarrow 1$ **to** S **do**
  $\boldsymbol{\epsilon} \leftarrow \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_{t_i}, t_i)$;
  $\hat{\mathbf{x}}_0 \leftarrow \frac{\mathbf{x}_{t_i} - \sqrt{1-\bar{\alpha}_{t_i}}\, \boldsymbol{\epsilon}}{\sqrt{\bar{\alpha}_{t_i}}}$;
  if $i < S$ **then**
    $\mathbf{x}_{t_{i+1}} \leftarrow \sqrt{\bar{\alpha}_{t_{i+1}}}\, \hat{\mathbf{x}}_0 + \sqrt{1 - \bar{\alpha}_{t_{i+1}}}\, \boldsymbol{\epsilon}$;
  end
  else
    $\mathbf{x}_0 \leftarrow \hat{\mathbf{x}}_0$;
  end
end
return $\mathbf{x}_0$

14.4 Architecture

14.4.1 UNet for Diffusion

The denoising model $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$ is typically a UNet architecture with three key characteristics:

Input: Noisy image $\mathbf{x}_t$ at resolution (e.g., 64×64, 256×256).

Time embedding: The time step t is embedded via sinusoidal positional encodings:

$$\text{PE}(t, 2i) = \sin(t/10000^{2i/d}), \quad \text{PE}(t, 2i+1) = \cos(t/10000^{2i/d}) \tag{14.37}$$

These are processed through an MLP to produce a time conditioning vector, which is added as a bias or via FiLM to intermediate features throughout the network.

Downsampling path: Residual blocks with group normalization and SiLU activations, interleaved with spatial downsampling (strided convolution or average pooling). At each resolution r (e.g., $64 \to 32 \to 16 \to 8$), the number of channels increases.

Bottleneck: Residual blocks with self-attention at the lowest resolution (8×8 or 16×16), capturing global context.

Upsampling path: Mirrors the downsampling path, with skip connections concatenating features from corresponding resolutions. Upsampling via nearest-neighbor interpolation followed by convolution.

Attention at multiple scales: Modern diffusion UNets include self-attention not just at the bottleneck but also at intermediate resolutions (e.g., 16×16, 32×32), significantly improving global coherence.

Cross-attention for conditioning: For text-to-image models, cross-attention layers are inserted after each self-attention block:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} \tag{14.38}$$

where $\mathbf{Q} = \mathbf{W}_Q \cdot \boldsymbol{\varphi}(\mathbf{z}_t)$ (spatial features), $\mathbf{K} = \mathbf{W}_K \cdot \boldsymbol{\tau}_{\boldsymbol{\theta}}(y)$, $\mathbf{V} = \mathbf{W}_V \cdot \boldsymbol{\tau}_{\boldsymbol{\theta}}(y)$ (text context).

14.4.2 Noise Schedules

The choice of noise schedule significantly affects sample quality:

- **Linear:** Simple but suboptimal; too much noise added early.
- **Cosine (Nichol & Dhariwal, 2021):** $\bar{\alpha}_t = \cos^2(\frac{t/T+s}{1+s} \cdot \frac{\pi}{2}) / \cos^2(\frac{s}{1+s} \cdot \frac{\pi}{2})$. Preserves information longer in early steps, improving quality.
- **Sigmoid:** Smooth transition, intermediate between linear and cosine.
- **Learned:** Parameterize β_t as a neural network, trained jointly with the denoiser (VDM, Kingma et al., 2021).

14.5 Pros and Cons

Pros:

- *Stable training:* Simple regression objective with no adversarial game. Training consistently converges.
- *High-quality samples:* Achieves state-of-the-art FID scores, surpassing GANs on many benchmarks.
- *Diverse generation:* The stochastic reverse process and mode-covering objective naturally cover all modes of the data distribution.
- *Principled likelihood:* Provides a well-defined bound on the data likelihood, unlike GANs.
- *Flexible conditioning:* Classifier-free guidance enables effective conditional generation without auxiliary models.
- *Zero-shot capabilities:* The iterative denoising process can be repurposed for inpainting, super-resolution, colorization, and other tasks without task-specific training.

Cons:

- *Slow sampling:* Generating a single sample requires T sequential neural network evaluations (100–1000), compared to one for GANs.
- *High compute cost:* Training requires many denoising steps and large architectures (UNets with attention), demanding significant GPU resources.
- *Discretization error:* The continuous SDE is approximated with discrete steps, introducing error that limits sample quality.
- *Score estimation error:* The model must learn the score function accurately across all noise levels, including very low noise where score estimation is challenging.
- *Sensitivity to schedule:* Performance depends heavily on the choice of noise schedule and number of timesteps.
- *Latent structure:* The mapping from noise to sample is not directly interpretable or manipulable without additional techniques like DDIM inversion.

14.6 Evolution

Diffusion models have rapidly evolved from theoretical constructs to the dominant generative modeling paradigm:

- **Score-based models (Song & Ermon, 2019):** Introduced score matching with multiple noise scales and Langevin dynamics sampling. Established the connection between adding noise and score estimation.

- **DDPM (Ho et al., 2020):** Simplified the training objective to noise prediction, demonstrated high-quality unconditional image generation, and popularized diffusion models.
- **Improved DDPM (Nichol & Dhariwal, 2021):** Learned variance parameters, cosine noise schedule, and importance sampling of timesteps. Substantially improved log-likelihood and sample quality.
- **DDIM (Song et al., 2021):** Non-Markovian forward process enabling deterministic and accelerated sampling. Enabled image inversion and editing.
- **Classifier guidance (Dhariwal & Nichol, 2021):** Showed that guided diffusion with classifier gradients surpasses GANs on ImageNet generation (FID). Launched the "diffusion beats GANs" era.
- **Classifier-free guidance (Ho & Salimans, 2021):** Eliminated the need for a classifier by jointly training conditional and unconditional models.
- **Latent Diffusion / Stable Diffusion (Rombach et al., 2022):** Moved diffusion to a compressed latent space, dramatically reducing compute while enabling high-resolution generation. Cross-attention conditioning for text-to-image.
- **DALL-E 2 (Ramesh et al., 2022):** Two-stage diffusion: CLIP latent prior + image decoder. Demonstrated remarkable text-to-image capabilities.
- **Imagen (Saharia et al., 2022):** Cascaded pixel-space diffusion with a frozen large language model (T5) as text encoder. Showed that scaling the text encoder is critical.
- **Consistency models (Song et al., 2023):** Distilled the diffusion trajectory into a single-step or few-step model, achieving competitive quality with orders-of-magnitude faster sampling.
- **Stable Diffusion XL (Podell et al., 2023):** Scaled latent diffusion with larger UNet, dual text encoders (CLIP + OpenCLIP), and additional conditioning signals, setting a new standard for open-source text-to-image generation.
- **Flux (Black Forest Labs, 2024):** Flow matching with a diffusion transformer (DiT) backbone, pushing text-to-image quality further with a simplified mathematical framework.

The diffusion framework has proven remarkably adaptable, underlying models for images, video, audio, molecules, and 3D content. The core insight—that iterative denoising decomposes hard generation problems into simple steps—has reshaped generative AI.

Chapter 15

Text-to-Image Generation

15.1 Motivation

The ability to generate photorealistic images from natural language descriptions represents a convergence of computer vision and natural language processing. Text-to-image generation requires a model to: (1) understand the semantic content, attributes, relationships, and style described in a text prompt, (2) generate a coherent visual scene consistent with all described elements, and (3) produce high-fidelity output at resolutions suitable for human consumption.

Early approaches used GANs conditioned on text embeddings. Modern state-of-the-art systems combine large language models or vision-language models with powerful generative backbones. The dominant paradigm has shifted from autoregressive over discrete tokens (DALL-E 1) to diffusion in a latent or CLIP embedding space (Stable Diffusion, DALL-E 2). Understanding this evolution requires examining each major architecture in detail.

15.2 Mathematical Formulation

15.2.1 DALL-E: Autoregressive over Discrete Tokens

DALL-E (Ramesh et al., 2021) treats text-to-image generation as autoregressive sequence modeling over discrete tokens, inspired by the success of GPT in language modeling.

Stage 1: Discrete VAE (dVAE)

The dVAE compresses a 256×256 RGB image into a 32×32 grid of discrete tokens from a learned codebook:

$$q(\mathbf{z} \mid \mathbf{x}) = \prod_{i=1}^{32} \prod_{j=1}^{32} q(z_{ij} \mid \mathbf{x}), \quad z_{ij} \in \{1, 2, \ldots, K\} \tag{15.1}$$

where $K = 8192$ is the codebook size. The encoder maps image patches to a distribution over codebook entries using a softmax layer. The decoder reconstructs the image from the codebook embeddings:

$$\hat{\mathbf{x}} = \text{Decoder}(E(z_{11}), E(z_{12}), \ldots, E(z_{32,32})) \tag{15.2}$$

where $E(k) \in \mathbb{R}^d$ is the embedding of codebook entry k. The dVAE is trained with the ELBO:

$$\mathcal{L}_{\text{dVAE}} = \text{KL}(q(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})) - \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x} \mid \mathbf{z})] \tag{15.3}$$

with $p(\mathbf{z}) = \text{Uniform}(K)$ and $p(\mathbf{x} \mid \mathbf{z})$ being a Gaussian likelihood with learned log-variance.

Stage 2: Autoregressive Transformer

Given a text caption tokenized via BPE into up to 256 tokens $(t_1, \ldots, t_{256})$ and the image tokens $(z_1, \ldots, z_{1024})$, the transformer models the joint distribution:

$$p(t_1, \ldots, t_{256}, z_1, \ldots, z_{1024}) = \prod_{i=1}^{256} p(t_i \mid t_{<i}) \prod_{j=1}^{1024} p(z_j \mid \mathbf{t}, z_{<j}) \tag{15.4}$$

The transformer is a sparse attention architecture with three kinds of attention masks:

- **Text-to-text:** Standard causal attention (each token attends to preceding text tokens).
- **Image-to-text:** Each image token attends to all text tokens (full bidirectional attention).
- **Image-to-image:** Each image token attends to preceding image tokens in raster-scan order (row-major).

The model has 12 billion parameters and is trained with cross-entropy loss over the image tokens. Sampling generates image tokens sequentially, which are then decoded by the dVAE decoder.

CLIP Reranking

To improve quality, DALL-E generates N candidate images and selects the best using CLIP (Contrastive Language-Image Pretraining). CLIP consists of an image encoder f_I and a text encoder f_T, trained to maximize the cosine similarity of matching image-text pairs:

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{N} \sum_{i=1}^{N} \log \frac{\exp(\cos(f_I(\mathbf{x}_i), f_T(\mathbf{y}_i))/\tau)}{\sum_{j=1}^{N} \exp(\cos(f_I(\mathbf{x}_i), f_T(\mathbf{y}_j))/\tau)} \tag{15.5}$$

Reranking selects $\hat{\mathbf{x}} = \arg\max_{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N)}} \cos(f_I(\mathbf{x}^{(i)}), f_T(\mathbf{y}))$.

15.2.2 DALL-E 2 (unCLIP)

DALL-E 2 (Ramesh et al., 2022) fundamentally redesigned the architecture around the insight that CLIP embeddings form a semantically rich intermediate space. The model has two stages:

Prior: Text-to-CLIP-Image-Embedding

The prior $P(\mathbf{z}_i \mid \mathbf{y})$ maps a text caption $\mathbf{y}$ to a CLIP image embedding $\mathbf{z}_i$. Two variants were explored:

Autoregressive prior: Model $P(\mathbf{z}_i \mid \mathbf{y})$ as an autoregressive model over the components of $\mathbf{z}_i$ (after PCA projection to ordered principal components).

Diffusion prior: A diffusion model in CLIP embedding space. The forward process adds noise to a CLIP image embedding:

$$\mathbf{z}_i^{(t)} = \sqrt{\bar{\alpha}_t}\, \mathbf{z}_i + \sqrt{1 - \bar{\alpha}_t}\, \boldsymbol{\epsilon} \tag{15.6}$$

The reverse process denoises conditioned on the CLIP text embedding $\mathbf{z}_t = f_T(\mathbf{y})$:

$$p_{\boldsymbol{\theta}}(\mathbf{z}_i^{(t-1)} \mid \mathbf{z}_i^{(t)}, \mathbf{z}_t) = \mathcal{N}(\mathbf{z}_i^{(t-1)}; \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{z}_i^{(t)}, t, \mathbf{z}_t), \boldsymbol{\Sigma}_{\boldsymbol{\theta}}) \tag{15.7}$$

The diffusion prior outperformed the autoregressive variant in sample quality while being comparably efficient. The model predicts the denoised $\mathbf{z}_i$ directly rather than the noise:

$$\mathcal{L}_{\text{prior}} = \mathbb{E}_{t,\mathbf{z}_i,\mathbf{z}_t}\left[\left\|\mathbf{z}_i - \hat{\mathbf{z}}_i(\mathbf{z}_i^{(t)}, t, \mathbf{z}_t)\right\|_2^2\right] \tag{15.8}$$

Decoder: CLIP-Image-Embedding-to-Image

The decoder $P(\mathbf{x} \mid \mathbf{z}_i)$ is a diffusion model that generates an image conditioned on the CLIP image embedding $\mathbf{z}_i$. The architecture is a UNet with cross-attention to $\mathbf{z}_i$ and optional conditioning on the CLIP text embedding $\mathbf{z}_t$:

$$\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \mathbf{z}_i, \mathbf{z}_t) \tag{15.9}$$

Starting from a sampled CLIP image embedding $\mathbf{z}_i \sim P(\mathbf{z}_i \mid \mathbf{y})$ (from the prior), the decoder generates the image. Classifier-free guidance is applied by randomly dropping $\mathbf{z}_i$ and $\mathbf{z}_t$ during training:

$$\hat{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \mathbf{z}_i, \mathbf{z}_t) = \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) + w\left(\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t, \mathbf{z}_i, \mathbf{z}_t) - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\right) \tag{15.10}$$

The unCLIP Insight

The term "unCLIP" is the inverse of CLIP: while CLIP maps (image, text) to a shared embedding space, unCLIP maps from this space back to an image. Since CLIP embeddings capture high-level semantics while being robust to irrelevant visual details, generating from these embeddings allows the model to focus on semantic accuracy in the prior and visual fidelity in the decoder, with the CLIP embedding serving as the bridge.

15.2.3 Stable Diffusion (Latent Diffusion Models)

Stable Diffusion (Rombach et al., 2022) combines a VAE for compression with a diffusion model in latent space. This gives it a crucial computational advantage over pixel-space diffusion, making text-to-image generation feasible on consumer GPUs.

Perceptual Compression via VAE

The autoencoder compresses a $H \times W \times 3$ image into a latent representation $\mathbf{z} \in \mathbb{R}^{h \times w \times c}$ with downsampling factor $f = H/h = W/w$ (typically $f = 8$). For a 512×512 input, the latent is $64 \times 64 \times 4$.

The autoencoder is trained with a combination of losses:

$$\mathcal{L}_{\text{AE}} = \mathcal{L}_{\text{rec}}(\mathbf{x}, \mathcal{D}(\mathcal{E}(\mathbf{x}))) + \mathcal{L}_{\text{LPIPS}}(\mathbf{x}, \mathcal{D}(\mathcal{E}(\mathbf{x}))) + \mathcal{L}_{\text{adv}}(\mathcal{D}(\mathcal{E}(\mathbf{x}))) \tag{15.11}$$

where $\mathcal{L}_{\text{rec}}$ is L1 or L2 reconstruction loss, $\mathcal{L}_{\text{LPIPS}}$ is the Learned Perceptual Image Patch Similarity (a perceptual loss using VGG features), and $\mathcal{L}_{\text{adv}}$ is a patch-based adversarial loss (similar to VQ-GAN) to encourage photorealism.

To regularize the latent space, a KL penalty toward a standard Gaussian is added (as in VAEs), or a vector quantization layer is used (as in VQ-GAN). Stable Diffusion uses the KL-regularized variant.

Latent Diffusion

The diffusion process operates entirely on the latent codes:

$$\mathbf{z}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{z}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{15.12}$$

where $\mathbf{z}_0 = \mathcal{E}(\mathbf{x})$ is the encoded latent. The denoising UNet predicts the noise:

$$\mathcal{L}_{\text{LDM}} = \mathbb{E}_{\mathcal{E}(\mathbf{x}),\boldsymbol{\epsilon},t,\mathbf{y}}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{z}_t, t, \boldsymbol{\tau}_{\boldsymbol{\theta}}(\mathbf{y}))\|_2^2\right] \tag{15.13}$$

Cross-Attention Conditioning

Text conditioning is implemented via cross-attention layers within the UNet. The text encoder $\boldsymbol{\tau}_{\boldsymbol{\theta}}$ (typically a CLIP text encoder) maps the prompt $\mathbf{y}$ to a sequence of token embeddings:

$$\boldsymbol{\tau}_{\boldsymbol{\theta}}(\mathbf{y}) = [\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_L] \in \mathbb{R}^{L \times d_{\text{text}}} \tag{15.14}$$

In each cross-attention layer, the spatial features from the UNet form the query, while the text embeddings form the keys and values:

$$\mathbf{Q} = \mathbf{W}_Q^{(i)} \cdot \boldsymbol{\varphi}_i(\mathbf{z}_t) \in \mathbb{R}^{(h \cdot w) \times d} \tag{15.15}$$

$$\mathbf{K} = \mathbf{W}_K^{(i)} \cdot \boldsymbol{\tau}_{\boldsymbol{\theta}}(\mathbf{y}) \in \mathbb{R}^{L \times d} \tag{15.16}$$

$$\mathbf{V} = \mathbf{W}_V^{(i)} \cdot \boldsymbol{\tau}_{\boldsymbol{\theta}}(\mathbf{y}) \in \mathbb{R}^{L \times d} \tag{15.17}$$

$$\text{CrossAttention}(\mathbf{z}_t, \mathbf{y}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V} \tag{15.18}$$

Cross-attention is inserted at multiple resolutions (64×64, 32×32, 16×16, 8×8 in the UNet), allowing the model to attend to text features at different spatial scales.

Classifier-Free Guidance

For text-to-image generation, classifier-free guidance is essential for strong text alignment:

$$\hat{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\mathbf{z}_t, \mathbf{y}) = \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{z}_t, \varnothing) + w\,(\boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{z}_t, \mathbf{y}) - \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{z}_t, \varnothing)) \tag{15.19}$$

Typical guidance scales are $w \in [5, 15]$ for good balance of fidelity and diversity.

15.2.4 Imagen: Large Frozen Text Encoders

Imagen (Saharia et al., 2022) demonstrated that scaling the text encoder matters more than scaling the image decoder. Key architectural choices:

Text Encoder

A frozen, pre-trained T5-XXL (11B parameters) serves as the text encoder. The text embeddings are obtained from the final encoder hidden states:

$$\boldsymbol{\tau}(\mathbf{y}) = \text{T5-XXL}(\mathbf{y}) \in \mathbb{R}^{L \times 4096} \tag{15.20}$$

The T5 encoder is not fine-tuned during training—it remains frozen. This is crucial because:

$$\text{FID} \propto (1/\text{text_encoder_size})^{\alpha} \tag{15.21}$$

The empirical finding was that increasing the text encoder from T5-small to T5-XXL improved FID on DrawBench more than using a larger diffusion UNet.

Cascaded Diffusion

Imagen uses a cascade of diffusion models at increasing resolutions:

- **Base model:** 64×64 text-to-image diffusion. UNet with cross-attention to T5 embeddings at multiple resolutions.
- **First super-resolution:** $64 \times 64 \to 256 \times 256$. UNet with cross-attention to T5 embeddings, conditioned on the low-resolution image.
- **Second super-resolution:** $256 \times 256 \to 1024 \times 1024$. Same architecture, no attention layers (purely convolutional) for efficiency.

Each super-resolution stage uses *noise conditioning augmentation*: Gaussian noise and Gaussian blur are applied to the low-resolution conditioning image during training:

$$\tilde{\mathbf{x}}_{\text{LR}} = \mathbf{x}_{\text{LR}} + \sigma\boldsymbol{\epsilon}, \quad \sigma \sim q(\sigma) \tag{15.22}$$

This prevents the model from relying too heavily on the low-resolution input and ensures robust upsampling even when the base model's output is imperfect.

Dynamic Thresholding

To prevent saturation artifacts at high guidance scales, dynamic thresholding clips pixel values to a range determined by a percentile of the current sample values:

$$\mathbf{x}_t^{\text{clipped}} = \text{clamp}\left(\mathbf{x}_t, -s, s\right), \quad s = c \cdot \text{percentile}(|\mathbf{x}_t|, p) \tag{15.23}$$

where $c > 1$ and p is a high percentile (e.g., 0.995). This prevents extreme values from accumulating across timesteps.

15.2.5 DALL-E 3: Caption Improvement

DALL-E 3 (Betker et al., 2023) introduced the insight that improving training captions dramatically improves image quality. A large language model (GPT-4) rewrites user prompts into detailed, descriptive captions before feeding them to the image generator:

$$\mathbf{y}_{\text{detailed}} = \text{LLM}(\mathbf{y}_{\text{user}}) \tag{15.24}$$

The image generator is then trained on $(\mathbf{x}, \mathbf{y}_{\text{detailed}})$ pairs. At inference, the user prompt is similarly expanded. Benefits:

- **Better alignment:** Detailed captions explicitly describe objects, attributes, spatial relationships, and style, improving compositionality.
- **Robustness:** The model sees consistent, well-written captions during training, reducing sensitivity to prompt phrasing.
- **Text rendering:** By including explicit text content in captions (e.g., "a sign that says 'HELLO'"), the model learns to render text correctly.

15.3 Evaluation Metrics

15.3.1 Fréchet Inception Distance (FID)

FID measures the distance between the distribution of real and generated images in the feature space of a pre-trained Inception v3 network. Let p_r and p_g be the real and generated distributions, each modeled as a multivariate Gaussian in Inception feature space with means $\boldsymbol{\mu}_r, \boldsymbol{\mu}_g$ and covariances $\boldsymbol{\Sigma}_r, \boldsymbol{\Sigma}_g$:

$$\text{FID} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|_2^2 + \text{Tr}\left(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2(\boldsymbol{\Sigma}_r\boldsymbol{\Sigma}_g)^{1/2}\right) \tag{15.25}$$

Lower FID indicates higher quality and diversity. The trace term captures the difference in feature covariances. FID is sensitive to both mode collapse (reduced diversity $\rightarrow$ smaller $\boldsymbol{\Sigma}_g$) and image quality (poor quality $\rightarrow$ distorted features $\rightarrow$ different $\boldsymbol{\mu}_g$).

15.3.2 CLIP Score

The CLIP score measures text-image alignment as the cosine similarity between the CLIP embeddings of the generated image and the input text:

$$\text{CLIP-Score}(\mathbf{x}, \mathbf{y}) = \cos(f_I(\mathbf{x}), f_T(\mathbf{y})) = \frac{\langle f_I(\mathbf{x}), f_T(\mathbf{y})\rangle}{\|f_I(\mathbf{x})\| \cdot \|f_T(\mathbf{y})\|} \tag{15.26}$$

Higher scores indicate better semantic alignment. A related metric, the *CLIP retrieval score*, measures whether the generated image is more similar to the input prompt than to random distractors.

15.4 Algorithm

Algorithm 15: Text-to-Image Generation with Stable Diffusion (DDIM + CFG)

Input: Text prompt $\mathbf{y}$, trained VAE $(\mathcal{E}, \mathcal{D})$, trained denoising UNet $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}$, text encoder $\boldsymbol{\tau}_{\boldsymbol{\theta}}$, guidance scale w, steps T

Output: Generated image $\mathbf{x}$

$\mathbf{c} \leftarrow \boldsymbol{\tau}_{\boldsymbol{\theta}}(\mathbf{y})$; // Encode text prompt

$\mathbf{c}_{\text{uncond}} \leftarrow \boldsymbol{\tau}_{\boldsymbol{\theta}}(\texttt{""})$; // Null text for CFG

$\mathbf{z}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$; // Sample initial noise in latent space

for $t \leftarrow T$ **to** 1 **do**

  // Classifier-free guidance

  $\boldsymbol{\epsilon}_{\text{cond}} \leftarrow \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{z}_t, t, \mathbf{c})$;

  $\boldsymbol{\epsilon}_{\text{uncond}} \leftarrow \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{z}_t, t, \mathbf{c}_{\text{uncond}})$;

  $\hat{\boldsymbol{\epsilon}} \leftarrow \boldsymbol{\epsilon}_{\text{uncond}} + w(\boldsymbol{\epsilon}_{\text{cond}} - \boldsymbol{\epsilon}_{\text{uncond}})$;

  // DDIM sampling step

  $\hat{\mathbf{z}}_0 \leftarrow \frac{\mathbf{z}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}}$;

  if $t > 1$ **then**

    $\mathbf{z}_{t-1} \leftarrow \sqrt{\bar{\alpha}_{t-1}}\,\hat{\mathbf{z}}_0 + \sqrt{1-\bar{\alpha}_{t-1}}\,\hat{\boldsymbol{\epsilon}}$;

  end

  else

    $\mathbf{z}_0 \leftarrow \hat{\mathbf{z}}_0$;

  end

end

$\mathbf{x} \leftarrow \mathcal{D}(\mathbf{z}_0)$; // Decode latent to pixel space

return $\mathbf{x}$

Algorithm 16: Training a Latent Diffusion Model with Classifier-Free Guidance

Input: Training pairs $\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^{m}$, VAE $(\mathcal{E}, \mathcal{D})$, UNet $\boldsymbol{\epsilon}_{\boldsymbol{\theta}}$, text encoder $\boldsymbol{\tau}_{\boldsymbol{\theta}}$, CFG dropout prob p_{drop}

Output: Trained denoising UNet

for *epoch* $\leftarrow 1$ **to** *max_ epochs* **do**
  for *each mini-batch* $\mathcal{B}$ **do**
    foreach $(\mathbf{x}, \mathbf{y}) \in \mathcal{B}$ **do**
      $\mathbf{z} \leftarrow \mathcal{E}(\mathbf{x})$; `// Encode to latent`
      $t \sim \mathcal{U}\{1, \ldots, T\}$;
      $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$;
      $\mathbf{z}_t \leftarrow \sqrt{\bar{\alpha}_t}\mathbf{z} + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$;
      `// Randomly drop text conditioning for CFG`
      if *rand*() $< p_{drop}$ **then**
        $\mathbf{c} \leftarrow \boldsymbol{\tau}_{\boldsymbol{\theta}}(\texttt{""})$;
      end
      else
        $\mathbf{c} \leftarrow \boldsymbol{\tau}_{\boldsymbol{\theta}}(\mathbf{y})$;
      end
      $\hat{\boldsymbol{\epsilon}} \leftarrow \boldsymbol{\epsilon}_{\boldsymbol{\theta}}(\mathbf{z}_t, t, \mathbf{c})$;
      $\mathcal{L} \leftarrow \|\boldsymbol{\epsilon} - \hat{\boldsymbol{\epsilon}}\|_2^2$;
    end
    Update $\boldsymbol{\theta}$ via gradient descent on $\frac{1}{|\mathcal{B}|}\sum \mathcal{L}$;
  end
end

15.5 Pros and Cons

15.5.1 Per-Model Comparison

DALL-E (2021):

+ Novel autoregressive approach; discrete tokens enable diverse operations; dVAE enables efficient tokenization

− 12B parameter transformer is enormous; sequential token generation is slow; raster-scan ordering lacks 2D structure

DALL-E 2 (2022):

+ Semantically rich CLIP embedding space; diffusion prior + decoder separation of concerns; superior text alignment

− Two-stage model is complex; CLIP embedding may miss fine details; slower than single-stage latent diffusion

Stable Diffusion (2022):

+ Runs on consumer GPUs (latent space efficiency); open-source and community-driven; extensive ecosystem (ControlNet, LoRA, IP-Adapter); faster than pixel-space diffusion

– VAE compression loses fine details; CLIP text encoder limits prompt understanding; hyperparameter-sensitive guidance

Imagen (2022):

+ Superior text understanding from T5-XXL; cascaded architecture naturally handles high resolution; dynamic thresholding improves stability

– Extremely computationally expensive (pixel-space diffusion at multiple scales); not open-source; frozen T5 limits domain adaptation

DALL-E 3 (2023):

+ Caption improvement via LLM dramatically improves text following; excellent spatial composition and text rendering

– Dependency on proprietary LLM; longer inference (LLM rewrite + generation); closed-source

15.6 Evolution

The evolution of text-to-image generation traces a dramatic arc from toy examples to photorealistic, prompt-controllable systems:

- **GAN-based (2016–2019):** StackGAN, AttnGAN, and DM-GAN used GANs conditioned on text embeddings. Image quality was limited, and text alignment was poor for complex scenes.
- **DALL-E (2021):** Demonstrated that a large autoregressive transformer over discrete image tokens could generate diverse, creative images from text. The discrete tokenization approach was novel but computationally expensive.
- **Motion to diffusion (2022):** DALL-E 2, Stable Diffusion, and Imagen all moved to diffusion-based generation. The shift from autoregressive to diffusion was driven by superior sample quality and the natural fit of iterative denoising for continuous image generation.
- **DALL-E 2 (2022):** Pioneered the CLIP-latent-as-bridge approach. The diffusion prior + decoder split enabled explicit semantic reasoning in the prior and photorealistic rendering in the decoder.
- **Stable Diffusion (2022):** Democratized text-to-image by running diffusion in latent space and releasing the model openly. The community rapidly developed tools for fine-tuning (DreamBooth, LoRA), controlled generation (ControlNet), and prompt engineering.
- **Imagen (2022):** Demonstrated the critical importance of scaling the text encoder rather than the image decoder. Used a frozen T5-XXL, achieving SOTA text alignment on benchmarks like DrawBench.
- **Midjourney (2022–2024):** Iterated rapidly through versions (v1 to v6), focusing on aesthetic quality and user experience. While architecture details are proprietary, it likely uses a variant of latent diffusion with aesthetic-aware fine-tuning.

- **DALL-E 3 (2023):** Introduced caption improvement via LLM rewriting, a simple but powerful technique that dramatically improved text following and compositional accuracy. This insight can benefit any text-to-image model.
- **SDXL (2023):** Scaled Stable Diffusion with a larger UNet, dual text encoders, and an additional refinement model, closing the gap with proprietary systems.
- **Flux (2024):** Introduced flow matching with a Diffusion Transformer (DiT) backbone, combining the stability of diffusion training with a simplified trajectory formulation.
- **DiT-based models (2023–2024):** The Diffusion Transformer architecture (DiT) replaces the UNet with a transformer operating on patchified latents, showing that vision transformers scale better than UNets for large diffusion models. This has become the standard architecture for state-of-the-art models.

The key architectural trends are clear: movement from autoregressive to diffusion, from pixel-space to latent-space, from small text encoders to (frozen) large language models, from UNets to transformers for the denoising backbone, and from training on raw captions to training on LLM-improved captions. The field continues to advance rapidly in sampling speed, controllability, and compositional accuracy.

Part V

Object Detection

Chapter 16

The RCNN Family: From Region Proposals to End-to-End Detection

16.1 Motivation

Object detection requires simultaneous localization and classification. Before deep convolutional networks became dominant, detection pipelines relied on sliding-window classifiers with hand-engineered features such as HOG (Histogram of Oriented Gradients) and SIFT (Scale-Invariant Feature Transform). A classifier—typically a linear SVM—was evaluated at every position and scale of a densely sampled image pyramid. This paradigm suffered from quadratic computational complexity in the number of window locations and scales, and hand-crafted features lacked the representational capacity to model high-level semantic concepts. The fundamental insight of the RCNN family was to replace the exhaustive sliding-window search with category-agnostic region proposals, and to replace hand-engineered features with features extracted by a deep convolutional neural network pre-trained on large-scale image classification. This shifted the computational burden from feature design to data-driven representation learning and enabled unprecedented detection accuracy on PASCAL VOC and MS COCO benchmarks.

16.2 RCNN

16.2.1 Mathematical Formulation

Let an input image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ be processed by a region proposal algorithm $\mathcal{P}$ that generates a set of candidate object regions $\mathcal{R} = \{R_1, R_2, \ldots, R_K\}$, where typically $K \approx 2000$. Each region $R_i = (x_i, y_i, w_i, h_i)$ is a bounding box in image coordinates. The original RCNN employs **Selective Search**, which constructs a hierarchical grouping of superpixels based on four similarity measures:

$$s_{\text{color}}(r_i, r_j) = \sum_{k=1}^{n_{\text{bins}}} \min(c_i^k, c_j^k) \tag{16.1}$$

$$s_{\text{texture}}(r_i, r_j) = \sum_{k=1}^{n_{\text{bins}}} \min(t_i^k, t_j^k) \tag{16.2}$$

$$s_{\text{size}}(r_i, r_j) = 1 - \frac{\text{size}(r_i) + \text{size}(r_j)}{\text{size}(\text{image})} \tag{16.3}$$

$$s_{\text{fill}}(r_i, r_j) = 1 - \frac{\text{size}(BB_{ij}) - \text{size}(r_i) - \text{size}(r_j)}{\text{size}(\text{image})} \tag{16.4}$$

where c_i^k, t_i^k are color and texture histograms of region r_i, and BB_{ij} is the tight bounding box around $r_i \cup r_j$. The combined similarity is:

$$s(r_i, r_j) = a_1 s_{\text{color}} + a_2 s_{\text{texture}} + a_3 s_{\text{size}} + a_4 s_{\text{fill}} \tag{16.5}$$

Each proposed region R_i is warped to a fixed input size 227×227 (the AlexNet input dimension) via anisotropic scaling (ignoring aspect ratio). The warped region $\mathbf{I}_i^{\text{warp}}$ is fed through a CNN Φ (AlexNet pre-trained on ImageNet) to produce a fixed-length feature vector:

$$\phi(R_i) = \Phi(\mathbf{I}_i^{\text{warp}}) \in \mathbb{R}^{4096} \tag{16.6}$$

taken from the pool5 (or fc7) layer activations.

Bounding Box Regression

Given a proposed box $P = (P_x, P_y, P_w, P_h)$ and a ground-truth box $G = (G_x, G_y, G_w, G_h)$, the goal is to learn a transformation that maps P to a predicted box $\hat{G}$ close to G. The transformation is parameterized as scale-invariant offsets:

$$t_x = \frac{G_x - P_x}{P_w}, \quad t_y = \frac{G_y - P_y}{P_h} \tag{16.7}$$

$$t_w = \log \frac{G_w}{P_w}, \quad t_h = \log \frac{G_h}{P_h} \tag{16.8}$$

The regressor predicts offsets $d_*(P) = \mathbf{w}_*^\top \phi_5(P)$ for $* \in \{x, y, w, h\}$, where $\phi_5(P)$ is the pool5 feature vector of the proposed region. The learning objective is regularized least squares:

$$\mathbf{w}_* = \arg\min_{\hat{\mathbf{w}}_*} \sum_{i=1}^{N} \sum_{*\in\{x,y,w,h\}} L_{\text{smooth}}^{\ell_1}\big(t_*^i - \hat{\mathbf{w}}_*^\top \phi_5(P^i)\big) + \lambda \left\|\hat{\mathbf{w}}_*\right\|_2^2 \tag{16.9}$$

where the smooth ℓ_1 loss (Huber loss) is defined as:

$$L_{\text{smooth}}^{\ell_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases} \tag{16.10}$$

This loss is less sensitive to outliers than ℓ_2 loss while being differentiable everywhere, unlike pure ℓ_1.

SVM Classification and Training Pipeline

RCNN training proceeds in three separate stages:

Stage 1 — CNN Fine-tuning. AlexNet (pre-trained on ImageNet 1000-way classification) is fine-tuned for $N + 1$ classes (N object classes + background) using warped region proposals. Regions with IoU ≥ 0.5 with any ground-truth box are treated as positives for that class; regions with IoU < 0.3 with all ground-truth boxes are negatives.

Stage 2 — SVM Training. The fine-tuned CNN is used as a fixed feature extractor. For each class c, a binary linear SVM is trained on pool5 features $\phi_5(R_i)$. The positive/negative definition differs from fine-tuning: only ground-truth boxes are positives for class c, and proposals with IoU < 0.3 are negatives. Hard negative mining is employed iteratively.

Stage 3 — Bounding Box Regressor Training. For each class, a ridge regression model is trained using pool5 features to predict the four transformation parameters (t_x, t_y, t_w, t_h). Only proposals with IoU ≥ 0.6 with ground-truth are used.

16.2.2 Intersection over Union and Non-Maximum Suppression

The **Intersection over Union** metric quantifies spatial overlap:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{\max(0, \min(x_A^r, x_B^r) - \max(x_A^l, x_B^l)) \cdot \max(0, \min(y_A^b, y_B^b) - \max(y_A^t, y_B^t))}{|A| + |B| - |A \cap B|} \tag{16.11}$$

Non-Maximum Suppression (NMS) removes duplicate detections. Given a set of detections $\mathcal{D} = \{(b_i, s_i)\}$ with boxes b_i and confidence scores s_i, the NMS algorithm is:

1. Sort $\mathcal{D}$ by confidence s_i in descending order.
2. Select the highest-scoring box b^* and add it to the final set $\mathcal{F}$.
3. Remove all boxes b_j from $\mathcal{D}$ for which $\text{IoU}(b^*, b_j) \geq \tau$ (typically $\tau = 0.5$).
4. Repeat steps 2–3 until $\mathcal{D}$ is empty.

16.2.3 Algorithm

Algorithm 17: RCNN Inference

Input: Image $\mathbf{I}$, region proposal function $\mathcal{P}$, CNN Φ, SVMs $\{S_c\}_{c=1}^{C}$, regressors $\{\mathcal{R}_c\}_{c=1}^{C}$

Output: Detections $\mathcal{D} = \{(c, b, s)\}$

$\mathcal{R} \leftarrow \mathcal{P}(\mathbf{I})$;
foreach $R \in \mathcal{R}$ **do**
  $\mathbf{I}^{\text{warp}} \leftarrow \text{Warp}(R, 227, 227)$;
  $\phi \leftarrow \Phi(\mathbf{I}^{\text{warp}})$;
  foreach $c \in \{1, \ldots, C\}$ **do**
    $s_c \leftarrow S_c(\phi)$;
  end
  $c^* \leftarrow \arg\max_c s_c$;
  if $s_{c^*} > 0$ **then**
    $(t_x, t_y, t_w, t_h) \leftarrow \mathcal{R}_{c^*}(\phi)$;
    $\hat{G}_x \leftarrow P_x + t_x P_w$;
    $\hat{G}_y \leftarrow P_y + t_y P_h$;
    $\hat{G}_w \leftarrow P_w \exp(t_w)$;
    $\hat{G}_h \leftarrow P_h \exp(t_h)$;
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(c^*, \hat{G}, s_{c^*})\}$;
  end
end
$\mathcal{D} \leftarrow \text{NMS}(\mathcal{D}, \tau = 0.5)$;
return $\mathcal{D}$;

16.2.4 Architecture

The RCNN pipeline is modular but not end-to-end differentiable. The CNN (AlexNet) has 5 convolutional layers followed by 3 fully-connected layers: conv1 $\to$ pool1 $\to$ conv2 $\to$ pool2 $\to$ conv3 $\to$ conv4 $\to$ conv5 $\to$ pool5 $\to$ fc6 $\to$ fc7 $\to$ fc8. The fc7 features (4096-d) are extracted and fed to class-specific SVMs and regressors. The three modules—region proposal, feature extraction, and classification/regression—are trained independently with no shared gradient flow.

16.2.5 Pros and Cons

Pros. Dramatic improvement over deformable parts models (DPM): 53.7% mAP vs. 33.4% on PASCAL VOC 2010. Leverages transfer learning from ImageNet pre-training. Bounding box regression refines localization.

Cons. Inference is extremely slow (47 seconds per image on GPU) because each of the 2000 region proposals goes through a full CNN forward pass independently—there is no shared computation. Multi-stage training (CNN fine-tuning $\to$ SVM training $\to$ regressor training) is cumbersome and requires storing extracted features to disk (hundreds of GB). Region proposals from Selective Search add a serial bottleneck (2 seconds per image). Deep features must be cached offline, precluding online fine-tuning.

16.3 Fast RCNN

16.3.1 Mathematical Formulation

Fast RCNN addresses the fundamental inefficiency of RCNN—redundant CNN computation on overlapping regions—by processing the entire image with a single convolutional forward pass, then extracting region features via a differentiable **RoI Pooling** layer.

RoI Pooling

Let the convolutional feature map be $\mathbf{X} \in \mathbb{R}^{C \times H' \times W'}$. An RoI (r, c, h, w) in image coordinates is projected to feature map coordinates by the spatial stride s (typically 16 for VGG-16 after conv5): $r' = \lfloor r/s \rfloor$, $c' = \lfloor c/s \rfloor$, $h' = \lfloor h/s \rfloor$, $w' = \lfloor w/s \rfloor$. The projected RoI is divided into an $H \times W$ grid (typically 7×7). For each grid cell (i, j), the pooling operation is:

$$y_{ijk} = \max_{(p,q) \in \mathrm{bin}(i,j)} x_{kpq} \tag{16.12}$$

where $\mathrm{bin}(i,j) = \left\{(p,q) \mid \lfloor i\frac{h'}{H} \rfloor \le p < \lceil (i+1)\frac{h'}{H} \rceil,\ \lfloor j\frac{w'}{W} \rfloor \le q < \lceil (j+1)\frac{w'}{W} \rceil \right\}$.

The backward pass of RoI pooling propagates gradients only to the argmax positions:

$$\frac{\partial L}{\partial x_{kpq}} = \sum_r \sum_{i,j} \mathbb{1}\big[(p,q) = (p^*, q^*) \text{ for bin } (i,j) \text{ in RoI } r\big] \cdot \frac{\partial L}{\partial y_{rijk}} \tag{16.13}$$

where $(p^*, q^*) = \arg\max_{(p,q) \in \mathrm{bin}(i,j)} x_{kpq}$.

Multi-Task Loss

Fast RCNN unifies classification and localization into a single end-to-end loss. For a given RoI with ground-truth class u and ground-truth box regression targets $v = (v_x, v_y, v_w, v_h)$, the loss is:

$$L(p, u, t^u, v) = L_{\text{cls}}(p, u) + \lambda \cdot \mathbb{1}[u \geq 1] \cdot L_{\text{loc}}(t^u, v) \tag{16.14}$$

where $p = (p_0, p_1, \ldots, p_C)$ is the discrete probability distribution over $C + 1$ classes (including background, $u = 0$), and $t^u = (t^u_x, t^u_y, t^u_w, t^u_h)$ is the predicted bounding box regression offsets for class u. λ is a balancing hyperparameter (typically $\lambda = 1$).

$$L_{\text{cls}}(p, u) = -\log p_u \tag{16.15}$$

$$L_{\text{loc}}(t^u, v) = \sum_{*\in\{x,y,w,h\}} L^{\ell_1}_{\text{smooth}}(t^u_* - v_*) \tag{16.16}$$

The indicator $\mathbb{1}[u \geq 1]$ ensures localization loss is computed only for non-background RoIs (background has $u = 0$ and no meaningful bounding box targets). The regression targets v are encoded as:

$$v_x = \frac{G_x - P_x}{P_w}, \quad v_y = \frac{G_y - P_y}{P_h}, \quad v_w = \log \frac{G_w}{P_w}, \quad v_h = \log \frac{G_h}{P_h} \tag{16.17}$$

Mini-Batch Sampling

During training, each mini-batch is formed by sampling $N = 2$ images uniformly and then sampling $R/N = 64$ RoIs from each image. Of these, 25% have IoU ≥ 0.5 with a ground-truth box (foreground, assigned to the highest-IoU class), and 75% have IoU $\in [0.1, 0.5)$ with all ground-truth boxes (background, $u = 0$). This hierarchical sampling leverages the fact that RoIs from the same image share convolutional computation.

16.3.2 Algorithm

Algorithm 18: Fast RCNN Training

Input: Images $\{\mathbf{I}_i\}$, CNN backbone Φ_{conv}, RoI head Φ_{roi}, classifier W_{cls}, regressor W_{reg}

Output: Updated parameters $\Theta = \{\theta_{\text{conv}}, \theta_{\text{roi}}, W_{\text{cls}}, W_{\text{reg}}\}$

for *each mini-batch of 2 images* **do**
  $\mathbf{X} \leftarrow \Phi_{\text{conv}}(\mathbf{I}_1, \mathbf{I}_2)$;
  $\mathcal{R} \leftarrow \text{SelectiveSearch}(\mathbf{I}_1) \cup \text{SelectiveSearch}(\mathbf{I}_2)$;
  Sample 64 RoIs from $\mathcal{R}$ with 1:3 foreground/background ratio;
  for $r \in$ *sampled RoIs* **do**
    $\mathbf{f}_r \leftarrow \text{RoIPool}(\mathbf{X}, r, 7 \times 7)$;
    $\mathbf{f}_r \leftarrow \Phi_{\text{roi}}(\mathbf{f}_r)$;
    $p_r \leftarrow \text{softmax}(W_{\text{cls}}\mathbf{f}_r + \mathbf{b}_{\text{cls}})$;
    $t_r \leftarrow W_{\text{reg}}\mathbf{f}_r + \mathbf{b}_{\text{reg}}$;
  end
  $L \leftarrow \frac{1}{N}\sum_r \big(L_{\text{cls}}(p_r, u_r) + \lambda \mathbb{1}[u_r \geq 1] L_{\text{loc}}(t_r^{u_r}, v_r)\big)$;
  $\Theta \leftarrow \Theta - \eta \nabla_\Theta L$;
end

16.3.3 Architecture

Fast RCNN replaces the final max-pooling layer of a pre-trained CNN (e.g., VGG-16) with an RoI pooling layer, followed by fully-connected layers that branch into a $(C+1)$-way softmax classifier and a $4C$-dimensional bounding box regressor. The SVD decomposition trick is applied to the large fully-connected layers ($W \approx U\Sigma_t V^\top$) to accelerate inference, reducing the parameter count by replacing one $N \times M$ weight matrix with two matrices of total size $N \times t + t \times M$ where $t \ll \min(N, M)$. This achieves 30% speedup with minimal accuracy loss.

16.3.4 Pros and Cons

Pros. Single-stage end-to-end training with a unified multi-task loss. Forward pass on the entire image once via shared convolutions— 0.3 s per image vs. 47 s for RCNN. No disk storage required for feature caching. Significantly higher mAP than RCNN (70.0% vs. 66.0% on VOC 2007) while being 150× faster at test time.

Cons. The region proposal step (Selective Search) remains external and non-differentiable, running on CPU and taking approximately 2 seconds per image—this dominates the inference time budget. RoI pooling performs quantization that misaligns features and bounding boxes (fixed in Mask RCNN via RoIAlign). Training still requires pre-computed region proposals.

16.4 Faster RCNN

16.4.1 Mathematical Formulation

Faster RCNN eliminates the external region proposal bottleneck by introducing a **Region Proposal Network (RPN)** that shares convolutional features with the detection network. The RPN is a fully convolutional network that simultaneously predicts objectness scores and bounding box regressions at each spatial location of a convolutional feature map.

Anchor Boxes

At each spatial location on the feature map (of stride s, typically 16 pixels), the RPN considers k reference boxes called **anchors**. Anchors are defined by a base size B (e.g., 16 on the feature map, corresponding to $B \cdot s$ pixels in the image), m scales $\{s_1, \ldots, s_m\}$, and n aspect ratios $\{r_1, \ldots, r_n\}$, giving $k = m \times n$ anchors per location. The canonical configuration uses $m = 3$ scales $\{128, 256, 512\}$ and $n = 3$ ratios $\{1{:}2, 1{:}1, 2{:}1\}$, yielding $k = 9$ anchors.

For each anchor $a = (a_x, a_y, a_w, a_h)$, the RPN outputs:

- A 2-way classification score $(p_{\text{fg}}, p_{\text{bg}})$ via softmax — whether the anchor contains an object.
- A 4-tuple regression offset (t_x, t_y, t_w, t_h) — transformation from anchor to predicted box $\hat{G}$:

$$\hat{G}_x = a_x + t_x a_w, \quad \hat{G}_y = a_y + t_y a_h, \quad \hat{G}_w = a_w \exp(t_w), \quad \hat{G}_h = a_h \exp(t_h) \tag{16.18}$$

Anchor Assignment and RPN Loss

An anchor is assigned a **positive** label ($p^* = 1$) if it has either (a) IoU > 0.7 with any ground-truth box, or (b) the highest IoU with a ground-truth box (to ensure at least one positive per ground-truth). An anchor is **negative** ($p^* = 0$) if its IoU is < 0.3 with all ground-truth boxes. Anchors with $0.3 \leq \text{IoU} \leq 0.7$ are ignored (no loss contribution).

The RPN multi-task loss for a mini-batch is:

$$L_{\text{RPN}}(\{p_i\}, \{t_i\}) = \frac{1}{N_{\text{cls}}} \sum_i L_{\text{cls}}(p_i, p_i^*) + \lambda \frac{1}{N_{\text{reg}}} \sum_i p_i^* \cdot L_{\text{reg}}(t_i, t_i^*) \tag{16.19}$$

where L_{cls} is binary cross-entropy, L_{reg} is smooth ℓ_1 loss, p_i^* is the ground-truth label (1 for positive, 0 for negative), and t_i^* is the ground-truth regression target from the positive anchor to the matched ground-truth box:

$$t_x^* = \frac{G_x - a_x}{a_w}, \quad t_y^* = \frac{G_y - a_y}{a_h}, \quad t_w^* = \log \frac{G_w}{a_w}, \quad t_h^* = \log \frac{G_h}{a_h} \tag{16.20}$$

N_{cls} is the mini-batch size (typically 256, balanced 1:1 positive/negative with padding) and N_{reg} is the number of anchor locations (2400). $\lambda \approx 10$ balances the two terms.

Training Procedure: 4-Step Alternating Training

1. Train RPN initialized with ImageNet-pre-trained weights. The shared convolutional layers are fine-tuned.

2. Train Fast RCNN detection network using proposals from the RPN (step 1, frozen). Shared conv layers are again initialized from ImageNet pre-training; RPN and Fast RCNN do not share conv layers at this stage.

3. Freeze the shared conv layers from step 2 and fine-tune only the RPN-specific layers. Now the two networks share convolutional features.

4. Freeze the shared conv layers and fine-tune only the Fast RCNN-specific layers (fc layers). This yields a unified network with shared convolutional computation.

Approximate Joint Training

An alternative to 4-step training merges the RPN and Fast RCNN losses into a single joint objective:

$$L = L_{\text{RPN}} + L_{\text{Fast RCNN}} \tag{16.21}$$

where gradients from the Fast RCNN head are treated as constant w.r.t. the proposal coordinates (ignoring the non-differentiable NMS step), enabling near end-to-end training in one pass.

16.4.2 Algorithm

Algorithm 19: Faster RCNN Inference with RPN

Input: Image $\mathbf{I}$, shared CNN Φ_{shared}, RPN head Φ_{rpn}, Fast RCNN head Φ_{det}
Output: Detections $\mathcal{D}$

$\mathbf{X} \leftarrow \Phi_{\text{shared}}(\mathbf{I})$;
foreach *spatial location* (i, j) *on* $\mathbf{X}$ **do**
  foreach *anchor* a_k *at* (i, j) **do**
    $p_k, t_k \leftarrow \Phi_{\text{rpn}}(\mathbf{X}[i, j])$;
    if $p_k(fg) > 0$ **then**
      $\hat{G}_k \leftarrow \text{decode}(t_k, a_k)$;
    end
  end
end
$\mathcal{P} \leftarrow \text{NMS}(\{\hat{G}_k \mid p_k(\text{fg}) > 0\}, \tau_{\text{rpn}} = 0.7)$;
$\mathcal{P} \leftarrow$ top-R $\mathcal{P}$ by score;
foreach $P \in \mathcal{P}$ **do**
  $\mathbf{f}_P \leftarrow \text{RoIPool}(\mathbf{X}, P, 7 \times 7)$;
  $p_P, t_P \leftarrow \Phi_{\text{det}}(\mathbf{f}_P)$;
  $c^* \leftarrow \arg\max_c p_P$;
  if $c^* > 0$ **then**
    $\hat{G}_P \leftarrow \text{decode}(t_P^{c^*}, P)$;
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(c^*, \hat{G}_P, p_P(c^*))\}$;
  end
end
$\mathcal{D} \leftarrow \text{NMS}(\mathcal{D}, \tau_{\text{det}} = 0.5)$;
return $\mathcal{D}$;

16.4.3 Architecture

Faster RCNN consists of three interconnected modules sharing a common convolutional backbone (VGG-16, ResNet-50/101, or ZF-Net): (1) the **RPN** implemented as a 3×3 conv layer followed by two sibling 1×1 conv layers for classification (outputting $2k$ channels) and regression (outputting $4k$ channels); (2) **RoI Pooling** to extract fixed-size features for each proposal; (3) the **Fast RCNN head** with fully-connected layers, a $(C+1)$-way softmax classifier, and per-class bounding box regressors. The entire pipeline is differentiable except for the NMS and proposal sampling steps, which are treated as fixed during backpropagation.

16.4.4 Pros and Cons

Pros. Fully end-to-end trainable object detector. Near real-time inference (5–17 fps depending on backbone; 200 ms per image). RPN proposals ($<$ 300 per image) are higher quality and more efficient than Selective Search. RPN + Fast RCNN share convolutional features, minimizing redundant computation. Achieves state-of-the-art accuracy on PASCAL VOC (76.4% mAP) and MS COCO.

Cons. Two-stage design still slower than single-shot detectors. Small object detection remains challenging because the feature map stride (16 pixels) limits the effective spatial resolution. Region proposals are computed on a single feature map, making the detector sensitive to object scale variation. NMS is a hard non-differentiable operation. Training requires careful multi-stage scheduling for stability.

16.4.5 Evolution of the RCNN Family

The RCNN lineage progressed from modular (RCNN) to semi-shared (Fast RCNN) to fully end-to-end (Faster RCNN), with each iteration removing a computational or optimization bottleneck. Key descendants include:

Feature Pyramid Networks (FPN, Lin et al. 2017). FPN extends Faster RCNN by constructing a top-down feature pyramid with lateral connections: $\mathbf{P}_\ell = \text{Conv}_{3\times3}(\mathbf{C}_\ell + \text{Upsample}(\mathbf{P}_{\ell+1}))$, enabling the RPN and detector heads to operate on multiple feature map resolutions. This dramatically improves small object detection by assigning anchors of different scales to different pyramid levels via $k = k_0 + \log_2(\sqrt{wh}/224)$, where k_0 is the target level for 224×224 objects.

Mask RCNN (He et al. 2017). Adds a parallel FCN branch to Faster RCNN for predicting 28×28 binary masks per RoI, enabling instance segmentation. Replaces RoI Pooling with **RoIAlign**—using bilinear interpolation instead of quantization to preserve spatial correspondence:

$$y_{ijk} = \sum_{(p,q)\in \text{bin}(i,j)} x_{kpq} \cdot \max(0, 1 - |p - \hat{p}|) \cdot \max(0, 1 - |q - \hat{q}|) \tag{16.22}$$

with a multi-task loss $L = L_{\text{cls}} + L_{\text{box}} + L_{\text{mask}}$, where L_{mask} is per-pixel sigmoid binary cross-entropy (decoupled from class prediction).

Cascade RCNN (Cai & Vasconcelos 2018). Trains a sequence of detectors with increasing IoU thresholds ($\tau = 0.5, 0.6, 0.7$), where each stage uses proposals regressed by the previous stage.

Subsequent directions. Region-based methods have been generalized to multi-task learning (keypoint detection, dense pose), video object detection via feature propagation, and integration with transformer-based architectures producing end-to-end set prediction (DETR). The mAP metric continues to be the standard evaluation: $\text{AP} = \int_0^1 p(r)\, dr$, where $p(r)$ is the precision at recall r, and mAP averages AP over all classes. Modern benchmarks report mAP_{50}, mAP_{75}, and $\text{mAP}_{[0.5:0.95]}$ (averaged over IoU thresholds from 0.5 to 0.95 in steps of 0.05).

Chapter 17

YOLO: You Only Look Once — Unified, Real-Time Object Detection

17.1 Motivation

Two-stage detectors (RCNN, Fast RCNN, Faster RCNN) decompose object detection into region proposal and region classification stages. While accurate, this paradigm imposes a fundamental latency floor: the region proposal step must complete before classification can begin, and proposals are processed sequentially or in batched groups. For applications requiring real-time perception—autonomous driving, drone navigation, video surveillance—throughput measured in frames per second (fps) is critical, and the two-stage overhead is prohibitive. YOLO reframes object detection as a single regression problem: given an image, predict bounding box coordinates and class probabilities directly from image pixels in one evaluation of a neural network. This end-to-end design enables inference at 45–155 fps while maintaining competitive accuracy, fundamentally trading spatial precision for temporal efficiency.

17.2 YOLOv1

17.2.1 Mathematical Formulation

YOLOv1 partitions the input image into an $S \times S$ grid (canonically $S = 7$). Each grid cell is responsible for detecting objects whose center falls within that cell. For each grid cell, the model predicts:

- B bounding boxes (canonically $B = 2$), each parameterized as (x, y, w, h, c).
- C conditional class probabilities $\Pr(\text{class}_i \mid \text{Object})$.

The coordinates (x, y) represent the bounding box center **relative to the grid cell boundaries** (normalized to $[0, 1]$). The dimensions (w, h) are **relative to the whole image** dimensions (also normalized to $[0, 1]$). The confidence score c is defined as:

$$c = \Pr(\text{Object}) \times \text{IoU}(\text{pred}, \text{truth}) \tag{17.1}$$

where $\Pr(\text{Object})$ reflects whether any object center falls in the cell (typically 1 if a ground-truth exists, 0 otherwise), and $\text{IoU}(\text{pred}, \text{truth})$ is the Intersection over Union between the predicted box and the ground-truth. During inference, the class-specific confidence for each box is:

$$\Pr(\text{class}_i) \times \text{IoU} = \Pr(\text{class}_i \mid \text{Object}) \times \Pr(\text{Object}) \times \text{IoU}(\text{pred}, \text{truth}) \tag{17.2}$$

The full output tensor has dimensions $S \times S \times (B \cdot 5 + C)$. For the PASCAL VOC configuration: $7 \times 7 \times (2 \cdot 5 + 20) = 7 \times 7 \times 30$.

17.2.2 Loss Function

The YOLOv1 loss function is a multi-part sum of squared errors, carefully weighted to balance localization, confidence, and classification objectives:

$$
\begin{aligned}
L = \lambda_{\text{coord}} & \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \Big[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \Big] \qquad (17.3) \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \Big[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \Big] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbb{1}_{i}^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \qquad (17.4)
\end{aligned}
$$

where:

- $\mathbb{1}_{ij}^{\text{obj}}$ is 1 if the j-th bounding box predictor in cell i is **responsible** for that prediction (among the B predictors in the cell, the one with highest current IoU with the ground-truth box).
- $\mathbb{1}_{ij}^{\text{noobj}}$ is 1 if there is no object in cell i, used for the no-object confidence penalty.
- $\mathbb{1}_{i}^{\text{obj}}$ is 1 if an object center falls in cell i.
- $\lambda_{\text{coord}} = 5$: upweights localization loss. Most grid cells contain no objects, pushing confidence scores toward zero, which can overpower the gradient from cells containing objects.
- $\lambda_{\text{noobj}} = 0.5$: downweights the confidence loss for cells without objects, preventing them from dominating the optimization.

The square root transformation on width and height $(\sqrt{w}, \sqrt{h})$ is critical: bounding box dimensions on large and small objects differ by orders of magnitude. Sum-of-squared errors weights large and small box errors equally in absolute terms, but a small deviation matters far more visually for a small object (e.g., 2-pixel offset on a 10-pixel box is a 20% error). The square root partially corrects for this scale sensitivity:

$$
\frac{\partial (\sqrt{w} - \sqrt{\hat{w}})^2}{\partial w} = \frac{\sqrt{w} - \sqrt{\hat{w}}}{\sqrt{w}} \qquad (17.5)
$$

The gradient magnitude is inversely proportional to $\sqrt{w}$, giving larger gradients for small box predictions than for large box predictions with the same absolute error.

17.2.3 Architecture

The YOLOv1 network consists of 24 convolutional layers (inspired by GoogLeNet's inception architecture) followed by 2 fully-connected layers. The convolutional layers alternate 1×1 reduction layers with 3×3 convolutional layers:

$$\begin{aligned}\text{Input}(448 \times 448 \times 3) &\to \text{Conv}(7 \times 7, 64, \text{stride } 2) \to \text{MaxPool}(2 \times 2, \text{stride } 2) \\ &\to \text{Conv}(3 \times 3, 192) \to \text{MaxPool}(2 \times 2, \text{stride } 2) \\ &\to \text{Conv}(1 \times 1, 128) \to \text{Conv}(3 \times 3, 256) \to \text{Conv}(1 \times 1, 256) \to \text{Conv}(3 \times 3, 512) \\ &\to \text{MaxPool}(2 \times 2, \text{stride } 2) \to [\text{4 repeats of } (1 \times 1, 3 \times 3) \text{ with 256-512 filters}] \\ &\to [\text{2 repeats of } (1 \times 1, 3 \times 3) \text{ with 512-1024 filters}] \\ &\to \text{Conv}(3 \times 3, 1024) \to \text{Conv}(3 \times 3, 1024, \text{stride } 2) \\ &\to \text{Conv}(3 \times 3, 1024) \to \text{Conv}(3 \times 3, 1024) \\ &\to \text{FC}(4096) \to \text{FC}(S \times S \times (B \cdot 5 + C))\end{aligned}$$

All convolutional layers use leaky ReLU activation: $\phi(x) = \max(0.1x, x)$. The input resolution 448×448 is twice the ImageNet pre-training resolution (224×224), requiring a high-resolution classifier fine-tuning stage.

17.2.4 Algorithm

Algorithm 20: YOLOv1 Inference

Input: Image $\mathbf{I} \in \mathbb{R}^{448\times448\times3}$, trained network Φ
Output: Detections $\mathcal{D}$

$\mathbf{T} \leftarrow \Phi(\mathbf{I}) \in \mathbb{R}^{S\times S\times(B\cdot5+C)}$;
$\mathcal{D} \leftarrow \emptyset$;
for $i \leftarrow 0$ **to** $S-1$ **do**
  for $j \leftarrow 0$ **to** $S-1$ **do**
    $p_{\text{class}} \leftarrow \sigma(\mathbf{T}[i, j, B\cdot5 \colon B\cdot5+C])$;
    for $b \leftarrow 0$ **to** $B-1$ **do**
      $(\hat{x}, \hat{y}, \hat{w}, \hat{h}, \hat{c}) \leftarrow \mathbf{T}[i, j, b\cdot5 \colon (b+1)\cdot5]$;
      $x \leftarrow (j + \sigma(\hat{x}))/S$;
      $y \leftarrow (i + \sigma(\hat{y}))/S$;
      $w \leftarrow \hat{w}^2$;
      $h \leftarrow \hat{h}^2$;
      $s_{\text{cls}} \leftarrow p_{\text{class}} \times \sigma(\hat{c})$;
      $c^* \leftarrow \arg\max_c s_{\text{cls}}$;
      if $s_{cls}[c^*] > \tau_{conf}$ **then**
        $\mathcal{D} \leftarrow \mathcal{D} \cup \{(c^*, (x, y, w, h), s_{\text{cls}}[c^*])\}$;
      end
    end
  end
end
$\mathcal{D} \leftarrow \text{NMS}(\mathcal{D}, \tau_{\text{iou}} = 0.5)$;
return $\mathcal{D}$;

17.2.5 Pros and Cons

Pros. Extremely fast: 45 fps on Titan X (base YOLO), 155 fps (Fast YOLO with 9 conv layers). Reasons about the image globally—the whole image is processed in one forward pass, giving the model contextual information about the entire scene and background. Generalizes well to new domains (natural images vs. artwork), unlike region-proposal methods that overfit to proposal characteristics. Learns generalizable representations of objects.

Cons. Strong spatial constraints: each grid cell predicts at most B objects of possibly one class, limiting detection of dense, overlapping, or clustered objects (e.g., flocks of birds, crowded pedestrians). Struggles with small objects because the coarse 7×7 grid provides limited spatial resolution. The loss function treats small and large bounding box errors equally (despite square-root correction), and the coordinate predictions (x, y) are unconstrained (can leave the grid cell during training, causing instability). Bounding box priors (aspect ratios, scales) are not used, forcing the network to learn diverse box shapes from scratch.

17.3 YOLOv2 / YOLO9000

17.3.1 Mathematical Formulation

YOLOv2 introduces several improvements that collectively raise mAP from 63.4% to 78.6% (VOC 2007) while maintaining real-time speed.

Batch Normalization

BatchNorm is added after every convolutional layer, normalizing activations:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}] + \epsilon}}, \quad y^{(k)} = \gamma^{(k)}\hat{x}^{(k)} + \beta^{(k)} \tag{17.6}$$

This regularizes the model (+2% mAP) and allows removing dropout without overfitting.

Anchor Boxes and Dimension Clusters

YOLOv2 removes the fully-connected layers (making the network fully convolutional) and introduces anchor boxes. The input resolution becomes 416×416 (to have a single center cell in the 13×13 feature map, since objects tend to center in images). Instead of hand-picking anchor dimensions, YOLOv2 runs k-means clustering on the training set bounding box dimensions using a distance metric based on IoU:

$$d(\text{box}, \text{centroid}) = 1 - \text{IoU}(\text{box}, \text{centroid}) \tag{17.7}$$

This metric penalizes distance proportional to box size mismatch rather than euclidean distance on width/height, which would give larger errors for larger boxes. Five clusters are chosen as the best trade-off between recall and model complexity.

Direct Location Prediction

To stabilize training (early iterations diverge when predicting unconstrained (t_x, t_y) offsets from anchors), YOLOv2 constrains the center prediction to lie within the grid cell using a sigmoid function:

$$b_x = \sigma(t_x) + c_x \tag{17.8}$$
$$b_y = \sigma(t_y) + c_y \tag{17.9}$$
$$b_w = p_w \exp(t_w) \tag{17.10}$$
$$b_h = p_h \exp(t_h) \tag{17.11}$$
$$\Pr(\text{object}) \times \text{IoU}(b, \text{object}) = \sigma(t_o) \tag{17.12}$$

where (c_x, c_y) is the cell offset from the top-left corner of the image (in feature map coordinates), and (p_w, p_h) is the prior anchor width and height. The sigmoid restricts the predicted center to within the cell, improving stability.

Passthrough Layer

To recover fine-grained features lost by downsampling, a passthrough layer reshapes a $26 \times 26 \times 512$ feature map into a $13 \times 13 \times 2048$ tensor and concatenates it with the final 13×13 features. This is analogous to residual connections with identity mappings in ResNet and provides 1% mAP improvement.

Multi-Scale Training

Every 10 batches, the network randomly selects a new input dimension from $\{320, 352, 384, 416, 448, 480,$ (all multiples of 32, the network's downsampling factor). This forces the network to learn to predict well across a range of input resolutions, providing a built-in speed-accuracy tradeoff at inference time.

17.3.2 Architecture

Darknet-19 backbone: 19 convolutional layers and 5 max-pooling layers. Global average pooling replaces the fully-connected prediction layer. The detection head operates on a 13×13 feature map with $k = 5$ anchors.

17.4 YOLOv3

17.4.1 Mathematical Formulation

YOLOv3 addresses the small-object limitation of previous versions through multi-scale predictions at three different strides (32, 16, and 8). Predictions are made at three scales using a Feature Pyramid Network structure:

$$\mathbf{P}_3 = \text{Conv}_{1\times1}(\mathbf{C}_3) + \text{Upsample}(\text{Conv}_{1\times1}(\mathbf{P}_4)) \tag{17.13}$$

where $\mathbf{C}_3$ and $\mathbf{C}_4$ are feature maps from the backbone at different depths.

At each scale, the network predicts 3 anchors (for 9 total clusters across scales, with larger anchors assigned to coarser feature maps). Instead of softmax classification, YOLOv3 uses independent logistic classifiers (sigmoid) for each class:

$$\Pr(\text{class}_i \mid \text{Object}) = \sigma(z_i) = \frac{1}{1 + e^{-z_i}} \tag{17.14}$$

This enables multi-label classification—an object can simultaneously be classified as both "person" and "man"—which is more appropriate for datasets with hierarchical or overlapping classes (e.g., Open Images).

Objectness Score

For each bounding box, the objectness score uses logistic regression. If a predicted box overlaps a ground-truth by more than any other prediction, its objectness target is 1. Predictions with IoU > 0.5 but not the best are ignored (no loss contribution).

17.4.2 Architecture

Darknet-53 backbone replaces Darknet-19, featuring 53 convolutional layers with residual (skip) connections. Each residual block consists of (1×1 conv $\rightarrow$ 3×3 conv $\rightarrow$ add identity):

$$\mathbf{x}_{\ell+1} = \mathbf{x}_\ell + \mathcal{F}(\mathbf{x}_\ell, \mathcal{W}_\ell) \tag{17.15}$$

Detection is performed at three scales: stride 32 (coarse, 13×13 for large objects), stride 16 (medium, 26×26), and stride 8 (fine, 52×52 for small objects).

17.5 YOLOv4

17.5.1 Mathematical Formulation

YOLOv4 introduces two categories of improvements: **Bag of Freebies** (training-time only) and **Bag of Specials** (architectural changes with small inference cost).

CIoU Loss

Complete IoU (CIoU) loss extends the standard IoU loss by considering three geometric factors: overlap area, center-point distance, and aspect ratio:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{\text{gt}})}{c^2} + \alpha v \tag{17.16}$$

where $\rho(\cdot, \cdot)$ is the Euclidean distance between box centers, c is the diagonal length of the smallest enclosing box covering both predicted and ground-truth boxes, and:

$$v = \frac{4}{\pi^2}\left(\arctan\frac{w^{\text{gt}}}{h^{\text{gt}}} - \arctan\frac{w}{h}\right)^2 \tag{17.17}$$

$$\alpha = \frac{v}{(1 - \text{IoU}) + v} \tag{17.18}$$

The term v measures aspect ratio consistency, and α is a positive trade-off parameter that prioritizes overlap over aspect ratio when IoU is small.

Mish Activation and Regularization

$$\text{Mish}(x) = x \cdot \tanh(\text{softplus}(x)) = x \cdot \frac{e^{\ln(1+e^x)} - e^{-\ln(1+e^x)}}{e^{\ln(1+e^x)} + e^{-\ln(1+e^x)}} \tag{17.19}$$

Mish is a smooth, non-monotonic activation function that consistently outperforms ReLU. Mosaic data augmentation combines 4 training images into one, enriching context and reducing the need for large mini-batches.

17.5.2 Architecture

CSPDarknet53 backbone with Cross-Stage Partial connections partitions the feature map into two parts: one goes through a dense block, the other is concatenated with the output. An SPP (Spatial Pyramid Pooling) block pools features at multiple kernel sizes $\{1 \times 1, 5 \times 5, 9 \times 9, 13 \times 13\}$ and concatenates them. PANet (Path Aggregation Network) augments the FPN with a bottom-up path, and SAM (Spatial Attention Module) applies spatial attention.

17.6 YOLOv5 Through YOLOv11: An Evolutionary Table

Table 17.1: Key innovations across YOLO versions.

Version	Key Innovations	Year	Backbone
YOLOv5	CSPNet, auto-learning anchors, mosaic augmentation, hyperparameter evolution via genetic algorithms	2020	CSP-Darknet
YOLOv6	RepVGG reparameterization, decoupled head, SimOTA label assignment, quantization-friendly	2022	EfficientRep
YOLOv7	E-ELAN (Extended ELAN), model reparameterization, auxiliary head (coarse-to-fine), planned architecture	2022	ELAN-based
YOLOv8	Anchor-free detection, C2f module (CSP bottleneck with two convolutions), decoupled classification-regression head, SIoU loss, instance segmentation support	2023	C2f-Darknet
YOLOv9	PGI (Programmable Gradient Information), GELAN architecture, reversible functions for information preservation across deep networks	2024	GELAN
YOLOv10	NMS-free training via consistent dual assignments (one-to-one + one-to-many), lightweight classification head, large-kernel convolutions, partial self-attention	2024	CSPNet variants
YOLOv11	Enhanced C3k2 blocks, improved attention mechanisms, dynamic head assignment, optimized for edge deployment	2025	C3k2-Darknet

17.7 YOLOv8: Anchor-Free Detection

17.7.1 Mathematical Formulation

YOLOv8 abandons anchors entirely. Instead, each grid cell predicts the bounding box center offset directly (anchor-free) and the box dimensions:

$$\hat{x}_{\text{center}} = \sigma(t_x) + c_x, \quad \hat{y}_{\text{center}} = \sigma(t_y) + c_y \tag{17.20}$$

where σ constrains the prediction to the grid cell. For box dimensions, YOLOv8 predicts the distances from the center to each of the four box edges (l, t, r, b):

$$\hat{l} = a_l \cdot \sigma(t_l), \quad \hat{t} = a_t \cdot \sigma(t_t), \quad \hat{r} = a_r \cdot \sigma(t_r), \quad \hat{b} = a_b \cdot \sigma(t_b) \tag{17.21}$$

where a_* are learnable anchor-free scales. The classification and regression branches are decoupled (separate convolutional heads), improving convergence by removing the conflict between classification and localization features. The task-aligned assigner (SimOTA) dynamically selects the k best predictions for each ground-truth based on classification and regression quality:

$$t = s^{\alpha} \times u^{\beta} \tag{17.22}$$

where s is classification score, u is IoU, and α, β are hyperparameters.

17.7.2 Algorithm

Algorithm 21: YOLOv8 Anchor-Free Inference

Input: Image $\mathbf{I}$, YOLOv8 network Φ, NMS threshold τ
Output: Detections $\mathcal{D}$

$\{\mathbf{P}_3, \mathbf{P}_4, \mathbf{P}_5\} \leftarrow \Phi(\mathbf{I})$ (multi-scale feature maps);
$\mathcal{D} \leftarrow \emptyset$;
foreach $\mathbf{P} \in \{\mathbf{P}_3, \mathbf{P}_4, \mathbf{P}_5\}$ **do**
  $\mathbf{O}_{\text{cls}} \leftarrow \Phi_{\text{cls-head}}(\mathbf{P})$;
  $\mathbf{O}_{\text{box}} \leftarrow \Phi_{\text{reg-head}}(\mathbf{P})$;
  foreach *cell* (c_x, c_y) *in* $\mathbf{P}$ **do**
    $(t_l, t_r, t_t, t_b) \leftarrow \mathbf{O}_{\text{box}}[c_x, c_y]$;
    $(\hat{x}_c, \hat{y}_c) \leftarrow (\sigma_x + c_x, \sigma_y + c_y)$;
    $(\hat{l}, \hat{r}, \hat{t}, \hat{b}) \leftarrow (a_l\sigma(t_l), a_r\sigma(t_r), a_t\sigma(t_t), a_b\sigma(t_b))$;
    $p_{\text{cls}} \leftarrow \sigma(\mathbf{O}_{\text{cls}}[c_x, c_y])$;
    for c *with* $p_{cls}(c) > \tau_{conf}$ **do**
      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(c, (\hat{x}_c - \hat{l}, \hat{y}_c - \hat{t}, \hat{x}_c + \hat{r}, \hat{y}_c + \hat{b}), p_{\text{cls}}(c))\}$;
    end
  end
end
$\mathcal{D} \leftarrow \text{NMS}(\mathcal{D}, \tau)$;
return $\mathcal{D}$;

17.8 Pros and Cons Across Versions

Overall Pros. Unmatched inference speed (45–160+ fps), enabling real-time video processing. Single unified architecture that jointly learns localization, confidence, and classification. Global reasoning about the entire image in one forward pass reduces false-positive background detections. Highly generalizable—YOLO models trained on natural images transfer to artistic domains better than region-based detectors. The evolutionary lineage demonstrates consistent adoption of state-of-the-art techniques (batch norm, residual connections, feature pyramids, attention mechanisms) into a fast, practical framework.

Overall Cons. Grid-based prediction imposes structural limits on the maximum number of detections per region. Small object detection remains challenging, especially in dense scenes, though multi-scale prediction partially mitigates this. The spatial grid is fixed at design time, limiting detection resolution inherently. Performance degrades with unusual aspect ratios or highly overlapping objects because each grid cell can predict a limited number of boxes. Training requires careful handling of positive/negative sample imbalance (most grid cells are background). Prior to YOLOv8, the anchor-based design required dataset-specific anchor selection.

17.9 Evolution of the YOLO Family

The trajectory of YOLO development reflects the broader evolution of deep learning:

Phase 1 — Engineering Simplicity (v1–v2). YOLOv1 established the single-shot paradigm. YOLOv2 added anchor boxes, batch normalization, and dimension priors—bridging the gap to two-stage accuracy while maintaining real-time speed.

Phase 2 — Multi-Scale Architecture (v3–v4). YOLOv3 introduced multi-scale predictions and a deeper backbone (Darknet-53). YOLOv4 systematized the optimization by categorizing improvements into training augmentations (Mosaic, CIoU loss, cosine annealing) and architectural innovations (CSPNet, PANet, Mish activation), creating a production-ready detector that could be trained on a single GPU.

Phase 3 — Anchor-Free and Automated (v5–v8). YOLOv5 brought the ecosystem to PyTorch with a focus on deployment (ONNX, TensorRT, CoreML exports). YOLOv6 and v7 explored efficient architectures (RepVGG, ELAN) and train-time reparameterization. YOLOv8 marked the definitive shift to anchor-free detection with a decoupled head, simplifying both training and deployment.

Phase 4 — Information-Theoretic and NMS-Free (v9–v11). YOLOv9 introduced Programmable Gradient Information (PGI) to address information bottleneck in deep networks, using reversible functions to guarantee that input information is preserved through the network. YOLOv10 eliminated the need for NMS at inference by training with consistent dual assignments—one-to-one matches produce the final output directly, while one-to-many matches provide rich supervision. YOLOv11 advances edge deployment efficiency through lightweight attention mechanisms and optimized block design.

Chapter 18

SSD: Single Shot MultiBox Detector — Multi-Scale Default Boxes

18.1 Motivation

The object detection landscape in 2015 presented a stark trade-off: Faster RCNN achieved state-of-the-art accuracy (76.4% mAP on PASCAL VOC 2007) at 7 fps, while YOLOv1 achieved real-time speed (45 fps) at the cost of accuracy (63.4% mAP), particularly on small objects. Faster RCNN's two-stage pipeline—region proposal followed by per-region classification—provides refined spatial localization but incurs the computational cost of resampling features for each proposal. YOLOv1's single-pass design eliminates resampling but suffers from coarse spatial resolution (7×7 grid, limited positive boxes per cell). SSD bridges this gap by predicting bounding boxes and class confidences at **multiple feature map resolutions** in a **single forward pass**, without any region proposal or pixel resampling step. The key insight is that feature maps at different depths in a convolutional network naturally represent information at different spatial scales: shallow layers with high resolution are well-suited for detecting small objects, while deep layers with large receptive fields but low resolution are appropriate for large objects.

18.2 Mathematical Formulation

18.2.1 Default Box Generation

SSD defines a set of **default boxes** (analogous to Faster RCNN's anchors) at each spatial location of each selected feature map. For a feature map of size $f \times f$ from m total feature maps used for detection, the scale of default boxes at that feature map is:

$$s_k = s_{\min} + \frac{s_{\max} - s_{\min}}{m - 1} \cdot (k - 1), \quad k \in \{1, 2, \ldots, m\} \tag{18.1}$$

where $s_{\min} = 0.2$ and $s_{\max} = 0.9$ (fractions of the input image dimension). For each scale s_k, a set of aspect ratios $a_r \in \{1, 2, 3, \frac{1}{2}, \frac{1}{3}\}$ defines the shape of each default box:

$$w_k^a = s_k \sqrt{a_r} \tag{18.2}$$

$$h_k^a = \frac{s_k}{\sqrt{a_r}} \tag{18.3}$$

For the aspect ratio $a_r = 1$, an additional default box with scale $s'_k = \sqrt{s_k \cdot s_{k+1}}$ is added to provide a finer coverage of scale space:

$$w'_k = h'_k = \sqrt{s_k \cdot s_{k+1}} \tag{18.4}$$

Each default box is centered at $(\frac{i+0.5}{|f_k|}, \frac{j+0.5}{|f_k|})$ for each grid cell (i, j) on the k-th feature map, where $|f_k|$ is the spatial dimension. The total number of default boxes is:

$$N_{\text{boxes}} = \sum_{k=1}^{m} f_k^2 \times n_k \tag{18.5}$$

where n_k is the number of default boxes per location at feature map k. For the canonical SSD300 configuration with feature maps of sizes 38×38, 19×19, 10×10, 5×5, 3×3, and 1×1 with $n_k \in \{4, 6, 6, 6, 4, 4\}$, the total is:

$$N_{\text{boxes}} = 38^2 \cdot 4 + 19^2 \cdot 6 + 10^2 \cdot 6 + 5^2 \cdot 6 + 3^2 \cdot 4 + 1^2 \cdot 4 = 5776 + 2166 + 600 + 150 + 36 + 4 = 8732 \tag{18.6}$$

18.2.2 Prediction and Encoding

At each feature map location, for each default box $d = (d^{cx}, d^{cy}, d^w, d^h)$, the model predicts:

- $C + 1$ class scores (including background) via a $(C + 1)$-way classification layer.
- 4 offset parameters $(\Delta cx, \Delta cy, \Delta w, \Delta h)$ via a bounding box regression layer.

The predicted offsets encode the transformation from a default box d to a predicted box ℓ using the same parameterization as Faster RCNN:

$$\ell^{cx} = d^w \cdot \Delta cx + d^{cx} \tag{18.7}$$
$$\ell^{cy} = d^h \cdot \Delta cy + d^{cy} \tag{18.8}$$
$$\ell^w = d^w \cdot \exp(\Delta w) \tag{18.9}$$
$$\ell^h = d^h \cdot \exp(\Delta h) \tag{18.10}$$

Conversely, the ground-truth encoding from a default box d to a matched ground-truth box g is:

$$\hat{g}^{cx} = \frac{g^{cx} - d^{cx}}{d^w}, \quad \hat{g}^{cy} = \frac{g^{cy} - d^{cy}}{d^h} \tag{18.11}$$
$$\hat{g}^w = \log \frac{g^w}{d^w}, \quad \hat{g}^h = \log \frac{g^h}{d^h} \tag{18.12}$$

18.2.3 Matching Strategy

During training, each ground-truth box is matched to the default box with the highest Jaccard overlap (IoU). Then, default boxes with IoU > 0.5 with any ground-truth are also matched. This differs critically from YOLOv1's "one predictor per cell" rule—multiple default boxes of varying scales and aspect ratios from overlapping spatial locations can all match the same ground-truth box, ensuring a rich set of positive training examples.

Let $\mathbf{X} \in \{0, 1\}^{N_{\text{boxes}} \times (C+1)}$ be the matching indicator matrix, where $x_{ij}^p = 1$ if default box i is matched to ground-truth j of class p. Only matched default boxes contribute to localization loss:

$$x_{ij}^{p} = \begin{cases} 1 & \text{if IoU}(d_i, g_j) \geq 0.5 \text{ and } p = \text{class}(g_j) \\ 0 & \text{otherwise} \end{cases} \tag{18.13}$$

18.2.4 Training Objective

The overall training loss is a weighted sum of localization loss and confidence loss:

$$L(x, c, \ell, g) = \frac{1}{N}\Big(L_{\text{conf}}(x, c) + \alpha \cdot L_{\text{loc}}(x, \ell, g)\Big) \tag{18.14}$$

where N is the number of matched default boxes (if $N = 0$, the loss is set to 0), and $\alpha = 1$ balances the two terms in practice.

Localization Loss

The localization loss is the smooth ℓ_1 loss between predicted offsets ℓ and ground-truth encoded offsets g, summed over all matched default boxes:

$$L_{\text{loc}}(x, \ell, g) = \sum_{i \in \text{Pos}} \sum_{m \in \{cx, cy, w, h\}} x_{ij}^{p} \cdot L_{\text{smooth}}^{\ell_1}\big(\ell_i^m - \hat{g}_j^m\big) \tag{18.15}$$

where Pos is the set of default boxes matched to a ground-truth, and:

$$L_{\text{smooth}}^{\ell_1}(z) = \begin{cases} 0.5z^2 & \text{if } |z| < 1 \\ |z| - 0.5 & \text{otherwise} \end{cases} \tag{18.16}$$

Confidence Loss

The confidence loss is the multinomial softmax loss over $C + 1$ classes:

$$L_{\text{conf}}(x, c) = -\sum_{i \in \text{Pos}} x_{ij}^{p} \log(\hat{c}_i^{p}) - \sum_{i \in \text{Neg}} \log(\hat{c}_i^{0}) \tag{18.17}$$

where $\hat{c}_i^{p}$ is the softmax-normalized confidence for class p:

$$\hat{c}_i^{p} = \frac{\exp(c_i^{p})}{\sum_{q=0}^{C} \exp(c_i^{q})} \tag{18.18}$$

and background is class 0. Neg is the set of negative (unmatched) default boxes.

18.2.5 Hard Negative Mining

After matching, the vast majority of default boxes are negatives (background), leading to severe class imbalance—typically 50–100 negatives per positive. To address this, SSD applies **hard negative mining**: sort all negative default boxes by their highest confidence loss (i.e., the confidence with which the model misclassifies them as foreground), and select the top k negatives such that the ratio of negatives to positives is at most $3:1$:

$$\mathcal{N}_{\text{hard}} = \arg\max_{S \subseteq \text{Neg}, |S| \leq 3N} \sum_{i \in S} \big(-\log(\hat{c}_i^{0})\big) \tag{18.19}$$

This focuses the optimization on the hardest (most confusing) negative examples, stabilizing training and preventing the confidence loss from being dominated by easy negatives.

18.2.6 Atrous Convolution in the Base Network

SSD converts the fc6 and fc7 fully-connected layers of VGG-16 into convolutional layers (fc6 $\rightarrow$ Conv(3×3, 1024) with dilation rate 6, fc7 $\rightarrow$ Conv(1×1, 1024)). The atrous (dilated) convolution expands the receptive field without increasing parameters or reducing spatial resolution:

$$y[\mathbf{p}] = \sum_{\mathbf{k} \in \mathcal{K}} x[\mathbf{p} + r \cdot \mathbf{k}] \cdot w[\mathbf{k}] \tag{18.20}$$

where r is the dilation rate. For a 3×3 kernel with dilation rate 6, the effective receptive field is 13×13:

$$k_{\text{eff}} = k + (k-1)(r-1) = 3 + (3-1)(6-1) = 13 \tag{18.21}$$

while using only 9 parameters (the same as a standard 3×3 conv).

18.3 Architecture

18.3.1 Base Network

SSD uses a truncated VGG-16 as the base feature extractor, modified as follows:

- The fc6 and fc7 layers are converted to convolutional layers: Conv6 (3×3, dilation=6, 1024 channels) and Conv7 (1×1, 1024 channels).
- Dropout and fc8 (classification) are removed.
- Pool5 is changed from $2 \times 2/2$ to $3 \times 3/1$ (to preserve spatial resolution).

18.3.2 Auxiliary Convolutional Layers

After the base network, four additional convolutional layers are appended to produce progressively smaller feature maps for multi-scale detection:

$$\begin{aligned}
&\text{Conv8_1: } 1 \times 1 \times 256 \rightarrow 3 \times 3 \times 512 \text{ (stride 2)} && (10 \times 10) \\
&\text{Conv9_1: } 1 \times 1 \times 128 \rightarrow 3 \times 3 \times 256 \text{ (stride 2)} && (5 \times 5) \\
&\text{Conv10_1: } 1 \times 1 \times 128 \rightarrow 3 \times 3 \times 256 \text{ (stride 1, no pad)} && (3 \times 3) \\
&\text{Conv11_1: } 1 \times 1 \times 128 \rightarrow 3 \times 3 \times 256 \text{ (stride 1, no pad)} && (1 \times 1)
\end{aligned}$$

18.3.3 Detection Heads

For each of the 6 feature maps used for detection, two separate 3×3 convolutional predictors are applied:

- **Classification head:** Outputs $n_k \times (C + 1)$ channels — for each default box, a score for each class plus background.
- **Localization head:** Outputs $n_k \times 4$ channels — for each default box, the four offset parameters.

The total parameter count for the detection heads is:

$$\sum_{k=1}^{m} (C + 5) \times n_k \times d_k \times 3 \times 3 \tag{18.22}$$

where d_k is the number of input channels at feature map k. This is a fully convolutional design—there are no fully-connected layers in the prediction heads, making inference efficient.

18.3.4 Speed-Accuracy Configurations

Table 18.1: SSD configurations and performance on PASCAL VOC 2007.

Model	**Input Size**	**FPS**	**mAP**	**Backbone**
SSD300	300×300	59	74.3%	VGG-16
SSD512	512×512	22	76.8%	VGG-16
SSD300-Res	300×300	—	75.8%	ResNet-101
SSD512-Res	512×512	—	78.5%	ResNet-101

18.4 Algorithm

Algorithm 22: SSD Single-Shot Multi-Scale Inference

Input: Image $\mathbf{I} \in \mathbb{R}^{300\times300\times3}$, trained SSD network Φ, NMS threshold τ
Output: Detections $\mathcal{D}$

$\{\mathbf{F}_1, \dots, \mathbf{F}_6\} \leftarrow \text{ExtractFeatureMaps}(\Phi, \mathbf{I})$;
$\mathcal{D} \leftarrow \emptyset$;
for $k \leftarrow 1$ **to** 6 **do**
  $\mathbf{O}_{\text{cls}}^{(k)} \leftarrow \text{Conv}_{\text{cls}}^{(k)}(\mathbf{F}_k)$;
  $\mathbf{O}_{\text{loc}}^{(k)} \leftarrow \text{Conv}_{\text{loc}}^{(k)}(\mathbf{F}_k)$;
  $f_k \leftarrow \dim(\mathbf{F}_k)$;
  for $i \leftarrow 0$ **to** $f_k - 1$ **do**
    for $j \leftarrow 0$ **to** $f_k - 1$ **do**
      $d^{cx} \leftarrow (i + 0.5)/f_k$, $d^{cy} \leftarrow (j + 0.5)/f_k$;
      for $b \leftarrow 0$ **to** $n_k - 1$ **do**
        $d^w \leftarrow s_k\sqrt{a_b}$, $d^h \leftarrow s_k/\sqrt{a_b}$;
        $p \leftarrow \text{softmax}(\mathbf{O}_{\text{cls}}^{(k)}[i, j, b])$;
        $(t_{cx}, t_{cy}, t_w, t_h) \leftarrow \mathbf{O}_{\text{loc}}^{(k)}[i, j, b]$;
        $\ell^{cx} \leftarrow d^w \cdot t_{cx} + d^{cx}$;
        $\ell^{cy} \leftarrow d^h \cdot t_{cy} + d^{cy}$;
        $\ell^w \leftarrow d^w \cdot \exp(t_w)$;
        $\ell^h \leftarrow d^h \cdot \exp(t_h)$;
        $c^* \leftarrow \arg\max_{c \geq 1} p_c$;
        if $p_{c^*} > \tau_{conf}$ **then**
          $\mathcal{D} \leftarrow \mathcal{D} \cup \{(c^*, (\ell^{cx}, \ell^{cy}, \ell^w, \ell^h), p_{c^*})\}$;
        end
      end
    end
  end
end
$\mathcal{D} \leftarrow \text{NMS}(\mathcal{D}, \tau = 0.45)$;
return top-K $\mathcal{D}$ by confidence;

18.4.1 Training Algorithm with Data Augmentation

Algorithm 23: SSD Training with Hard Negative Mining

Input: Dataset $\mathcal{T} = \{(\mathbf{I}_i, \{\mathbf{g}_j\})\}$, SSD model parameters Θ, learning rate η
Output: Trained parameters Θ

Initialize Θ with VGG-16 pre-trained on ILSVRC classification;
for *epoch* $e = 1, \ldots, E$ **do**
  for *each mini-batch* **do**
    Sample images, apply data augmentation (random crop, expansion, photometric distortion, horizontal flip);
    Extract feature maps $\{\mathbf{F}_1, \ldots, \mathbf{F}_6\}$ via forward pass;
    Compute all predicted offsets and class scores;
    Match each ground-truth to default boxes (IoU ≥ 0.5);
    Perform hard negative mining (top-3 negatives per positive);
    $L \leftarrow \frac{1}{N}(L_{\text{conf}} + \alpha L_{\text{loc}})$;
    $\Theta \leftarrow \Theta - \eta \cdot \nabla_\Theta L$;
  end
  Decay η according to step schedule;
end
return Θ;

18.5 Data Augmentation

SSD's data augmentation strategy is central to its accuracy—it applies a cascade of transformations to diversify the training distribution:

1. **Random sample** one of the following operations with equal probability:
   - Use the entire original input image.
   - Sample a patch whose minimum Jaccard overlap with any object is one of $\{0.1, 0.3, 0.5, 0.7, 0.9\}$.
   - Sample a patch randomly.
2. **Random expansion:** Place the sampled patch on a canvas of the original image size filled with the mean pixel value, creating "zoom-out" effects.
3. **Photometric distortions:** Randomly adjust brightness, contrast, hue, and saturation in a randomized sequence.
4. **Horizontal flip** with probability 0.5.
5. Resize to fixed 300×300 (or 512×512).

This pipeline significantly increases the diversity of object scales, positions, and appearances seen during training, acting as a regularizer that improves generalization—particularly for small and occluded objects.

18.6 Pros and Cons

Pros. True single-shot design: one forward pass produces all detections without region proposal, feature resampling, or post-classification refinement. Multi-scale feature maps naturally handle objects at different sizes without explicit pyramid architectures—the feature hierarchy already encodes scale variation. Simpler than two-stage detectors: no RPN, no RoI pooling, no post-hoc feature cropping. Fast inference (59 fps at 300×300, 22 fps at 512×512) with competitive accuracy (74.3%/76.8% mAP on VOC 2007). Fully convolutional design means the same architecture works with any input resolution (with minor adaptation). Data augmentation pipeline is a strong feature regularizer.

Cons. Small object detection remains weak: the shallowest feature map used for detection (Conv4_3, stride 8, resolution 38×38) still provides limited spatial granularity for very small objects. The default box design requires careful manual configuration of scales and aspect ratios—these hyperparameters are dataset-dependent and suboptimal choices degrade performance. Feature maps used for detection are independent of each other; information from higher-resolution maps does not flow to lower-resolution maps (unlike FPN's top-down pathway), causing a disconnect between the classification of small and large instances of the same object. Hard negative mining is a heuristic that introduces a non-differentiable sampling step, breaking end-to-end differentiability. The loss function weights localization and confidence equally by default ($\alpha = 1$), which may not be optimal across all scales and classes.

18.7 Evolution of Single-Shot Detection

DSSD (Deconvolutional SSD, Fu et al. 2017). Adds a deconvolution module after the SSD architecture to create a top-down feature pyramid: $\mathbf{P}_\ell = \text{Deconv}(\mathbf{P}_{\ell+1}) \odot \mathbf{F}_\ell + \mathbf{b}$, where $\odot$ denotes element-wise multiplication (gated fusion). The base network is upgraded from VGG-16 to ResNet-101. DSSD achieves 81.5% mAP on VOC 2007 but at significantly reduced speed (6 fps), demonstrating that feature fusion helps small object detection at the cost of compute.

RetinaNet (Lin et al. 2017). Addresses the class imbalance problem at its root via **Focal Loss** instead of hard negative mining. Focal loss dynamically scales the cross-entropy loss, down-weighting the contribution of easy examples:

$$\text{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t) \tag{18.23}$$

where $p_t = p$ if $y = 1$, $p_t = 1 - p$ otherwise; α_t balances positive/negative examples; and $\gamma \geq 0$ is the focusing parameter. For a well-classified example ($p_t \approx 1$), $(1-p_t)^\gamma \approx 0$, so the loss contribution is negligible. For a misclassified example ($p_t \ll 1$), the modulating factor is near 1. This removes the need for hard negative mining entirely, creating a fully differentiable pipeline. RetinaNet with ResNet-101-FPN backbone achieves 39.1 AP on COCO (single-model, single-scale).

FCOS (Fully Convolutional One-Stage, Tian et al. 2019). Eliminates the need for pre-defined anchor boxes entirely (anchor-free). Each location directly predicts a 4D vector (ℓ, t, r, b) representing distances to the four sides of the bounding box. A "centerness" branch predicts the normalized distance from the location to the object center:

$$\text{centerness} = \sqrt{\frac{\min(\ell, r)}{\max(\ell, r)} \times \frac{\min(t, b)}{\max(t, b)}} \tag{18.24}$$

This suppresses low-quality detections far from the object center during NMS. FCOS achieves comparable accuracy to anchor-based methods with simpler design.

DETR (DEtection TRansformer, Carion et al. 2020). A paradigm shift replacing the entire convolutional detection pipeline with a transformer encoder-decoder architecture. DETR treats object detection as a direct set prediction problem using a bipartite matching loss (Hungarian algorithm) between predicted and ground-truth boxes. The loss is:

$$L_{\text{Hungarian}}(y, \hat{y}) = \sum_{i=1}^{N} \Big[-\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbb{1}_{\{c_i \neq \varnothing\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}(i)}) \Big] \tag{18.25}$$

where $\hat{\sigma}$ is the optimal permutation and $\mathcal{L}_{\text{box}}$ combines ℓ_1 and GIoU loss. DETR removes NMS, anchor design, and all hand-crafted components, but requires long training schedules and struggles with small objects—limitations addressed by subsequent deformable DETR.

DETR variants: Deformable DETR (Zhu et al. 2021) introduces deformable attention to focus on sparse spatial locations, drastically accelerating convergence and improving small object detection. DINO (Zhang et al. 2023) further improves training efficiency via contrastive denoising, mixed query selection, and a novel look-forward-twice scheme.

The arc from SSD to DETR represents the progression from hand-designed spatial priors (default boxes, matching heuristics) toward learned spatial reasoning (transformers, set-based losses), while the overall single-shot prediction philosophy—one forward pass, all detections—remains constant.

Part VI

Vision Transformers & Multimodal Models

Chapter 19

ViT: Vision Transformer

19.1 Motivation

Convolutional neural networks (CNNs) dominated computer vision for nearly a decade, owing their effectiveness to strong inductive biases: locality (pixels close in space are related), translation equivariance (weight sharing via convolution), and hierarchical feature extraction through successive pooling. Meanwhile, the Transformer architecture revolutionized natural language processing through self-attention, which models pairwise interactions between all tokens without built-in spatial priors. The Vision Transformer (ViT) asks a bold question: can a pure attention-based architecture succeed on visual recognition tasks without any convolution? The answer, demonstrated by Dosovitskiy et al. (2021), is yes—provided sufficient training data is available. The key insight is that an image can be treated as a sequence of flattened patches, analogous to tokens in a sentence, and processed by a standard Transformer encoder. This unification of vision and language under the same architectural paradigm marked a inflection point: today, Transformers are the default backbone for both modalities, enabling multimodal models like CLIP and unified sequence-to-sequence frameworks.

19.2 Mathematical Formulation

19.2.1 Image to Patch Sequence

Let an input image be $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, where H, W are spatial dimensions and C is the number of channels (typically $C = 3$ for RGB). The image is partitioned into a grid of non-overlapping patches of size $P \times P$, yielding

$$N = \frac{HW}{P^2}$$

patches. Each patch $\mathbf{x}_p^i \in \mathbb{R}^{P^2 \cdot C}$ is flattened into a vector. A learnable linear projection $\mathbf{E} \in \mathbb{R}^{(P^2C) \times D}$ maps each patch vector to dimension D, producing patch embeddings $\mathbf{x}_p^i \mathbf{E} \in \mathbb{R}^D$. This is equivalent to a 2D convolution with kernel size $P \times P$ and stride P, a subtle point: ViT's "patch embedding" is a convolutional operation with no further convolutions downstream.

Following BERT, a learnable classification token $\mathbf{x}_{\text{class}} \in \mathbb{R}^D$ is prepended to the sequence. Its final hidden state serves as the image representation for classification. Learnable 1D position embeddings $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$ are added to inject spatial information. The input to the Transformer is:

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}};\ \mathbf{x}_p^1 \mathbf{E};\ \mathbf{x}_p^2 \mathbf{E};\ \ldots;\ \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}}.$$

19.2.2 Multi-Head Self-Attention (MSA)

At each layer ℓ, the input $\mathbf{z}_{\ell-1} \in \mathbb{R}^{(N+1)\times D}$ is first layer-normalized, then projected to queries, keys, and values via learned weight matrices. For a single attention head i:

$$[\mathbf{Q}_i,\ \mathbf{K}_i,\ \mathbf{V}_i] = \mathrm{LN}(\mathbf{z}_{\ell-1})\,\mathbf{W}^{(i)}_{QKV},$$

where $\mathbf{W}^{(i)}_{QKV} \in \mathbb{R}^{D\times 3D_h}$ and $D_h = D/h$ is the per-head dimension (with h heads). The scaled dot-product attention is:

$$\mathrm{head}_i = \mathrm{softmax}\!\left(\frac{\mathbf{Q}_i\mathbf{K}_i^\top}{\sqrt{D_h}}\right)\mathbf{V}_i.$$

The $\sqrt{D_h}$ scaling prevents saturation of the softmax for large D_h, keeping gradients in a reasonable regime. Multi-head attention concatenates all heads and projects back to dimension D:

$$\mathrm{MSA}(\mathbf{z}_{\ell-1}) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\,\mathbf{W}_O,$$

with $\mathbf{W}_O \in \mathbb{R}^{D\times D}$. The residual connection yields $\mathbf{z}'_\ell = \mathrm{MSA}(\mathrm{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}$.

19.2.3 Transformer Encoder Block

The full Transformer block comprises MSA followed by an MLP, each preceded by layer normalization and wrapped with residual connections:

$$\begin{aligned}\mathbf{z}'_\ell &= \mathrm{MSA}(\mathrm{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1},\\ \mathbf{z}_\ell &= \mathrm{MLP}(\mathrm{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell.\end{aligned}$$

The MLP consists of two linear layers with a GELU activation and hidden size $4D$:

$$\mathrm{MLP}(\mathbf{u}) = \mathbf{W}_2\ \mathrm{GELU}(\mathbf{W}_1\mathbf{u} + \mathbf{b}_1) + \mathbf{b}_2,$$

with $\mathbf{W}_1 \in \mathbb{R}^{D\times 4D}$, $\mathbf{W}_2 \in \mathbb{R}^{4D\times D}$. This is repeated for L layers. The final classification head extracts the [CLS] token from layer L and projects it via a linear layer:

$$\hat{\mathbf{y}} = \mathrm{softmax}(\mathbf{z}^0_L\,\mathbf{W}_{\mathrm{cls}}).$$

19.2.4 Position Embeddings

Learned 1D position embeddings $\mathbf{E}_{\mathrm{pos}} \in \mathbb{R}^{(N+1)\times D}$ are initialized from a Gaussian distribution and trained jointly with the rest of the network. At fine-tuning on higher-resolution images (yielding more patches), the position embeddings are interpolated (e.g., bicubic) to match the new sequence length. This simple scheme works surprisingly well, though 2D-aware position embeddings—encoding both row and column coordinates—provide marginal improvements. Notably, the sinusoidal position encoding from the original Transformer can also be used, but learned embeddings are the standard choice.

19.2.5 Self-Attention as Generalized Convolution

Consider the self-attention matrix $\mathbf{A} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{D_h}) \in \mathbb{R}^{(N+1)\times(N+1)}$. Each row $\mathbf{a}_i$ is a convex combination weight vector: output at position i is a weighted sum of all values. For a CNN with kernel size k, the equivalent attention would have $\mathbf{a}_{i,j} \neq 0$ only for j within the $k \times k$ neighborhood of i. ViT learns these weights from data without the locality constraint—it is a superset of convolution. However, this flexibility comes at the cost of sample efficiency: the quadratic complexity $\mathcal{O}(N^2)$ in sequence length and the lack of built-in spatial priors demand large-scale pretraining.

19.2.6 Hybrid Architecture

A hybrid ViT replaces the raw image patches with feature maps extracted by a CNN (e.g., ResNet-50). The CNN acts as a feature extractor; its output feature map of spatial dimensions $H_f \times W_f$ is flattened into a sequence of $H_f W_f$ tokens. This injects CNN-like locality into the input representation. The subsequent Transformer encoder operates identically. Hybrid models typically require less pretraining data than pure ViT for comparable accuracy, bridging the gap between inductive and learned priors.

19.3 Algorithm

Algorithm 24: Vision Transformer (ViT)

Input: Image $\mathbf{x} \in \mathbb{R}^{H\times W\times C}$, patch size P, embedding dim D, depth L, heads h
Output: Prediction $\hat{\mathbf{y}} \in \mathbb{R}^K$ (for K classes)

$N \leftarrow HW/P^2$;
$\mathbf{z}_0^0 \leftarrow \mathbf{x}_{\text{class}}$ (learnable);
for $i \leftarrow 1$ **to** N **do**
  $\mathbf{p}^i \leftarrow \text{flatten}(\mathbf{x}_{(i-1)P:iP,\ldots})$;
  $\mathbf{z}_0^i \leftarrow \mathbf{p}^i\mathbf{E}$ (linear projection);
end
$\mathbf{z}_0 \leftarrow [\mathbf{z}_0^0; \mathbf{z}_0^1; \ldots; \mathbf{z}_0^N] + \mathbf{E}_{\text{pos}}$;
for $\ell \leftarrow 1$ **to** L **do**
  $\mathbf{u} \leftarrow \text{LN}(\mathbf{z}_{\ell-1})$;
  for $m \leftarrow 1$ **to** h **do**
    $[\mathbf{Q}_m, \mathbf{K}_m, \mathbf{V}_m] \leftarrow \mathbf{u}\mathbf{W}_{QKV}^{(m)}$;
    $\mathbf{A}_m \leftarrow \text{softmax}(\mathbf{Q}_m\mathbf{K}_m^\top/\sqrt{D_h})$;
    $\text{head}_m \leftarrow \mathbf{A}_m\mathbf{V}_m$;
  end
  $\mathbf{z}'_\ell \leftarrow \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}_O + \mathbf{z}_{\ell-1}$;
  $\mathbf{z}_\ell \leftarrow \mathbf{W}_2\ \text{GELU}(\mathbf{W}_1\text{LN}(\mathbf{z}'_\ell) + \mathbf{b}_1) + \mathbf{b}_2 + \mathbf{z}'_\ell$;
end
$\hat{\mathbf{y}} \leftarrow \text{softmax}(\mathbf{z}_L^0\mathbf{W}_{\text{cls}})$;
return $\hat{\mathbf{y}}$

19.4 Architecture

19.4.1 ViT Variants

Three standard configurations follow the scaling conventions of the NLP Transformer (BERT):

- **ViT-Base**: $L = 12$, $D = 768$, $h = 12$, MLP hidden 3072. Parameters: 86M.
- **ViT-Large**: $L = 24$, $D = 1024$, $h = 16$, MLP hidden 4096. Parameters: 307M.
- **ViT-Huge**: $L = 32$, $D = 1280$, $h = 16$, MLP hidden 5120. Parameters: 632M.

The patch size P determines the sequence length: $P = 16$ on 224×224 images yields $N = 14^2 = 196$ tokens (plus [CLS]). Smaller patches ($P = 14$, $P = 8$) increase resolution and accuracy but quadratically increase computational cost. The [CLS] token is not strictly necessary; global average pooling (GAP) over all patch tokens can replace it, offering slightly better performance in some settings.

19.4.2 Self-Attention Complexity

For N patches and embedding dimension D, the computational complexity of MSA is:

$$\mathcal{O}_{\text{MSA}} = 4ND^2 + 2N^2D.$$

The N^2D term (quadratic in sequence length) dominates for large N, making ViT computationally expensive for high-resolution images. Swin Transformer later addressed this via window-based attention (Section 19.6). The ND^2 term (quadratic in model dimension) dominates for very wide models.

19.4.3 Pre-training and Fine-tuning

Pre-training is conducted on large labeled datasets: ImageNet-21k (14M images, 21k classes), JFT-300M (300M images, 18k annotated classes). Standard supervised cross-entropy loss is used. At fine-tuning, the classification head is replaced by a new zero-initialized $D \times K_{\text{down}}$ layer, and the model is trained on the target dataset at higher resolution (e.g., 384^2 or 512^2), with position embeddings interpolated and optionally fine-tuned. The AdamW optimizer with cosine learning rate decay and linear warmup is the standard recipe, supplemented by strong data augmentation (RandAugment, MixUp) and regularization (weight decay, stochastic depth).

19.5 Pros and Cons

Advantages:

1. **Unified architecture**: ViT uses the same Transformer backbone as NLP, enabling seamless multimodal integration (e.g., CLIP, BLIP). This drastically reduces engineering complexity for systems that process both images and text.

2. **Global context from layer one**: Self-attention models pairwise interactions between all patches at every layer, unlike CNNs whose receptive field grows gradually. This benefits tasks requiring long-range reasoning.

3. **Scalability**: ViT follows power-law scaling similar to NLP Transformers—larger models and more data consistently improve performance without architectural changes. ViT-22B shows no saturation.

4. **Flexibility at input level**: The same architecture processes images at any resolution by adjusting patch count. In contrast, CNNs with fully connected heads require fixed input size.

5. **No hand-crafted inductive bias**: The absence of convolution forces the model to learn spatial structure from data—but with sufficient data, this yields stronger, data-driven representations.

Disadvantages:

1. **Data hunger**: ViT underperforms CNNs on small and medium datasets (ImageNet-1k from scratch) due to the lack of locality and translation equivariance priors. ResNet outperforms ViT-B until pre-training dataset sizes exceed roughly 10–30M images.

2. **Quadratic complexity**: $\mathcal{O}(N^2)$ self-attention makes ViT prohibitively expensive for high-resolution images (e.g., medical images at 2048^2). Window-based variants partially mitigate this.

3. **No built-in scale invariance**: Unlike CNNs with pyramidal feature hierarchies, ViT operates at a single resolution throughout all layers (isotropic architecture). Multi-scale understanding must be learned entirely from position embeddings.

4. **Poor fine-grained performance**: ViT struggles with dense prediction tasks (segmentation, depth estimation) without architectural modifications. Specialized designs like Swin or SegFormer are preferred.

5. **Position embedding interpolation loss**: When fine-tuning at a resolution different from pre-training, the interpolated position embeddings introduce subtle misalignment that can degrade performance.

19.6 Evolution

The ViT lineage has diversified rapidly:

DeiT (Data-efficient Image Transformers, Touvron et al., 2020): Introduces knowledge distillation from a CNN teacher to a ViT student, using a dedicated distillation token alongside the [CLS] token. The distillation loss $\mathcal{L}_{\text{distill}} = \text{KL}(\text{softmax}(\mathbf{z}_{\text{distill}}/\tau) \| \text{softmax}(\mathbf{y}_{\text{teacher}}/\tau)$ enables training ViT on ImageNet-1k from scratch with state-of-the-art accuracy. This showed that the data-efficiency gap was bridgeable without massive datasets.

Swin Transformer (Liu et al., 2021): Introduces hierarchical feature maps (like CNNs) and shifted window attention. Patch merging between stages reduces spatial resolution while increasing channels (4×, 8×, 16×, 32× downsampling). Window-based

self-attention confines computation to local $M \times M$ windows ($\mathcal{O}(NM^2)$ instead of $\mathcal{O}(N^2)$), and shifted windows between consecutive layers allow cross-window communication. Swin became the backbone of choice for vision beyond classification.

PVT (Pyramid Vision Transformer, Wang et al., 2021): Spatial-reduction attention (SRA) uses strided convolutions to reduce key and value spatial dimensions, achieving hierarchical features with linear complexity $\mathcal{O}(NR)$ where $R \ll N$ is the reduced resolution.

MAE (Masked Autoencoders, He et al., 2022): A self-supervised pre-training method: randomly mask a high fraction ($\tilde{7}5\%$) of patches, encode only visible patches with ViT, and reconstruct masked patches with a lightweight decoder. The asymmetric design (heavy encoder, light decoder) makes training efficient. MAE pre-training produces features competitive with supervised pre-training and established the "masked image modeling" paradigm.

DINO (Caron et al., 2021): Self-supervised learning via self-distillation with no labels. A student ViT matches the output of a teacher ViT (EMA of student) on different augmented views. The resulting features exhibit emergent segmentation capabilities, with attention maps delineating object boundaries without pixel-level supervision.

ViT-22B (Dehghani et al., 2023): Scaled to 22 billion parameters, trained on a massive dataset, demonstrating that ViT follows predictable scaling laws. The largest ViT achieved new state-of-the-art on numerous benchmarks, confirming that architecture simplicity is an asset at scale.

DINOv2: Extends DINO to larger scale with curated data, producing general-purpose visual features that match or exceed weakly-supervised features across tasks without fine-tuning.

Trend: The field moves toward larger pre-training with improved data, self-supervised objectives (MAE, DINO, I-JEPA), and architectural innovations that reduce complexity while retaining the Transformer's representational power. ViT's unification of vision and NLP under the Transformer umbrella remains its most enduring contribution.

Chapter 20

CLIP: Contrastive Language-Image Pre-training

20.1 Motivation

Traditional computer vision models are trained to classify images into a fixed, predefined set of categories (e.g., the 1,000 ImageNet classes). Expanding to new categories requires collecting additional labeled data and retraining, a costly and inflexible process. Moreover, the richness of visual concepts far exceeds any finite label ontology—images contain objects, attributes, scenes, actions, and relationships that cannot be exhaustively enumerated. CLIP (Contrastive Language-Image Pre-training, Radford et al., 2021) proposes a radical alternative: learn visual representations by aligning images with their natural language descriptions. By training on 400 million image-text pairs harvested from the web, CLIP builds a joint embedding space where semantically similar images and texts lie close together. The result is a model capable of zero-shot classification on any category expressible in natural language, without a single labeled training example for that category. This paradigm shift connects vision and language at the representation level, enabling flexible, open-vocabulary recognition that powers downstream applications from image retrieval to text-to-image generation (DALL-E 2, Stable Diffusion).

20.2 Mathematical Formulation

20.2.1 Joint Embedding Space

CLIP learns two encoders: an image encoder f_θ (ViT or ResNet) and a text encoder g_ϕ (Transformer), which respectively map images and text captions to a shared d-dimensional embedding space. Given a batch of N image-text pairs $\{(\mathbf{x}_i, \mathbf{t}_i)\}_{i=1}^N$:

$$\mathbf{I}_i = \frac{f_\theta(\mathbf{x}_i)}{\|f_\theta(\mathbf{x}_i)\|_2} \in \mathbb{S}^{d-1},$$
$$\mathbf{T}_i = \frac{g_\phi(\mathbf{t}_i)}{\|g_\phi(\mathbf{t}_i)\|_2} \in \mathbb{S}^{d-1},$$

where $\mathbb{S}^{d-1}$ denotes the unit hypersphere in $\mathbb{R}^d$. The ℓ_2 normalization ensures that inner products become cosine similarities, confining representations to a compact manifold and stabilizing training.

20.2.2 InfoNCE Loss

The core of CLIP is the InfoNCE (Information Noise-Contrastive Estimation) objective, applied symmetrically to both modalities. For a temperature parameter $\tau > 0$, define

the pairwise similarity scores:

$$s_{ij} = \frac{\langle \mathbf{I}_i, \mathbf{T}_j \rangle}{\tau} = \frac{\mathbf{I}_i^\top \mathbf{T}_j}{\tau}.$$

The image-to-text contrastive loss is:

$$\mathcal{L}_{\mathrm{I}\to\mathrm{T}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp(s_{ii})}{\sum_{j=1}^{N} \exp(s_{ij})}.$$

Symmetrically, the text-to-image loss is:

$$\mathcal{L}_{\mathrm{T}\to\mathrm{I}} = -\frac{1}{N}\sum_{j=1}^{N} \log \frac{\exp(s_{jj})}{\sum_{i=1}^{N} \exp(s_{ij})}.$$

The total CLIP loss averages both directions:

$$\mathcal{L}_{\mathrm{CLIP}} = \frac{1}{2}\big(\mathcal{L}_{\mathrm{I}\to\mathrm{T}} + \mathcal{L}_{\mathrm{T}\to\mathrm{I}}\big).$$

This objective maximizes the similarity of the N correct (diagonal) pairs while minimizing the similarity of the $N^2 - N$ incorrect (off-diagonal) pairs within the batch. The temperature τ is a learnable scalar (clipped to prevent scaling beyond $\log 100$), log-parameterized as $\tau = \exp(t)$ with t initialized to $\log(1/0.07)$.

20.2.3 Information-Theoretic Interpretation

The InfoNCE loss is a lower bound on the mutual information between image and text representations. Let $I(\mathbf{I}; \mathbf{T})$ denote the mutual information. For any critic function $f(\mathbf{I}, \mathbf{T})$, we have:

$$I(\mathbf{I}; \mathbf{T}) \geq \mathbb{E}_{p(\mathbf{I},\mathbf{T})}\left[\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp(f(\mathbf{I}_i, \mathbf{T}_i))}{\frac{1}{N}\sum_{j=1}^{N} \exp(f(\mathbf{I}_i, \mathbf{T}_j))}\right].$$

With $f(\mathbf{I}_i, \mathbf{T}_j) = \mathbf{I}_i^\top \mathbf{T}_j / \tau$, maximizing $\mathcal{L}_{\mathrm{CLIP}}$ (i.e., minimizing $-\mathcal{L}_{\mathrm{CLIP}}$) tightens this lower bound, implicitly maximizing the mutual information between the two modalities. This connection explains why CLIP learns semantically meaningful alignments: the optimization pressure to distinguish true pairs from false ones forces the encoders to capture shared semantic content.

20.2.4 Zero-Shot Classification

Given an image $\mathbf{x}$ and a set of K candidate class names $\{c_1, \ldots, c_K\}$, CLIP constructs text prompts for each class. A prompt template transforms the class name: e.g., `"a photo of a {`c_k`}"`. The text encoder produces embeddings $\mathbf{T}_1^c, \ldots, \mathbf{T}_K^c$ for these prompts. The zero-shot prediction is:

$$\hat{y} = \arg\max_{k \in \{1,\ldots,K\}} \frac{\langle \mathbf{I}, \mathbf{T}_k^c \rangle}{\tau} = \arg\max_k \cos(\mathbf{I}, \mathbf{T}_k^c).$$

Equivalently, this is a linear classifier in the embedding space where the weight vector for class k is $\mathbf{T}_k^c$ and the bias is zero. The temperature τ scales the logits, effectively controlling the confidence (a fixed τ would produce the same ranking). Prompt engineering significantly impacts accuracy: adding contextual phrases (e.g., "a photo of a {}", "a {}, a type of pet") or using prompt ensembles (averaging embeddings of multiple templates) can improve zero-shot accuracy by several percentage points.

20.2.5 Gradient Analysis

Consider the gradient of $\mathcal{L}_{\mathrm{I}\to\mathrm{T}}$ with respect to the image embedding $\mathbf{I}_i$:

$$\frac{\partial \mathcal{L}_{\mathrm{I}\to\mathrm{T}}}{\partial \mathbf{I}_i} = \frac{1}{\tau N}\left(-\mathbf{T}_i + \sum_{j=1}^{N} \frac{\exp(s_{ij})}{\sum_{k=1}^{N}\exp(s_{ik})}\,\mathbf{T}_j\right).$$

The gradient pulls $\mathbf{I}_i$ toward its positive text partner $\mathbf{T}_i$ (first term) and pushes it away from all negatives $\mathbf{T}_j$ ($j \neq i$), weighted by their relative similarity (the softmax distribution). This is a form of contrastive attraction-repulsion dynamics. The temperature τ modulates the hardness of negatives: smaller τ sharpens the softmax, making the model focus more on hard negatives (those with high similarity), while larger τ smooths the distribution toward uniform weighting.

20.3 Algorithm

Algorithm 25: CLIP Contrastive Pre-training

Input: Batch of N image-text pairs $\{(\mathbf{x}_i, \mathbf{t}_i)\}$, temperature τ, encoders f_θ, g_ϕ
Output: Loss $\mathcal{L}_{\mathrm{CLIP}}$

for $i \leftarrow 1$ **to** N **do**
  $\mathbf{I}_i \leftarrow f_\theta(\mathbf{x}_i)$;
  $\mathbf{T}_i \leftarrow g_\phi(\mathbf{t}_i)$;
  $\mathbf{I}_i \leftarrow \mathbf{I}_i / \|\mathbf{I}_i\|_2$;
  $\mathbf{T}_i \leftarrow \mathbf{T}_i / \|\mathbf{T}_i\|_2$;
end
$\mathbf{S} \leftarrow [s_{ij}]$ where $s_{ij} = \mathbf{I}_i^\top \mathbf{T}_j$;
$\mathbf{S} \leftarrow \mathbf{S} \cdot \exp(t)$ (scale by temperature);
$\mathcal{L}_{\mathrm{I}\to\mathrm{T}} \leftarrow -\frac{1}{N}\sum_{i=1}^{N} \log \mathrm{softmax}(\mathbf{S}_{i,:})_i$;
$\mathcal{L}_{\mathrm{T}\to\mathrm{I}} \leftarrow -\frac{1}{N}\sum_{j=1}^{N} \log \mathrm{softmax}(\mathbf{S}_{:,j}^\top)_j$;
$\mathcal{L}_{\mathrm{CLIP}} \leftarrow \frac{1}{2}(\mathcal{L}_{\mathrm{I}\to\mathrm{T}} + \mathcal{L}_{\mathrm{T}\to\mathrm{I}})$;
Minimize $\mathcal{L}_{\mathrm{CLIP}}$ w.r.t. (θ, ϕ, t);

20.4 Architecture

20.4.1 Image Encoder

CLIP experiments with two image encoder families:

- **ResNet**: Modified ResNet-50 and ResNet-101 with attention pooling instead of global average pooling. The attention pooling layer computes $\mathbf{I} = \sum_{h,w} \alpha_{h,w}\mathbf{x}_{h,w}$ where $\alpha = \mathrm{softmax}(\mathrm{MLP}(\mathbf{x}_{\mathrm{global}})^\top \mathbf{x}_{h,w})$. Also tested: EfficientNet-style scaling (ResNet-$D \times E$ where D is depth multiplier, E is width multiplier). ResNet-50×64 (640M params) is the largest variant.
- **Vision Transformer (ViT)**: Standard ViT with minor modifications—an additional layer normalization before the projection head, and a slightly different initialization scheme. ViT-L/14 (304M parameters, patch size 14, input 336^2) is a common choice.

The final linear projection from the encoder's output (pooled feature) to the joint embedding space of dimension d is applied after the encoder backbone. The embedding dimension d varies: 512 for ResNet-50, 768 for ViT-B, 1024 for ViT-L.

20.4.2 Text Encoder

The text encoder is a Transformer (GPT-2 style) with the following specifications:

- 12 layers, 8 attention heads, embedding dimension 512, MLP hidden 2048 (63M parameters).
- Input text is lowercased, tokenized with byte-pair encoding (BPE) using a vocabulary of 49,152 tokens.
- Maximum sequence length: 77 tokens, with a special [EOS] token marking the end.
- The representation is obtained from the final layer's hidden state at the [EOS] position, layer-normalized, then linearly projected to the joint embedding space.
- The text encoder is trained from scratch, not initialized from pre-trained language models.

20.4.3 Training Configuration

CLIP is trained on the WIT (WebImageText) dataset: 400 million (image, text) pairs collected from publicly available internet sources. Training uses:

- Batch size: 32,768 (extremely large, requiring distributed training across hundreds of GPUs/TPUs).
- Optimizer: Adam with decoupled weight decay (AdamW), $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-6}$.
- Learning rate: cosine schedule with 2,000-step linear warmup, peak LR $\approx 5 \times 10^{-4}$ scaled linearly with batch size.
- Training epochs: 32 on WIT (approximately 12.8 billion image-text pairs seen).
- Mixed-precision training (FP16) for memory efficiency.
- The large batch size is critical: InfoNCE benefits from many negative samples, and $N \gg 1$ provides a rich set of negatives per batch.

20.4.4 Prompt Engineering and Ensembling

For zero-shot inference, the base prompt is `"a photo of a {label}"`. CLIP also employs prompt ensembling: for a given label, multiple templates are filled (e.g., `"a blurry photo of a {}"`, `"a close-up photo of a {}"`, `"a {label}, a type of {category}"` for fine-grained datasets). The final text embedding is the mean of all template embeddings: $\mathbf{T}_k^c = \frac{1}{M} \sum_{m=1}^{M} \mathbf{T}_{k,m}^c$, where M is the number of templates. This averaging suppresses template-specific biases.

20.5 Pros and Cons

Advantages:

1. **Zero-shot transfer**: CLIP achieves competitive accuracy on over 30 vision datasets without any dataset-specific training labels, including fine-grained classification (Stanford Cars, Food-101), OCR (SST-2), and action recognition (UCF-101). On ImageNet, CLIP ViT-L/14 matches the fully supervised ResNet-50 baseline in zero-shot mode.

2. **Robustness to distribution shift**: CLIP models are significantly more robust to natural distribution shifts (ImageNet-A, ImageNet-R, ImageNet-Sketch) than standard ImageNet-trained models. The effective robustness (improvement beyond what accuracy predicts) suggests that CLIP learns more transferable, less spurious features.

3. **Open vocabulary**: The text-based classification head can be defined on-the-fly, enabling flexible inference for custom taxonomies, hierarchical labels, or attribute-based queries without retraining.

4. **Unified visual-semantic space**: The shared embedding space enables cross-modal retrieval (image-to-text, text-to-image) and zero-shot image captioning by nearest-neighbor retrieval.

5. **Scaling behavior**: Both model size and dataset size contribute to smooth performance improvements, with no observed plateau, indicating further gains from scaling are possible.

Disadvantages:

1. **Fine-grained limitations**: CLIP underperforms on tasks requiring fine spatial reasoning (e.g., counting objects, reading text, distinguishing left from right), suggesting that its global representation may lose detailed spatial information.

2. **Out-of-distribution blindness**: On truly out-of-distribution data like MNIST (handwritten digits), CLIP performs near chance level (zero-shot accuracy $\approx$ 15–20%). The web training data does not prepare it for synthetic or abstract visual patterns.

3. **Prompt sensitivity**: Performance varies substantially with prompt phrasing. Differences of 5–10 percentage points are common, requiring human-in-the-loop prompt optimization that partially defeats the zero-shot convenience.

4. **Societal biases**: Since WIT reflects internet data distributions, CLIP inherits harmful stereotypes and associations (e.g., gender-occupation correlations, racial biases). Deploying CLIP-based systems risks amplifying these biases.

5. **Computational cost**: Training requires hundreds of GPU-days or TPU-days, making full-scale reproduction expensive. Efficient fine-tuning of pre-trained CLIP models partially mitigates this.

6. **No generative capability**: CLIP is purely discriminative (contrastive). It aligns representations but cannot generate images or captions. Downstream integration with generative models (diffusion, autoregressive) is needed for multi-modal generation.

20.6 Evolution

SigLIP (Sigmoid Loss for Language-Image Pre-training, Zhai et al., 2023): Replaces the softmax-based InfoNCE loss with a simpler sigmoid-based pairwise loss. For each pair $(\mathbf{I}_i, \mathbf{T}_j)$, the loss is:

$$\mathcal{L}_{\text{SigLIP}} = -\frac{1}{N^2}\sum_{i=1}^{N}\sum_{j=1}^{N}\log\sigma(z_{ij}\cdot(\mathbf{I}_i^\top\mathbf{T}_j + b)/\tau),$$

where $z_{ij} = 1$ if $i = j$ (positive), -1 otherwise (negative). This decouples batch size dependence: each pair is treated independently, enabling smaller batch training without loss of quality. SigLIP achieves comparable or better performance while being more memory-efficient.

LiT (Locked-image Tuning, Zhai et al., 2022): Demonstrates that fine-tuning only the text encoder while keeping a pre-trained, frozen image encoder yields better performance than joint fine-tuning. The insight is that image representations from good pre-trained models are rich enough; calibrating text to align with them is sufficient. LiT achieves state-of-the-art with fewer trainable parameters.

OpenCLIP (Ilharco et al., 2022): An open-source reproduction of CLIP, training on LAION-400M (a filtered subset of Common Crawl) and LAION-2B. OpenCLIP validates and extends the original findings, providing publicly available models and training code. Scaling to 2B samples with ViT-G/14 achieves performance exceeding the original CLIP.

MetaCLIP (Xu et al., 2023): Introduces metadata-based curation of the training data. Instead of training on all web image-text pairs equally, MetaCLIP uses metadata (e.g., creation dates, textual overlap, cross-modal consistency scores) to filter and balance the training distribution, reducing noise while preserving diversity.

CoCa (Contrastive Captioner, Yu et al., 2022): Unifies contrastive and generative objectives. A single encoder-decoder model is trained with both contrastive loss (on unimodal encoder outputs) and captioning loss (autoregressive decoder). The decoder is decoupled into unimodal and image-attended components, enabling both embedding extraction and generation from one model.

Fashion for diffusion: CLIP embeddings serve as conditioning signals in text-to-image models. Stable Diffusion uses CLIP text encoder embeddings as conditioning for the UNet denoiser. DALL-E 2 uses CLIP image embeddings as an intermediate representation: text $\rightarrow$ CLIP image embedding $\rightarrow$ image decoder.

CLIP-Benchmark: The community has developed extensive evaluation suites measuring zero-shot classification, retrieval, and robustness across hundreds of datasets, establishing CLIP as the de facto standard for vision-language pre-training evaluation.

Trend: The field moves from pure contrastive to hybrid contrastive-generative objectives (CoCa, BLIP-2), from softmax-based to sigmoid-based losses (SigLIP), and toward data quality over quantity (MetaCLIP, DataComp). CLIP's core insight—that natural

language supervision is a scalable alternative to fixed label sets—remains foundational to modern multimodal AI.

Chapter 21

BLIP: Bootstrapping Language-Image Pre-training

21.1 Motivation

CLIP demonstrated that contrastive learning on web-scale image-text pairs produces powerful multimodal representations. However, web-crawled image-text data is inherently noisy: captions are often incomplete, irrelevant, or syntactically broken. Training on such data introduces a ceiling on representation quality, as the learning signal is contaminated by mismatched pairs and impoverished textual descriptions. BLIP (Bootstrapping Language-Image Pre-training, Li et al., 2022) addresses this through two complementary mechanisms: (1) a unified multimodal architecture (MED) that jointly handles contrastive learning, image-text matching, and caption generation, and (2) CapFilt (Captioning and Filtering), a bootstrapping framework where the model generates synthetic captions for web images and filters out noisy pairs, iteratively improving the training data. This removes the need for expensive human annotation and produces cleaner data from raw web crawl, leading to state-of-the-art vision-language representations. BLIP-2 extends this further by introducing the Q-Former, a lightweight module that bridges frozen large-scale image and language models, enabling zero-shot instruction following without fine-tuning the LLM.

21.2 Mathematical Formulation

21.2.1 Multimodal Mixture of Encoder-Decoder (MED)

BLIP's architecture, MED, comprises four components that can be flexibly combined:

1. **Unimodal Image Encoder** f_{img}: A ViT that produces image features. Given image $\mathbf{x}$, the output is a sequence of patch embeddings $\mathbf{v} = [v_{\text{cls}}, v_1, \ldots, v_N] \in \mathbb{R}^{(N+1)\times D}$, where v_{cls} is the [CLS] embedding used for contrastive and matching objectives.

2. **Unimodal Text Encoder** f_{txt}: A BERT-base model. For text input $\mathbf{t}$, the [CLS] token embedding is used: $\mathbf{u}_{\text{cls}} = f_{\text{txt}}(\mathbf{t}) \in \mathbb{R}^D$.

3. **Image-Grounded Text Encoder** $f_{\text{txt}}^{\text{img}}$: Augments the text encoder with cross-attention layers inserted between self-attention and feed-forward layers. The cross-attention attends to image features $\mathbf{v}$:

$$\mathbf{y} = \text{CrossAttn}(\text{SelfAttn}(\mathbf{h}), \mathbf{v}),$$

   where $\mathbf{h}$ is the hidden representation of the text token sequence. This produces contextualized embeddings that fuse visual and textual information, used for image-text matching.

4. **Image-Grounded Text Decoder** f_{dec}: Replaces the bidirectional self-attention in the text encoder with causal (autoregressive) self-attention. The decoder generates text token by token, attending to image features via cross-attention:

$$P(\mathbf{t} \mid \mathbf{x}) = \prod_{\ell=1}^{L} P(t_\ell \mid t_{<\ell}, \mathbf{x}).$$

Parameter sharing: f_{txt}, $f_{\text{txt}}^{\text{img}}$, and f_{dec} share all parameters except the self-attention layers (bidirectional vs. causal) and cross-attention layers (present in $f_{\text{txt}}^{\text{img}}$ and f_{dec}, absent in f_{txt}). The self-attention and feed-forward layers are shared, making MED parameter-efficient.

21.2.2 Pre-training Objectives

BLIP is pre-trained with three losses, jointly optimized:

Image-Text Contrastive (ITC)

Same formulation as CLIP (Section 20.2.2). Let $\mathbf{I} = v_{\text{cls}} / \|v_{\text{cls}}\|_2$ and $\mathbf{T} = \mathbf{u}_{\text{cls}} / \|\mathbf{u}_{\text{cls}}\|_2$. The contrastive loss with temperature τ is:

$$\mathcal{L}_{\text{ITC}} = \frac{1}{2}\Big(\mathcal{L}_{\text{I}\to\text{T}} + \mathcal{L}_{\text{T}\to\text{I}}\Big),$$

with symmetric InfoNCE as defined in equation (20.3). This aligns the unimodal representations in a shared embedding space, facilitating retrieval and zero-shot transfer.

Image-Text Matching (ITM)

ITM is a binary classification task: given an image $\mathbf{x}$ and text $\mathbf{t}$, predict whether they match. The image-grounded text encoder $f_{\text{txt}}^{\text{img}}$ processes both modalities and produces a fused [CLS] embedding $\mathbf{z} \in \mathbb{R}^D$. A linear layer predicts the matching probability:

$$\hat{p} = \sigma(\mathbf{w}_{\text{ITM}}^\top \mathbf{z} + b_{\text{ITM}}).$$

The loss is binary cross-entropy:

$$\mathcal{L}_{\text{ITM}} = -\mathbb{E}_{(\mathbf{x},\mathbf{t})}\Big[y \log \hat{p} + (1-y)\log(1-\hat{p})\Big],$$

where $y = 1$ for matched pairs and $y = 0$ for negative pairs. To avoid trivial negatives, BLIP uses **hard negative mining**: given a positive pair $(\mathbf{x}_i, \mathbf{t}_i)$, the negative sample is selected from the batch as:

$$\mathbf{t}_j^{\text{hard}} = \arg\max_{j \neq i} \frac{\langle \mathbf{I}_i, \mathbf{T}_j \rangle}{\tau},$$

i.e., the text that has highest contrastive similarity with image $\mathbf{x}_i$ but is not its ground-truth caption. This forces ITM to learn fine-grained alignment beyond what contrastive learning captures.

Language Modeling (LM)

The image-grounded text decoder f_{dec} generates captions autoregressively. Given image $\mathbf{x}$ and prefix text tokens $t_{<k}$, the model predicts the next token:

$$\mathcal{L}_{\text{LM}} = -\sum_{k=1}^{L} \log P(t_k \mid t_{<k}, \mathbf{x}),$$

where the probability is obtained from a softmax over the vocabulary applied to the decoder's output hidden state at position k. This enables caption generation and, during CapFilt, synthetic caption creation.

The total pre-training loss is a weighted sum:

$$\mathcal{L}_{\text{MED}} = \mathcal{L}_{\text{ITC}} + \mathcal{L}_{\text{ITM}} + \mathcal{L}_{\text{LM}}.$$

21.2.3 CapFilt: Captioning and Filtering

CapFilt bootstraps cleaner training data from noisy web pairs. It has two sub-modules, both initialized from the pre-trained MED and fine-tuned on a small set of high-quality human-annotated data (e.g., COCO captions):

- **Captioner** ($\mathcal{C}$): Fine-tuned f_{dec} on the LM objective using clean image-caption pairs. For each web image $\mathbf{x}_w$, it generates a synthetic caption $\hat{\mathbf{t}}_w$ via beam search (typically beam size 3–5). The captioner may also use nucleus sampling ($p = 0.9$) to produce diverse captions.
- **Filter** ($\mathcal{F}$): Fine-tuned $f_{\text{txt}}^{\text{img}}$ on the ITM objective. For each (image, caption) pair—both original web text and synthesized captions—the filter predicts a matching score. Pairs with scores below a threshold are discarded.

The bootstrapping process:

1. Pre-train MED on noisy web data.
2. Fine-tune $\mathcal{C}$ and $\mathcal{F}$ on clean data.
3. Apply $\mathcal{C}$ to all web images, generating new captions $\{\hat{\mathbf{t}}_w^m\}_{m=1}^M$ (multiple candidates per image).
4. Apply $\mathcal{F}$ to filter both original and synthetic pairs. High-scoring synthetic captions replace or augment the original noisy text.
5. Re-train MED on the cleaned dataset.
6. Optionally iterate: the improved MED yields better $\mathcal{C}$ and $\mathcal{F}$.

This creates a virtuous cycle: better captions improve the model, and a better model produces better captions and filtering.

21.3 Algorithm

Algorithm 26: CapFilt: Bootstrapping Data Cleaning

Input: Noisy web dataset $\mathcal{D}_w$, clean dataset $\mathcal{D}_c$, MED model
Output: Improved image-text pairs $\mathcal{D}_{\text{clean}}$

Pre-train MED on $\mathcal{D}_w$ with $\mathcal{L}_{\text{MED}}$;
Fine-tune Captioner $\mathcal{C}$ on $\mathcal{D}_c$ with $\mathcal{L}_{\text{LM}}$;
Fine-tune Filter $\mathcal{F}$ on $\mathcal{D}_c$ with $\mathcal{L}_{\text{ITM}}$;
for *each image* $\mathbf{x}_w$ *in* $\mathcal{D}_w$ **do**
    $\{\hat{\mathbf{t}}_w^m\} \leftarrow \mathcal{C}(\mathbf{x}_w)$ (beam search, M candidates);
    for *each candidate* $\hat{\mathbf{t}}_w^m$ **do**
        $s_m \leftarrow \mathcal{F}(\mathbf{x}_w, \hat{\mathbf{t}}_w^m)$ (matching score);
    end
    $\hat{\mathbf{t}}_w^* \leftarrow$ caption with highest s_m above threshold;
    Add $(\mathbf{x}_w, \hat{\mathbf{t}}_w^*)$ to $\mathcal{D}_{\text{clean}}$;
end
Re-train MED on $\mathcal{D}_{\text{clean}}$ with $\mathcal{L}_{\text{MED}}$;

21.4 Architecture

21.4.1 BLIP-2: Q-Former

BLIP-2 introduces a fundamentally different approach: instead of jointly training large vision and language models end-to-end, it interposes a lightweight Querying Transformer (Q-Former) between a **frozen** image encoder and a **frozen** large language model (LLM). This decoupling dramatically reduces training cost while leveraging the capabilities of pre-trained giants.

The Q-Former consists of two transformer sub-modules sharing self-attention layers:

- **Image Transformer**: Interacts with frozen image features $\mathbf{v} \in \mathbb{R}^{N \times D_{\text{img}}}$ through cross-attention layers. A set of M learnable query tokens $\mathbf{Q} \in \mathbb{R}^{M \times D_q}$ (where $M \ll N$, typically $M = 32$) serves as the information bottleneck. The cross-attention layers compute:

$$\mathbf{Q}' = \text{CrossAttn}(\mathbf{Q}, \mathbf{v}, \mathbf{v}) = \text{softmax}\left(\frac{\mathbf{Q}_q(\mathbf{v}_k)^\top}{\sqrt{D_q}}\right)\mathbf{v}_v,$$

  where $\mathbf{Q}_q$ are query projections of $\mathbf{Q}$, and $\mathbf{v}_k, \mathbf{v}_v$ are key/value projections of $\mathbf{v}$. Effectively, the queries *extract* relevant visual information.

- **Text Transformer**: Processes text input with shared self-attention layers but its own cross-attention (to queries for ITG loss). The text can serve both as input (for matching) and output (for generation).

BLIP-2 pre-training (Stage 1) optimizes the Q-Former with three objectives, all op-

erating on the query representations extracted from images:

$\mathcal{L}_{\text{ITC}}$: contrastive alignment between query output and text [CLS],

$\mathcal{L}_{\text{ITM}}$: binary matching (hard negatives),

$\mathcal{L}_{\text{ITG}}$: image-grounded text generation: queries attend to image, then text decoder generates captio

In the ITG task, the queries are fed as visual prefixes to the text decoder via causal self-attention, forcing the queries to compress visual information into a format the text decoder can consume.

Stage 2 connects the Q-Former to a frozen LLM (e.g., OPT, FlanT5). The output query embeddings $\mathbf{Q}_{\text{out}} \in \mathbb{R}^{M \times D_q}$ from Stage 1 are projected by a linear layer $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{D_q \times D_{\text{LLM}}}$ to match the LLM's input dimension, then prepended to the text token embeddings as a *visual prompt*:

$$\text{LLM input} = [\mathbf{Q}_{\text{out}}\mathbf{W}_{\text{proj}};\ \mathbf{t}_1\mathbf{W}_e;\ \ldots;\ \mathbf{t}_L\mathbf{W}_e].$$

The LLM generates text autoregressively, attending to both the visual prefix and text context. Only the projection $\mathbf{W}_{\text{proj}}$ is trained; the LLM remains frozen. This enables BLIP-2 to inherit the LLM's reasoning and generation capabilities without catastrophic forgetting.

21.4.2 Training Details

BLIP pre-training uses:

- Datasets: COCO, Visual Genome, Conceptual Captions, Conceptual 12M, SBU Captions (14M images total for pre-training; COCO for CapFilt fine-tuning).
- Image encoder: ViT-B/16 (86M) or ViT-L/16 (304M).
- Text encoder/decoder: BERT-base (110M) with added cross-attention layers.
- Optimizer: AdamW, learning rate 10^{-4}, cosine decay, batch size 2,880.
- Image resolution: 384×384 for ViT-L.

BLIP-2 pre-training uses larger frozen models: ViT-g/14 from EVA-CLIP (1B params) or ViT-L from CLIP, and LLMs like OPT-6.7B or FlanT5-XXL (11B). The Q-Former itself has only 188M parameters, making Stage 2 fine-tuning extremely efficient.

21.4.3 Instruction Tuning (InstructBLIP)

InstructBLIP extends BLIP-2 by fine-tuning the Q-Former on instruction-formatted data. The model receives both an image and a natural language instruction (e.g., `"Describe this image in detail."` or `"What is unusual about this image?"`), and must produce the appropriate response. The Q-Former additionally takes the instruction token sequence as input (via self-attention with the queries), enabling instruction-conditioned visual feature extraction. Training data is a mixture of 26 publicly available vision-language datasets converted to instruction format. InstructBLIP achieves state-of-the-art zero-shot performance across diverse vision-language tasks, including visual question answering, image captioning, and visual reasoning.

21.5 Pros and Cons

Advantages:

1. **Unified framework**: MED handles understanding (contrastive, matching) and generation (captioning) in a single architecture with shared parameters, unlike CLIP (contrastive only) or VL-T5 (generation only).

2. **Noisy data robustness**: CapFilt significantly improves data quality without human annotation, creating a self-improving loop. The synthetic captions from CapFilt improve downstream tasks by 1–2% over the original web text.

3. **Modularity (BLIP-2)**: By freezing large pre-trained models and connecting them via a lightweight Q-Former, BLIP-2 leverages the best of both worlds: visual representation from CLIP/EVA and language understanding from LLMs, without expensive joint training.

4. **Efficiency**: BLIP-2's two-stage training requires training only 188M parameters (Q-Former + projection), yet achieves performance comparable to models 10× larger trained end-to-end.

5. **Instruction following (InstructBLIP)**: Instruction tuning generalizes to unseen tasks, enabling flexible multimodal interaction without task-specific fine-tuning.

6. **Hard negative mining**: BLIP's ITM objective with hard negatives forces fine-grained multimodal alignment beyond the global alignment of contrastive learning, improving visual grounding for tasks like visual question answering.

Disadvantages:

1. **Training complexity**: MED requires balancing three loss terms (ITC, ITM, LM), each with different scales and convergence rates. Hyperparameter tuning is non-trivial.

2. **Captioner bias**: CapFilt's synthetic captions can amplify biases present in the clean fine-tuning data. If COCO captions have a particular style (e.g., describing only foreground objects), Web images will inherit this narrowness.

3. **Information bottleneck (Q-Former)**: The fixed number of query tokens ($M = 32$) limits the amount of visual detail that can be communicated to the LLM. Fine-grained tasks (e.g., counting, reading small text) suffer.

4. **Frozen encoder limitations**: BLIP-2's reliance on frozen encoders means it cannot improve visual or language representations beyond their pre-trained quality. If the image encoder misidentifies an object, the Q-Former cannot correct it.

5. **Hallucination**: When connected to LLMs, BLIP-2 can generate plausible but factually incorrect descriptions, inheriting the hallucination tendencies of the base LLM.

6. **Caption length and diversity**: CapFilt generates captions of typical training-set length; it does not produce long, detailed descriptions without dedicated training (e.g., on LLaVA-style data).

21.6 Evolution

BLIP-2 (Li et al., 2023): The Q-Former architecture as described above, enabling frozen-model integration. Key insight: the visual-to-language interface can be learned efficiently with a small bottleneck model. BLIP-2 with OPT-6.7B achieves state-of-the-art on zero-shot VQAv2 and image captioning benchmarks.

InstructBLIP (Dai et al., 2023): Instruction-tuned variant of BLIP-2. Instruction text is fed to the Q-Former alongside queries, enabling task-adaptive visual extraction. Trained on 26 datasets converted to instruction format. Achieves zero-shot generalization to tasks not seen during instruction tuning, including visual dialog and spatial reasoning.

LLaVA (Liu et al., 2023): A simpler alternative to Q-Former: uses a single linear projection layer to map visual features to the LLM's token embedding space. While architecturally simpler than Q-Former, LLaVA achieves competitive performance through high-quality instruction data (GPT-4 generated multimodal conversations). This suggests that the bottleneck design, while theoretically sound, may not be strictly necessary if training data is rich enough.

MiniGPT-4 (Zhu et al., 2023): Connects a frozen ViT (BLIP-2) and a frozen Vicuna LLM through a single linear projection layer, trained on 5M aligned image-text pairs. Demonstrates that even minimal learnable interfaces suffice when base models are sufficiently strong.

Qwen-VL (Bai et al., 2023): A family of multimodal LLMs jointly trained from the start (not frozen). Uses a ViT initialized from OpenCLIP and a Qwen LLM, trained end-to-end with a three-stage curriculum. Supports multi-image input and bounding-box-grounded conversation.

BLIP-3 (xGen-MM, 2024): Scales BLIP to larger data and models, adding interleaved image-text training. The architecture converges toward a simpler design: ViT + projector + LLM, moving away from the Q-Former complexity toward the effective simplicity demonstrated by LLaVA.

Coca (Contrastive Captioner, Yu et al., 2022): A contemporaneous approach that unifies contrastive and generative objectives in a single decoder-only architecture, achieving both classification and captioning in one model. Pre-dates BLIP but shares the motivation of unifying understanding and generation.

Trend: The BLIP lineage demonstrates a clear trajectory: from complex unified architectures (MED) to modular, frozen-model integration (Q-Former) to simple projectors (LLaVA-style). The key insight—that effective vision-language models can be built by bridging strong pre-trained components—has become the dominant paradigm. The remaining frontier is scaling: larger datasets, longer contexts, and deeper integration beyond simple caption-level alignment.

Part VII

Specialized Vision Tasks

Chapter 22

Optical Character Recognition

22.1 Motivation

Optical Character Recognition (OCR) is the task of extracting machine-readable text from images. Its applications span document digitization (scanning books, invoices, receipts), scene text understanding (street signs, license plates, storefronts), handwriting recognition (historical archives, medical prescriptions), and assistive technology (reading text aloud for visually impaired users). Unlike generic object recognition, OCR faces unique challenges: (i) text appears in arbitrary fonts, sizes, colors, and orientations; (ii) scene text is often deformed by perspective, curvature, and occlusions; (iii) character sequences must be recognized in correct order, making spatial arrangement critical; (iv) the output space is combinatorial—any sequence of characters from the alphabet. Traditional OCR pipelines decomposed the problem into binarization, character segmentation, and per-character recognition, with heuristic post-processing for lexicon-based correction. Deep learning replaced these fragile, stage-error-compounding pipelines with end-to-end trainable models that learn feature extraction, sequence modeling, and decoding jointly. Modern OCR has bifurcated into two predominant paradigms: two-stage (detection followed by recognition) and end-to-end (single model from pixel to text), with Transformer-based architectures increasingly unifying both.

22.2 Mathematical Formulation

22.2.1 Text Detection: Differential Binarization (DBNet)

Text detection localizes text regions in an image. DBNet (Liao et al., 2020) is a segmentation-based detector that produces both a probability map $\mathbf{P} \in [0,1]^{H \times W}$ (text vs. non-text at each pixel) and a threshold map $\mathbf{T} \in \mathbb{R}^{H \times W}$. The binarization is made differentiable to enable end-to-end training:

Standard binarization (hard threshold):

$$\mathbf{B}_{i,j} = \begin{cases} 1 & \text{if } \mathbf{P}_{i,j} \geq t, \\ 0 & \text{otherwise.} \end{cases}$$

Differentiable binarization (soft approximation with temperature k):

$$\hat{\mathbf{B}}_{i,j} = \sigma\big(k(\mathbf{P}_{i,j} - \mathbf{T}_{i,j})\big) = \frac{1}{1 + \exp(-k(\mathbf{P}_{i,j} - \mathbf{T}_{i,j}))}.$$

As $k \to \infty$, $\hat{\mathbf{B}} \to \mathbf{B}$ (hard threshold). At test time, a fixed threshold (e.g., $t = 0.3$) on $\mathbf{P}$ produces the binary map; the threshold map $\mathbf{T}$ is supervised during training to learn adaptive per-pixel thresholds. The loss is a weighted combination of probability map loss, threshold map loss, and binary map loss:

$$\mathcal{L}_{\text{DBNet}} = \mathcal{L}_p + \alpha \mathcal{L}_t + \beta \mathcal{L}_b,$$

where $\mathcal{L}_p$ and $\mathcal{L}_b$ use binary cross-entropy and $\mathcal{L}_t$ uses ℓ_1 distance between $\mathbf{T}$ and the computed ideal threshold.

22.2.2 Text Recognition with CRNN+CTC

The most widely deployed recognition architecture is CRNN (Convolutional Recurrent Neural Network) with CTC (Connectionist Temporal Classification) decoding. An image patch $\mathbf{x} \in \mathbb{R}^{H' \times W' \times C}$ is processed by a CNN backbone (e.g., VGG, ResNet) followed by a feature squasher: a row-wise max-pooling or a dedicated reshape operation that extracts a sequence of feature vectors $\mathbf{h} = (h_1, \ldots, h_T)$ where T is the sequence length (proportional to W'). A bidirectional LSTM (BiLSTM) models sequential dependencies:

$$\overrightarrow{h}_t = \mathrm{LSTM}(h_t, \overrightarrow{h}_{t-1}), \qquad \overleftarrow{h}_t = \mathrm{LSTM}(h_t, \overleftarrow{h}_{t+1}).$$

The concatenated hidden states $\tilde{h}_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]$ are mapped via a linear layer $W_{\mathrm{ctc}} \in \mathbb{R}^{2D_{\mathrm{hidden}} \times (|\mathcal{V}|+1)}$ to logits over the vocabulary $\mathcal{V}$ plus a blank token $\varnothing$:

$$\mathbf{z}_t = \tilde{h}_t W_{\mathrm{ctc}} + \mathbf{b}_{\mathrm{ctc}}, \qquad P(a_t = k \mid \mathbf{x}) = \mathrm{softmax}(\mathbf{z}_t)_k.$$

CTC defines the conditional probability of a label sequence $\mathbf{y} = (y_1, \ldots, y_U)$ given the input. Let $\mathcal{B}$ be the many-to-one mapping that removes blank tokens and merges consecutive identical non-blank tokens (so $\texttt{a}\varnothing\texttt{b}\varnothing\texttt{b} \to \texttt{abb}$ and $\texttt{a}\varnothing\texttt{a}\varnothing \to \texttt{aa}$). For a length-$T$ alignment $\mathbf{a} = (a_1, \ldots, a_T)$ with each $a_t \in \mathcal{V} \cup \{\varnothing\}$:

$$\mathcal{B}(\mathbf{a}) = \mathbf{y} \iff \text{removing } \varnothing \text{ and merging duplicates yields } \mathbf{y}.$$

The probability of the label sequence is the sum over all consistent alignments:

$$P(\mathbf{y} \mid \mathbf{x}) = \sum_{\mathbf{a} \in \mathcal{B}^{-1}(\mathbf{y})} \prod_{t=1}^{T} P(a_t \mid \mathbf{x}).$$

The number of valid alignments grows exponentially with T and U, but the forward-backward algorithm computes $P(\mathbf{y} \mid \mathbf{x})$ in $\mathcal{O}(TU)$ using dynamic programming. Define forward variables $\alpha_t(s)$ as the total probability of all alignments of length t that match the prefix of $\mathbf{y}$ up to position s (with blanks interleaved). The recurrence:

$$\alpha_t(s) = \big(\alpha_{t-1}(s) + \alpha_{t-1}(s-1) + [y_s \neq y_{s-1}] \cdot \alpha_{t-1}(s-2)\big) \cdot P(a_t = y'_s \mid \mathbf{x}),$$

where y' is y with blanks inserted between all positions and at both ends. The CTC loss is the negative log-likelihood:

$$\mathcal{L}_{\mathrm{CTC}} = -\log P(\mathbf{y} \mid \mathbf{x}) = -\log \sum_{s=1}^{|\mathbf{y}'|} \alpha_T(s).$$

At inference, greedy decoding selects $\hat{\mathbf{a}}_t = \arg\max_k P(a_t = k \mid \mathbf{x})$ and applies $\mathcal{B}$, or beam search with a language model for improved accuracy.

22.2.3 Attention-Based Sequence Recognition

An alternative to CTC is attention-based encoder-decoder models. The encoder (CNN+BiLSTM or ViT) produces hidden states $\mathbf{h} = [h_1, \ldots, h_T]$. The decoder (GRU or LSTM) generates the output sequence autoregressively. At step t, the context vector is computed via Bahdanau attention:

$$e_{t,i} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a s_{t-1} + \mathbf{U}_a h_i), \qquad \alpha_{t,i} = \text{softmax}(\mathbf{e}_t)_i, \qquad \mathbf{c}_t = \sum_{i=1}^{T} \alpha_{t,i} h_i.$$

The decoder state and output:

$$s_t = \text{GRU}(s_{t-1}, [\mathbf{c}_t; \text{Embedding}(y_{t-1})]), \qquad P(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_o[\mathbf{c}_t; s_t] + \mathbf{b}_o).$$

The training loss is cross-entropy: $\mathcal{L}_{\text{Attn}} = -\sum_t \log P(y_t^* \mid y_{<t}^*, \mathbf{x})$.

22.2.4 TrOCR: Pure Transformer OCR

TrOCR (Li et al., 2021) simplifies the pipeline by using a ViT encoder and a Transformer decoder with cross-attention, eliminating RNNs and CTC entirely. The image $\mathbf{x}$ is partitioned into patches: $\mathbf{z}_0 = \text{ViT}(\mathbf{x}) \in \mathbb{R}^{N \times D}$. The decoder generates text tokens autoregressively:

$$P(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{U} P(y_t \mid y_{<t}, \mathbf{z}_0).$$

Each decoder layer includes self-attention (causal) followed by cross-attention to $\mathbf{z}_0$:

$$\mathbf{c}_t = \text{CrossAttn}(\text{SelfAttn}(\mathbf{d}_{<t}), \mathbf{z}_0, \mathbf{z}_0),$$

where $\mathbf{d}_{<t}$ are the decoder hidden states up to position $t-1$. The model is pre-trained on large synthetic datasets (e.g., SynthText, MJSynth with billions of word images), then fine-tuned on real data.

22.2.5 Thin-Plate Spline (TPS) Rectification

For irregular text (curved, perspective-distorted), a spatial transformer network (STN) can rectify the image before recognition. TPS defines a smooth warp between control points. Given K target control points $\{\mathbf{p}'_k\}_{k=1}^K$ on a canonical grid (typically $K = 20$ on the image boundary), the STN predicts the corresponding source control points $\{\mathbf{p}_k\}$ from the input image via a localization CNN. The TPS interpolation computes the warp:

$$\mathbf{p}' = \mathbf{T} + \mathbf{M}\mathbf{p} + \sum_{k=1}^{K} \mathbf{w}_k \, \phi(\|\mathbf{p} - \mathbf{p}_k\|),$$

where $\phi(r) = r^2 \log r$ is the radial basis function kernel. The coefficients are solved to exactly map the predicted control points to the target grid, producing a rectified image that can be processed by a standard recognizer.

22.3 Algorithm

Algorithm 27: CRNN with CTC for Text Recognition

Input: Image $\mathbf{x}$, CRNN model with CNN encoder E, BiLSTM R, CTC head
Output: Recognized text $\hat{\mathbf{y}}$

$\mathbf{F} \leftarrow E(\mathbf{x})$ (CNN feature map, shape $C' \times H' \times W'$);
$\mathbf{h} \leftarrow \text{map_to_sequence}(\mathbf{F})$ (reshape to $T \times D_{\text{cnn}}$);
$\overrightarrow{\mathbf{h}}, \overleftarrow{\mathbf{h}} \leftarrow \text{BiLSTM}(\mathbf{h})$;
$\tilde{\mathbf{h}} \leftarrow [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$ for $t = 1, \ldots, T$;
$\mathbf{z} \leftarrow \tilde{\mathbf{h}}\mathbf{W}_{\text{ctc}} + \mathbf{b}_{\text{ctc}}$ (logits $\in \mathbb{R}^{T \times (|\mathcal{V}|+1)}$);
$\mathbf{P} \leftarrow \text{softmax}(\mathbf{z})$ (frame-wise probabilities);
for *training* **do**
  $\mathcal{L} \leftarrow -\log \sum_{\mathbf{a}:\mathcal{B}(\mathbf{a})=\mathbf{y}} \prod_t P(a_t \mid \mathbf{x})$ (CTC forward-backward);
end
for *inference (greedy)* **do**
  $\hat{\mathbf{a}} \leftarrow (\arg\max_k \mathbf{P}_{1,k}, \ldots, \arg\max_k \mathbf{P}_{T,k})$;
  $\hat{\mathbf{y}} \leftarrow \mathcal{B}(\hat{\mathbf{a}})$ (collapse blanks and duplicates);
end

22.4 Architecture

22.4.1 Detection Architectures

- **CTPN (Connectionist Text Proposal Network)**: Uses VGG-16 features, a 3×3 sliding window generates fixed-width ($16 \times W$) proposals at each spatial position. Vertical anchors of $k = 10$ heights capture text of varying scales. The proposals are connected by a BiLSTM to model the sequential nature of text, then refined via bounding box regression and text/non-text classification.

- **EAST (Efficient and Accurate Scene Text Detector)**: An FCN-based detector that directly predicts per-pixel score maps (text presence) and geometry maps (axis-aligned boxes or rotated rectangles/quadrangles). No proposal stage, no post-processing beyond thresholding and NMS. Output channels: 1 (score) + 4 (AABB) or 1 + 5 (rotated box: distance to four boundaries + rotation angle) or 1 + 8 (quadrangle vertices).

- **DBNet (Differentiable Binarization)**: Backbone (ResNet-18/50 with FPN) $\rightarrow$ probability map $\mathbf{P}$ + threshold map $\mathbf{T} \rightarrow$ approximate binary map $\hat{\mathbf{B}} \rightarrow$ contour extraction and box formation. The differentiable binarization allows gradients to flow through the binarization step, improving detection of text boundaries.

22.4.2 Recognition Architectures

- **CRNN**: CNN (VGG/ResNet) + BiLSTM (2 layers, 256 hidden units each) + CTC. Simple, fast, and effective for regular (horizontal) text. Input height normalized to 32 pixels, maintaining aspect ratio.

- **ASTER**: Adds TPS rectification network before CRNN recognition. The STN predicts 20 control points; TPS warps the image to a canonical view. Recognition uses BiLSTM with attention-based decoder instead of CTC, enabling modeling of longer sequences and 2D attention.
- **SATRN (Self-Attention Text Recognition Network)**: Uses a Transformer encoder that applies self-attention directly to the 2D CNN feature map (without squeezing to 1D sequence), preserving 2D spatial structure. A Transformer decoder with cross-attention generates text. This enables modeling irregular text with curved/vertical layout.
- **TrOCR**: ViT encoder + Transformer decoder. Pre-trained on synthetic data (hundreds of millions of word images rendered with diverse fonts, colors, and backgrounds). Fine-tuned on real datasets. Performance scales with ViT size and pre-training data volume.
- **Donut** (Document Understanding Transformer): An encoder-decoder Transformer for document parsing. Input: document image. Output: JSON parse tree. No OCR-specific heuristics—the model learns to read and structure text simultaneously. Pre-trained on synthetic documents.

22.4.3 Metrics

- **Character Error Rate (CER)**: CER $= (S + D + I)/N$, where S substitutions, D deletions, I insertions via edit distance, N total reference characters.
- **Word Error Rate (WER)**: Same as CER but computed at word level (splitting on whitespace). A single character error makes the whole word incorrect, making WER harsher.
- **Normalized Edit Distance**: $1 - \text{CER}$, ranging from 0 (completely wrong) to 1 (perfect).
- **Detection metrics**: Precision, recall, F1-score at IoU thresholds (typically 0.5).
- **End-to-end metrics**: Word-level F1 (a detection is correct if IoU > 0.5 and the recognized text exactly matches ground truth, case-insensitive).

22.5 Pros and Cons

Advantages (Two-stage: Detection + Recognition):

1. **Modularity**: Detection and recognition can be independently improved, swapped, and hyperparameter-tuned. A better detector benefits all downstream recognizers.
2. **Interpretability**: Detection outputs (bounding boxes) provide spatial feedback for debugging. Recognition errors can be traced to detector misses or recognizer failures.
3. **Efficiency on long documents**: Detection isolates text regions, enabling per-region recognition with fixed input size. Full-page recognition avoids the quadratic complexity of attention over the entire page.

4. **CTC simplicity**: CTC requires no attention, no autoregressive decoding, and no teacher forcing. It is simple to implement, parallelize, and integrate with beam search and language models.

Advantages (End-to-end Transformers):

1. **No error propagation**: A single model removes the risk of cascading errors (detection failure → recognition failure). Errors are globally optimized.
2. **Unified architecture**: Same Transformer design as other vision and language tasks, enabling co-training and transfer learning (e.g., from image captioning to OCR).
3. **Handles diverse layouts**: Attention mechanisms naturally attend to different text regions without explicit detection, adapting to complex, multi-column layouts.
4. **Scale-friendly**: Performance improves smoothly with model size and data, following Transformer scaling laws without architectural changes.

Disadvantages:

1. **Two-stage latency**: Detection + recognition incurs sequential latency. Real-time applications (video OCR) struggle.
2. **CTC alignment constraints**: CTC assumes conditional independence between output tokens given the input. Language model integration (beam search) is needed, but the basic CTC output is character-level, not word-level.
3. **Attention drift**: Attention-based decoders can misalign (attend to wrong image regions) for long or highly curved text, producing hallucinated or truncated output.
4. **Synthetic domain gap**: Models trained on synthetic text (MJSynth, SynthText) may not generalize well to unusual fonts, extreme lighting, or handwritten text. Real data fine-tuning is usually required.
5. **Computational cost of end-to-end Transformers**: Processing a full document page at native resolution with global self-attention is computationally prohibitive ($\mathcal{O}(HW)$ patches). Patch reduction or hierarchical attention is necessary.
6. **Language-specific challenges**: Scripts without clear word boundaries (Chinese, Japanese), right-to-left text (Arabic, Hebrew), and mixed-script documents pose additional complexity not handled by English-centric benchmarks.

22.6 Evolution

Early deep learning OCR: The introduction of CRNN+CTC (Shi et al., 2015) was the breakthrough. It unified feature extraction, sequence modeling, and transcription in a single differentiable pipeline, replacing the multi-stage heuristic approach and achieving state-of-the-art on scene text benchmarks with far less engineering.

ASTER (Shi et al., 2018): Combined TPS rectification with attention-based sequence-to-sequence recognition. The rectification network learns to normalize irregular text before recognition, dramatically improving accuracy on curved and perspective-distorted text. This established the "rectify then recognize" paradigm.

SATRN (Lee et al., 2020): Applied the Transformer's self-attention mechanism to the 2D feature map without squeezing it to a 1D sequence. This preserves vertical spatial information, enabling recognition of multi-line and vertically oriented text. A shallow CNN extracts features; the Transformer encoder applies 2D-aware positional encoding; the Transformer decoder attends to these features.

TrOCR (Li et al., 2021): Demonstrated that a pure Transformer (ViT encoder + autoregressive decoder) solves OCR without any OCR-specific inductive biases, given sufficient synthetic pre-training data. The simplicity of the approach and the scaling behavior mirror ViT's story in image classification.

Donut (Kim et al., 2022): Extended the end-to-end Transformer approach to document understanding—not just reading text, but parsing the visual structure of documents (tables, forms, key-value pairs). Input: image. Output: JSON with field-value pairs. No OCR intermediate; the model learns to read and structure end-to-end.

Nougat (Blecher et al., 2023): Specialized Donut-style architecture for academic PDF processing. Converts PDF page images to LaTeX/markdown markup, recovering mathematical notation, tables, and document structure. Trained on arXiv papers paired with their LaTeX source.

PaddleOCR (Baidu, 2020–present): A production-grade OCR system with modular design: detection (DBNet), recognition (CRNN or SVTR), layout analysis, and post-processing. Supports 80+ languages. The SVTR (Single Visual Transformer) recognition model applies lightweight local attention in a pyramid structure, achieving fast and accurate recognition for industrial deployment.

MGP-STR (2023): A multi-granularity prediction STR that fuses character-level, subword-level, and word-level predictions for improved recognition. Cross-attention over multiple ViT feature resolutions captures fine-grained character shapes.

Trend: OCR is converging toward fully Transformer-based, end-to-end architectures that handle detection, recognition, and document structure jointly. The scaling behavior (more data → better accuracy) and simplicity (no OCR-specific heuristics) favor the unified approach. However, for latency-critical and resource-constrained applications, CNN+CTC pipelines remain competitive. The frontier is multilingual, handwritten, and video OCR, where temporal consistency can improve frame-by-frame recognition.

Chapter 23

Pose Estimation

23.1 Motivation

Human pose estimation—the task of localizing anatomical keypoints (joints) in images—is a fundamental problem in computer vision with far-reaching applications: action recognition (identifying sports gestures, dance moves), motion capture and animation (driving virtual characters), healthcare (gait analysis, rehabilitation monitoring), human-computer interaction (gesture-based interfaces, sign language translation), sports analytics (biomechanics of athletic performance), and autonomous driving (predicting pedestrian intent). Unlike object detection, which returns coarse bounding boxes, pose estimation requires pixel-level localization of typically $K = 17$ skeletal keypoints (COCO format: nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles), with the additional complexity of person-person and person-scene occlusions, varying body proportions, clothing, and camera viewpoints. The problem has evolved from pictorial structures (hand-crafted part templates with spring-like spatial constraints) to deep heatmap regression (hourglass, HRNet) to the recent Transformer-based architectures (ViTPose) that leverage large-scale pre-training. Two paradigms coexist: top-down (detect person, then estimate pose within the crop) and bottom-up (detect all keypoints, then group them into individuals), trading accuracy for speed in crowd scenarios.

23.2 Mathematical Formulation

23.2.1 Problem Definition

Given an input image $\mathbf{I} \in \mathbb{R}^{H\times W\times 3}$, the goal is to predict the locations of K keypoints for all M persons present. Formally, the ground truth is:

$$\mathcal{Y} = \{(\mathbf{p}_k^m, v_k^m)\}_{k=1}^K \qquad \mathbf{p}_k^m = (x_k^m, y_k^m) \in \mathbb{R}^2,$$

where $m = 1, \ldots, M$ indexes persons and $v_k^m \in \{0, 1, 2\}$ is the visibility flag: 0 (not labeled), 1 (labeled but occluded), 2 (labeled and visible). In top-down methods, M is determined by a person detector; in bottom-up methods, M is inferred from detected keypoints.

23.2.2 Heatmap Regression

The dominant regression paradigm generates a spatial probability distribution (heatmap) for each keypoint rather than directly regressing coordinates. For keypoint k of a person, the ground-truth heatmap $\mathbf{H}_k^* \in \mathbb{R}^{H'\times W'}$ is created by placing an unnormalized 2D Gaussian centered at (x_k, y_k):

$$\mathbf{H}_k^*(i,j) = \exp\left(-\frac{(i-x_k')^2 + (j-y_k')^2}{2\sigma^2}\right), \qquad x_k' = y_k' = \frac{H'}{H}x_k,$$

where (H', W') are the output dimensions (typically $H' = H/4$, $W' = W/4$ for HRNet) and σ controls the spatial extent of the Gaussian (typically 2 pixels at the output resolution). The model predicts $\hat{\mathbf{H}}_k \in \mathbb{R}^{H' \times W'}$, and the loss is mean squared error:

$$\mathcal{L}_{\text{heatmap}} = \frac{1}{K} \sum_{k=1}^{K} \left\| \hat{\mathbf{H}}_k - \mathbf{H}_k^* \right\|_F^2.$$

At inference, the keypoint location is typically the argmax:

$$\hat{\mathbf{p}}_k = \arg\max_{(i,j)} \hat{\mathbf{H}}_k(i, j).$$

For sub-pixel precision, the expectation over the softmax-normalized heatmap (the "soft-argmax") can be used:

$$\hat{\mathbf{p}}_k = \sum_{i=1}^{H'} \sum_{j=1}^{W'} \mathbf{P}_k(i, j) \cdot \begin{pmatrix} i \\ j \end{pmatrix}, \quad \mathbf{P}_k = \text{softmax}(\hat{\mathbf{H}}_k \cdot \beta),$$

where β is a temperature parameter controlling the softness. This is fully differentiable and has been shown to improve coordinate accuracy.

23.2.3 Why Heatmaps Outperform Direct Coordinate Regression

Direct coordinate regression $\hat{\mathbf{p}} = f_\theta(\mathbf{I})$ maps high-dimensional input to $2K$ real numbers. This is an ill-posed inverse problem: multiple drastically different body configurations can project to nearly identical appearance when limbs are foreshortened or occluded. The network must learn a highly nonlinear, discontinuous function—joints jump discontinuously between adjacent frames when switching occlusion states. Heatmaps, by contrast, represent uncertainty spatially: each output pixel encodes the likelihood of a joint being at that location, rendering the mapping locally smooth. Occluded joints produce low-confidence, broadly distributed heatmaps rather than wrong pinpoint coordinates. The heatmap represents a posterior distribution $P(\mathbf{p}_k \mid \mathbf{I})$ approximated by a spatial grid, and the model learns to reduce variance by placing mass at the correct location.

23.2.4 Stacked Hourglass Networks

The Stacked Hourglass (Newell et al., 2016) uses a symmetric encoder-decoder topology repeated N times (typically $N = 8$), with each hourglass producing intermediate heatmap predictions that serve as input to the next hourglass. The hourglass module processes features at multiple scales:

Single hourglass: The input feature map is processed through a series of residual blocks. At each downsampling step (max-pooling by 2), a branch preserves the original resolution features. After reaching the lowest resolution (4×4 for typical 256×256 input), upsampling (nearest neighbor) with skip connections restores the full resolution. At each scale, the skip connection is concatenated with the upsampled features, and a 1×1 convolution matches channels. The output of the hourglass is a set of K heatmaps.

Intermediate supervision: Each hourglass output is supervised with the ground-truth heatmaps, and the predicted heatmaps are remapped to the feature dimension via

another 1×1 convolution and added to the features of the next hourglass (along with the original input features). The total loss is:

$$\mathcal{L}_{\text{HG}} = \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}_{\text{heatmap}}^{(n)}.$$

Intermediate supervision combats gradient vanishing in deep recurrent architectures and enables each hourglass to refine previous predictions.

23.2.5 HRNet: High-Resolution Representation Learning

HRNet (Sun et al., 2019) challenges the encoder-decoder paradigm by maintaining a high-resolution stream throughout the entire network, rather than recovering it from a low-resolution bottleneck. The architecture maintains S parallel sub-networks of decreasing resolution (typically $S = 4$), with repeated multi-scale fusion:

Resolution streams: Let $\mathbf{F}_s^r$ denote the feature map at resolution stream $r \in \{1, \ldots, S\}$ after stage s. Resolution r has dimensions $(H/2^{r-1}, W/2^{r-1})$. A new low-resolution stream is added at each stage by strided convolutions from the highest-resolution stream.

Multi-scale fusion: After each convolutional block, all streams exchange information. For a target resolution r, features from all source resolutions r' are transformed and aggregated:

$$\mathbf{F}_s^r \leftarrow \mathbf{F}_s^r + \sum_{r' \neq r} \mathcal{T}_{r' \to r}(\mathbf{F}_s^{r'}),$$

where the transformation $\mathcal{T}_{r' \to r}$ is:

- Strided 3×3 convolution (for $r' < r$, downsampling): halving resolution repeatedly until matching.
- Bilinear upsampling + 1×1 convolution (for $r' > r$, upsampling): doubling resolution repeatedly.

The output of HRNet is the feature map from the highest-resolution stream, from which K heatmaps are predicted via a 1×1 convolution. The key insight is that HRNet never loses high-resolution spatial information: features at all scales are enriched by each other, but the finest-resolution representation persists intact.

23.2.6 OpenPose: Part Affinity Fields

OpenPose (Cao et al., 2017) is a bottom-up method that detects all keypoints for all persons simultaneously and then assembles them into individual skeletons. The model predicts two types of output:

Part Confidence Maps (PCMs): For each keypoint k, a set of heatmaps $\mathbf{S}^k$ where the value at each pixel is the maximum over all persons of the Gaussian response:

$$\mathbf{S}_k^*(p) = \max_m \exp\left(-\frac{\|p - \mathbf{p}_k^m\|^2}{\sigma^2} \right).$$

The max operator (not sum) ensures that overlapping persons each produce a distinct peak.

Part Affinity Fields (PAFs): For each limb c connecting keypoints k_1 and k_2, a 2D vector field $\mathbf{L}_c^* \in \mathbb{R}^{2\times H'\times W'}$ encodes the direction of the limb. At a point p that lies on the limb (within a distance threshold), the vector is the unit direction:

$$\mathbf{L}_c^*(p) = \begin{cases} \mathbf{v}/\left\|\mathbf{v}\right\| & \text{if } p \text{ lies on limb } c, \\ \mathbf{0} & \text{otherwise,} \end{cases}$$

where $\mathbf{v} = \mathbf{p}_{k_2}^m - \mathbf{p}_{k_1}^m$. For points near but not exactly on the limb, the vector is set to zero.

The loss combines PCM and PAF predictions:

$$\mathcal{L} = \sum_{k=1}^{K}\sum_{p}\left\|\hat{\mathbf{S}}_k(p) - \mathbf{S}_k^*(p)\right\|_2^2 + \sum_{c=1}^{C}\sum_{p}\left\|\hat{\mathbf{L}}_c(p) - \mathbf{L}_c^*(p)\right\|_2^2,$$

where the PAF loss is weighted spatially—points on limbs have higher weight because the non-limb regions would dominate otherwise.

Greedy bipartite matching for association: Given all detected keypoint candidates (local maxima in PCMs above a confidence threshold), the goal is to connect them into person skeletons. For a limb c connecting keypoints k_1 and k_2, and candidate keypoints $\mathbf{d}_{k_1}$ and $\mathbf{d}_{k_2}$ at positions p_{k_1} and p_{k_2}, the association score is the line integral of the PAF along the segment:

$$E = \int_{u=0}^{1} \hat{\mathbf{L}}_c\big(p(u)\big) \cdot \frac{p_{k_2} - p_{k_1}}{\|p_{k_2} - p_{k_1}\|}\, du, \quad p(u) = (1-u)p_{k_1} + up_{k_2}.$$

The integral is approximated by sampling N equidistant points along the segment. This score measures how well the predicted limb direction field aligns with the candidate connection. Association is formulated as a K-partite graph matching problem, solved greedily for each limb independently using maximum-weight bipartite matching. Candidate connections are selected if their PAF score exceeds a threshold.

23.2.7 ViTPose: Vision Transformer for Pose Estimation

ViTPose (Xu et al., 2022) applies a plain ViT to pose estimation with a simple decoder. The image $\mathbf{x}$ is partitioned into patches, passed through a ViT encoder (pre-trained with MAE), and the output token sequence is reshaped and upsampled to produce heatmaps:

$$\mathbf{z} = \text{ViT}(\text{PatchEmbed}(\mathbf{x})) \in \mathbb{R}^{N\times D},$$
$$\hat{\mathbf{H}} = \text{Decoder}(\text{Reshape}(\mathbf{z})) \in \mathbb{R}^{K\times H'\times W'},$$

where the decoder is a single deconvolution layer or a lightweight FPN. The simplicity contrasts with the complex architectures (Hourglass, HRNet) and yet achieves state-of-the-art when the ViT backbone is sufficiently large (ViT-H/14, ViT-G/14). The key is the backbone's representational capacity from large-scale pre-training (MAE on ImageNet), which generalizes to dense prediction without task-specific architectural innovations.

23.2.8 Object Keypoint Similarity (OKS)

The standard COCO metric is the average precision (AP) based on OKS, the keypoint analog of IoU for bounding boxes. For ground-truth person m and predicted person n:

$$\mathrm{OKS}(m,n) = \frac{\sum_{k=1}^{K} \exp\!\left(-\frac{d_k^{mn}}{2s_m^2\kappa_k^2}\right)\delta(v_k^m > 0)}{\sum_{k=1}^{K}\delta(v_k^m > 0)},$$

where $d_k^{mn} = \|\hat{\mathbf{p}}_k^n - \mathbf{p}_k^m\|_2$ is the Euclidean distance, $s_m = \sqrt{w_m h_m}$ is the scale of person m (square root of detection bounding box area), and κ_k is a per-keypoint constant reflecting annotation difficulty (e.g., $\kappa_{\text{hip}} < \kappa_{\text{wrist}}$ because hips are more consistently localized). The exponential weighting produces a soft match: close predictions $\to$ OKS $\to 1$, far predictions $\to$ OKS $\to 0$. AP is reported at OKS thresholds: AP^{50} (OKS > 0.50), AP^{75} (OKS > 0.75), and the primary metric AP is averaged over 10 thresholds $[0.50, 0.55, \ldots, 0.95]$.

23.2.9 Associative Embedding

An alternative to PAFs for bottom-up association: along with heatmaps, the network outputs a D-dimensional embedding vector (a "tag") for each detected keypoint. The loss encourages tags of keypoints belonging to the same person to be close, and tags from different persons to be far apart:

$$\mathcal{L}_{\text{AE}} = \frac{1}{M}\sum_{m}\frac{1}{K}\sum_{k}\|\mathbf{e}_k^m - \bar{\mathbf{e}}^m\|_2^2 + \frac{1}{M^2}\sum_{m}\sum_{n\neq m}\exp\left(-\frac{\|\bar{\mathbf{e}}^m - \bar{\mathbf{e}}^n\|_2^2}{2\sigma^2}\right),$$

where $\bar{\mathbf{e}}^m = \frac{1}{K}\sum_k \mathbf{e}_k^m$ is the mean tag for person m. The first term pulls within-person tags together (variance within group); the second term pushes between-person means apart (separation between groups). At test time, keypoints are clustered by their tag similarity (e.g., greedy grouping or HDBSCAN), forming person instances.

23.3 Algorithm

Algorithm 28: HRNet for Pose Estimation (Top-Down)

Input: Image $\mathbf{I}$, HRNet model, K keypoints
Output: Keypoint heatmaps $\hat{\mathbf{H}}_k$, person instances

$\mathbf{F} \leftarrow \text{HRNet_Stem}(\mathbf{I})$ (2 strided convs, $H/4 \times W/4$);
for *stage* $s \leftarrow 1$ **to** 4 **do**
  Add new lower-resolution stream (if $s \geq 2$) via strided conv;
  Apply residual blocks to each resolution stream;
  for *each target resolution* r **do**
    for *each source resolution* $r' \neq r$ **do**
      $\mathbf{F}_r \leftarrow \mathbf{F}_r + \mathcal{T}_{r' \to r}(\mathbf{F}_{r'})$ (multi-scale fusion);
    end
  end
end
$\hat{\mathbf{H}} \leftarrow \text{Conv}_{1\times1}(\mathbf{F}_{\text{high-res}}) \in \mathbb{R}^{K \times H/4 \times W/4}$;
for *training* **do**
  $\mathcal{L} \leftarrow \frac{1}{K} \sum_k \left\| \hat{\mathbf{H}}_k - \mathbf{H}_k^* \right\|_F^2$;
end
for *inference (top-down)* **do**
  Detect persons $\rightarrow$ bounding boxes $\rightarrow$ crop $\mathbf{I}_b$;
  $\hat{\mathbf{H}}^b \leftarrow \text{HRNet}(\mathbf{I}_b)$;
  $\hat{\mathbf{p}}_k \leftarrow \arg\max_{(i,j)} \hat{\mathbf{H}}_k^b(i,j)$ for each k;
  Transform back to original image coordinates;
end

23.4 Architecture

23.4.1 Top-Down Pipeline

1. **Person detection**: A standard object detector (Faster R-CNN, YOLO) detects all persons, producing bounding boxes with high recall (low confidence threshold, e.g., 0.3).

2. **Single-person pose estimation**: Each detection crop is resized to a fixed size (e.g., 256×192 or 384×288) and processed by the pose model. The predicted heatmaps are resized to the original crop size and adjusted to full-image coordinates. The OKS metric is computed using the detection bounding box scale s.

3. **Box refinement (optional)**: The initial detection box can be refined by fitting it to the predicted keypoints, cropping a tighter region, and re-running pose estimation (two-stage refinement).

23.4.2 Bottom-Up Pipeline

1. **Keypoint detection**: The network processes the full image and outputs PCMs (per-keypoint heatmaps) and either PAFs or associative embeddings.
2. **Peak extraction**: Local maxima in each PCM above a threshold form candidate keypoints.
3. **Grouping**: For PAF-based: greedy bipartite matching per limb. For associative embeddings: clustering by tag similarity.
4. **Person assembly**: Starting from a root keypoint (e.g., nose), iteratively connect to neighboring keypoints via the limb graph (a tree-structured kinematic skeleton).

23.4.3 Key Model Architectures

- **CPM (Convolutional Pose Machines, Wei et al., 2016)**: Sequential CNNs with intermediate supervision. Each stage receives the image features and the heatmap predictions from the previous stage, enabling iterative refinement. Pioneered the multi-stage heatmap prediction paradigm.
- **Stacked Hourglass**: Symmetric UNet-like design, $N = 8$ hourglasses stacked, residual blocks throughout. 256×256 input, 64×64 heatmaps. Strong performance but computationally heavy.
- **HRNet-W32/W48**: Width variants with 32 and 48 channels in the highest-resolution stream (channels double at each lower resolution). HRNet-W48 achieves state-of-the-art with moderate parameters ($\sim$63M). Input 384×288, output 96×72 heatmaps.
- **ViTPose-B/L/H**: ViT backbones with a single deconvolution decoder. ViTPose-H ($\sim$632M) with MAE pre-training achieves top COCO AP without bells and whistles. Input 256×192 for Base/Large, 384×288 for Huge.
- **DWPose**: A distilled version of ViTPose using a lightweight backbone (e.g., MobileNetV3) trained to mimic the ViTPose teacher's heatmaps, achieving real-time performance with minimal accuracy drop.

23.5 Pros and Cons

Advantages (Top-down):

1. **Superior accuracy**: Top-down methods consistently outperform bottom-up on standard benchmarks (COCO, MPII) because the person crop normalizes scale and reduces the search space from M persons to 1 person, simplifying the learning problem.
2. **Scale normalization**: Cropping and resizing to a fixed input size removes scale variation, which is especially beneficial for small persons.
3. **Modular**: Any person detector can be paired with any pose estimator. Improvements in detection (e.g., stronger backbones) directly boost pose accuracy.

4. **Heatmap quality**: Single-person heatmaps are cleaner and more localized than multi-person heatmaps, leading to higher OKS precision.

Advantages (Bottom-up):

1. **Inference speed independent of person count**: Bottom-up methods process the image once regardless of the number of persons. Top-down runs the pose model M times, making it slow for crowded scenes (e.g., $M > 30$ in a concert photo).

2. **No detection dependency**: Errors from the person detector (missed persons, duplicate detections) do not propagate to pose estimation.

3. **Natural handling of truncation**: At image boundaries, some keypoints of a person may be cropped; bottom-up methods simply fail to detect those keypoints without affecting the rest.

Common Disadvantages:

1. **Occlusion**: Severe occlusion (multiple persons overlapping, objects blocking limbs) remains challenging. Heatmaps become multi-modal or flat, and limb association becomes ambiguous.

2. **Rare poses**: Unusual poses (gymnastics, yoga) or viewpoints (top-down, from below) are underrepresented in training data and poorly estimated.

3. **Ambiguous left/right**: Symmetric body parts (left vs. right hand) are often confused, especially when appearance alone is insufficient and context is lost in cropped regions.

4. **Annotation cost**: Accurate keypoint annotation requires trained annotators and is expensive. At COCO scale, 250K person instances were annotated; scaling beyond this is bottlenecked by annotation cost, driving interest in semi-supervised and self-supervised methods.

5. **3D ambiguity**: 2D pose estimation inherently struggles with depth ambiguity. Lifting 2D poses to 3D (a separate task) often relies on strong priors about bone lengths and plausible joint angles.

6. **Real-time constraints**: For video applications (30+ FPS), even efficient top-down pipelines (detection + pose) require careful engineering (model distillation, quantization, framework optimization).

23.6 Evolution

CPM (Convolutional Pose Machines, 2016): Established the sequential refinement paradigm: stage-wise CNNs where each stage takes the image and previous-stage heatmaps, enabling multi-scale, progressive localization. Intermediate supervision at each stage prevents gradient vanishing.

Stacked Hourglass (2016): Unified multi-scale processing and repeated bottom-up/top-down inference in a symmetric architecture with skip connections. $N = 8$ hourglasses stacked transformed the field, dominating benchmarks for several years. Demonstrated that repeated refinement with intermediate supervision is crucial for fine localization.

OpenPose (2017): Popularized real-time multi-person pose estimation via PAFs. The two-branch network (PCMs + PAFs) and greedy matching enabled bottom-up pose estimation at interactive frame rates. OpenPose became the standard tool for research and applications, with open-source code and a large community.

HRNet (2019): The pivotal architectural innovation: maintaining high-resolution representations throughout (no bottleneck) while exchanging multi-scale information via repeated fusion. HRNet dramatically improved accuracy, especially for small-scale keypoints, and became the go-to backbone for pose estimation and other dense prediction tasks (semantic segmentation, facial landmark detection).

ViTPose (2022): Demonstrated that a plain ViT with a trivial decoder matches or exceeds HRNet, provided the ViT is pre-trained at scale (MAE). This shifted focus from architecture engineering to pre-training strategies. ViTPose-H (ViT-Huge + MAE) set new state-of-the-art, validating the "larger backbone, simpler head" philosophy.

DWPose (2023): Distilled ViTPose to lightweight backbones, achieving real-time performance (30+ FPS on COCO) with minimal accuracy degradation. Two-stage distillation: whole-body heatmap distillation followed by visible keypoint fine-tuning. Enables deployment on edge devices.

TokenPose (2021): Introduced token-based representation for keypoints. Each keypoint is assigned a learnable token; these tokens interact with visual tokens via Transformer layers, directly outputting heatmaps or coordinates. The token representation naturally handles keypoint dependencies and occlusion reasoning.

SimCC (2022): A simple coordinate classification approach: discretizes the X and Y axes into bins and classifies each keypoint into horizontal and vertical bins separately. A lightweight MLP head replaces heatmap decoding. Achieves competitive accuracy with far fewer output parameters than heatmaps (no spatial upsampling).

Pose Estimation in the Wild (3D, multi-view): Extending 2D pose to 3D has seen the emergence of transformer-based lifting (PoseFormer), multi-view triangulation with epipolar constraints (EpipolarPose), and implicit neural representations (Neural Body) that directly model the 3D human shape from sparse views.

Trend: The field is moving from hand-crafted multi-scale architectures (Hourglass, HRNet) to general-purpose Transformers (ViTPose) with scale-driven improvements. The focus in bottom-up methods is shifting from PAF-based matching to associative embeddings and learned grouping (End-to-End Multi-Person Pose Estimation with Transformers, e.g., PETR). The frontier is full-body estimation (body + hands + face, 133 keypoints), temporal consistency in video, and 3D lifting from monocular images, with self-supervised pre-training (MAE, DINO) increasingly driving progress.

Chapter 24

Object Tracking

24.1 Motivation

Object tracking is the task of maintaining the identity of objects across consecutive frames in a video sequence. Unlike object detection, which processes each frame independently without establishing temporal correspondence, tracking must solve the **data association** problem: linking detections of the same object across time. The core challenge lies in the ambiguity introduced by occlusions, appearance changes, scale variation, fast motion, illumination shifts, and interactions among multiple objects.

The economic and societal impact of robust visual tracking spans surveillance and security (monitoring suspicious behavior in crowded scenes), autonomous driving (tracking pedestrians, cyclists, and vehicles to predict future trajectories), sports analytics (player tracking for performance analysis and tactical evaluation), wildlife monitoring (tracking individual animals for behavioral ecology studies), and human-computer interaction (gesture tracking for augmented and virtual reality). Crucially, tracking provides the temporal continuity that transforms per-frame perception into spatio-temporal understanding.

Formally, the **Single Object Tracking** (SOT) setting assumes that the target is identified in the first frame by a bounding box b_1, and the tracker must localize the same object in all subsequent frames $t = 2, \dots, T$. No re-identification module is required since identity is fixed. In contrast, **Multiple Object Tracking** (MOT) must discover, track, and maintain identities for all objects of interest in a video, requiring both a detector to propose candidate locations and an association mechanism to link detections into consistent trajectories. This chapter develops the mathematical foundations of both paradigms, culminating in a unified treatment of modern deep tracking architectures.

24.2 Mathematical Formulation

24.2.1 Problem Definition

Let a video be a sequence of frames $\{I_t\}_{t=1}^{T}$ indexed by time. For SOT, given an initial axis-aligned bounding box $b_1 = (x_1, y_1, w_1, h_1)$ defining the target extent in I_1, we seek to estimate b_t for all $t = 2, \dots, T$. The tracker must be *causal* (online): b_t may depend only on $\{I_s, b_s\}_{s=1}^{t}$.

For MOT, let $\mathcal{D}_t = \{d_{t,1}, \dots, d_{t,N_t}\}$ denote the set of detections at frame t, each $d_{t,i} = (x, y, w, h, s_{t,i})$ where $s_{t,i}$ is a detection confidence score. A trajectory τ_k is an ordered sequence of detections associated with object k: $\tau_k = (\dots, d_{t,k}, d_{t+1,k}, \dots)$. The goal is to partition $\bigcup_t \mathcal{D}_t$ into trajectories while respecting temporal ordering and non-overlap constraints (each detection assigned to at most one trajectory).

24.2.2 Single Object Tracking: Siamese Formulation

The Siamese tracking paradigm, pioneered by SiamFC [**?**], casts tracking as similarity learning. Let z denote the *exemplar* image (a crop centered on the target in the first frame) and x denote the *search* image (a larger crop in the current frame centered on the previous location). A fully convolutional Siamese network $\varphi(\cdot)$ extracts features from both inputs. The response map is computed via cross-correlation:

$$f(z, x; \theta) = \varphi(z) \star \varphi(x) + b \cdot \mathbf{1} \tag{24.1}$$

where $\star$ denotes the cross-correlation operator and $b \in \mathbb{R}$ is a learned scalar bias. The output $f(z, x) \in \mathbb{R}^{H \times W}$ is a score map whose maximum indicates the target location. At inference time, the target position is estimated as:

$$\hat{p}_t = \arg\max_{(i,j)} f(z, x_t)_{ij} \tag{24.2}$$

and the bounding box is obtained by mapping the peak location back to image coordinates through the stride of the network.

Training. Training pairs (z, x, y) are constructed from video sequences where z and x are extracted from frames separated by up to $T_{\max}$ frames apart. The ground-truth label map $y \in \{+1, -1\}^{H \times W}$ has $y[u] = +1$ for positions within radius R of the true target center, and -1 otherwise (Gaussian-weighted labels are also common). The per-position logistic loss is:

$$\ell(y, v) = \log\left(1 + \exp(-yv)\right) \tag{24.3}$$

and the total training loss is the mean over all positions in the score map:

$$\mathcal{L}_{\text{SiamFC}}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{(z,x,y) \in \mathcal{D}} \frac{1}{HW} \sum_{u \in \mathbb{Z}_2} \ell(y[u], f(z, x)[u]) \tag{24.4}$$

Optimization uses SGD with momentum. Training data is generated from large-scale video detection datasets (ILSVRC VID, YouTube-BB) by sampling random frame pairs and extracting exemplar and search crops at the appropriate locations and scales.

SiamRPN extension. SiamRPN [**?**] augments the Siamese backbone with a Region Proposal Network (RPN) composed of two branches. The classification branch produces $2k$ scores per anchor (k anchors, each classified as foreground or background), and the regression branch produces $4k$ coordinate refinements per anchor. Let $A_{w \times h \times k}$ denote the set of anchor boxes densely tiled over the response map. The RPN output is:

$$\mathcal{A}^{\text{cls}}_{w \times h \times 2k} = [\varphi(z)]_{\text{cls}} \star [\varphi(x)]_{\text{cls}} \tag{24.5}$$

$$\mathcal{A}^{\text{reg}}_{w \times h \times 4k} = [\varphi(z)]_{\text{reg}} \star [\varphi(x)]_{\text{reg}} \tag{24.6}$$

The anchor with the highest classification score is selected, and its coordinates are refined by the regression branch. Scale and aspect ratio robustness are achieved through the anchor design, eliminating the need for exhaustive multi-scale search.

Deeper backbones and modern variants. SiamRPN++ [**?**] addresses the challenge of using deep backbones (ResNet-50) in Siamese trackers. The key innovations are: (i) spatial-aware sampling to break the translation invariance of deep CNNs, (ii) depth-wise cross-correlation to reduce parameters while preserving channel-wise information,

and (iii) multi-layer feature aggregation, where feature maps from conv3, conv4, and conv5 are each processed by an RPN head and the outputs are weighted and summed:

$$\mathcal{A}_{\text{agg}} = \sum_{l \in \{3,4,5\}} \alpha_l \cdot \mathcal{A}_l \tag{24.7}$$

with learnable or fixed fusion weights α_l.

Transformer-based trackers such as OSTrack [**?**] and MixFormer [**?**] reformulate tracking as a template-search feature matching problem via attention. The template tokens $\mathbf{z} \in \mathbb{R}^{N_z \times D}$ and search tokens $\mathbf{x} \in \mathbb{R}^{N_x \times D}$ are concatenated and processed by a Transformer encoder with self-attention and cross-attention. The one-stream (early fusion) paradigm jointly processes both template and search, while the two-stream paradigm processes them separately and fuses later. The score map is derived from the attention weights or a dedicated prediction head.

24.2.3 Multiple Object Tracking: Filtering and Association

Kalman Filter for Motion Prediction

The Kalman filter [**?**] provides an optimal recursive estimator for linear dynamical systems with Gaussian noise. In tracking, it models the motion of each object as a constant-velocity process and predicts its location in subsequent frames.

State representation. In SORT [**?**], the state vector is:

$$\mathbf{x} = [u, v, s, r, \dot{u}, \dot{v}, \dot{s}]^\top \in \mathbb{R}^7 \tag{24.8}$$

where (u, v) is the center of the bounding box, s is the scale (area), $r = h/w$ is the aspect ratio (assumed constant, so $\dot{r}$ is excluded), and $\dot{u}, \dot{v}, \dot{s}$ are the respective velocities. The measurement vector is $\mathbf{z} = [u, v, s, r]^\top \in \mathbb{R}^4$, representing the observed detection values.

Prediction step. The state and covariance evolve according to a linear constant-velocity model:

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}\hat{\mathbf{x}}_{t-1} \tag{24.9}$$

$$\mathbf{P}_{t|t-1} = \mathbf{F}\mathbf{P}_{t-1}\mathbf{F}^\top + \mathbf{Q}_t \tag{24.10}$$

where the state transition matrix $\mathbf{F} \in \mathbb{R}^{7\times7}$ encodes the motion model:

$$\mathbf{F} = \begin{bmatrix} \mathbf{I}_4 & \Delta t \cdot \mathbf{I}_{4\times3} \\ \mathbf{0}_{3\times4} & \mathbf{I}_3 \end{bmatrix} \tag{24.11}$$

with Δt being the time step between frames. The process noise covariance $\mathbf{Q}_t \in \mathbb{R}^{7\times7}$ quantifies the uncertainty in the motion model, modeled as piecewise white noise:

$$\mathbf{Q}_t = \sigma_q^2 \begin{bmatrix} \frac{\Delta t^4}{4}\mathbf{I}_{4\times3} & \frac{\Delta t^3}{2}\mathbf{I}_{4\times3} \\ \frac{\Delta t^3}{2}\mathbf{I}_{4\times3} & \Delta t^2\mathbf{I}_3 \end{bmatrix} \tag{24.12}$$

where σ_q^2 is the process noise intensity.

Update step. Given a measurement $\mathbf{z}_t$, the innovation (measurement residual) is:

$$\tilde{\mathbf{y}}_t = \mathbf{z}_t - \mathbf{H}\hat{\mathbf{x}}_{t|t-1} \tag{24.13}$$

with observation matrix $\mathbf{H} = [\mathbf{I}_4, \mathbf{0}_{4\times 3}] \in \mathbb{R}^{4\times 7}$ extracting the position components. The innovation covariance is:

$$\mathbf{S}_t = \mathbf{H}\mathbf{P}_{t|t-1}\mathbf{H}^\top + \mathbf{R}_t \tag{24.14}$$

where $\mathbf{R}_t = \mathrm{diag}(\sigma_u^2, \sigma_v^2, \sigma_s^2, \sigma_r^2)$ is the measurement noise covariance. The optimal Kalman gain is:

$$\mathbf{K}_t = \mathbf{P}_{t|t-1}\mathbf{H}^\top \mathbf{S}_t^{-1} \tag{24.15}$$

which minimizes the trace of the posterior covariance. Finally, the state and covariance are updated:

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t \tilde{\mathbf{y}}_t \tag{24.16}$$

$$\mathbf{P}_t = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\mathbf{P}_{t|t-1} \tag{24.17}$$

The posterior covariance $\mathbf{P}_t$ is in Joseph form for numerical stability. The predicted bounding box in image coordinates is obtained from $\hat{\mathbf{x}}_{t|t-1}$ by extracting (u, v, s, r) and converting s to width and height via $w = \sqrt{s \cdot r}$ and $h = \sqrt{s/r}$.

Hungarian Algorithm for Data Association

Given the set of M tracked objects (with Kalman predictions) and N detections, we construct a cost matrix $\mathbf{C} \in \mathbb{R}^{M\times N}$ where c_{ij} encodes the dissimilarity between track i and detection j. The association problem is a minimum-weight bipartite matching:

$$\min_{x_{ij}\in\{0,1\}} \sum_{i=1}^{M}\sum_{j=1}^{N} c_{ij}x_{ij} \quad \text{s.t.} \quad \sum_i x_{ij} \le 1, \;\; \sum_j x_{ij} \le 1 \tag{24.18}$$

In SORT, the cost is simply $c_{ij} = 1 - \mathrm{IoU}(b_i^{\mathrm{pred}}, b_j^{\mathrm{det}})$, the intersection-over-union between the Kalman-predicted bounding box of track i and detection j. The Hungarian algorithm [**?**] solves this assignment problem in $\mathcal{O}(n^3)$ time. Unmatched tracks increment an age counter and are deleted after $A_{\max}$ consecutive misses. Unmatched detections with confidence above a threshold initialize new tentative tracks, which are promoted to confirmed status after $M_{\min}$ consecutive detections.

DeepSORT extension. DeepSORT [**?**] augments the motion-based association with appearance information. A convolutional neural network $\psi(\cdot)$ trained for person re-identification extracts an appearance feature vector $\psi(d_j) \in \mathbb{R}^D$ for each detection (an ℓ_2-normalized embedding). Each track maintains a gallery $\mathcal{G}_i = \{\mathbf{e}_i^{(1)}, \ldots, \mathbf{e}_i^{(L)}\}$ of its last $L = 100$ appearance descriptors.

The Mahalanobis distance provides a probabilistic measure of motion compatibility:

$$d^{(1)}(i, j) = (\mathbf{z}_j - \mathbf{H}\hat{\mathbf{x}}_i)^\top \mathbf{S}_i^{-1} (\mathbf{z}_j - \mathbf{H}\hat{\mathbf{x}}_i) \tag{24.19}$$

where $\mathbf{S}_i$ is the innovation covariance of track i. Measurements with $d^{(1)}(i, j) > \chi^2_\alpha(4)$ are gated out (excluded from association) at confidence level α.

The appearance distance is the minimum cosine distance between the detection embedding and the track gallery:

$$d^{(2)}(i,j) = \min_{\mathbf{e}\in\mathcal{G}_i} \left\{ 1 - \frac{\psi(d_j)^\top \mathbf{e}}{\|\psi(d_j)\| \cdot \|\mathbf{e}\|} \right\} \tag{24.20}$$

The combined association cost is a convex combination:

$$c_{ij} = \lambda\, d^{(1)}(i,j) + (1-\lambda)\, d^{(2)}(i,j) \tag{24.21}$$

with $\lambda \in [0,1]$ typically set to 0.98 (strong reliance on motion, with appearance breaking ties). DeepSORT introduces **cascade matching**, where tracks are matched in order of decreasing age (newer tracks matched first) to prioritize frequently seen objects.

24.2.4 Tracking Metrics

Multiple Object Tracking Accuracy (MOTA) [**?**] is the primary metric:

$$\mathrm{MOTA} = 1 - \frac{\sum_t (\mathrm{FN}_t + \mathrm{FP}_t + \mathrm{IDSW}_t)}{\sum_t \mathrm{GT}_t} \tag{24.22}$$

where FN_t, FP_t, IDSW_t are false negatives, false positives, and identity switches at frame t, and GT_t is the number of ground-truth objects. MOTA can be negative if errors exceed ground-truth count.

Multiple Object Tracking Precision (MOTP) measures localization accuracy:

$$\mathrm{MOTP} = \frac{\sum_t \sum_{i\in\mathcal{M}_t} \mathrm{IoU}(b_i^{\mathrm{pred}}, b_i^{\mathrm{gt}})}{|\mathcal{M}|} \tag{24.23}$$

where $\mathcal{M}_t$ is the set of matched pairs at frame t. IDF1 [**?**] evaluates identity preservation as the F1 score of the optimal assignment between predicted and ground-truth trajectories (ratio of correctly identified detections to average of total predicted and total ground-truth detections). The Higher Order Tracking Accuracy (HOTA) [**?**] decomposes tracking performance into detection accuracy and association accuracy:

$$\mathrm{HOTA} = \sqrt{\mathrm{DetA} \cdot \mathrm{AssA}} \tag{24.24}$$

where DetA measures detection quality and AssA measures temporal association quality across multiple localization thresholds, providing a more balanced evaluation than MOTA.

24.3 Algorithms

24.4 Architecture

SiamFC architecture. The Siamese backbone φ in the original SiamFC is a fully convolutional network with the following structure: Conv1 (11×11, stride 2, 96 filters) → max-pool (3×3, stride 2) → Conv2 (5×5, 256 filters) → max-pool (3×3, stride 2) → Conv3 (3×3, 384 filters) → Conv4 (3×3, 384 filters) → Conv5 (3×3, 256 filters). No padding is used, ensuring fully convolutional behavior. The template branch and search branch share weights by construction (Siamese). The cross-correlation is computed as a convolution where f_z serves as the kernel over f_x, producing the score map of spatial size 17×17 for a 255×255 search image with stride 8.

Algorithm 29: SORT: Simple Online Realtime Tracking

Input : Video frames $\{I_t\}_{t=1}^{T}$, object detector $\mathcal{F}_{\text{det}}$
Output: Trajectories $\mathcal{T} = \{\tau_1, \dots, \tau_K\}$

Initialize track set $\mathcal{T} \leftarrow \emptyset$;
for $t = 1$ **to** T **do**
  $\mathcal{D}_t \leftarrow \mathcal{F}_{\text{det}}(I_t)$ `// detect objects`
  foreach *track* $\tau_k \in \mathcal{T}$ **do**
    Predict state: $\hat{\mathbf{x}}_{t|t-1}^{(k)} \leftarrow \mathbf{F}\hat{\mathbf{x}}_{t-1}^{(k)}$;
    Predict covariance: $\mathbf{P}_{t|t-1}^{(k)} \leftarrow \mathbf{F}\mathbf{P}_{t-1}^{(k)}\mathbf{F}^{\top} + \mathbf{Q}$;
  Construct cost matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{T}| \times |\mathcal{D}_t|}$ with $c_{ij} = 1 - \text{IoU}(b_i^{\text{pred}}, b_j^{\text{det}})$;
  $\mathcal{M} \leftarrow \text{Hungarian}(\mathbf{C})$ `// optimal assignment`
  foreach *matched pair* $(i, j) \in \mathcal{M}$ **do**
    Update track i with detection j via Kalman update;
    $\mathcal{G}_i \leftarrow \mathcal{G}_i \cup \{\psi(d_{t,j})\}$;
  foreach *unmatched track* i **do**
    Increment age a_i;
    if $a_i > A_{\max}$ **then**
      Remove track i
  foreach *unmatched detection* j *with* $s_{t,j} > \tau_{init}$ **do**
    Initialize new track: $\hat{\mathbf{x}} \leftarrow [u, v, s, r, 0, 0, 0]^{\top}$, $\mathbf{P} \leftarrow \mathbf{P}_0$;
return $\mathcal{T}$

Algorithm 30: SiamFC: Siamese Fully Convolutional Tracker

Input : Video frames $\{I_t\}_{t=1}^{T}$, initial box $b_1 = (x_1, y_1, w_1, h_1)$, network φ, stride s
Output: Bounding boxes $\{b_t\}_{t=2}^{T}$

Extract exemplar z from I_1 at b_1, resize to 127×127;
Compute template feature: $f_z \leftarrow \varphi(z)$;
Initialize position $p_1 \leftarrow (x_1 + w_1/2, y_1 + h_1/2)$, scale $S_1 \leftarrow \sqrt{w_1 h_1}$;
for $t = 2$ **to** T **do**
  Crop search region at p_{t-1} with size $255 \times 255 \times S_{t-1}$, resize to 255×255: x;
  Compute search feature: $f_x \leftarrow \varphi(x)$;
  Score map: $R \leftarrow f_z \star f_x + b \cdot \mathbf{1}$;
  Find peak: $(i^*, j^*) \leftarrow \arg\max_{i,j} R_{ij}$;
  Map to image coordinates: displacement $\Delta \leftarrow s \cdot ((i^*, j^*) - (H/2, W/2))$;
  Update position: $p_t \leftarrow p_{t-1} + \Delta$;
  Update scale via multi-scale search (optional);
  $b_t \leftarrow$ box centered at p_t with size S_t;
return $\{b_t\}_{t=2}^{T}$

SORT/DeepSORT pipeline. The MOT pipeline is modular: (1) a generic object detector (typically YOLO or Faster R-CNN pre-trained on a detection dataset) extracts bounding boxes per frame, (2) the Kalman filter predicts the next state for each active track, (3) the Hungarian algorithm solves the bipartite assignment, (4) track management creates and terminates tracks. DeepSORT injects an appearance model between the detector and the association stage: a wide residual network extracts a 128-dimensional ℓ_2-normalized feature vector from each detection crop, and these are used in the cosine distance component of the association cost.

Joint detection and embedding. JDE [**?**] and FairMOT [**?**] fuse detection and re-identification into a single network. The shared backbone (DLA-34 or HRNet) produces a feature map, from which three heads branch: (i) a heatmap head predicting object centers, (ii) a box size head predicting width and height offsets, and (iii) an embedding head producing dense identity features. The detection loss is a variant of CenterNet (focal loss for center heatmap, ℓ_1 for box regression). The identity loss is computed only at ground-truth object centers, treating each identity as a class in a cross-entropy formulation. This eliminates the need for a separate re-ID network and reduces latency.

Transformer trackers. TrackFormer [**?**] extends DETR by maintaining "track queries" that are propagated across frames. At each frame, a Transformer decoder processes image features together with track queries from the previous frame and a set of learned object queries for new detections. Self-attention over queries handles occlusion and re-identification implicitly. MOTR [**?**] adopts a similar approach with an additional track query memory bank. The loss function combines Hungarian-matched detection losses (classification + bounding box regression) with identity consistency penalties across time.

24.5 Pros and Cons

SiamFC and Siamese trackers. *Advantages:* (i) Extreme efficiency: template extraction is a one-time cost, enabling inference at 80–100 FPS on GPU. (ii) No online adaptation required, avoiding drift from erroneous updates. (iii) Training is purely offline with large-scale video data; no fine-tuning at test time. (iv) Fully convolutional nature enables efficient batched processing. *Disadvantages:* (i) Cannot recover from drift: if the tracker loses the target, it has no recovery mechanism since the template is fixed. (ii) Scale adaptation is limited; multi-scale search adds computational overhead. (iii) Semantic ambiguity: distractors similar to the template (other instances of the same class) cause confusion. (iv) Fixed template ignores appearance changes due to deformation, illumination, and out-of-plane rotation.

SORT and DeepSORT. *Advantages:* (i) Simplicity: minimal assumptions (constant velocity, linear measurement) yet effective in many scenarios. (ii) Real-time performance: Kalman filter update is $\mathcal{O}(1)$ per track, Hungarian is $\mathcal{O}(n^3)$ but n is typically small. (iii) Modular design allows swapping any detector. (iv) DeepSORT's appearance features significantly reduce identity switches. *Disadvantages:* (i) Constant velocity assumption fails for highly non-linear motion (rapid turns, acceleration). (ii) SORT has no re-identification; if a track is lost, a new track with a new ID is created. (iii) Kalman filter is sensitive to parameter choices ($\mathbf{Q}, \mathbf{R}$). (iv) DeepSORT's separate appearance network adds latency and memory overhead. (v) Both rely heavily on detector quality; missed detections cascade.

Transformer trackers. *Advantages:* (i) End-to-end trainable, avoiding hand-crafted heuristics. (ii) Attention naturally handles long-range temporal dependencies. (iii) Object queries provide a principled mechanism for maintaining identity. *Disadvantages:* (i) Computationally expensive, especially with many queries over many frames. (ii) Training requires large-scale video datasets with identity annotations. (iii) Inference latency limits real-time applications.

24.6 Evolution

The evolution of object tracking can be traced through several conceptual shifts. In the SOT domain, the progression is: **SiamFC** (2016) → **SiamRPN** (2018, adds region proposal) → **SiamRPN++** (2019, deeper backbones, depth-wise correlation) → **SiamMask** (2019, adds segmentation) → **Ocean** (2020, object-aware anchor-free) → **STARK** (2021, Transformer with spatio-temporal attention) → **OSTrack** (2022, one-stream Transformer with early fusion) → **MixFormer** (2022, mixed attention modules). The trend is toward end-to-end training with minimal inductive bias, leveraging the representational power of Transformers.

In the MOT domain: **SORT** (2016, Kalman + IoU) → **DeepSORT** (2017, adds appearance) → **JDE** (2019, joint detection and embedding) → **FairMOT** (2021, anchor-free joint learning) → **CenterTrack** (2020, track-by-detection via CenterNet) → **ByteTrack** (2021, uses low-confidence detections) → **BoT-SORT** (2022, better Kalman state representation and camera motion compensation) → **OC-SORT** (2023, observation-centric with recovery from occlusion) → **Transformer trackers** (TrackFormer, MOTR, 2021–2023). ByteTrack's key insight—that low-confidence detections often correspond to occluded objects and should not be discarded but used in a secondary association step—improved MOTA significantly on standard benchmarks without additional model complexity.

The broader trend is a convergence toward **joint detection and tracking** in a single end-to-end architecture, eliminating the fragmentation of detector, feature extractor, motion model, and association algorithm. This mirrors the broader deep learning philosophy of replacing engineered pipelines with learned components.

Chapter 25

Open-Vocabulary Detection and Segmentation

25.1 Motivation

Conventional visual recognition systems operate under a **closed-set** assumption: the set of object categories is known at training time, and the model learns a fixed classifier head $\mathbf{W} \in \mathbb{R}^{C \times D}$ that maps feature vectors to one of C class logits. At test time, the model is incapable of recognizing categories beyond its training vocabulary. In practice, however, the visual world contains a long tail of rare, novel, and compositional categories. An autonomous vehicle may need to recognize a "fallen tree blocking the road"; a manufacturing robot may need to detect a "cracked weld on a chassis"; a visual search engine must index millions of diverse concepts. Retraining a detector for each new vocabulary is prohibitively expensive in terms of data, compute, and time.

Open-Vocabulary (OV) recognition [**?**] breaks the closed-set barrier by allowing classification via natural language descriptions at test time. Rather than learning a fixed classification matrix, OV models learn to map image regions and text descriptions into a shared embedding space where semantic similarity is measured by cosine distance or dot product. A model trained on a handful of base categories can generalize to thousands of novel categories specified only by their textual names—a form of zero-shot generalization.

The fundamental enabler is the advent of large-scale vision-language pre-training, most prominently CLIP [**?**] and ALIGN [**?**], which align image and text representations across hundreds of millions of web-scale image-text pairs. Adapting these foundation models to the dense prediction tasks of detection and segmentation—where the output is a structured prediction over spatial coordinates rather than a single image-level label—constitutes the core challenge and opportunity of open-vocabulary visual recognition.

25.2 Mathematical Formulation

25.2.1 From Closed-Set to Open-Vocabulary Classification

In a closed-set detector, given a region feature $\mathbf{f} \in \mathbb{R}^D$ extracted by a backbone and RoI pooling, the class prediction is:

$$p(c \mid \mathbf{f}) = \text{softmax}_c(\mathbf{W}\mathbf{f} + \mathbf{b}) \tag{25.1}$$

where $\mathbf{W} = [\mathbf{w}_1, \ldots, \mathbf{w}_C]^\top$ contains a learned prototype for each of the C training categories. In the open-vocabulary setting, $\mathbf{W}$ is replaced by text embeddings derived from category names. Given a set of category descriptions $\{T_1, \ldots, T_C\}$ (e.g., T_c = "a photo of a {category name}"), a text encoder ψ_{text} produces embeddings:

$$\mathbf{t}_c = \psi_{\text{text}}(T_c) \in \mathbb{R}^D, \quad \mathbf{T} = [\mathbf{t}_1, \ldots, \mathbf{t}_C]^\top \in \mathbb{R}^{C \times D} \tag{25.2}$$

The prediction is then:

$$p(c \mid \mathbf{f}) = \text{softmax}_c\left(\tau \cdot \mathbf{f}^\top \mathbf{t}_c\right) = \frac{\exp(\tau \cdot \langle \mathbf{f}, \mathbf{t}_c \rangle)}{\sum_{c'=1}^{C} \exp(\tau \cdot \langle \mathbf{f}, \mathbf{t}_{c'} \rangle)} \tag{25.3}$$

where τ is a learned temperature parameter controlling the sharpness of the softmax distribution. The critical property is modularity: at test time, $\mathbf{T}$ can be recomputed for an entirely different set of category names, and the visual feature extractor remains unchanged.

25.2.2 CLIP: Vision-Language Pre-training Foundation

CLIP [**?**] trains an image encoder ψ_{img} (ResNet or ViT) and a text encoder ψ_{text} (Transformer) jointly on 400M image-text pairs using a contrastive objective. Given a batch of B image-text pairs $\{(I_i, T_i)\}_{i=1}^{B}$, the image and text embeddings are:

$$\mathbf{v}_i = \psi_{\text{img}}(I_i), \quad \mathbf{u}_i = \psi_{\text{text}}(T_i) \tag{25.4}$$

both ℓ_2-normalized to unit length. The contrastive loss maximizes the cosine similarity of the B correct pairs while minimizing that of the $B^2 - B$ incorrect pairs:

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{2B} \sum_{i=1}^{B} \left[\log \frac{\exp(\tau \cdot \langle \mathbf{v}_i, \mathbf{u}_i \rangle)}{\sum_{j=1}^{B} \exp(\tau \cdot \langle \mathbf{v}_i, \mathbf{u}_j \rangle)} + \log \frac{\exp(\tau \cdot \langle \mathbf{u}_i, \mathbf{v}_i \rangle)}{\sum_{j=1}^{B} \exp(\tau \cdot \langle \mathbf{u}_i, \mathbf{v}_j \rangle)} \right] \tag{25.5}$$

The first term is the image-to-text matching loss; the second is symmetric text-to-image. This is equivalent to a multi-class softmax over the batch dimension for each modality, with learnable temperature τ initialized as $\exp(\tau) \approx 1/0.07$. The resulting embedding space exhibits remarkable zero-shot transfer: an image of a previously unseen category aligns with its textual description by virtue of the semantic structure learned across millions of concepts.

25.2.3 Open-Vocabulary Object Detection

ViLD: Vision-Language Distillation for Detection

ViLD [**?**] adapts a standard two-stage detector (Mask R-CNN) to open-vocabulary detection by distilling CLIP's visual knowledge into the detector's region-level features. The architecture comprises three components:

Teacher: A pre-trained CLIP image encoder $\psi_{\text{img}}^{\text{CLIP}}$ produces embeddings for cropped image regions. For a ground-truth object bounding box at coordinates b, a masked crop x_{crop} is obtained by zeroing out pixels outside the box, and its CLIP embedding is $\mathbf{v}_{\text{crop}} = \psi_{\text{img}}^{\text{CLIP}}(x_{\text{crop}})$.

Student: The detector's RoI head produces region features $\mathbf{f}(b) \in \mathbb{R}^D$ for each proposal. The distillation loss aligns student features with teacher embeddings:

$$\mathcal{L}_{\text{distill}} = \frac{1}{N} \sum_{i=1}^{N} \left\| \mathbf{f}(b_i) - \mathbf{v}_{\text{crop},i} \right\|_1 \tag{25.6}$$

where $\|\cdot\|_1$ denotes the ℓ_1 loss, found to be more stable than ℓ_2 for distillation. Additionally, the student must maintain region-level classification accuracy on base categories (those with detection annotations), so a standard detection loss $\mathcal{L}_{\text{det}}$ is jointly optimized:

$$\mathcal{L}_{\text{ViLD}} = \mathcal{L}_{\text{det}} + \lambda_{\text{distill}} \mathcal{L}_{\text{distill}} \tag{25.7}$$

At inference, region features are compared to text embeddings (computed from CLIP's text encoder for arbitrary category names) via cosine similarity to determine the class. This enables zero-shot detection of novel categories.

GLIP: Grounded Language-Image Pre-training

GLIP [**?**] reformulates object detection as **phrase grounding**: given an image and a text prompt containing concatenated category names (e.g., "person. bicycle. car. ... airplane."), the task is to ground each noun phrase to the corresponding bounding box. This unifies detection data (bounding boxes with category labels) and grounding data (image-text pairs with phrase-to-region correspondences) in a single training framework.

The architecture consists of an image encoder ψ_{img} (DyHead or Swin Transformer), a text encoder ψ_{text} (BERT), and a **deep fusion** module that injects cross-modal information at multiple layers. At each fusion layer ℓ, image features $\mathbf{F}_\ell^{\text{img}} \in \mathbb{R}^{H_\ell \times W_\ell \times D}$ and text features $\mathbf{F}_\ell^{\text{text}} \in \mathbb{R}^{L \times D}$ are processed by cross-attention:

$$\mathbf{F}_\ell^{\text{img}} \leftarrow \mathbf{F}_\ell^{\text{img}} + \text{CrossAttn}_{\text{img}\to\text{text}}(\mathbf{F}_\ell^{\text{img}}, \mathbf{F}_\ell^{\text{text}}) \tag{25.8}$$

$$\mathbf{F}_\ell^{\text{text}} \leftarrow \mathbf{F}_\ell^{\text{text}} + \text{CrossAttn}_{\text{text}\to\text{img}}(\mathbf{F}_\ell^{\text{text}}, \mathbf{F}_\ell^{\text{img}}) \tag{25.9}$$

The prediction head computes alignment scores between region proposals and word tokens. For N region proposals with features $\{\mathbf{o}_i\}_{i=1}^N$ and L text tokens with features $\{\mathbf{p}_j\}_{j=1}^L$, the alignment logit is:

$$S_{ij} = \mathbf{o}_i^\top \mathbf{p}_j + b_i^{\text{box}} \tag{25.10}$$

where b_i^{box} is a box-level bias. The prediction for region i is:

$$\hat{\mathbf{c}}_i = \sigma(\mathbf{S}_i) \tag{25.11}$$

where $\mathbf{S}_i = [S_{i1}, \ldots, S_{iL}]^\top$. Each word token that corresponds to a noun phrase anchors a binary classification target: a region is positive for a noun phrase if it overlaps sufficiently with the ground-truth bounding box associated with that phrase.

The training loss is the **grounding loss**, combining localization (focal loss over region-word pairs) and box regression:

$$\mathcal{L}_{\text{GLIP}} = \mathcal{L}_{\text{loc}} + \mathcal{L}_{\text{reg}} \tag{25.12}$$

With massive pre-training on 27M grounding data triples (image, text, bounding boxes), GLIP achieves strong zero-shot detection performance.

Grounding DINO: DETR with Text Conditioning

Grounding DINO [**?**] integrates text conditioning into a DETR-style Transformer detector. The key architectural innovations are:

Feature enhancer: A cross-modality fusion module that processes image features and text features through a sequence of deformable self-attention, text-to-image cross-attention, and image-to-text cross-attention layers.

Language-guided query selection: The top-K encoder features are selected as initial object queries, where the selection is guided by dot-product similarity with the aggregated text feature (mean pooling of sentence-level text embeddings). This ensures that queries are initialized in semantically relevant regions.

Cross-modality decoder: Each decoder layer performs self-attention among queries, then cross-attention to image features, then cross-attention to text features. The output of the i-th query $\mathbf{q}_i$ is used for both bounding box regression and per-token classification. The box prediction head outputs $(\hat{x}, \hat{y}, \hat{w}, \hat{h})$ in normalized coordinates. The classification head computes:

$$\hat{p}_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{t}_j)}{\sum_k \exp(\mathbf{q}_i^\top \mathbf{t}_k)} \tag{25.13}$$

where $\mathbf{t}_j$ is the text embedding of the j-th token. The set-based Hungarian matching loss penalizes both box error and classification error against ground-truth bounding boxes and their associated text phrases.

OWL-ViT: Simple and Scalable Open-Vocabulary Detection

OWL-ViT [**?**] presents a minimalist approach: take a ViT image encoder and a corresponding text encoder from CLIP, add lightweight objectness and bounding box regression heads on top of ViT patch tokens, and fine-tune with self-training on web-scale image-text data. The objectness head ψ_{obj} predicts:

$$o_{ij} = \sigma(\psi_{\text{obj}}(\mathbf{p}_{ij})) \tag{25.14}$$

for each patch token $\mathbf{p}_{ij}$, representing the probability that an object's bounding box is centered at that patch. The box head ψ_{box} predicts a bounding box relative to the patch location:

$$\mathbf{b}_{ij} = \psi_{\text{box}}(\mathbf{p}_{ij}) \in \mathbb{R}^4 \tag{25.15}$$

For classification, the patch token is ℓ_2-normalized and compared to text embeddings of category names:

$$p(c \mid \mathbf{p}_{ij}) = \text{softmax}_c(\tau \cdot \langle \mathbf{p}_{ij}, \mathbf{t}_c \rangle) \tag{25.16}$$

OWLv2 [**?**] scales this further with more self-training data and token-level ℓ_2 normalization to improve generalization.

25.2.4 Open-Vocabulary Semantic Segmentation

Semantic segmentation requires per-pixel class predictions. The open-vocabulary extension replaces the pixel-level classification matrix with text embedding comparisons.

LSeg: Language-Driven Semantic Segmentation

LSeg [**?**] aligns per-pixel features with CLIP text embeddings. Given an image I, a dense prediction backbone (DPT or ResNet with dilated convolutions) produces a dense feature map $\mathbf{F} \in \mathbb{R}^{H \times W \times D}$. For a set of C categories with text descriptions $\{T_c\}_{c=1}^{C}$, CLIP's text encoder (frozen) yields text embeddings $\mathbf{T} = [\mathbf{t}_1, \ldots, \mathbf{t}_C] \in \mathbb{R}^{C \times D}$. The per-pixel logits are:

$$s_{ij}^{(c)} = \tau \cdot \langle \mathbf{F}_{ij}, \mathbf{t}_c \rangle \tag{25.17}$$

and the training loss is pixel-wise cross-entropy against ground-truth semantic labels on the base (seen) categories:

$$\mathcal{L}_{\text{LSeg}} = -\frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} \log \frac{\exp(\tau \langle \mathbf{F}_{ij}, \mathbf{t}_{y_{ij}} \rangle)}{\sum_{c=1}^{C_{\text{base}}} \exp(\tau \langle \mathbf{F}_{ij}, \mathbf{t}_c \rangle)} \tag{25.18}$$

At test time, the text embeddings $\mathbf{T}$ are replaced with those for novel categories, and predictions are obtained via $\arg\max_c \langle \mathbf{F}_{ij}, \mathbf{t}_c \rangle$. The key assumption is that the feature extractor learns to produce embeddings that are aligned with CLIP's joint vision-language space, enabling zero-shot transfer.

OpenSeg: Scaling with Weak Localization

OpenSeg [**?**] scales open-vocabulary segmentation by leveraging ALIGN [**?**] caption data with weak localization supervision from a class-agnostic region proposal network (Mask R-CNN proposals). For each image-caption pair, region proposals are classified via ALIGN's text-image similarity, and regions with high alignment scores are used as pseudo-ground-truth masks. The segmentation model is then trained on these class-agnostic masks with an ALIGN-based classification head, enabling scaling to millions of captioned images without manual segmentation annotations.

ODISE: Diffusion Model Priors for Segmentation

ODISE [**?**] exploits the rich internal representations of a pre-trained text-to-image diffusion model (Stable Diffusion) for open-vocabulary panoptic segmentation. The key insight is that the UNet of a diffusion model, trained to denoise images conditioned on text prompts, contains semantic and spatial priors that transfer to dense prediction tasks. ODISE extracts multi-scale UNet features and combines them with a lightweight segmentation head. Given a text prompt describing the target categories, the UNet's cross-attention maps provide a strong prior for segmentation. The output is a panoptic segmentation (semantic classes for "stuff" regions + instance IDs for "thing" regions), all classified via text embedding comparisons.

SAM + CLIP: A Modular Zero-Shot Pipeline

The Segment Anything Model (SAM) [**?**] is a promptable segmentation model trained on the SA-1B dataset of 11M images and 1.1B masks. SAM takes an image and a prompt (points, boxes, or masks) and produces a segmentation mask. The architecture consists of a ViT image encoder, a prompt encoder, and a lightweight mask decoder that fuses image embeddings with prompt embeddings via cross-attention:

$$\mathbf{m} = \text{MaskDecoder}(\mathbf{F}_{\text{img}}, \mathbf{F}_{\text{prompt}}) \tag{25.19}$$

SAM is class-agnostic: it produces high-quality masks without assigning any semantic label. For open-vocabulary segmentation, a two-stage pipeline couples SAM with CLIP: (i) SAM generates mask proposals for the entire image (using a grid of point prompts), (ii) each mask is cropped from the image, fed through CLIP's image encoder to obtain a region embedding, and (iii) the embedding is matched to text embeddings of the desired categories:

$$\hat{c}(m) = \arg\max_c \langle \psi_{\text{img}}^{\text{CLIP}}(\text{crop}(I, m)), \psi_{\text{text}}(T_c) \rangle \tag{25.20}$$

This modular approach achieves competitive zero-shot segmentation without any segmentation-specific training, demonstrating the power of composing pre-trained foundation models.

25.3 Algorithms

Algorithm 31: ViLD: Knowledge Distillation for Open-Vocabulary Detection

Input : Detection dataset $\mathcal{D}_{\text{base}}$, CLIP teacher $\psi_{\text{img}}^{\text{CLIP}}$, student detector Φ_{det}
Output: Open-vocabulary detector

Initialize Φ_{det} with pre-trained weights (R50-FPN);
for *epoch* $= 1$ **to** E **do**
  foreach *batch* $(I, \{b_i, c_i\}_{i=1}^N)$ *in* $\mathcal{D}_{base}$ **do**
    RoI features: $\mathbf{f}_i \leftarrow \Phi_{\text{det}}(I, b_i)$ for $i = 1, \ldots, N$;
    Teacher embeddings: $\mathbf{v}_i \leftarrow \psi_{\text{img}}^{\text{CLIP}}(\text{crop}(I, b_i))$ for $i = 1, \ldots, N$;
    $\mathcal{L}_{\text{distill}} \leftarrow \frac{1}{N} \sum_i \|\mathbf{f}_i - \mathbf{v}_i\|_1$;
    $\mathcal{L}_{\text{det}} \leftarrow \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}}$ on base categories;
    $\mathcal{L} \leftarrow \mathcal{L}_{\text{det}} + \lambda_{\text{distill}} \mathcal{L}_{\text{distill}}$;
    Update Φ_{det} via SGD;
Compute text embeddings $\mathbf{t}_c \leftarrow \psi_{\text{text}}(T_c)$ for all target categories;
Replace classifier head weights $\mathbf{W}$ with $\mathbf{T}$;
return Φ_{det} *(with text embedding classifier)*

25.4 Architecture

ViLD. The student detector is a standard Mask R-CNN with a ResNet-50-FPN backbone. The RoI features are extracted after the box head (2 fully-connected layers) and projected to match the CLIP feature dimension via a learnable projection layer. The CLIP teacher uses a ViT-B/32 or ResNet-50 visual encoder, frozen during distillation. Mask crops are processed by the CLIP teacher with the background zeroed out to focus attention on the object, a technique known as masked pooling.

GLIP. The image encoder uses a DyHead (Dynamic Head) or Swin-B Transformer backbone, providing multi-scale feature pyramids. The text encoder is BERT-base, producing contextualized token embeddings. The deep fusion module applies 6–12 layers

Algorithm 32: Open-Vocabulary Segmentation via SAM + CLIP

Input : Image I, target categories $\{c_1, \ldots, c_K\}$, SAM model, CLIP model
Output: Labeled segmentation masks

$\mathcal{M} \leftarrow$ SAM.generate_all_masks(I) `// using grid of point prompts`
Text embeddings: $\mathbf{t}_k \leftarrow \psi_{\text{text}}^{\text{CLIP}}(\text{“a photo of a } c_k\text{”})$ for $k = 1, \ldots, K$;
foreach *mask* $m \in \mathcal{M}$ **do**
    $x_{\text{crop}} \leftarrow \text{crop}(I, \text{bounding box of } m)$;
    $\mathbf{v} \leftarrow \psi_{\text{img}}^{\text{CLIP}}(x_{\text{crop}})$;
    $k^* \leftarrow \arg\max_k \langle \mathbf{v}, \mathbf{t}_k \rangle$;
    $\text{label}(m) \leftarrow c_{k^*}$;
return $\{(m, \mathit{label}(m)) \mid m \in \mathcal{M}, \max_k \langle \mathbf{v}, \mathbf{t}_k \rangle > \tau\}$

of bi-directional cross-attention, enabling early and deep interaction between visual and linguistic features. The prediction is made at each pyramid level using a shared bounding box head and region-word alignment head.

Grounding DINO. The architecture mirrors Deformable DETR but replaces the object query initialization and decoder design with text-aware variants. The feature enhancer is a stack of deformable attention layers interleaved with cross-modal attention. The decoder takes $N = 900$ object queries per image. Each decoder layer contains: (i) deformable self-attention over queries, (ii) deformable cross-attention to image features, (iii) standard cross-attention to text features. The output is fed to an MLP for box prediction and a dot-product with text tokens for classification.

SAM + CLIP. SAM's ViT-H image encoder produces a 64×64 feature map from a 1024×1024 input. The prompt encoder encodes point coordinates with positional encoding and masks with a lightweight CNN. The mask decoder is a two-layer Transformer decoder with cross-attention to image features and self-attention among output tokens. CLIP's ViT-L/14 encodes 336×336 crops, producing a 768-dimensional embedding. The pipeline is entirely zero-shot and model-agnostic.

25.5 Pros and Cons

Distillation-based methods (ViLD, OpenSeg). *Advantages:* (i) Leverage strong pre-trained vision-language models without re-training them. (ii) The detector architecture is unchanged; only the classification head is modified. (iii) Teacher embeddings can be pre-computed for all training images, adding minimal training overhead. *Disadvantages:* (i) Distillation is lossy: the student may not fully recover the teacher's representational quality, especially for fine-grained distinctions. (ii) Requires detection annotations for base categories; cannot train from caption data alone. (iii) The teacher is frozen and may have been trained on whole images, not regions, leading to a domain gap (the "whole-image bias" of CLIP).

Grounding-based methods (GLIP, Grounding DINO). *Advantages:* (i) Unifies detection and grounding data, enabling massive weak supervision. (ii) Deep cross-modal fusion enables richer interaction between language and vision. (iii) Can leverage web-scale image-text data without bounding box annotations. *Disadvantages:* (i) High computational cost: image-text cross-attention scales quadratically. (ii) Phrase extraction from captions is noisy and may introduce false associations. (iii) Training on large-scale

grounding data is resource-intensive.

Modular pipelines (SAM + CLIP). *Advantages:* (i) Zero-shot: no task-specific training required. (ii) Composable: each component can be upgraded independently. (iii) SAM provides high-quality, class-agnostic masks. *Disadvantages:* (i) Slow: running SAM inference plus multiple CLIP forward passes per image. (ii) Mask crop classification is coarser than pixel-level classification. (iii) SAM may fail on thin structures or highly overlapping instances; CLIP may misclassify ambiguous crops.

General limitations. (i) The vocabulary is bounded by the text encoder's knowledge: CLIP cannot recognize concepts absent from its pre-training data distribution. (ii) Text prompt engineering is crucial: "a photo of a cat" may yield different embeddings than "cat" or "a cat walking." (iii) The domain gap between web images and specialized imagery (medical, satellite) limits zero-shot performance.

25.6 Evolution

The evolution of open-vocabulary recognition follows a trajectory from post-hoc adaptation toward end-to-end grounded training. In detection: **OVR-CNN** (2021, region-text matching with captioned images) → **ViLD** (2022, CLIP distillation into detectors) → **Detic** (2022, classifiers trained on image-level labels for extended vocabulary) → **GLIP** (2022, phrase grounding at scale, 27M grounded data) → **Grounding DINO** (2023, DETR + text conditioning) → **OWL-ViT / OWLv2** (2022/2023, simple ViT + text, web-scale self-training) → **T-Rex2** (2024, visual prompting for open-vocabulary counting and detection).

In segmentation: **LSeg** (2022, pixel-CLIP alignment) → **OpenSeg** (2022, scaling with ALIGN and proposals) → **OVSeg** (2023, adapting CLIP for masked crops) → **ODISE** (2023, diffusion models as dense priors) → **SAM** (2023, promptable class-agnostic segmentation) → **SAM + CLIP** (2023, composing foundation models) → **Grounded-SAM** (2023, adding Grounding DINO for detection) → **SAM2** (2024, extending SAM to video with memory-based prompting) → **EVF-SAM** (2024, early vision-language fusion in SAM).

The overarching trend is the convergence of detection, grounding, and segmentation into unified vision-language architectures capable of any-vocabulary dense prediction. The proliferation of large-scale pre-training data, the maturation of Transformer architectures, and the availability of strong foundation models (CLIP, SAM, Stable Diffusion, LLMs) have collapsed what were once distinct research subfields into variations on a single theme: **aligning visual features with language in a shared representational space, then projecting to the output structure (bounding boxes, masks, or pixel labels) via lightweight, task-specific heads.**

Part VIII

Graph Neural Networks

Chapter 26

Graph Neural Networks

26.1 Motivation

The data structures that dominate deep learning—grids (images, audio spectrograms) and sequences (text, time series)—possess well-defined, regular topologies that enable parameter sharing through translation invariance (CNNs) or sequential recurrence (RNNs, Transformers). However, a vast class of real-world data is inherently **graph-structured**: molecules are atoms connected by chemical bonds, social networks are individuals linked by friendships, citation networks are papers connected by references, knowledge graphs are entities linked by typed relations, and recommender systems involve users interacting with items across a bipartite graph. In each case, the relational structure is as informative as the entities themselves, and the topology is irregular: nodes have varying numbers of neighbors, and there is no natural notion of spatial ordering or translation.

Graph Neural Networks (GNNs) [**?**] [**?**] extend deep learning to graph domains by defining operations that respect the graph topology—specifically, **permutation invariance** (the output of a node update is invariant to the ordering of its neighbors) and **permutation equivariance** (reordering input nodes reorders output representations identically). These properties are satisfied by the message-passing paradigm, which aggregates information from each node's neighborhood and updates its representation iteratively.

The applications of GNNs span drug discovery (predicting molecular properties, generating novel compounds), recommender systems (graph-based collaborative filtering), traffic forecasting (sensor networks on road graphs), program analysis (abstract syntax trees as graphs), physical simulation (particle interactions as dynamic graphs), and social network analysis (community detection, influence prediction). This chapter develops the mathematical theory of graph representation learning from spectral graph theory through attention-based architectures.

26.2 Mathematical Formulation

26.2.1 Graphs: Notation and Fundamentals

A graph is a pair $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where $\mathcal{V} = \{v_1, \ldots, v_N\}$ is the node set and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the edge set. The adjacency matrix $\mathbf{A} \in \{0,1\}^{N \times N}$ encodes connectivity: $A_{ij} = 1$ if $(v_i, v_j) \in \mathcal{E}$, else 0. For weighted or attributed graphs, $A_{ij} \in \mathbb{R}_{\geq 0}$. Node features are collected in $\mathbf{X} \in \mathbb{R}^{N \times d}$, where $\mathbf{x}_i \in \mathbb{R}^d$ is the feature vector of node v_i. When available, edge features can be represented as $\mathbf{E} \in \mathbb{R}^{N \times N \times d_e}$.

The **degree matrix** $\mathbf{D} = \mathrm{diag}(d_1, \ldots, d_N)$ has entries $d_i = \sum_j A_{ij}$. The neighborhood of node v_i is $\mathcal{N}(i) = \{v_j \in \mathcal{V} : (v_i, v_j) \in \mathcal{E}\}$.

The **graph Laplacian** is defined as:

$$\mathbf{L} = \mathbf{D} - \mathbf{A} \tag{26.1}$$

Its symmetric normalized form is:

$$\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2} \tag{26.2}$$

The Laplacian is a central object in spectral graph theory: it is symmetric positive semi-definite, so it admits an eigendecomposition $\mathbf{L}_{\text{sym}} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ where $\mathbf{U} \in \mathbb{R}^{N\times N}$ is orthogonal and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_N)$ with $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_N$. The eigenvectors form an orthonormal basis for functions on the graph, analogous to the Fourier basis on Euclidean domains.

26.2.2 Message Passing Neural Networks: A Unified Framework

The Message Passing Neural Network (MPNN) [**?**] framework abstracts most GNNs into two operations per layer. At layer ℓ, each node v maintains a hidden representation $\mathbf{h}_v^{(\ell)} \in \mathbb{R}^{d_\ell}$. The layer transition is:

$$\mathbf{m}_v^{(\ell)} = \text{AGGREGATE}^{(\ell)}\left(\left\{\mathbf{h}_u^{(\ell-1)} : u \in \mathcal{N}(v)\right\}\right) \tag{26.3}$$

$$\mathbf{h}_v^{(\ell)} = \text{UPDATE}^{(\ell)}\left(\mathbf{h}_v^{(\ell-1)}, \mathbf{m}_v^{(\ell)}\right) \tag{26.4}$$

The AGGREGATE function must be permutation-invariant (order of neighbors does not matter). After L message-passing layers, each node's representation $\mathbf{h}_v^{(L)}$ captures information from its L-hop neighborhood (the L-step receptive field). A **READOUT** function computes a graph-level representation from node embeddings:

$$\mathbf{h}_{\mathcal{G}} = \text{READOUT}\left(\{\mathbf{h}_v^{(L)} : v \in \mathcal{V}\}\right) \tag{26.5}$$

26.2.3 Graph Convolutional Network (GCN)

Spectral Motivation and Derivation

The spectral graph convolution of a signal $\mathbf{x} \in \mathbb{R}^N$ (one feature dimension over all nodes) with a filter g_θ is defined in the Fourier domain:

$$g_\theta \star \mathbf{x} = \mathbf{U}\, g_\theta(\mathbf{\Lambda})\, \mathbf{U}^\top \mathbf{x} \tag{26.6}$$

where $g_\theta(\mathbf{\Lambda}) = \text{diag}(g_\theta(\lambda_1), \ldots, g_\theta(\lambda_N))$ is a diagonal matrix of filtered eigenvalues. Direct computation requires the eigendecomposition of $\mathbf{L}_{\text{sym}}$ (cost $\mathcal{O}(N^3)$) and dense matrix multiplication (cost $\mathcal{O}(N^2)$), which is infeasible for large graphs.

Defferrard et al. [**?**] approximate $g_\theta(\mathbf{\Lambda})$ with a K-th order Chebyshev polynomial expansion:

$$g_\theta(\mathbf{\Lambda}) \approx \sum_{k=0}^{K} \theta_k T_k(\tilde{\mathbf{\Lambda}}), \quad \tilde{\mathbf{\Lambda}} = \frac{2}{\lambda_{\max}}\mathbf{\Lambda} - \mathbf{I} \tag{26.7}$$

where T_k is the Chebyshev polynomial of order k, defined recursively: $T_0(x) = 1$, $T_1(x) = x$, $T_k(x) = 2xT_{k-1}(x) - T_{k-2}(x)$. Substituting into (26.6) and using $\tilde{\mathbf{L}}_{\text{sym}} = \mathbf{U}\tilde{\mathbf{\Lambda}}\mathbf{U}^\top = \frac{2}{\lambda_{\max}}\mathbf{L}_{\text{sym}} - \mathbf{I}$:

$$g_\theta \star \mathbf{x} \approx \sum_{k=0}^{K} \theta_k T_k(\tilde{\mathbf{L}}_{\text{sym}})\mathbf{x} \tag{26.8}$$

This is a K-localized filter: the output at node v depends only on nodes within K hops, since $T_k(\tilde{\mathbf{L}}_{\text{sym}})$ is a polynomial of degree k in the (local) Laplacian.

Kipf and Welling [**?**] further simplify by: (i) setting $K = 1$ (linear approximation), (ii) assuming $\lambda_{\max} \approx 2$, giving $\tilde{\mathbf{L}}_{\text{sym}} \approx \mathbf{L}_{\text{sym}} - \mathbf{I} = -\mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$, and (iii) constraining $\theta = \theta_0 = -\theta_1$ to reduce parameters:

$$g_\theta \star \mathbf{x} \approx \theta_0 T_0(\tilde{\mathbf{L}}_{\text{sym}})\mathbf{x} + \theta_1 T_1(\tilde{\mathbf{L}}_{\text{sym}})\mathbf{x} \tag{26.9}$$

$$= \theta_0 \mathbf{x} + \theta_1 \tilde{\mathbf{L}}_{\text{sym}}\mathbf{x} \tag{26.10}$$

$$= \theta_0 \mathbf{x} - \theta_1 \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}\mathbf{x} \tag{26.11}$$

$$\approx \theta \left(\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}\right)\mathbf{x} \tag{26.12}$$

The matrix $\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ has eigenvalues in $[0, 2]$. Repeated application of this operator leads to numerical instability (exploding/vanishing). The **renormalization trick** mitigates this by adding self-loops and renormalizing:

$$\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}, \quad \tilde{\mathbf{D}}_{ii} = \sum_j \tilde{A}_{ij}, \quad \hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2} \tag{26.13}$$

Generalizing to multi-channel input $\mathbf{H}^{(\ell-1)} \in \mathbb{R}^{N \times d_{\ell-1}}$ with a learned weight matrix $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_{\ell-1} \times d_\ell}$ and a non-linearity σ:

$$\mathbf{H}^{(\ell)} = \sigma\left(\hat{\mathbf{A}}\,\mathbf{H}^{(\ell-1)}\,\mathbf{W}^{(\ell)}\right) \tag{26.14}$$

A 2-layer GCN for semi-supervised node classification produces logits:

$$\mathbf{Z} = \text{softmax}\left(\hat{\mathbf{A}}\ \text{ReLU}\left(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}^{(1)}\right)\mathbf{W}^{(2)}\right) \tag{26.15}$$

The loss is computed only over labeled nodes $\mathcal{V}_L$:

$$\mathcal{L} = -\sum_{v \in \mathcal{V}_L} \sum_{c=1}^{C} Y_{vc} \log Z_{vc} \tag{26.16}$$

Crucially, the adjacency matrix $\hat{\mathbf{A}}$ propagates information from labeled nodes through unlabeled neighbors, enabling semi-supervised learning without explicitly modeling the manifold.

GCN as a Special Case of Message Passing

In MPNN terms, a GCN layer with $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$ has:

$$\text{AGGREGATE: } \mathbf{m}_v^{(\ell)} = \sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\tilde{d}_v \tilde{d}_u}} \mathbf{h}_u^{(\ell-1)} \tag{26.17}$$

$$\text{UPDATE: } \mathbf{h}_v^{(\ell)} = \sigma\left(\mathbf{W}^{(\ell)}\mathbf{m}_v^{(\ell)}\right) \tag{26.18}$$

The aggregation weights are fixed by the graph structure (normalized adjacency), not learned. This is both a strength (inductive bias from topology) and a limitation (cannot learn edge importance).

26.2.4 GraphSAGE: Inductive Representation Learning

GraphSAGE [**?**] addresses the transductive limitation of GCN (which requires the full graph at training time) by learning a function that generates embeddings for **any** node based on its local neighborhood features, enabling generalization to unseen nodes and entirely new graphs.

The forward propagation for node v at layer ℓ is:

$$\mathbf{h}_{\mathcal{N}(v)}^{(\ell)} = \text{AGGREGATE}^{(\ell)}\left(\{\mathbf{h}_u^{(\ell-1)} : u \in \mathcal{N}(v)\}\right) \tag{26.19}$$

$$\mathbf{h}_v^{(\ell)} = \sigma\left(\mathbf{W}^{(\ell)} \cdot \left[\mathbf{h}_v^{(\ell-1)} \| \, \mathbf{h}_{\mathcal{N}(v)}^{(\ell)}\right]\right) \tag{26.20}$$

where $\|$ denotes concatenation and $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_\ell \times 2d_{\ell-1}}$. Three permutation-invariant aggregation functions are proposed:

Mean aggregator:

$$\mathbf{h}_{\mathcal{N}(v)}^{(\ell)} = \text{MEAN}\left(\{\mathbf{h}_u^{(\ell-1)} : u \in \mathcal{N}(v)\}\right) \tag{26.21}$$

This is essentially the GCN aggregator sans the learned linear transformation before aggregation.

LSTM aggregator: Apply an LSTM to a random permutation of the neighbor embeddings. While LSTMs are not permutation-invariant in theory, the random permutation regularizes the model, and empirical performance is strong. The LSTM processes the sequence $(\mathbf{h}_{u_1}, \mathbf{h}_{u_2}, \ldots, \mathbf{h}_{u_{|\mathcal{N}(v)|}})$ where $(u_1, \ldots)$ is a random shuffle.

Pooling aggregator:

$$\mathbf{h}_{\mathcal{N}(v)}^{(\ell)} = \max\left(\{\sigma(\mathbf{W}_{\text{pool}}\mathbf{h}_u^{(\ell-1)} + \mathbf{b}) : u \in \mathcal{N}(v)\}\right) \tag{26.22}$$

where max is element-wise maximum over the neighbor set. This is permutation-invariant and empirically performs well.

GraphSAGE further introduces **neighbor sampling**: for each node v, a fixed-size set $\mathcal{N}_S(v)$ of S neighbors is sampled uniformly. This bounds the computational cost per node and enables mini-batch training on large graphs.

26.2.5 Graph Attention Network (GAT)

GAT [**?**] learns edge-specific weights via an attention mechanism, allowing the model to assign different importance to different neighbors. For a node v and neighbor $u \in \mathcal{N}(v)$, an attention coefficient e_{vu} is computed:

$$e_{vu} = \text{LeakyReLU}\left(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_v \| \mathbf{W}\mathbf{h}_u]\right) \tag{26.23}$$

where $\mathbf{W} \in \mathbb{R}^{d' \times d}$ is a shared linear transformation, $\mathbf{a} \in \mathbb{R}^{2d'}$ is a learnable attention vector, and $\|$ denotes concatenation. The coefficients are normalized across neighbors via softmax:

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in \mathcal{N}(v)} \exp(e_{vk})} \tag{26.24}$$

The output representation for node v is a weighted combination:

$$\mathbf{h}'_v = \sigma\left(\sum_{u\in\mathcal{N}(v)} \alpha_{vu}\mathbf{W}\mathbf{h}_u\right) \tag{26.25}$$

Multi-head attention. To stabilize training, K independent attention heads are computed, and their outputs are concatenated (or averaged for the final layer):

$$\mathbf{h}'_v = \Big\|_{k=1}^{K} \sigma\left(\sum_{u\in\mathcal{N}(v)} \alpha_{vu}^{(k)}\mathbf{W}^{(k)}\mathbf{h}_u\right) \tag{26.26}$$

Each head attends to a different subspace of the feature space, analogous to multi-head attention in Transformers. GAT is fully local (no global graph structure required) and intrinsically inductive.

26.2.6 Graph Isomorphism Network (GIN)

Xu et al. [**?**] prove that the expressive power of GNNs is bounded by the Weisfeiler-Lehman (WL) graph isomorphism test. A GNN can distinguish two non-isomorphic graphs if and only if its aggregation function is **injective** (like the WL test's hash function). Among common aggregation functions, only the sum is injective on a multiset (mean and max are not). GIN's update rule is:

$$\mathbf{h}_v^{(k)} = \mathrm{MLP}^{(k)}\left((1+\varepsilon^{(k)})\mathbf{h}_v^{(k-1)} + \sum_{u\in\mathcal{N}(v)} \mathbf{h}_u^{(k-1)}\right) \tag{26.27}$$

where $\varepsilon^{(k)}$ is a learnable scalar or a fixed constant (e.g., 0), and $\mathrm{MLP}^{(k)}$ is a multi-layer perceptron. The sum aggregation guarantees maximum expressiveness under the WL hierarchy. For graph-level readout, GIN concatenates node embeddings from all layers (to capture information at multiple resolutions):

$$\mathbf{h}_{\mathcal{G}} = \mathrm{CONCAT}\left(\mathrm{SUM}\left(\{\mathbf{h}_v^{(k)} : v\in\mathcal{V}\}\right) \;\middle|\; k = 0, 1, \ldots, K\right) \tag{26.28}$$

26.2.7 Training Considerations

Node classification. For semi-supervised node classification on a single graph, full-batch gradient descent is feasible for medium-sized graphs ($N \lesssim 10^5$). The loss is computed only on labeled training nodes, while the graph Laplacian diffuses gradients to unlabeled nodes.

Large-scale training. For graphs with millions of nodes, mini-batch sampling is necessary. Methods include: (i) **GraphSAINT** [**?**]: random walk-based sampler that constructs mini-batches by sampling subgraphs, (ii) **Cluster-GCN** [**?**]: graph clustering followed by sampling of clusters, (iii) **neighbor sampling**: recursively sample S neighbors per node as in GraphSAGE.

Link prediction. To predict missing edges, an encoder $\mathrm{ENC}(\mathcal{G})$ produces node embeddings, and a decoder predicts edge scores: $p(A_{ij}=1) = \sigma(\mathbf{h}_i^\top \mathbf{h}_j)$ or $p(A_{ij}=1) = \mathrm{MLP}([\mathbf{h}_i \| \mathbf{h}_j])$. Training uses negative sampling: for each positive edge (i,j), sample K negative edges (i,j') and minimize the binary cross-entropy.

Graph classification/regression. The READOUT function aggregates node embeddings into a graph-level embedding, followed by an MLP classifier. Common READOUT functions are SUM (sensitive to graph size), MEAN (size-invariant), and MAX (identifies most prominent nodes). The choice depends on the task: for molecular property prediction, SUM is preferred because the number of atoms matters.

26.2.8 The Over-Smoothing Problem

As the number of GCN layers L increases, node representations converge to the same vector regardless of input features. Formally, for a connected graph with Laplacian $\mathbf{L}_{\text{sym}}$, repeated application of the GCN operator (without non-linearity) exponentially converges to the projection onto the eigenspace of $\lambda_1 = 0$ (the constant eigenvector):

$$\lim_{L\to\infty} \hat{\mathbf{A}}^L \mathbf{x} = \frac{\tilde{\mathbf{D}}^{1/2}\mathbf{1}\mathbf{1}^\top \tilde{\mathbf{D}}^{1/2}}{2m+N}\mathbf{x} \tag{26.29}$$

where $2m+N = \sum_i \tilde{d}_i$. All node representations collapse to a single point, destroying discriminability. This limits practical GCN depth to 2–4 layers, in stark contrast to CNNs that benefit from depth. Remedies include residual connections, normalization layers, and the use of attention (GAT) or skip connections to preserve local information.

26.3 Algorithms

Algorithm 33: Graph Convolutional Network (GCN) — Forward Propagation

Input : Adjacency $\mathbf{A} \in \{0,1\}^{N\times N}$, features $\mathbf{X} \in \mathbb{R}^{N\times d}$, weights $\{\mathbf{W}^{(1)}, \ldots, \mathbf{W}^{(L)}\}$, labels $\mathbf{Y}$ for $\mathcal{V}_L \subset \mathcal{V}$
Output: Node predictions $\mathbf{Z} \in \mathbb{R}^{N\times C}$

Compute $\tilde{\mathbf{A}} \leftarrow \mathbf{A} + \mathbf{I}$;
$\tilde{\mathbf{D}}_{ii} \leftarrow \sum_j \tilde{A}_{ij}$ for $i = 1, \ldots, N$;
$\hat{\mathbf{A}} \leftarrow \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$;
$\mathbf{H}^{(0)} \leftarrow \mathbf{X}$;
for $\ell = 1$ **to** $L-1$ **do**
  $\mathbf{H}^{(\ell)} \leftarrow \text{ReLU}\left(\hat{\mathbf{A}}\mathbf{H}^{(\ell-1)}\mathbf{W}^{(\ell)}\right)$;
$\mathbf{Z} \leftarrow \text{softmax}\left(\hat{\mathbf{A}}\mathbf{H}^{(L-1)}\mathbf{W}^{(L)}\right)$;
$\mathcal{L} \leftarrow -\sum_{v\in\mathcal{V}_L}\sum_{c=1}^{C} Y_{vc}\log Z_{vc}$;
Update weights via Adam on $\nabla_{\mathbf{W}}\mathcal{L}$;
return Z

26.4 Architecture

GCN. The canonical GCN uses symmetric normalization of the adjacency with self-loops. The architecture is shallow (2–3 layers) due to over-smoothing. Each layer applies a linear transformation (learned weight matrix) followed by ReLU, preceded by the fixed

Algorithm 34: Graph Attention Network (GAT) — Single Layer Forward

Input : Node features $\mathbf{H} \in \mathbb{R}^{N \times d}$, adjacency $\mathbf{A}$, heads K, attention vector $\mathbf{a}$, weight matrix $\mathbf{W}$
Output: Updated node features $\mathbf{H}' \in \mathbb{R}^{N \times Kd'}$

foreach *node* $v \in \mathcal{V}$ **do**
  $\mathbf{z}_v \leftarrow \mathbf{W}\mathbf{h}_v$;
foreach *head* $k = 1$ **to** K **do**
  foreach *edge* $(v, u) \in \mathcal{E}$ **do**
    $e_{vu}^{(k)} \leftarrow \text{LeakyReLU}\left(\mathbf{a}^{(k)\top}[\mathbf{z}_v \| \mathbf{z}_u]\right)$;
  foreach *node* $v \in \mathcal{V}$ **do**
    $\alpha_{vu}^{(k)} \leftarrow \frac{\exp(e_{vu}^{(k)})}{\sum_{w \in \mathcal{N}(v)} \exp(e_{vw}^{(k)})}$ for $u \in \mathcal{N}(v)$;
    $\mathbf{h}_v^{(k)} \leftarrow \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu}^{(k)} \mathbf{z}_u\right)$;
$\mathbf{h}'_v \leftarrow \|_{k=1}^{K} \mathbf{h}_v^{(k)}$ for all v;
return $\mathbf{H}'$

Algorithm 35: GraphSAGE: Mini-Batch Training

Input : Graph $\mathcal{G}$, features $\mathbf{X}$, depth L, sample sizes $S_1, \ldots, S_L$, batch size B
Output: Trained model parameters

for *epoch* $= 1$ **to** E **do**
  Sample mini-batch $\mathcal{B} \subset \mathcal{V}$ of B nodes;
  for $\ell = L$ **to** 1 **do**
    Sample S_ℓ neighbors for each node in the current set;
    Aggregate features from sampled neighbors at layer ℓ;
  Forward propagate from layer 1 to L;
  Compute loss only on $\mathcal{B}$;
  Backpropagate gradients;

aggregation (normalized adjacency multiplication). The final layer produces C logits with softmax. This architecture is transductive: it requires the full graph adjacency at both training and inference time.

GraphSAGE. ArgsGAGE is an encoder-decoder design for inductive learning. The encoder (GraphSAGE layers) produces node embeddings from local neighborhood features; the decoder varies by task (e.g., dot product for link prediction, softmax for node classification). The key architectural difference from GCN is the explicit separation of neighbor aggregation and self-representation updating, along with neighbor sampling for computational efficiency.

GAT. GAT layers replace the fixed normalized aggregation of GCN with learned attention weights. The multi-head design concatenates outputs of parallel attention heads, with each head learning independent linear transformations and attention vectors. Averaging replaces concatenation in the final layer. GAT is inductive and does not require knowledge of the full graph structure, only the local neighborhood per node.

GIN. GIN uses sum aggregation with a 2-layer MLP after aggregation. The ε parameter controls the weight of the self-representation relative to neighbor sum. ReLU batch normalization follows each MLP. GIN is designed to be maximally expressive under the WL test.

Graphormer [?]. A Transformer adapted to graphs: (i) **centrality encoding**: node degree added as a learnable embedding to input features, (ii) **spatial encoding**: shortest-path distance between nodes encoded as a learnable bias in the attention matrix, $A_{ij} = \frac{(\mathbf{h}_i\mathbf{W}_Q)(\mathbf{h}_j\mathbf{W}_K)^\top}{\sqrt{d}} + b_{\mathrm{SPD}(i,j)}$, (iii) **edge encoding**: edge features integrated via a learnable attention bias $\mathbf{c}_{ij} = \frac{1}{M}\sum_{m=1}^{M} \mathbf{w}_{E_m} \odot \mathbf{e}_{ij,m}$, where M is the number of edge feature dimensions. These encodings inject graph structural priors into the otherwise structure-agnostic Transformer.

26.5 Pros and Cons

GCN. *Advantages:* (i) Elegant spectral derivation with strong theoretical grounding. (ii) Fixed, non-learned aggregation is parameter-efficient and computationally fast. (iii) Semi-supervised learning naturally leverages graph topology. (iv) The renormalization trick is simple yet effective for numerical stability. *Disadvantages:* (i) Transductive: cannot generalize to unseen nodes or new graphs without retraining. (ii) Fixed aggregation weights cannot adapt to task-specific neighbor importance. (iii) Over-smoothing limits its depth to 2–4 layers. (iv) The spectral approximation ($K = 1$, $\lambda_{\max} \approx 2$) may be insufficiently expressive for complex graphs.

GraphSAGE. *Advantages:* (i) Inductive: generalizes to unseen nodes and graphs. (ii) Neighbor sampling enables training on billion-scale graphs. (iii) Flexible aggregator choices (mean, LSTM, pooling) allow task-specific adaptation. *Disadvantages:* (i) Sampling introduces variance; the inductive bias is weaker than full-graph GCN. (ii) Fixed sample size may miss important long-range dependencies. (iii) Performance depends on the quality of the sampling scheme.

GAT. *Advantages:* (i) Learned attention weights provide adaptive aggregation, potentially ignoring noisy edges. (ii) Multi-head attention captures multiple types of neighborhood interactions. (iii) Computationally efficient: attention is computed per edge with shared parameters. (iv) Inductive and interpretable (attention weights indicate important neighbors). *Disadvantages:* (i) More parameters than GCN; can overfit on small

datasets. (ii) Static attention pattern: attention weights depend only on feature values, not on the broader graph context (GATv2 addresses this by computing attention as a function of both nodes jointly before LeakyReLU). (iii) Over-smoothing still occurs with depth.

GIN. *Advantages:* (i) Theoretically maximally expressive among MPNNs (matches WL test). (ii) Simple architecture with clear design rationale. *Disadvantages:* (i) Sum aggregation is sensitive to graph size; generalization to graphs of different sizes can be problematic. (ii) Empirically, the WL-test upper bound is sometimes insufficient for real-world tasks that require counting substructures or detecting higher-order graph properties.

General GNN limitations. (i) MPNNs cannot count cycles, distinguish regular graphs of the same degree, or detect higher-order structural patterns (bounded by 1-WL). (ii) Over-squashing: information from exponentially many nodes must be compressed into a fixed-size vector, causing a bottleneck for long-range dependencies. (iii) Scalability: full-batch GCNs cannot handle graphs with $> 10^5$ nodes without specialized sampling.

26.6 Evolution

GNN research has progressed through several conceptual phases. The **spectral era**: spectral networks [**?**] (2014, first graph convolution via Laplacian eigenvectors) → ChebNet [**?**] (2016, Chebyshev polynomial approximation) → GCN [**?**] (2017, first-order approximation with renormalization), establishing the connection between spectral filtering and spatial aggregation.

The **spatial era**: GraphSAGE [**?**] (2017, inductive learning, neighbor sampling) → GAT [**?**] (2018, learned attention over edges) → GIN [**?**] (2019, theoretical expressiveness analysis) → GATv2 [**?**] (2022, dynamic attention, fixing the static attention limitation).

The **beyond-MPNN era**: addressing the limitations of the message-passing framework. Higher-order GNNs [**?**] operate on k-tuples of nodes ($k > 2$) to achieve k-WL expressiveness. Subgraph GNNs [**?**] process subgraphs extracted from the original graph. Positional encoding approaches inject node positional information (Laplacian eigenvectors, random walk probabilities, shortest-path distances) to break the symmetry that limits WL-expressiveness.

The **graph Transformer era**: Graphormer [**?**] (2021, spatial/centrality encoding) → SAN [**?**] (2021, spectral attention with learned positional encodings) → GraphGPS [**?**] (2022, a blueprint combining MPNNs for local structure with Transformers for global attention). These models address the over-squashing problem by enabling direct, long-range node interactions through full-graph attention.

The **scaling era**: efforts to scale GNNs to web-scale graphs (billions of nodes and edges) via advanced sampling (GraphSAINT [**?**], Cluster-GCN [**?**]), graph partitioning, and distributed training. Concurrently, the adaptation of geometric deep learning principles (invariance, equivariance) to 3D molecular graphs (E(n) Equivariant GNNs [**?**]) and the integration of GNNs with large language models (GraphGPT, InstructGLM) represent the frontier.

Part IX

Reinforcement Learning

Chapter 27

Reinforcement Learning Fundamentals

27.1 Motivation

Reinforcement Learning (RL) [**?**] addresses a problem fundamentally different from supervised learning: an agent learns to make sequential decisions through **interaction** with an environment, receiving only scalar **reward signals** that evaluate the quality of its actions, without explicit supervision about which action is correct. This trial-and-error learning paradigm is directly inspired by behavioral psychology (Thorndike's Law of Effect) and optimal control theory, and it is the natural framework for any problem where the agent's actions affect future states and rewards.

The mathematical power of RL lies in its generality. Supervised learning can be seen as a special case where the reward is the negative loss and the agent's actions are predictions. But RL extends far beyond: game playing (AlphaGo defeated world champions in Go, a game with 10^{170} possible board states), robotics (learning dexterous manipulation policies for multi-fingered hands), dialogue systems (RLHF aligning language models with human preferences), autonomous driving, resource allocation in data centers, and scientific discovery (optimizing chemical reactions, controlling fusion plasmas). In each case, the core challenge is the **credit assignment problem**: determining which actions in a long sequence were responsible for an eventual success or failure, and the **exploration-exploitation dilemma**: balancing the need to explore unknown state-action pairs against the desire to exploit known high-reward actions.

This chapter develops the mathematical foundations of RL—Markov decision processes, Bellman optimality, policy gradients, and modern deep RL algorithms—from first principles through convergence properties, with particular attention to the intersection of RL and deep learning that has powered recent breakthroughs.

27.2 Mathematical Formulation

27.2.1 Markov Decision Processes

A Markov Decision Process (MDP) [**?**] is the formal framework for RL. An MDP is a 5-tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- $\mathcal{S}$: the state space.
- $\mathcal{A}$: the action space.
- $P(s' \mid s, a)$: the transition probability distribution, $\forall s, s' \in \mathcal{S}, a \in \mathcal{A}$.
- $R(s, a)$: the reward function, $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$.
- $\gamma \in [0, 1)$: the discount factor (the case $\gamma = 1$ is reserved for finite-horizon or ergodic settings).

The **Markov property** states that the future depends only on the current state, not on the history: $P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t)$. This is both a mathematical convenience and a realistic assumption when states are sufficiently informative.

A **trajectory** τ of length T (possibly infinite) is:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, s_2, \dots, s_T) \tag{27.1}$$

where $s_0 \sim \rho_0$ (initial state distribution), $a_t \sim \pi(\cdot \mid s_t)$, $s_{t+1} \sim P(\cdot \mid s_t, a_t)$, and $r_t = R(s_t, a_t)$. The **return** (cumulative discounted reward) from time t is:

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots \tag{27.2}$$

The discount factor γ serves a dual purpose: (i) it ensures the infinite sum converges (since R is assumed bounded), and (ii) it captures the time value of reward—immediate rewards are preferred over delayed rewards, a modeling choice that often reflects real-world preferences.

27.2.2 Value Functions and Bellman Equations

A **policy** $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to a probability distribution over actions: $\pi(a \mid s)$ is the probability of taking action a in state s. Policy determines behavior; the goal of RL is to find an optimal policy.

The **state value function** $V^\pi(s)$ is the expected return starting from state s and following policy π:

$$V^\pi(s) = \mathbb{E}_{\tau \sim \pi} \left[G_t \mid s_t = s \right] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t = s \right] \tag{27.3}$$

The **action value function** $Q^\pi(s, a)$ is the expected return starting from state s, taking action a, and thereafter following π:

$$Q^\pi(s, a) = \mathbb{E}_{\tau \sim \pi} \left[G_t \mid s_t = s, a_t = a \right] \tag{27.4}$$

The **advantage function** quantifies the relative benefit of taking action a in state s compared to the average policy behavior:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \tag{27.5}$$

Bellman expectation equations. By the Markov property and the definition of V^π, we derive a recursive relationship:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[R(s, a) + \gamma \, V^\pi(s') \mid s \right] \tag{27.6}$$

$$= \sum_{a \in \mathcal{A}} \pi(a \mid s) \left[R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \, V^\pi(s') \right] \tag{27.7}$$

Analogously, for Q^π:

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \sum_{a' \in \mathcal{A}} \pi(a' \mid s') \, Q^\pi(s', a') \tag{27.8}$$

In vector form for finite state spaces, let $\mathbf{V}^\pi \in \mathbb{R}^{|\mathcal{S}|}$ be the vector of state values, $\mathbf{R}^\pi \in \mathbb{R}^{|\mathcal{S}|}$ with $R^\pi(s) = \sum_a \pi(a \mid s) R(s,a)$, and $\mathbf{P}^\pi \in \mathbb{R}^{|\mathcal{S}|\times|\mathcal{S}|}$ the policy-averaged transition matrix with $P^\pi(s,s') = \sum_a \pi(a \mid s) P(s' \mid s,a)$. Then:

$$\mathbf{V}^\pi = \mathbf{R}^\pi + \gamma \mathbf{P}^\pi \mathbf{V}^\pi \tag{27.9}$$

Since $\gamma < 1$, the matrix $\mathbf{I} - \gamma \mathbf{P}^\pi$ is invertible (its eigenvalues have magnitude $\leq \gamma < 1$), and the unique solution is:

$$\mathbf{V}^\pi = (\mathbf{I} - \gamma \mathbf{P}^\pi)^{-1} \mathbf{R}^\pi \tag{27.10}$$

This is the fundamental linear equation underlying all value-based RL. For infinite state spaces (the continuous case), this generalizes to the integral operator form.

Bellman optimality equations. An optimal policy π^* maximizes $V^\pi(s)$ for all $s \in \mathcal{S}$. The optimal value function is:

$$V^*(s) = \max_\pi V^\pi(s) = \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s,a)\, V^*(s') \right] \tag{27.11}$$

The optimal action-value function is:

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \max_{a' \in \mathcal{A}} Q^*(s',a') \tag{27.12}$$

The optimal policy is deterministic: $\pi^*(s) = \arg\max_a Q^*(s,a)$. However, in practice, stochastic policies are often used for exploration and robustness.

27.2.3 Value-Based RL: Q-Learning and DQN

Tabular Q-learning [?] is the classic algorithm for learning Q^* without a model of the environment. The update rule for a transition (s_t, a_t, r_t, s_{t+1}) is:

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha \underbrace{\left[r_t + \gamma \max_{a'} Q(s_{t+1},a') - Q(s_t,a_t) \right]}_{\text{TD error } \delta_t} \tag{27.13}$$

where $\alpha \in (0,1]$ is the learning rate. This is a stochastic approximation to the Bellman optimality equation: the target $r_t + \gamma \max_{a'} Q(s_{t+1},a')$ is a biased but consistent estimate of the expected optimal value. Convergence to Q^* is guaranteed under standard Robbins-Monro conditions ($\sum_t \alpha_t = \infty, \sum_t \alpha_t^2 < \infty$) and the assumption that every state-action pair is visited infinitely often.

Deep Q-Network (DQN). [?] The state space in realistic domains (e.g., video games) is far too large for tabular representation. DQN approximates Q^* with a neural network $Q_\theta(s,a)$ parameterized by θ. Two key innovations stabilize training:

(i) Experience Replay. Transitions (s,a,r,s') are stored in a replay buffer $\mathcal{D}$ with capacity N_{replay}. At each training step, a mini-batch of B transitions is sampled uniformly from $\mathcal{D}$. This breaks the strong temporal correlations present in online interaction data and improves sample efficiency by reusing data.

(ii) Target Network. A separate target network Q_{θ^-} with parameters θ^- is used to compute the TD target, while the online network Q_θ is used for action selection. θ^- is updated periodically (every C steps) by copying $\theta^- \leftarrow \theta$, and is held fixed between

updates. This prevents the "moving target" problem where the target depends on the same parameters being optimized.

The DQN loss at iteration i is:

$$\mathcal{L}_{\text{DQN}}(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma \max_{a'} Q_{\theta_i^-}(s', a') - Q_{\theta_i}(s, a)\right)^2\right] \tag{27.14}$$

Gradients are computed with respect to θ_i only through the $Q_{\theta_i}(s, a)$ term; the target is treated as constant.

Double DQN. [?] DQN's max operator introduces a systematic overestimation bias. By Jensen's inequality, $\mathbb{E}[\max_{a'} Q(s', a')] \geq \max_{a'} \mathbb{E}[Q(s', a')]$, so the expected maximum of noisy estimates overestimates the true maximum. Double DQN decouples action selection from action evaluation:

$$y_t^{\text{DDQN}} = r_t + \gamma\, Q_{\theta^-}\left(s_{t+1}, \arg\max_{a'} Q_\theta(s_{t+1}, a')\right) \tag{27.15}$$

The online network selects the action, and the target network evaluates it. This simple change significantly reduces overestimation and improves performance.

Dueling DQN. [?] The Q-function is decomposed into a state value and state-dependent action advantages:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s, a'; \theta, \alpha)\right) \tag{27.16}$$

where θ are shared parameters, α are advantage stream parameters, and β are value stream parameters. The average subtraction ensures identifiability (the advantage of the optimal action is forced to zero on average). This architecture learns which states are valuable without needing to learn the effect of each action for every state, improving efficiency when many actions have similar consequences.

Prioritized Experience Replay. [?] Not all transitions are equally informative. Transitions with high TD error δ_i are more "surprising" and should be replayed more frequently. The sampling probability is:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \quad p_i = |\delta_i| + \epsilon \tag{27.17}$$

where $\epsilon > 0$ ensures all transitions have non-zero probability, and α controls the degree of prioritization ($\alpha = 0$ recovers uniform sampling). Importance-sampling weights $w_i = (N \cdot P(i))^{-\beta}$ correct the bias introduced by non-uniform sampling, with β annealed to 1 over training.

27.2.4 Policy Gradient Methods

Value-based methods derive a policy implicitly from a learned value function ($a = \arg\max_a Q(s, a)$). Policy-based methods directly parameterize the policy $\pi_\theta(a \mid s)$ and optimize the expected return via gradient ascent.

Policy gradient theorem. [?] Let $J(\theta)$ be the expected return under policy π_θ:

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right] = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] \tag{27.18}$$

where $R(\tau) = \sum_t \gamma^t r_t$ is the trajectory return. We derive $\nabla_\theta J(\theta)$:

$$\nabla_\theta J(\theta) = \nabla_\theta \int R(\tau)\, p_\theta(\tau)\, d\tau \tag{27.19}$$

$$= \int R(\tau)\, \nabla_\theta p_\theta(\tau)\, d\tau \tag{27.20}$$

$$= \int R(\tau)\, p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, d\tau \tag{27.21}$$

$$= \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\, \nabla_\theta \log p_\theta(\tau)\right] \tag{27.22}$$

where we used the "log-derivative trick": $\nabla_\theta p_\theta = p_\theta \nabla_\theta \log p_\theta$. The trajectory log-probability decomposes as:

$$\log p_\theta(\tau) = \log \rho_0(s_0) + \sum_{t=0}^{T} \left[\log \pi_\theta(a_t \mid s_t) + \log P(s_{t+1} \mid s_t, a_t)\right] \tag{27.23}$$

Since transition dynamics P and initial distribution ρ_0 do not depend on θ, their gradients vanish:

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \tag{27.24}$$

Thus the policy gradient is:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau)\right] \tag{27.25}$$

This is the **REINFORCE** algorithm [**?**]. Intuitively, it increases the log-probability of actions that were part of trajectories with high total return, and decreases those that led to low return. The use of the full trajectory return $R(\tau)$ introduces high variance, since many actions in a trajectory have little to do with the eventual outcome.

Variance reduction with baselines. Subtracting a baseline $b(s_t)$ that does not depend on the action preserves the gradient expectation (since $\mathbb{E}_{a\sim\pi}[\nabla_\theta \log \pi_\theta(a \mid s) \cdot b(s)] = b(s)\nabla_\theta \sum_a \pi_\theta(a \mid s) = 0$) while reducing variance:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, (R(\tau) - b(s_t))\right] \tag{27.26}$$

A natural choice for the baseline is the state value function $V^\pi(s_t)$, giving the advantage estimator:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A^\pi(s_t, a_t)\right] \tag{27.27}$$

27.2.5 Actor-Critic Methods

Actor-critic methods [**?**] combine the strengths of value-based and policy-based RL: the **actor** (π_θ) selects actions, and the **critic** (V_ϕ or Q_ϕ) evaluates them, providing a low-variance advantage estimate.

Advantage Actor-Critic (A2C). The critic V_ϕ approximates the true value function. The TD error serves as an unbiased estimate of the advantage:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \tag{27.28}$$

One can show that $\mathbb{E}[\delta_t \mid s_t] = A^\pi(s_t, a_t)$ (assuming $V_\phi = V^\pi$, though in practice it's an approximation). The A2C updates are:

$$\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \delta_t \tag{27.29}$$

$$\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \frac{1}{2}\delta_t^2 \tag{27.30}$$

The critic minimizes the squared TD error (mean-squared Bellman error), and the actor maximizes the advantage-weighted log-likelihood.

Generalized Advantage Estimation (GAE). [**?**] The n-step TD error is:

$$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{27.31}$$

GAE defines the advantage as an exponentially weighted sum of k-step TD errors:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l\, \delta_{t+l}^V \tag{27.32}$$

where $\lambda \in [0, 1]$ controls the bias-variance trade-off. $\lambda = 0$ gives the one-step TD error (low variance, high bias if V is inaccurate). $\lambda = 1$ gives the Monte Carlo return minus the baseline (unbiased, high variance). Intermediate values interpolate smoothly. The discount γ controls the effective horizon; $\gamma\lambda$ appears as the combined discount applied to future TD errors.

27.2.6 Proximal Policy Optimization (PPO)

PPO [**?**] addresses the instability of policy gradient methods by constraining the size of policy updates. It is a first-order approximation to Trust Region Policy Optimization (TRPO) [**?**] that is simpler to implement and empirically more effective.

Let $\pi_{\theta_{\text{old}}}$ be the policy before the update, and π_θ the current policy. The probability ratio is:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{27.33}$$

The **clipped surrogate objective** prevents destructively large policy updates:

$$\mathcal{L}^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t \right) \right] \tag{27.34}$$

The clipping parameter ε (typically 0.2) bounds the ratio to $[1-\varepsilon, 1+\varepsilon]$. The intuition: when $\hat{A}_t > 0$ (action was better than expected), the objective wants $r_t > 1$ (increase probability), but the clip prevents r_t from exceeding $1+\varepsilon$, avoiding excessively large increases. When $\hat{A}_t < 0$ (action was worse), the objective wants $r_t < 1$ (decrease probability), but the clip prevents r_t from falling below $1-\varepsilon$. The min operator selects the more pessimistic bound, ensuring the objective is a lower bound on the unclipped objective.

The full PPO loss combines the clipped surrogate, a value function loss (squared error against empirical returns), and an entropy bonus to encourage exploration:

$$\mathcal{L}(\theta,\phi) = \mathbb{E}_t\left[\mathcal{L}^{\text{CLIP}}(\theta) - c_1\underbrace{(V_\phi(s_t) - V_t^{\text{target}})^2}_{\mathcal{L}^{\text{VF}}} + c_2\underbrace{H(\pi_\theta(\cdot \mid s_t))}_{\text{entropy bonus}}\right] \tag{27.35}$$

where $H(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s)\log\pi(a \mid s)$ is the policy entropy, and c_1, c_2 are coefficients. The entropy bonus prevents premature convergence to a deterministic policy, maintaining exploration throughout training.

PPO updates are performed using multiple epochs of mini-batch SGD on data collected under $\pi_{\theta_{\text{old}}}$ (typically 3–10 epochs before collecting new data). This is in contrast to TRPO's conjugate gradient optimization, which requires computing and inverting Fisher information matrices.

27.2.7 Maximum Entropy RL: Soft Actor-Critic (SAC)

SAC [**?**] augments the standard RL objective with an entropy term, encouraging stochastic policies that not only maximize reward but also maximize entropy (remain exploratory):

$$J_{\text{SAC}}(\pi) = \sum_{t=0}^{T}\mathbb{E}_{(s_t,a_t)\sim\rho_\pi}\left[r(s_t,a_t) + \alpha\, H(\pi(\cdot \mid s_t))\right] \tag{27.36}$$

where α (the "temperature") controls the stochasticity of the optimal policy. The soft Q-function and soft value function satisfy modified Bellman equations:

$$Q_{\text{soft}}(s,a) = r(s,a) + \gamma\mathbb{E}_{s'}\left[V_{\text{soft}}(s')\right] \tag{27.37}$$

$$V_{\text{soft}}(s) = \mathbb{E}_{a\sim\pi}\left[Q_{\text{soft}}(s,a) - \alpha\log\pi(a \mid s)\right] \tag{27.38}$$

SAC is an off-policy actor-critic algorithm with a stochastic actor, enabling sample-efficient learning on continuous action spaces while maintaining robust exploration through entropy maximization.

27.2.8 Reinforcement Learning from Human Feedback (RLHF)

RLHF [**?**] [**?**] is the technique underlying the alignment of modern large language models (ChatGPT, Claude, Gemini) with human values and preferences. The pipeline consists of three stages:

Stage 1: Supervised Fine-Tuning (SFT). A pre-trained language model π_{LM} is fine-tuned on a dataset of high-quality human demonstrations $\mathcal{D}_{\text{SFT}} = \{(x,y)\}$ where x is a prompt and y is a desirable response. The model π_{SFT} is obtained by maximizing $\mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{SFT}}}[\log\pi(y \mid x)]$.

Stage 2: Reward Modeling. Human annotators compare pairs of responses (y_w, y_l) to prompt x, indicating which is preferred ($y_w \succ y_l$, where y_w is the preferred response). The Bradley-Terry model [**?**] posits that the probability of preferring y_w over y_l is:

$$P(y_w \succ y_l \mid x) = \frac{\exp(r_\phi(x,y_w))}{\exp(r_\phi(x,y_w)) + \exp(r_\phi(x,y_l))} = \sigma(r_\phi(x,y_w) - r_\phi(x,y_l)) \tag{27.39}$$

where $r_\phi(x, y)$ is the scalar reward predicted by a neural network (initialized from π_{SFT} with a value head). The reward model is trained by minimizing the negative log-likelihood:

$$\mathcal{L}_R(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right] \tag{27.40}$$

Stage 3: RL Fine-Tuning. The language model π_θ (initialized from π_{SFT}) is fine-tuned to maximize the reward predicted by r_ϕ, while regularized by a KL divergence penalty to prevent it from diverging too far from the SFT model (which would lead to reward hacking—generating high-reward but nonsensical text):

$$\max_\theta \ \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_\theta(\cdot|x)}\left[r_\phi(x, y) - \beta \ \text{KL}\left(\pi_\theta(\cdot \mid x) \,\middle\|\, \pi_{\text{SFT}}(\cdot \mid x)\right)\right] \tag{27.41}$$

where β controls the strength of the KL penalty. Optimization uses PPO, where the advantage is defined using the reward model's output minus the KL penalty, and the value function critic estimates this penalized reward. The KL divergence can be computed analytically per token: $\text{KL}(\pi_\theta \| \pi_{\text{SFT}}) = \sum_a \pi_\theta(a \mid s)(\log \pi_\theta(a \mid s) - \log \pi_{\text{SFT}}(a \mid s))$.

The RLHF formulation elegantly solves the problem of aligning a generative model with complex, ill-defined human preferences that cannot be captured by a simple loss function.

27.3 Algorithms

Algorithm 36: Deep Q-Network (DQN) with Experience Replay

Input : Environment with state space $\mathcal{S}$, action space $\mathcal{A}$
Output: Trained Q-network Q_θ

Initialize replay buffer $\mathcal{D}$ with capacity N;
Initialize online network Q_θ and target network Q_{θ^-} with $\theta^- \leftarrow \theta$;
for *episode* $= 1$ **to** M **do**
  $s \leftarrow$ initial state;
  for $t = 1$ **to** T **do**
    Select a via ε-greedy: $a = \begin{cases} \text{random action} & \text{with prob } \varepsilon \\ \arg\max_a Q_\theta(s, a) & \text{otherwise} \end{cases}$;
    Execute a, observe r, s', done;
    Store $(s, a, r, s', \text{done})$ in $\mathcal{D}$;
    Sample random minibatch $\mathcal{B}$ of B transitions from $\mathcal{D}$;
    foreach *transition* $(s_j, a_j, r_j, s'_j, d_j)$ *in* $\mathcal{B}$ **do**
      $y_j \leftarrow \begin{cases} r_j & \text{if } d_j \\ r_j + \gamma \max_{a'} Q_{\theta^-}(s'_j, a') & \text{otherwise} \end{cases}$;
    Update θ by SGD on $\frac{1}{B}\sum_j (y_j - Q_\theta(s_j, a_j))^2$;
    Every C steps: $\theta^- \leftarrow \theta$;
    $s \leftarrow s'$;
return Q_θ

Algorithm 37: Proximal Policy Optimization (PPO)

Input : Policy π_θ, value function V_ϕ, environment, hyperparameters $\varepsilon, c_1, c_2, N_{\text{steps}}, N_{\text{epochs}}$

Output: Optimized policy π_θ

Initialize θ, ϕ;

for *iteration* $= 1$ **to** M **do**

  Collect N_{steps} transitions $\{(s_t, a_t, r_t, s_{t+1})\}$ using π_θ;

  Compute returns: $G_t \leftarrow r_t + \gamma r_{t+1} + \cdots + \gamma^{T-t} V_\phi(s_T)$ `// truncated returns`

  Compute advantage estimates: $\hat{A}_t \leftarrow \text{GAE}(r_t, V_\phi(s_t), V_\phi(s_{t+1}), \gamma, \lambda)$;

  for *epoch* $= 1$ **to** N_{epochs} **do**

    Sample minibatches from the collected trajectory;

    foreach *minibatch* **do**

      Compute $r_t(\theta) \leftarrow \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$;

      $\mathcal{L}^{\text{CLIP}} \leftarrow \frac{1}{B}\sum_t \min(r_t \hat{A}_t, \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t)$;

      $\mathcal{L}^{\text{VF}} \leftarrow \frac{1}{B}\sum_t (V_\phi(s_t) - G_t)^2$;

      $\mathcal{L}^{\text{entropy}} \leftarrow \frac{1}{B}\sum_t H(\pi_\theta(\cdot \mid s_t))$;

      $\mathcal{L} \leftarrow -\mathcal{L}^{\text{CLIP}} + c_1 \mathcal{L}^{\text{VF}} - c_2 \mathcal{L}^{\text{entropy}}$;

      Update θ, ϕ via Adam;

  $\theta_{\text{old}} \leftarrow \theta$;

return π_θ

Algorithm 38: REINFORCE with Baseline (Policy Gradient)

Input : Policy π_θ, baseline V_ϕ, environment, learning rates $\alpha_\theta, \alpha_\phi$

Output: Optimized policy π_θ

for *episode* $= 1$ **to** M **do**

  Generate trajectory $\tau = (s_0, a_0, r_0, \ldots, s_T)$ using π_θ;

  $R \leftarrow 0$;

  for $t = T$ **to** 0 **do**

    $R \leftarrow r_t + \gamma R$;

    $\delta_t \leftarrow R - V_\phi(s_t)$;

    $\theta \leftarrow \theta + \alpha_\theta \, \gamma^t \, \delta_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$;

    $\phi \leftarrow \phi - \alpha_\phi \, \delta_t \, \nabla_\phi V_\phi(s_t)$;

return π_θ

27.4 Architecture

DQN architecture. The original DQN [**?**] for Atari games takes raw $84 \times 84 \times 4$ pixel inputs (a stack of 4 consecutive frames to provide motion information) and passes them through three convolutional layers: Conv1 (8×8, stride 4, 32 filters, ReLU) $\rightarrow$ Conv2 (4×4, stride 2, 64 filters, ReLU) $\rightarrow$ Conv3 (3×3, stride 1, 64 filters, ReLU). The flattened features are fed into a fully connected layer (512 units, ReLU) and then the output layer with $|\mathcal{A}|$ units (one Q-value per action). The network outputs all Q-values in a single forward pass (rather than requiring $|\mathcal{A}|$ separate network evaluations with the action as input), which is efficient for discrete action spaces up to a few hundred actions.

Dueling DQN architecture. After the shared convolutional backbone and the 512-unit FC layer, the network splits into two streams: (i) a value stream (one fully connected layer to 512 units, then one to 1 scalar $V(s)$) and (ii) an advantage stream (one FC to 512 units, then one to $|\mathcal{A}|$ scalars $A(s, a)$). The final Q-values are produced by the aggregation formula in (27.16).

Actor-Critic architecture. In A2C/A3C for continuous control, the actor and critic typically share a convolutional or MLP backbone and diverge at the final layers. For Atari, the architecture mirrors DQN but with two output heads: the policy head (softmax over $|\mathcal{A}|$ actions) and the value head (a single scalar $V(s)$). For continuous control (MuJoCo, robotics), the backbone is an MLP with 2–3 hidden layers of 64–256 units each, with the actor outputting the mean and log-standard-deviation of a Gaussian policy, and the critic outputting a scalar value.

PPO architecture. PPO reuses the same architecture as A2C, typically with orthogonal initialization of weights. The actor outputs the parameters of an action distribution: for discrete actions, a categorical distribution via softmax; for continuous actions, a diagonal Gaussian parameterized by mean $\mu_\theta(s)$ and log-standard-deviation $\log \sigma_\theta$ (often a state-independent learnable vector). The entropy bonus in (27.35) is computed as $H = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$ for discrete or $H = \frac{1}{2} \sum_i (1 + \log(2\pi\sigma_i^2))$ for continuous Gaussian policies.

SAC architecture. SAC uses two Q-networks (the minimum of two estimates is used for the target, reducing overestimation bias), one policy network, and one target value network. The policy outputs mean and log-standard-deviation of a squashed Gaussian: $a = \tanh(\mu_\theta(s) + \sigma_\theta(s) \odot \epsilon)$ with $\epsilon \sim \mathcal{N}(0, \mathbf{I})$. The tanh squashing bounds actions to $[-1, 1]$, and the log-probability is adjusted by the change-of-variables formula: $\log \pi(a \mid s) = \log \mathcal{N}(u \mid \mu, \sigma^2) - \sum_i \log(1 - \tanh^2(u_i))$ where $a = \tanh(u)$.

27.5 Pros and Cons

Value-based methods (DQN and variants). *Advantages:* (i) Off-policy: can learn from any data, enabling high sample efficiency via experience replay. (ii) Simple and theoretically grounded: directly approximates the optimal Q-function. (iii) Deterministic policy (argmax) is stable during deployment. *Disadvantages:* (i) Max operator induces overestimation bias (partially addressed by Double DQN). (ii) Cannot handle continuous action spaces efficiently (the argmax becomes an optimization problem). (iii) Implicit policy is deterministic; no principled exploration mechanism beyond ε-greedy. (iv) Sample inefficiency from uniform replay: most transitions are uninformative (prioritized replay helps).

Policy gradient methods (REINFORCE, A2C). *Advantages:* (i) Naturally handle continuous action spaces via parameterized distributions. (ii) Can learn stochastic policies, important for partially observable environments and game theory. (iii) Directly optimize the objective of interest (expected return). *Disadvantages:* (i) On-policy: requires fresh data from the current policy, sample-inefficient. (ii) High variance in gradient estimates (mitigated by baselines and GAE). (iii) Sensitive to learning rate and other hyperparameters. (iv) Convergence to local optima; the policy landscape is often highly non-convex.

PPO. *Advantages:* (i) Stable and reliable: the clipped objective prevents catastrophic updates, making PPO the default algorithm for many applications. (ii) First-order: no costly Hessian computations (unlike TRPO). (iii) Good sample complexity for an on-policy algorithm. (iv) Widely adopted: extensive empirical validation across domains. *Disadvantages:* (i) Still on-policy, so sample efficiency is lower than off-policy alternatives (SAC). (ii) Sensitive to the clipping parameter ε and the number of epochs per data batch. (iii) Code-level complexity: many implementation details (GAE, advantage normalization, value clipping, learning rate annealing) are critical for performance.

SAC. *Advantages:* (i) Off-policy: high sample efficiency, crucial for real-world robotics. (ii) Maximum entropy framework provides automatic exploration tuning. (iii) Robust to hyperparameter variation. (iv) State-of-the-art on continuous control benchmarks. *Disadvantages:* (i) More complex to implement than PPO. (ii) Requires reward scaling/tuning for optimal performance. (iii) May struggle in environments with sparse rewards without additional exploration incentives.

RLHF. *Advantages:* (i) Aligns models with complex, subjective human preferences that are difficult to specify programmatically. (ii) Scalable: the reward model can be trained on large-scale human comparison data. (iii) KL penalty prevents reward hacking. *Disadvantages:* (i) Expensive and time-consuming: requires collecting human preference data for each domain. (ii) Reward model is imperfect and may have blind spots (reward overoptimization). (iii) PPO + KL penalty is a delicate optimization; hyperparameters require careful tuning. (iv) The alignment tax: RLHF may reduce the model's raw capabilities.

27.6 Evolution

The evolution of deep reinforcement learning can be charted through a sequence of conceptual breakthroughs. The **tabular era** (1950s–1990s) established the mathematical foundations: Bellman equations (1957), dynamic programming, TD learning [**?**], and Q-learning [**?**]. The **deep RL breakthrough**: DQN [**?**] (2015, Atari from pixels) demonstrated that neural networks could approximate value functions in high-dimensional state spaces, combining three innovations (replay, target network, CNN).

The **policy gradient renaissance**: REINFORCE [**?**] → TRPO [**?**] (2015, trust regions for stable policy updates) → PPO [**?**] (2017, clipped surrogate, dominant on-policy algorithm) → A2C/A3C [**?**] (2016, asynchronous parallel training for stability).

The **off-policy actor-critic era**: DDPG [**?**] (2015, DQN adapted for continuous actions via deterministic policy gradient) → TD3 [**?**] (2018, clipped double Q-learning, delayed policy updates) → SAC [**?**] (2018, maximum entropy, stochastic policy, state-of-the-art sample efficiency).

The **model-based RL era**: World Models [**?**] (2018, learning environment dynamics

in latent space) → Dreamer [**?**] (2020, planning in latent space via RSSM) → MuZero [**?**] (2020, learning both dynamics and value without an explicit simulator, achieving superhuman performance in Atari, Go, chess, and shogi with a single algorithm).

The **RL as sequence modeling**: Decision Transformer [**?**] (2021) casts RL as a conditional sequence modeling problem: given a desired return R, previous states, actions, and returns-to-go, a Transformer autoregressively predicts the next action. This eliminates the need for Bellman backups and temporal difference learning, reframing RL as supervised learning on trajectory data.

The **RLHF era**: Christiano et al. [**?**] (2017, deep RL from human preferences) → InstructGPT/ChatGPT [**?**] (2022, RLHF for language models at scale) → Constitutional AI [**?**] (2022, AI feedback instead of human feedback) → DPO [**?**] (2023, direct preference optimization eliminates the need for explicit reward modeling and RL, reparameterizing the RLHF objective as a closed-form binary classification loss on the preference data).

The frontier continues to expand rapidly: multi-agent RL (learning cooperative and competitive behaviors), offline RL (learning from static datasets without online interaction), goal-conditioned RL, hierarchical RL, and the integration of RL with large language and vision-language models for embodied AI and generalist agents.